METHODS, DRAWN.

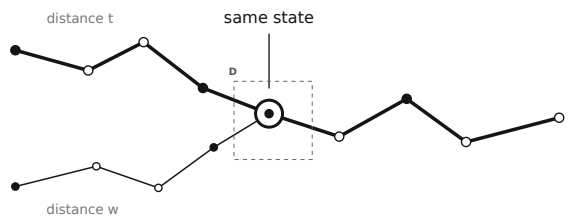
ENHANCED THEMATIC EDITION

225 mechanism plates / 14 visual families / enlarged vector details

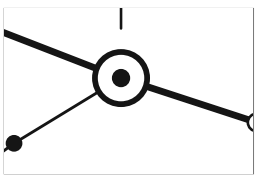
02 / TRACKS / COLLISIONS

M022

Tame/wild collision recovery



D / OPERATION DETAIL



Vector detail / 1.93× main-view scale

HOW THE OPERATION WORKS

A tame walk starts from a known scalar; a wild walk starts from the target point. Their meeting relates recorded distances and yields a candidate scalar.

READ THE THREE STAGES

01 tame walk
02 shared curve point
03 wild walk

WHAT WAS ACTUALLY DONE

collision.py groups same-x entries, tries tame/wild assignments and derives candidates. Because x alone has two point parities, the final scalar must pass exact public-key verification.

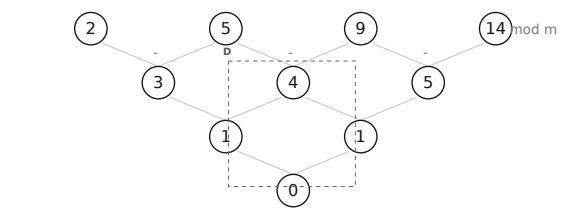
ORIGINAL FINDING / IMPLEMENTED

Same-point opposite-herd distances yield a scalar when the walk origins and distances are valid. No real target collision is established here.

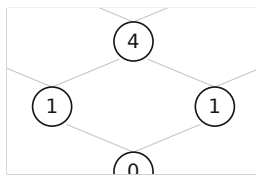
13 / LAYERS / DIFFERENCES

M202

Finite modular difference triangles



D / OPERATION DETAIL



Vector detail / 1.21× main-view scale

HOW THE OPERATION WORKS

Repeatedly subtract neighboring residues modulo m; finite rows shrink by one entry at each level.

READ THE THREE STAGES

01 finite residue row
02 neighbor subtraction
03 shrinking triangle

WHAT WAS ACTUALLY DONE

The experiment follows declared polynomial/exponential residue families through their full finite triangles. Reaching zero on supplied samples does not determine an unknown continuation without a recurrence assumption.

ORIGINAL FINDING / EXECUTED RECORD

Exact finite-row experiments track polynomial/exponential residues through all levels; they do not reconstruct an unknown puzzle key.

A closer view of each operation, with its mechanism, execution record, and original finding. Line hierarchy, grayscale, family motifs, and matched detail windows keep the visual language consistent.

Vector PDF + editable SVG + high-resolution PNG

THEMATIC INDEX

Each family has its own motif and operator vocabulary.



01 / COVERAGE / VERIFICATION

Verification, sampling and live search



02 / TRACKS / COLLISIONS

Kangaroo, interval ECDLP and execution backends



03 / VOLUMES / DECOMPOSITION

CTheory decompositions and scalar geometry



04 / TRACES / VALIDATION

Predictors, held-out validation and corrective audits



05 / BUCKETS / TRANSFER

Root-difference, carry, interpolation and near-miss routes



06 / REGISTERS / GENERATORS

Generator reconstruction, masking and deterministic sequences



07 / BITS / NEIGHBORHOODS

Sparse keys, bit walks and synthesis orbits



08 / SETS / TOPOLOGY

HP-RTE, topology and pattern-avoidance experiments



09 / SIGNATURES / CONSTRAINTS

ECDSA, nonce relations and signature-data searches



10 / GRAPHS / RELATIONS

On-chain, metadata and clue relationships



11 / SEEDS / ENTROPY

Weak-entropy wallet and PRNG research



12 / RESIDUES / STRUCTURE

Exploratory number theory, statistics and literature-inspired probes



13 / LAYERS / DIFFERENCES

Modular differences and general transformation frameworks

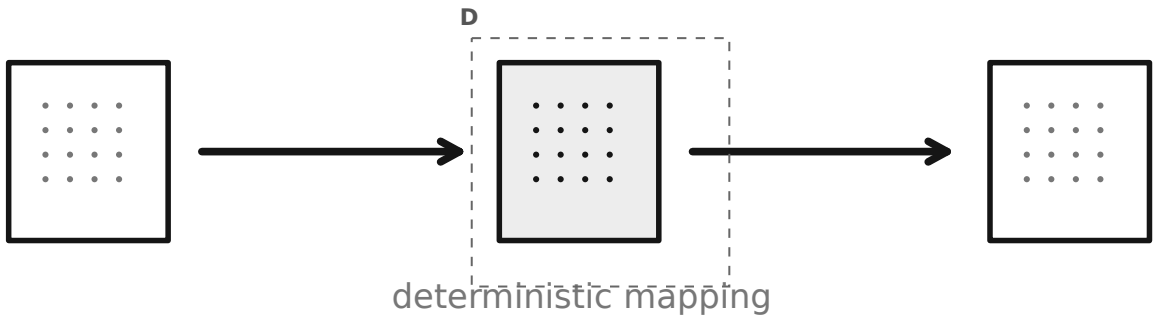


14 / CIRCUITS / RESOURCES

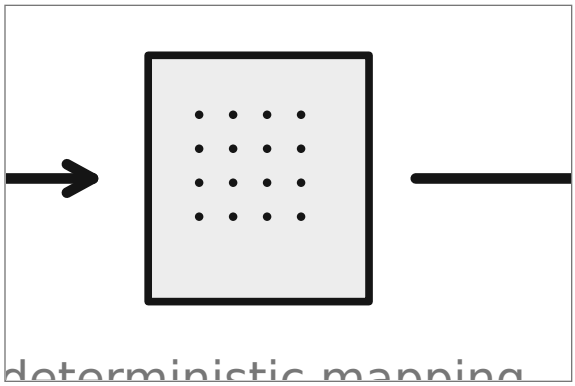
Shor, classical factoring and quantum-resource research

M001

Public puzzle corpus verification



D / OPERATION DETAIL



Vector detail / 1.38× main-view scale

HOW THE OPERATION WORKS

A candidate scalar determines a public point and address hash. Recomputing those values detects an incorrect published key or mismatched database row.

READ THE THREE STAGES

01 private scalar
02 public point
03 address hash

WHAT WAS ACTUALLY DONE

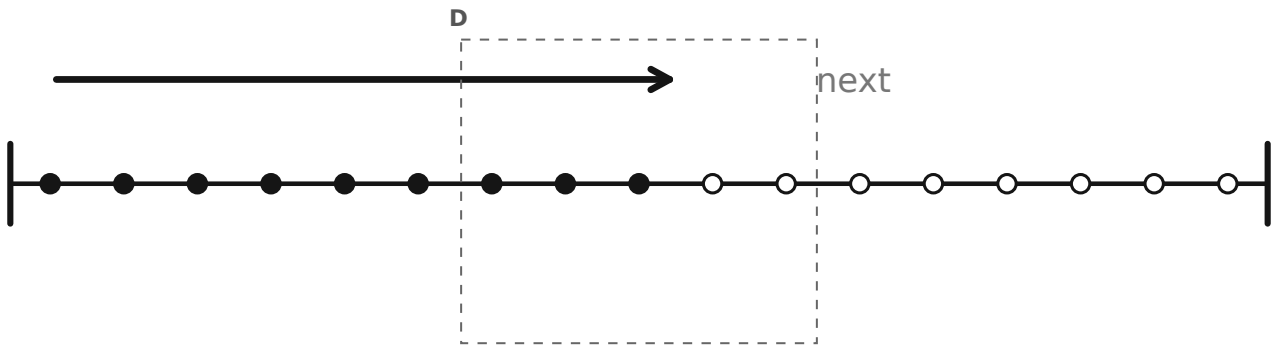
Read filled private-key rows, derive the recorded public-key format, and compare with stored point/HASH160 fields. Empty rows remain unknown; passing the check does not discover their keys.

ORIGINAL FINDING / EXECUTED RECORD

Recorded public scalars are checked against secp256k1 points and HASH160; validates the dataset, not a prediction.

M002

Small-range exhaustive address search

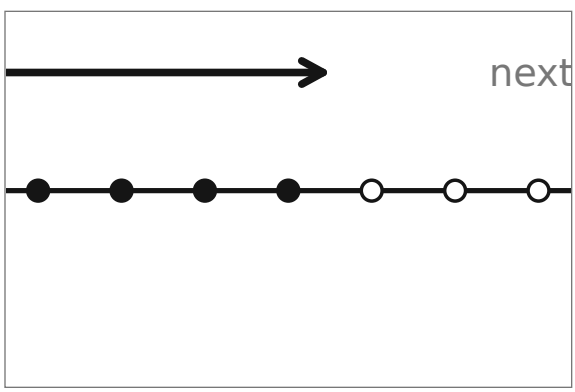


01 interval start

02 check each value

03 interval end

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Enumerate every integer in a genuinely small interval and accept only an address match. Exhaustion gives a complete answer for that interval.

READ THE THREE STAGES

01 interval start
02 check each value
03 interval end

WHAT WAS ACTUALLY DONE

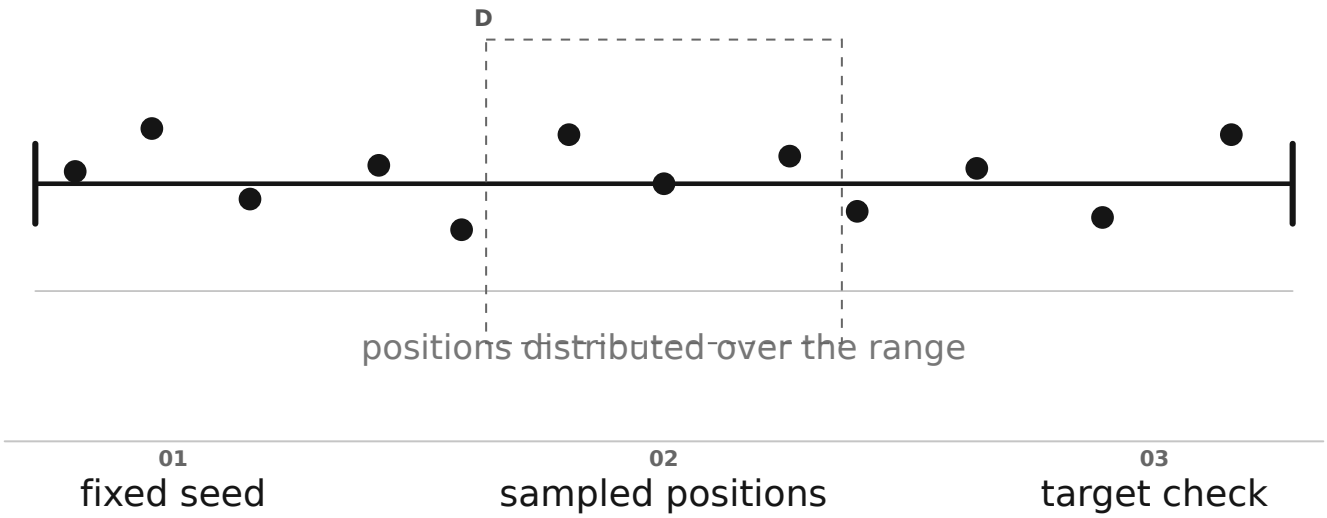
The educational tests scan puzzles 1-10 directly. Their keys are already public and used afterward to validate the results; this demonstrates a correct checker at small scale.

ORIGINAL FINDING / EXECUTED RECORD

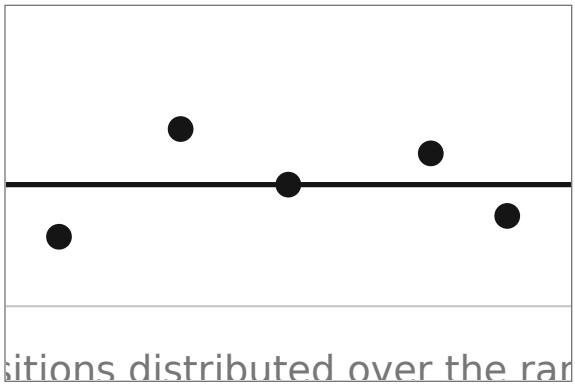
Puzzles 1-10 scanned completely; each yielded the expected published key. Small-range recovery is not an unknown-key breakthrough.

M003

Full-range seeded sampling backtests



D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

A seeded sampler chooses a reproducible order of candidates. The verifier stops when a candidate produces the target address.

READ THE THREE STAGES

01 fixed seed
02 sampled positions
03 target check

WHAT WAS ACTUALLY DONE

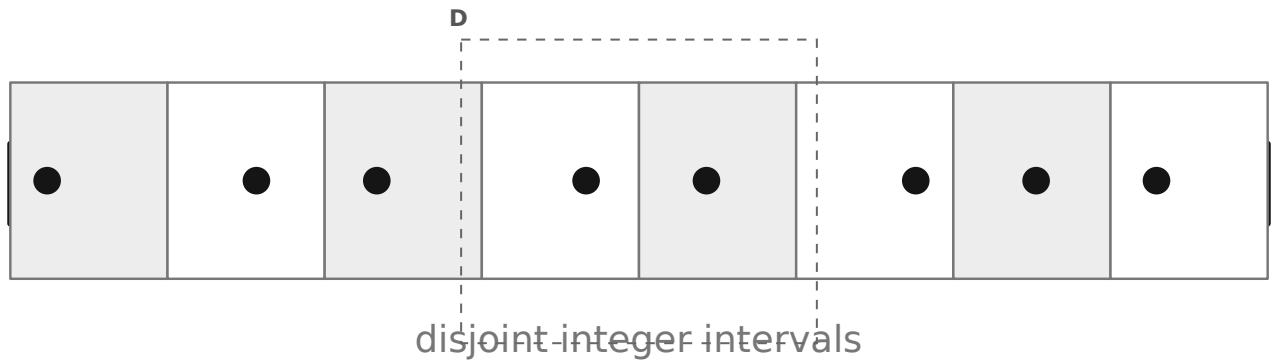
Saved subprocess backtests run the actual checker on small solved puzzles and compare recovered values afterward. Counts describe work until a match, not a learned shortcut.

ORIGINAL FINDING / EXECUTED RECORD

Actual address checkers recovered small published puzzles; seeded search order does not establish predictive narrowing.

M004

Uniform within-shard representatives

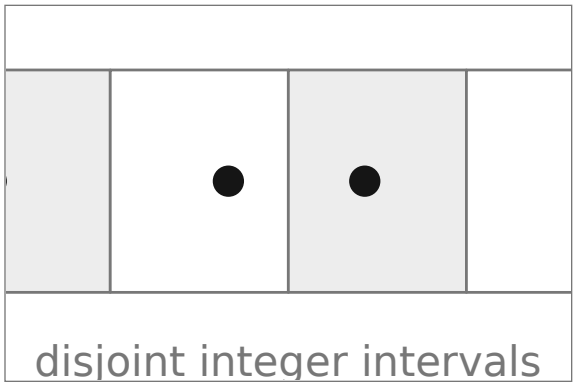


01 whole interval

02 one draw per shard

03 spread coverage

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Divide an interval into nonoverlapping shards and select one integer uniformly within each. This spreads a batch over the advertised range.

READ THE THREE STAGES

01 whole interval
02 one draw per shard
03 spread coverage

WHAT WAS ACTUALLY DONE

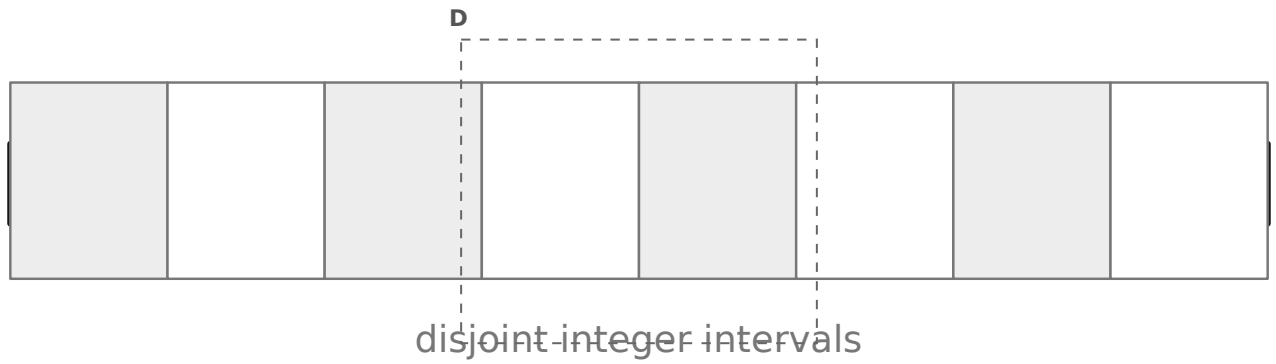
The corrected generator uses integer randrange calls. One record is one sampled candidate plus its shard boundaries; no other integer in that shard is thereby checked.

ORIGINAL FINDING / EXECUTED RECORD

Samples one integer per disjoint interval; representative checks do not exhaust their shard.

M005

Exact integer partition coverage

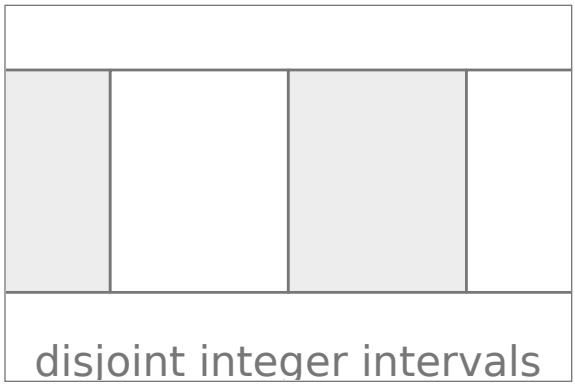


01
integer bounds

02
adjoining shards

03
no gaps

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Integer division places adjoining boundaries so the first shard starts at the lower limit and the last ends at the upper limit.

READ THE THREE STAGES

01 integer bounds
02 adjoining shards
03 no gaps

WHAT WAS ACTUALLY DONE

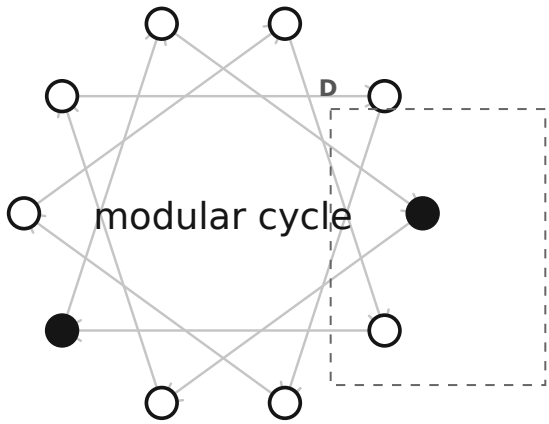
The corrected partitions function distributes the interval across the requested record count. Tests enumerate small examples to check every integer appears exactly once in the intervals.

ORIGINAL FINDING / EXECUTED RECORD

Gap-free inclusive intervals replace biased single-key records. Exhaustive small-range checks verify the partition arithmetic.

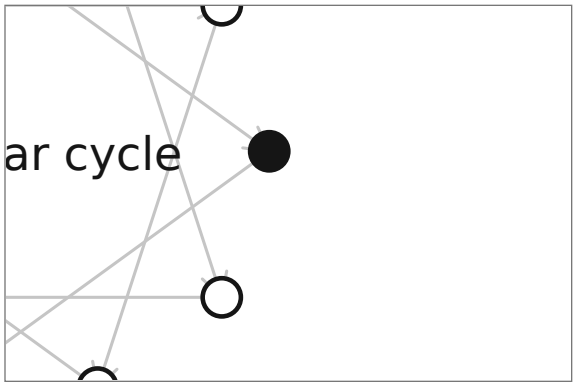
M006

No-repeat affine shard permutations



01	02	03
shard offsets	coprime stride	visit once

D / OPERATION DETAIL



Vector detail / 1.25× main-view scale

HOW THE OPERATION WORKS

An affine permutation visits offsets using (offset + index*stride) modulo shard size. A coprime stride prevents repetition before exhausting the shard.

READ THE THREE STAGES

01 shard offsets
02 coprime stride
03 visit once

WHAT WAS ACTUALLY DONE

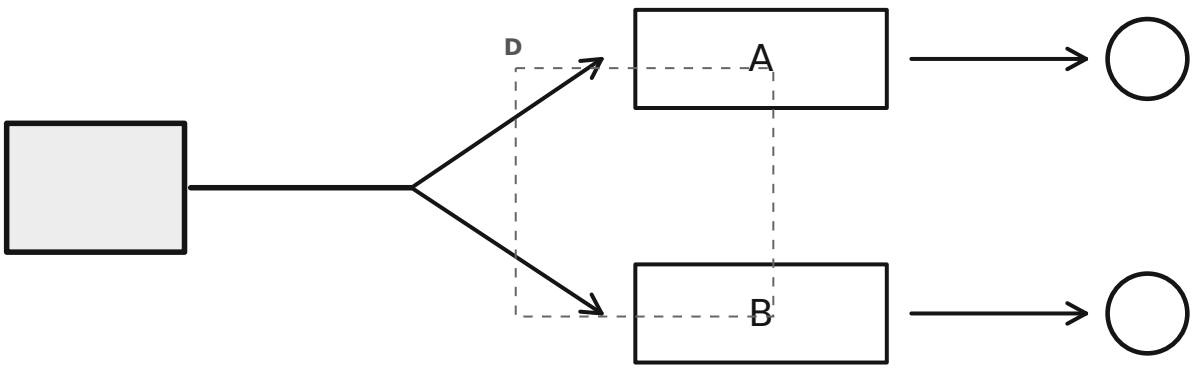
The samplers store per-shard offsets, strides and a traversal position. They reconstruct candidates without storing all previously visited keys; this improves coverage accounting, not target likelihood.

ORIGINAL FINDING / IMPLEMENTED

Coprime strides enumerate each shard without a visited-key set; retains full support.

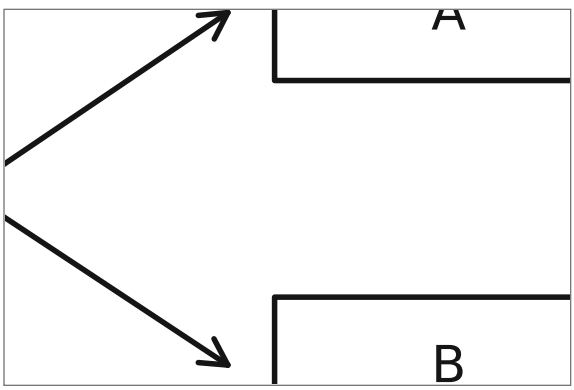
M007

One-million sample-key verification



01	02	03
stored samples	compressed / full	two hash checks

D / OPERATION DETAIL



Vector detail / 1.38× main-view scale

HOW THE OPERATION WORKS

Derive both public-key serializations for each stored sample and compare their HASH160 values with the decoded target hash.

READ THE THREE STAGES

01 stored samples
02 compressed / full
03 two hash checks

WHAT WAS ACTUALLY DONE

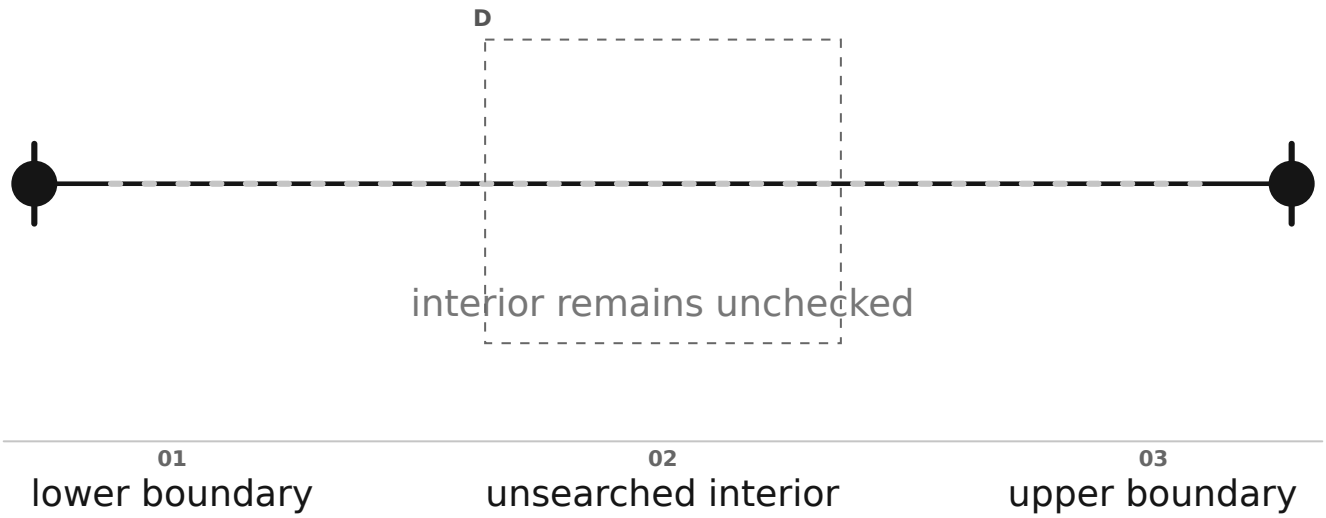
The verifier read the million-record JSONL file, padded each sample_key to 32 bytes, and performed two comparisons per scalar. The saved report records no match.

ORIGINAL FINDING / EXECUTED RECORD

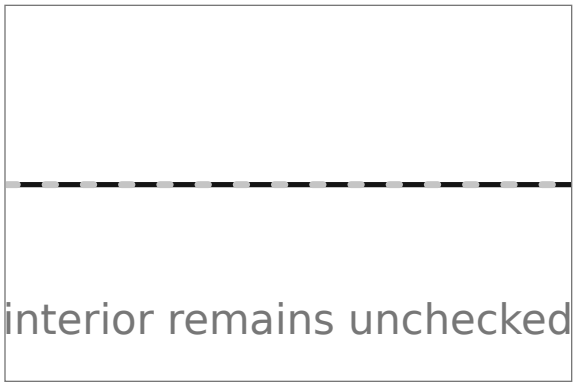
One million explicit samples checked in both public-key serializations; no target match.

M008

Range-start and range-end verification



D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

A range boundary is itself a possible private scalar and can be checked exactly like a sample. Checking it does not search its neighboring integers.

READ THE THREE STAGES

01 lower boundary
02 unsearched interior
03 upper boundary

WHAT WAS ACTUALLY DONE

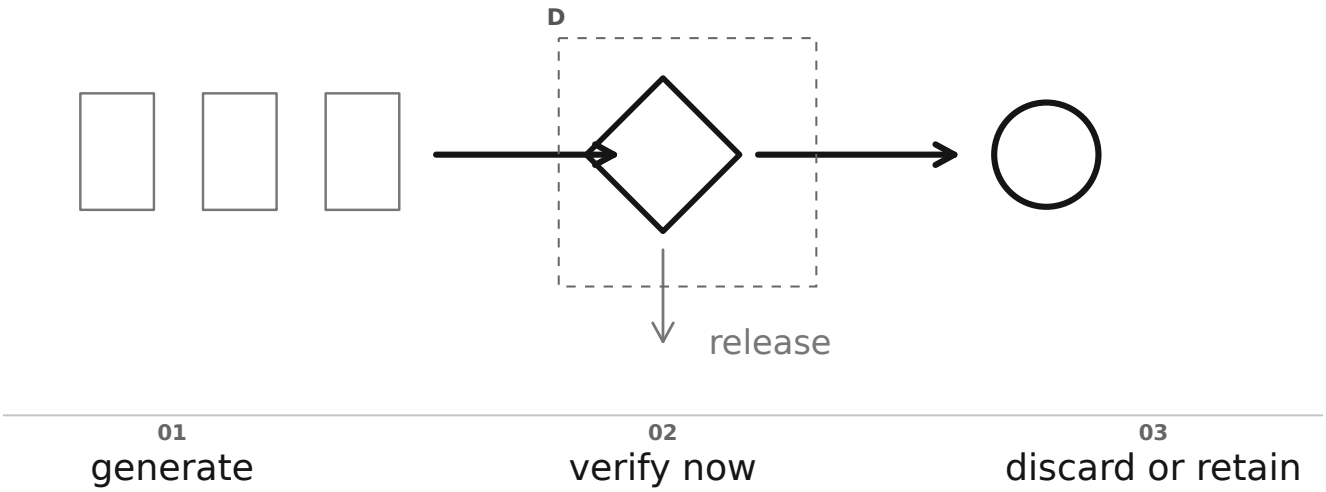
The endpoint run read range_start and range_end from every record and checked both address formats. It performed two million scalar checks, separately from the earlier samples.

ORIGINAL FINDING / EXECUTED RECORD

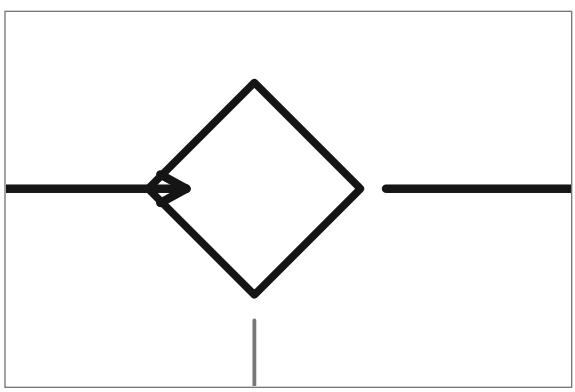
Two million endpoint entries checked in both address formats; no target match. Interiors were not searched.

M009

Generate-and-verify streaming batches



D / OPERATION DETAIL



Vector detail / 1.38× main-view scale

HOW THE OPERATION WORKS

Generate a candidate, verify it immediately, then discard it unless it matches. Memory does not grow with the batch size.

READ THE THREE STAGES

01 generate
02 verify now
03 discard or retain

WHAT WAS ACTUALLY DONE

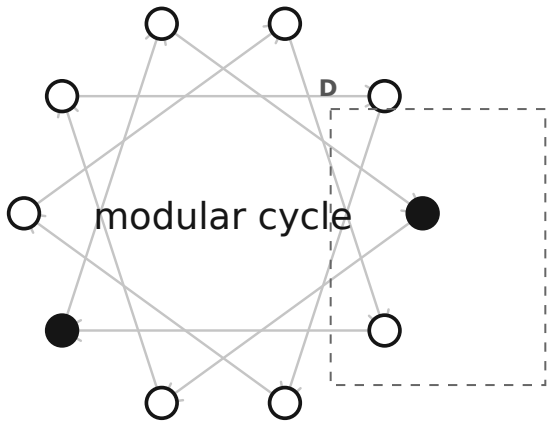
stream_check.py samples one key per partition and saves aggregate counts, a deterministic stream digest and any matches. The seed permits regeneration without a large candidate file.

ORIGINAL FINDING / EXECUTED RECORD

One million additional samples checked immediately without a large dataset; no matches in the saved batch.

M010

Continuous affine full-support search

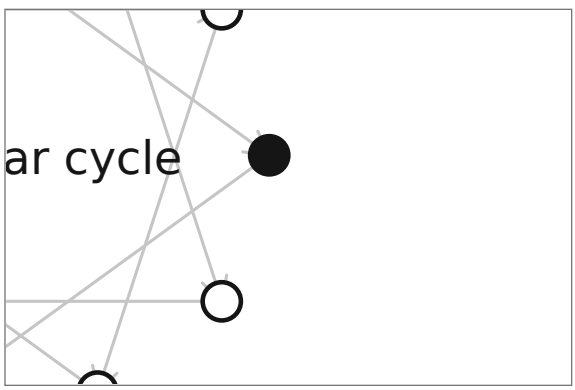


01
full offset ring

02
odd stride

03
complete cycle

D / OPERATION DETAIL



Vector detail / 1.25× main-view scale

HOW THE OPERATION WORKS

Apply one odd-stride affine permutation to all 2^70 interval offsets. Each offset occurs once over a complete cycle.

READ THE THREE STAGES

01 full offset ring
02 odd stride
03 complete cycle

WHAT WAS ACTUALLY DONE

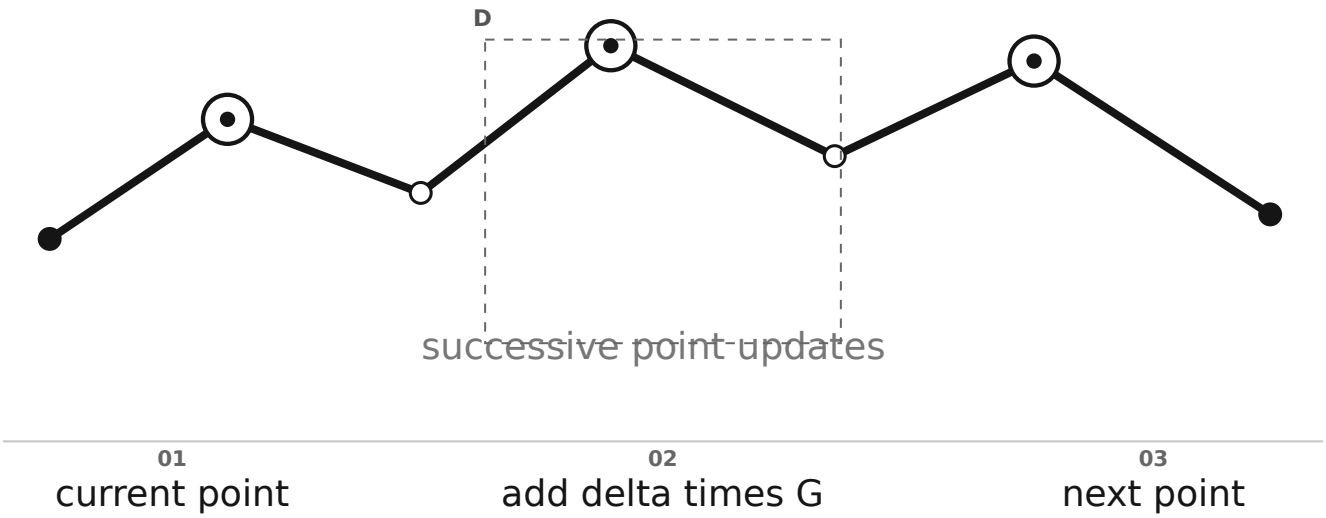
continuous_stream.py generates keys indefinitely until stopped, matched or exhausted. It checkpoints counters and permutation parameters; the process performs actual address checks, not predictor training.

ORIGINAL FINDING / LIVE SNAPSHOT

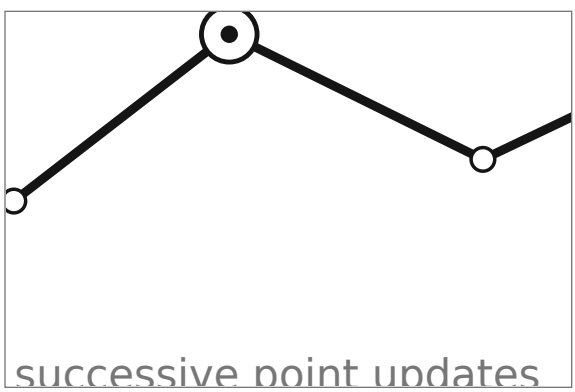
Checkpointed local worker checks an affine permutation of the 70-unknown-bit interval. Status is mutable and must be time-stamped.

M011

Incremental public-point verification



D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

If the next scalar differs by delta, its public point is the old point plus delta*G. Reusing that point avoids repeated full scalar multiplication.

READ THE THREE STAGES

01 current point
02 add delta times G
03 next point

WHAT WAS ACTUALLY DONE

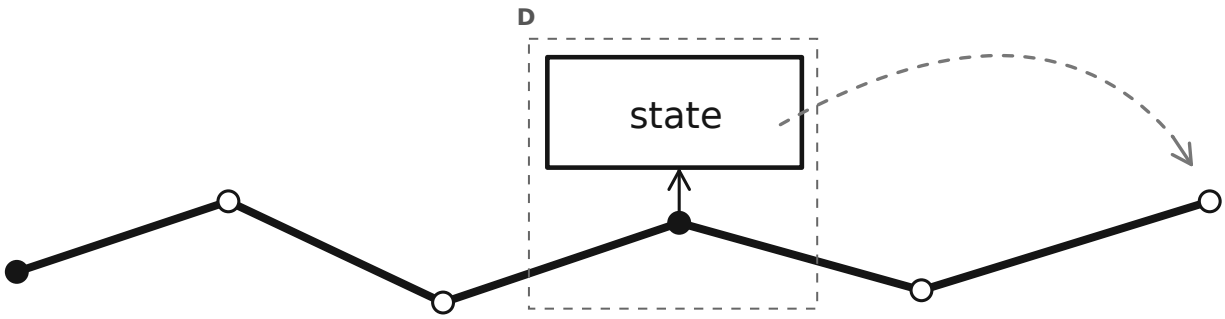
point_walk.py caches the two step points needed by the affine traversal, including wraparound. It still serializes and hashes every point; the candidate sequence is unchanged.

ORIGINAL FINDING / EXECUTED RECORD

Same scalar sequence verified using cached point increments; 900 comparison cases and paired benchmarks passed.

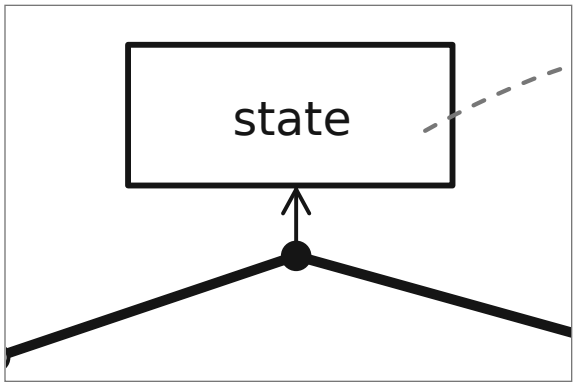
M012

Checkpoint resume and scalar cross-checks



01	02	03
walk position	save state	resume and cross-check

D / OPERATION DETAIL



Vector detail / 1.27× main-view scale

HOW THE OPERATION WORKS

Saving traversal state allows a restart at the next unchecked position. Independent scalar multiplication can detect incremental-point drift.

READ THE THREE STAGES

01 walk position
02 save state
03 resume and cross-check

WHAT WAS ACTUALLY DONE

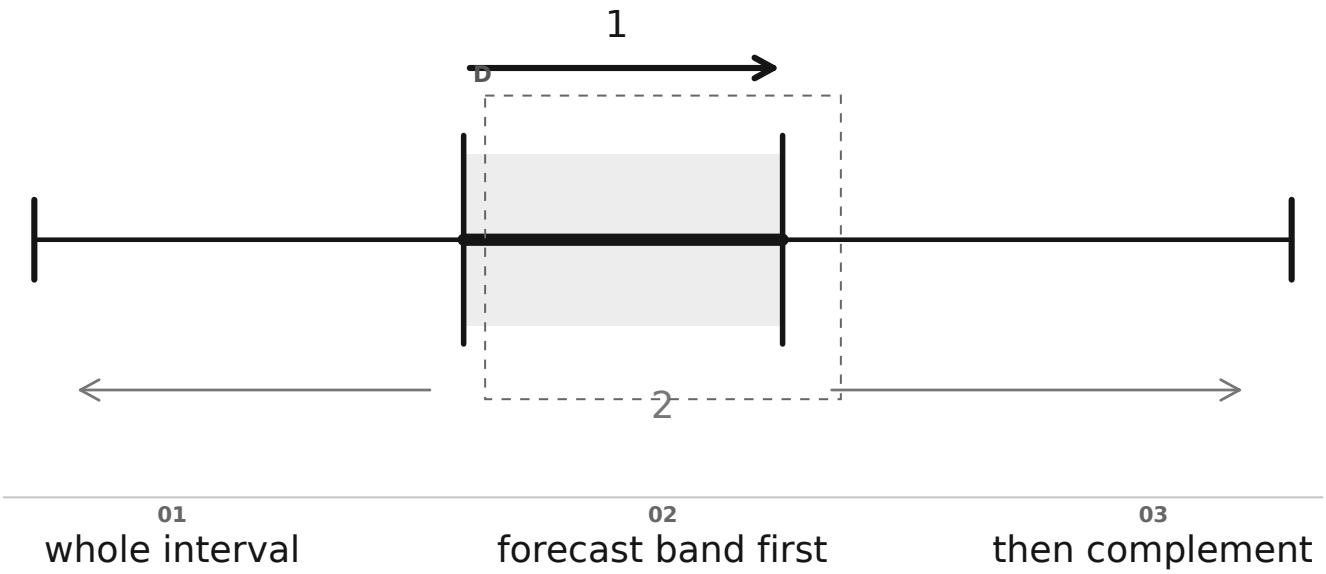
The worker writes an atomic status file and retains its offset/stride across restarts. The optimized version checks the first and each 10,000th point against the original verifier.

ORIGINAL FINDING / IMPLEMENTED

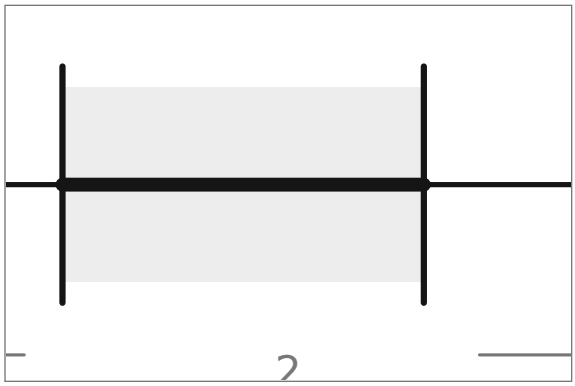
Live optimization resumes the exact saved position and compares against full scalar multiplication every 10,000 entries.

M013

FUSED priority-band traversal with fallback



D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Search a forecast band first, then its complement. This can improve order under a good predictor without permanently excluding any key.

READ THE THREE STAGES

01 whole interval
02 forecast band first
03 then complement

WHAT WAS ACTUALLY DONE

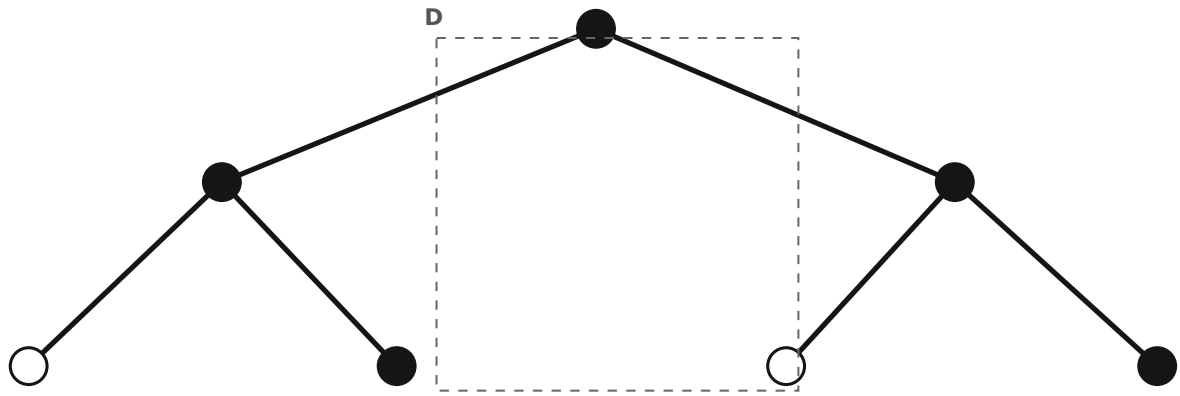
The fixed-budget educational comparison used affine permutations within the band and fallback regions. It checked bounded prefixes, not every integer in the forecast band.

ORIGINAL FINDING / EXECUTED RECORD

Fixed-budget test retained complementary regions; 286,720 checks across both methods found no keys.

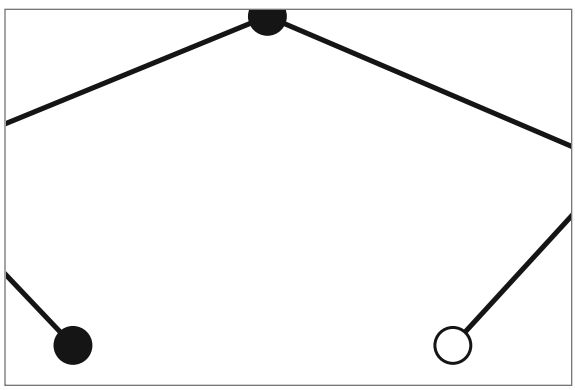
M014

Advanced selective FUSED/Delta2 adapter



01	02	03
root / coefficient	remainder choices	candidate intervals

D / OPERATION DETAIL



Vector detail / 0.98× main-view scale

HOW THE OPERATION WORKS

Turn predicted root/coefficient/remainder combinations into scalar intervals and sample those hypotheses instead of the entire range.

READ THE THREE STAGES

01 root / coefficient
02 remainder choices
03 candidate intervals

WHAT WAS ACTUALLY DONE

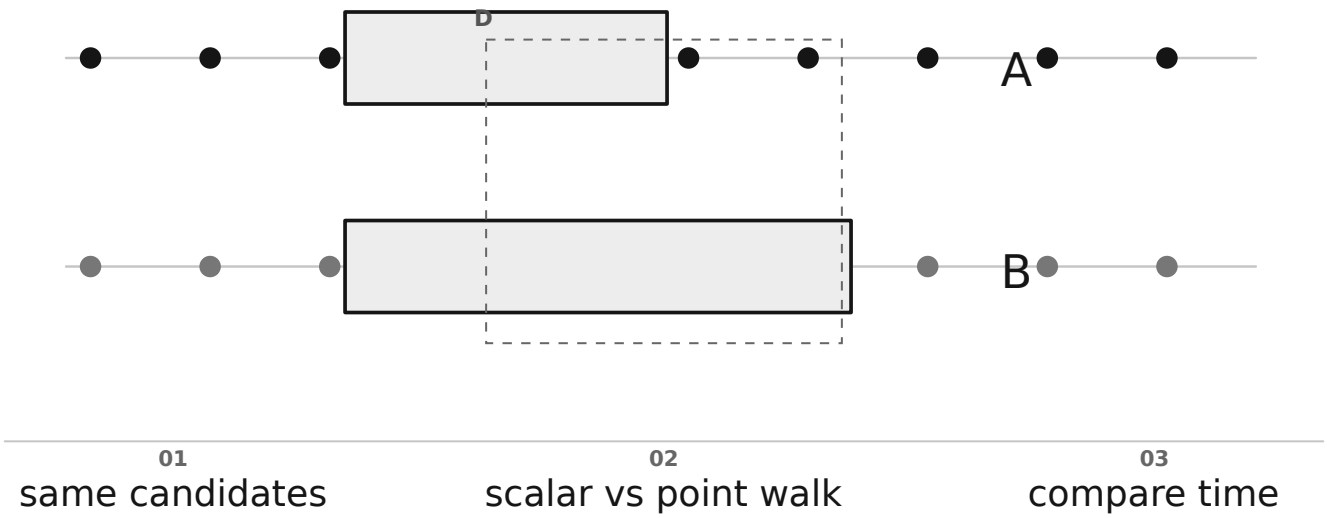
The adapter froze training data, retained its natural hypothesis list and checked one representative per shard. A later containment check found that most known evaluation keys were outside those selected intervals.

ORIGINAL FINDING / EXECUTED RECORD

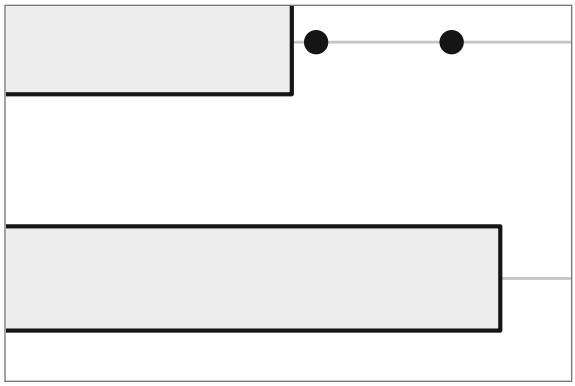
Selected 12,515 hypothesis shards across puzzles 30-64; only 1/35 known targets was contained.

M015

CPU scalar versus point-walk benchmarking



D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Compare two verifiers on identical scalar streams. Matching outputs isolate performance changes from candidate-selection differences.

READ THE THREE STAGES

01 same candidates
02 scalar vs point walk
03 compare time

WHAT WAS ACTUALLY DONE

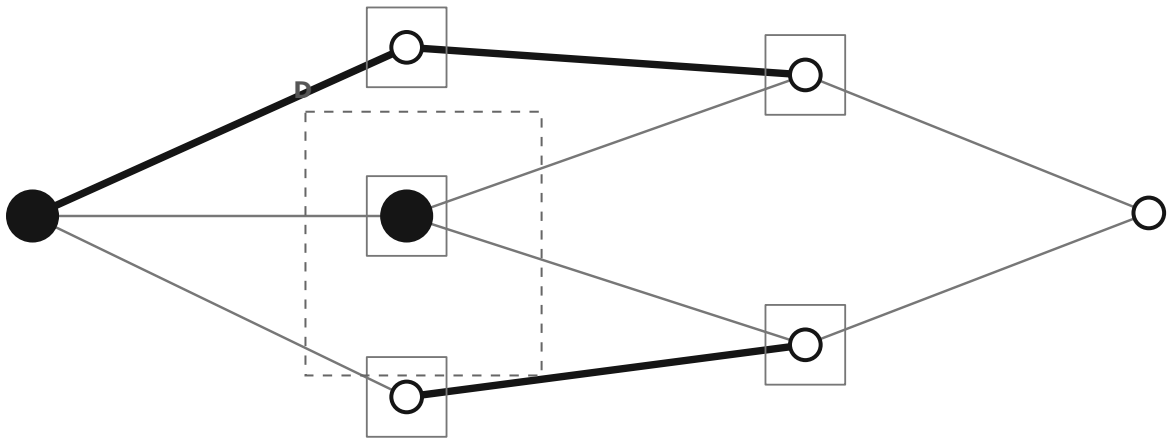
The point-walk benchmark compared exact points and hashes, then timed alternating paired trials with matching stream digests. It measures local throughput, not a higher probability per tested key.

ORIGINAL FINDING / EXECUTED RECORD

Measured acceleration is computational, not evidence of a more likely candidate distribution.

M016

Mock distributed key-generation network

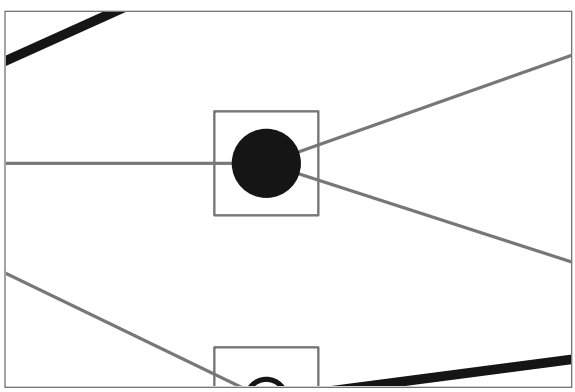


01 coordinator

02 worker pull queues

03 checker nodes

D / OPERATION DETAIL



Vector detail / 1.30× main-view scale

HOW THE OPERATION WORKS

A coordinator hands work units to generator nodes while optional checker nodes consume results. Pull-based scheduling balances unequal workers.

READ THE THREE STAGES

01 coordinator
02 worker pull queues
03 checker nodes

WHAT WAS ACTUALLY DONE

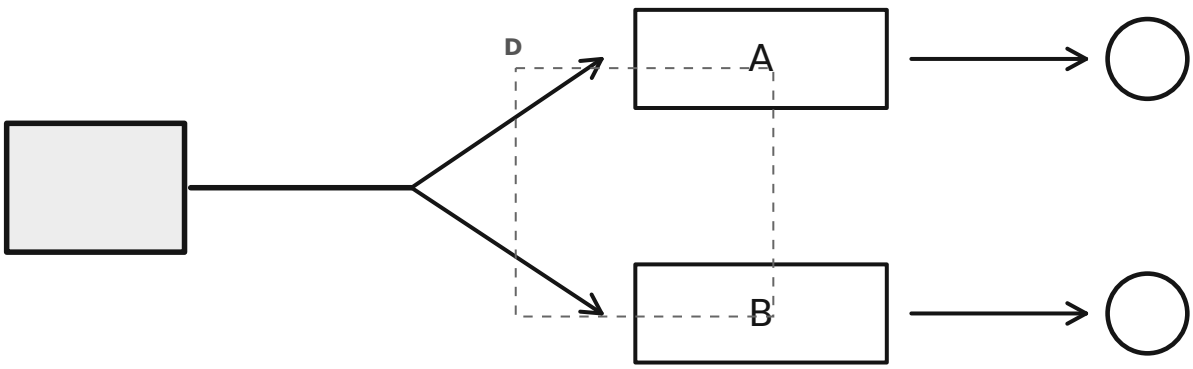
mock_keyspace_network.py runs bounded practice batches using processes or threads. Its counters and coverage checks demonstrate orchestration; the mock network is not evidence of a full 71-bit sweep.

ORIGINAL FINDING / IMPLEMENTED

Pull-based process/thread workers and checker queues exercise scheduling on bounded mock work; not a full puzzle solve.

M017

32-byte padding and dual-format P2PKH checks

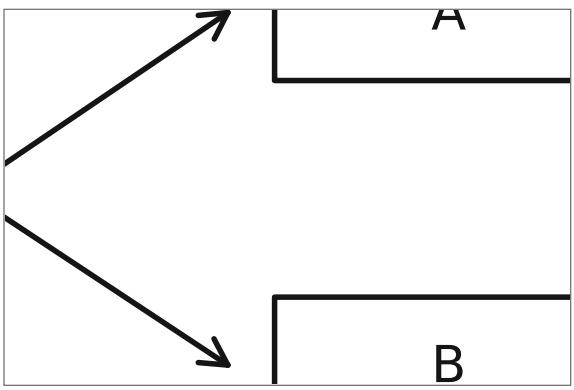


01
32-byte scalar

02
public-key encodings

03
20-byte hash

D / OPERATION DETAIL



Vector detail / 1.38× main-view scale

HOW THE OPERATION WORKS

A scalar is encoded as exactly 32 big-endian bytes, multiplied by G, serialized and hashed. P2PKH comparisons use the address's decoded 20-byte hash.

READ THE THREE STAGES

01 32-byte scalar
02 public-key encodings
03 20-byte hash

WHAT WAS ACTUALLY DONE

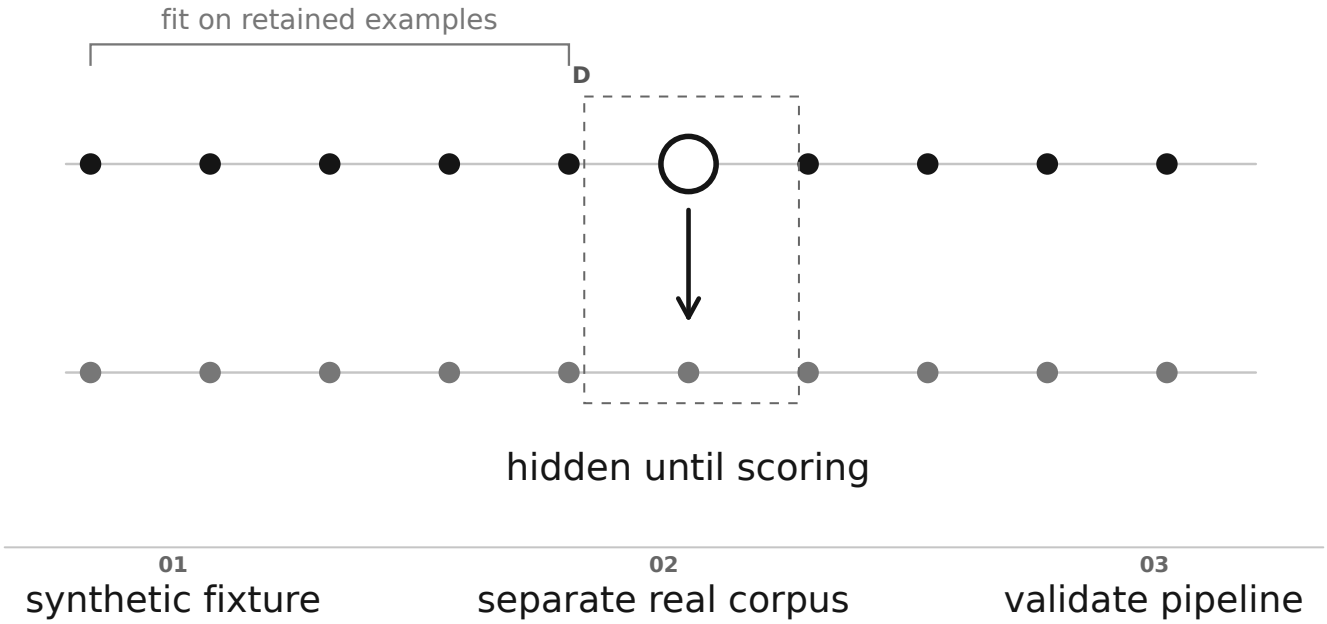
Existing checkers use key.to_bytes(32,'big'), known scalar-one vectors and compressed/uncompressed formats. They compare HASH160 directly rather than Base58-encoding every generated address, an equivalent target check.

ORIGINAL FINDING / EXECUTED RECORD

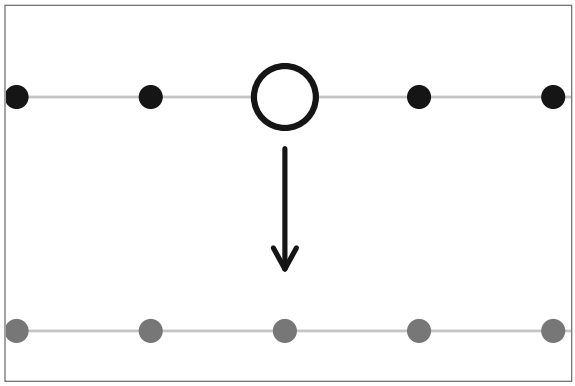
Left-padded scalar inputs, compressed/uncompressed public keys and known scalar-one address vectors guard verifier correctness.

M018

Synthetic fixtures versus disclosed puzzle data



D / OPERATION DETAIL



Vector detail / 1.12× main-view scale

HOW THE OPERATION WORKS

A synthetic fixture can validate pipeline logic because its generation rule is known. It must remain separate from actual published puzzle secrets.

READ THE THREE STAGES

01 synthetic fixture
02 separate real corpus
03 validate pipeline

WHAT WAS ACTUALLY DONE

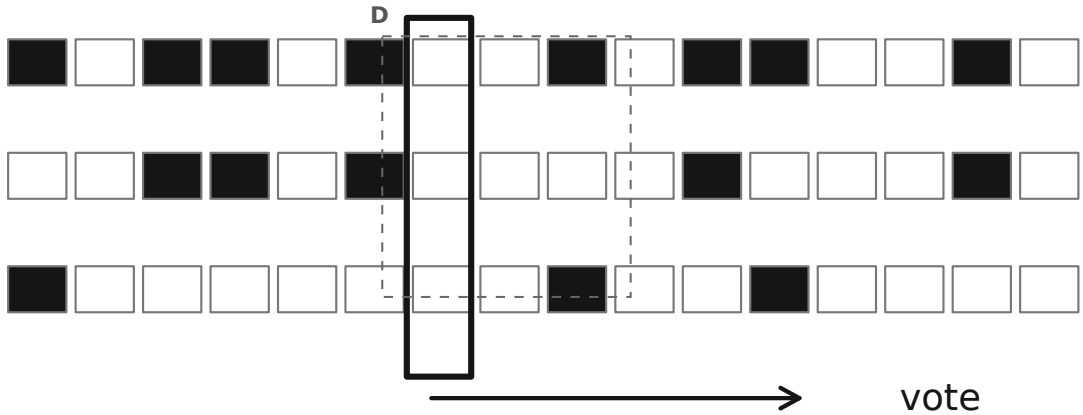
PEC37 tools generate synthetic chains and compare them with disclosed puzzle data. Successful synthetic reconstruction tests the fixture rule; it cannot populate undisclosed puzzle-key rows.

ORIGINAL FINDING / EXECUTED RECORD

PEC37 stores synthetic 1-160 chains separately and compares them with actual disclosed keys. Synthetic keys are not puzzle recoveries.

M019

Empirical bit-vote and position forecasting

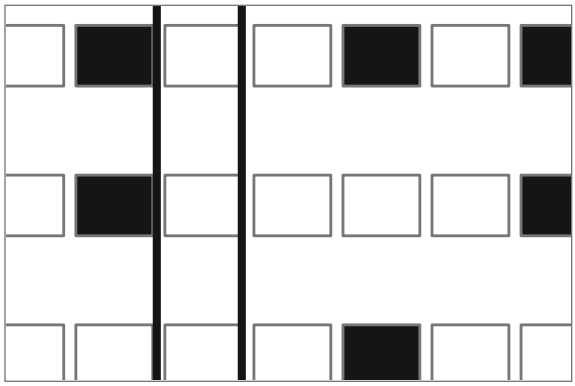


01 training bit rows

02 column majority

03 later-key check

D / OPERATION DETAIL



Vector detail / 1.32× main-view scale

HOW THE OPERATION WORKS

Train simple position trends or majority votes for fractional bit positions, then evaluate them on later known keys.

READ THE THREE STAGES

01 training bit rows
02 column majority
03 later-key check

WHAT WAS ACTUALLY DONE

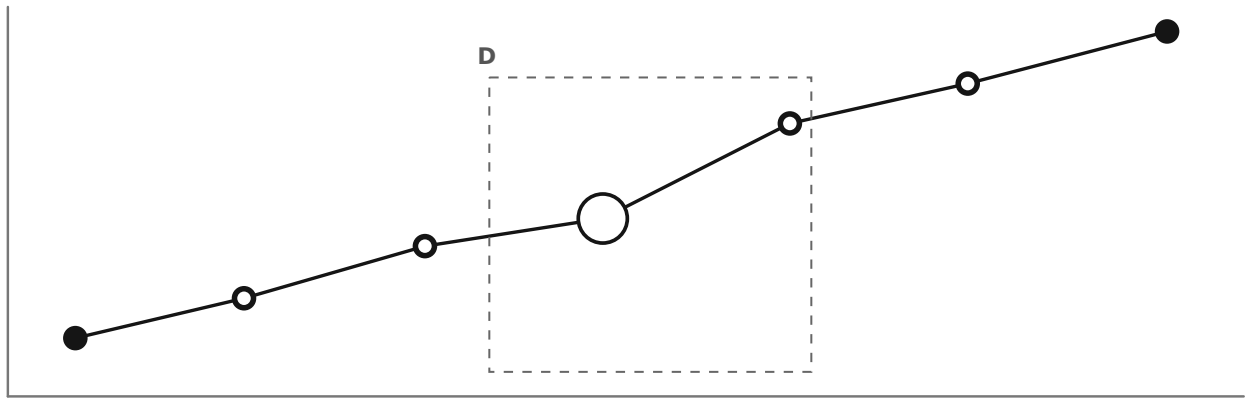
empirical_key_forensics.py uses disclosed keys, exact rational regressions and bit-vote diagnostics. It measures retrospective prediction accuracy; it does not search unknown Bitcoin addresses.

ORIGINAL FINDING / EXECUTED RECORD

Retrospective generator-forensics tests fractional-bit votes, exact regressions and baseline predictors on disclosed keys.

M020

Five-index scale/transition analysis

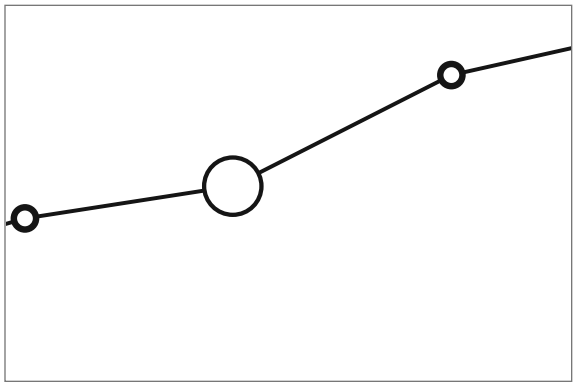


01 normalized positions

02 five-index gaps

03 compare transitions

D / OPERATION DETAIL



Vector detail / 1.17× main-view scale

HOW THE OPERATION WORKS

Normalize every-fifth-puzzle keys to comparable interval positions and ask whether one predicts the next better than a baseline.

READ THE THREE STAGES

01 normalized positions
02 five-index gaps
03 compare transitions

WHAT WAS ACTUALLY DONE

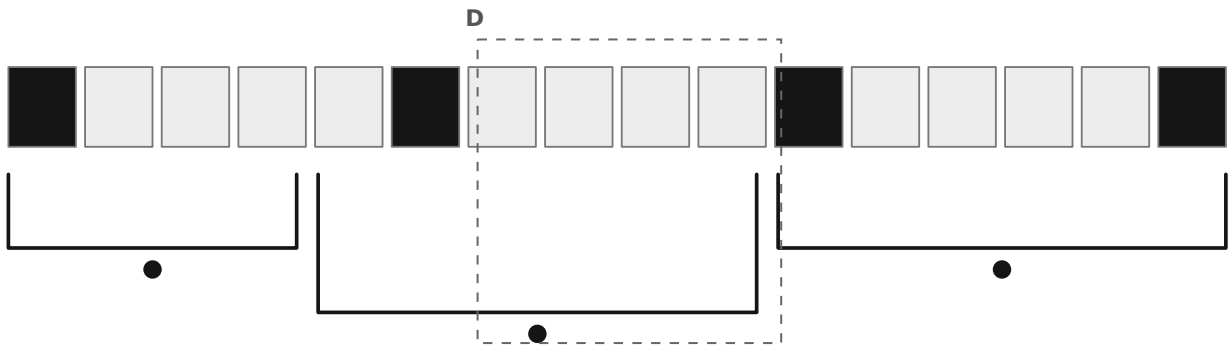
The downloaded result file records lag correlation, permutation tests, prefix Hamming distances and held-out transition errors. These are saved statistical results, not additional private-key discoveries.

ORIGINAL FINDING / EXECUTED RECORD

Normalized positions, lag correlations, Hamming similarity and held-out transition forecasts are recorded; no new key is recovered.

M021

Binary distinguished-point parsing

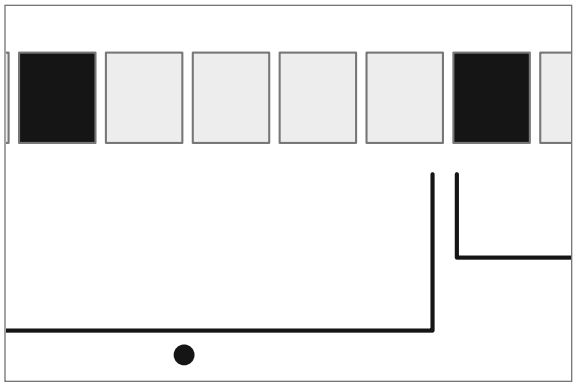


01 binary dump

02 field boundaries

03 point and distance

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Distinguished-point dumps encode selected curve points and possibly walk distances. Recovering their layout is necessary before collision algebra has meaning.

READ THE THREE STAGES

01 binary dump
02 field boundaries
03 point and distance

WHAT WAS ACTUALLY DONE

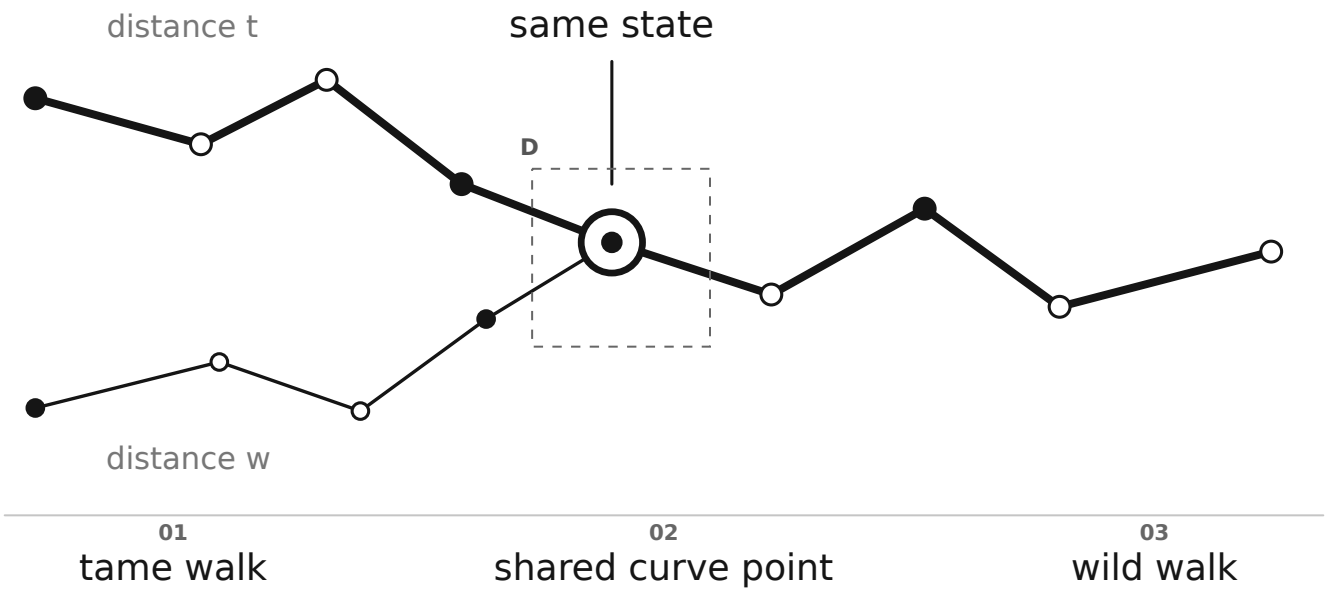
The parser scans bit/byte layouts, checks candidate x-coordinates and extracts metadata slices. Later scripts test interpretations; a parseable integer is not automatically a valid walk distance.

ORIGINAL FINDING / EXECUTED RECORD

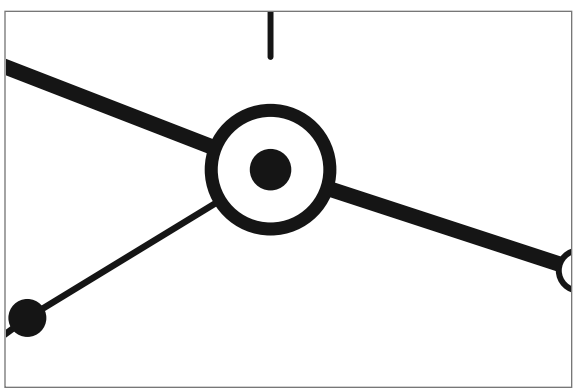
Extract curve x-coordinates, candidate metadata and distance fields from the supplied binary. Interpretation of metadata remains a hypothesis.

M022

Tame/wild collision recovery



D / OPERATION DETAIL



Vector detail / 1.93× main-view scale

HOW THE OPERATION WORKS

A tame walk starts from a known scalar; a wild walk starts from the target point. Their meeting relates recorded distances and yields a candidate scalar.

READ THE THREE STAGES

01 tame walk
02 shared curve point
03 wild walk

WHAT WAS ACTUALLY DONE

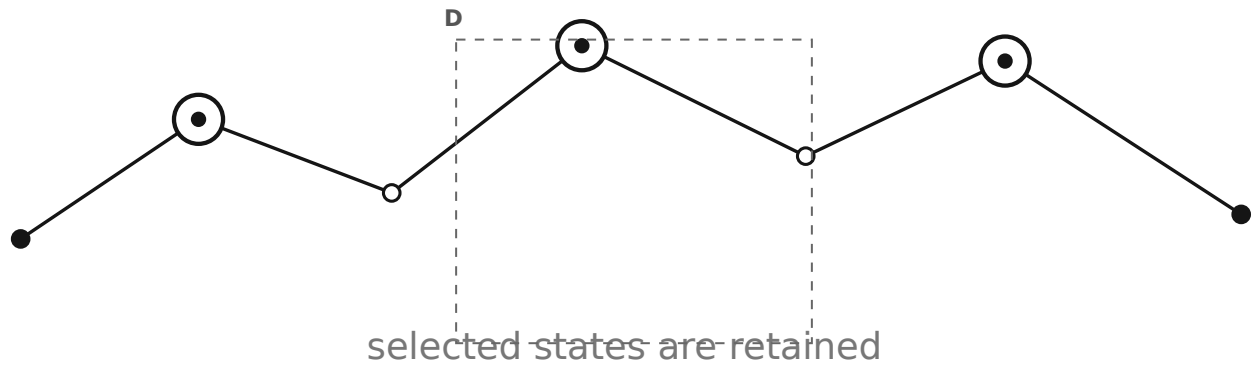
collision.py groups same-x entries, tries tame/wild assignments and derives candidates. Because x alone has two point parities, the final scalar must pass exact public-key verification.

ORIGINAL FINDING / IMPLEMENTED

Same-point opposite-herd distances yield a scalar when the walk origins and distances are valid. No real target collision is established here.

M023

Single-process bounded Pollard kangaroo

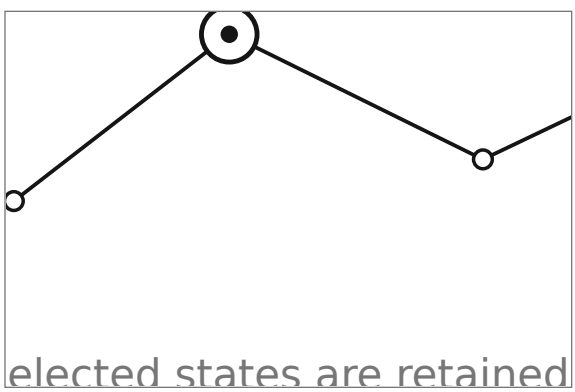


01 bounded interval

02 deterministic jumps

03 distinguished points

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Pollard kangaroo uses deterministic pseudo-random jumps so tame and wild walks can merge. Distinguished points reduce the amount of stored walk data.

READ THE THREE STAGES

01 bounded interval
02 deterministic jumps
03 distinguished points

WHAT WAS ACTUALLY DONE

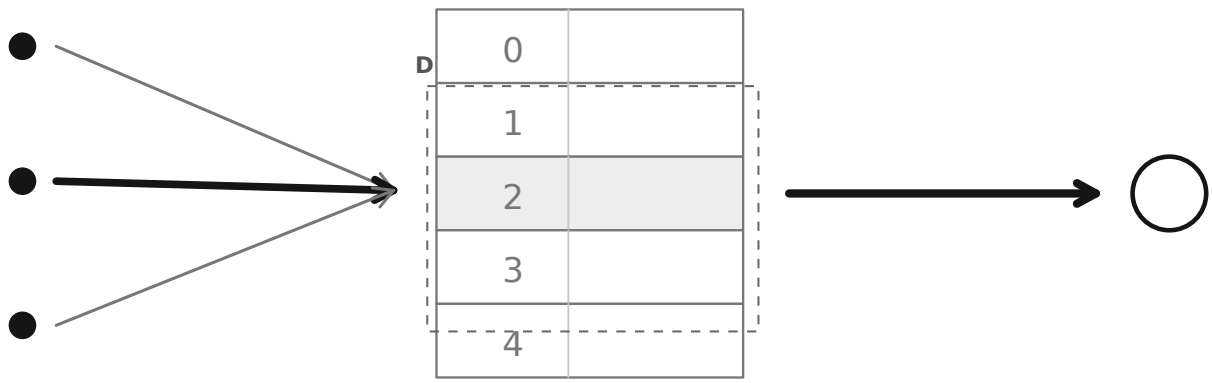
kangaroo.py implements bounded CPU walks, jump tables, point selection and verification. It requires the target public point and a budget; an address hash cannot initialize its wild walk.

ORIGINAL FINDING / IMPLEMENTED

Standard interval discrete-log implementation for targets with known public points.

M024

Separated tame and wild phases

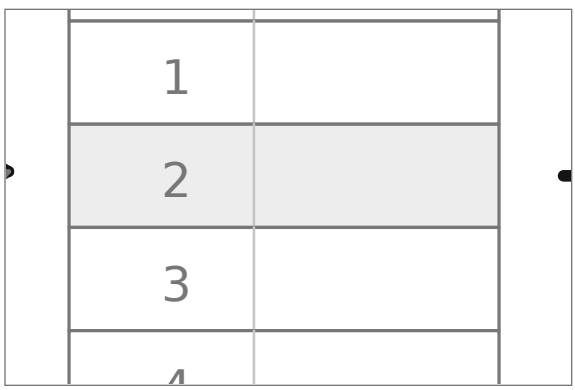


01
tame phase

02
saved point table

03
wild lookup

D / OPERATION DETAIL



Vector detail / 1.40× main-view scale

HOW THE OPERATION WORKS

Build the tame table once, save it, and later walk from the target until a compatible distinguished point is reached.

READ THE THREE STAGES

01 tame phase
02 saved point table
03 wild lookup

WHAT WAS ACTUALLY DONE

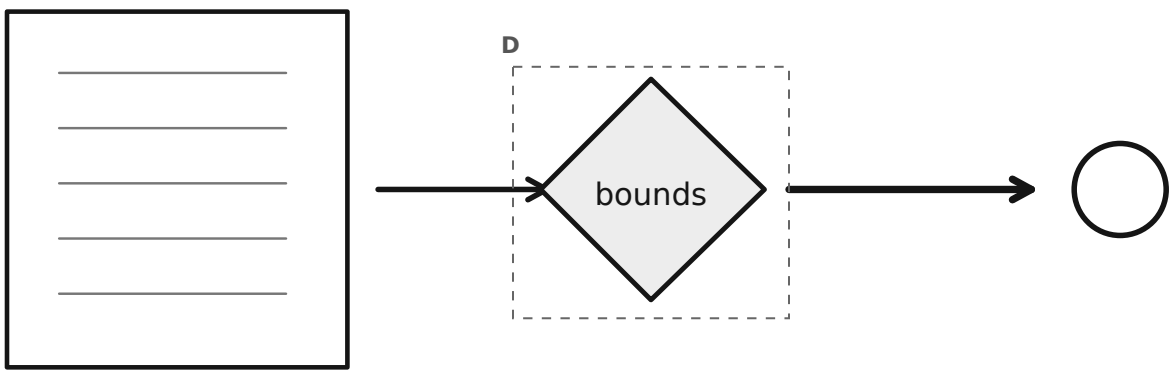
phase.py exposes separate tame and wild commands and table export/loading. This supports separate machines or stages without converting a missing public key into an ECDLP target.

ORIGINAL FINDING / IMPLEMENTED

Persist a tame distinguished-point table and search from the target in a separate phase.

M025

JeanLucPons / RCKangaroo export

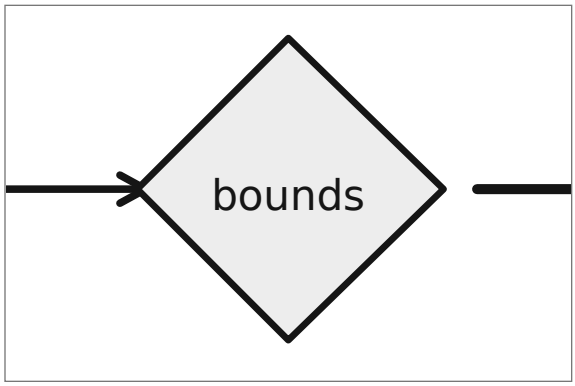


01
point and bounds

02
solver manifest

03
external solver

D / OPERATION DETAIL



Vector detail / 1.37× main-view scale

HOW THE OPERATION WORKS

External interval solvers need bounds, a target point and runtime settings in their expected configuration format.

READ THE THREE STAGES

01 point and bounds
02 solver manifest
03 external solver

WHAT WAS ACTUALLY DONE

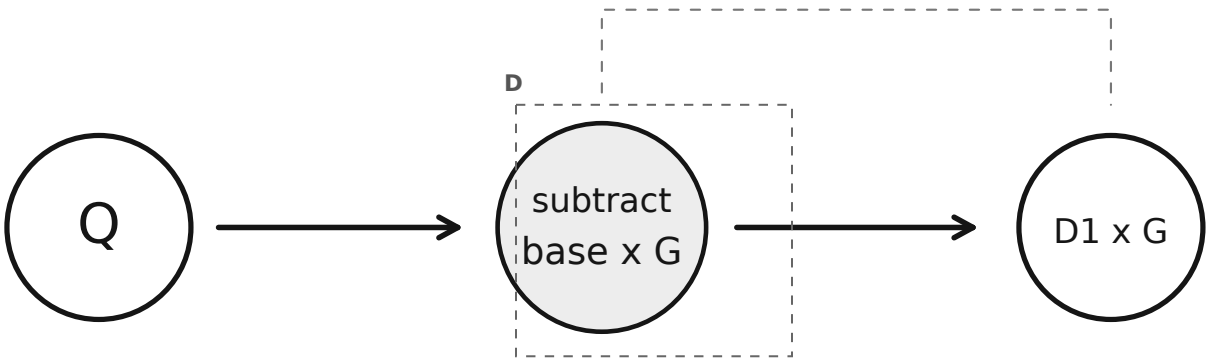
Export scripts write JeanLucPons/RCKangaroo inputs from puzzle or slot metadata. They prepare jobs; whether the external executable ran and found a match requires separate logs.

ORIGINAL FINDING / IMPLEMENTED

Generate solver configuration files for public-key interval targets; configuration existence is not execution evidence.

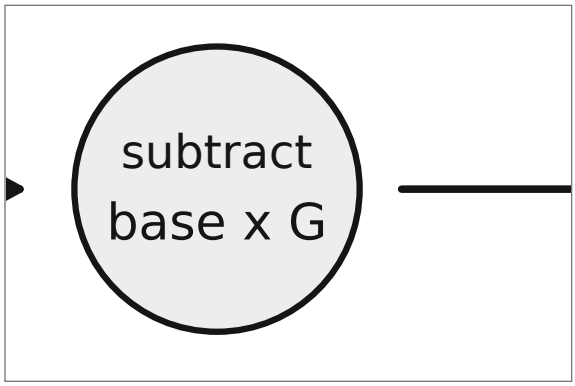
M026

Direct D1 interval kangaroo



01	02	03
public point Q	subtract base times G	D1 target point

D / OPERATION DETAIL



Vector detail / 1.37× main-view scale

HOW THE OPERATION WORKS

If k=base+D1, then Q-base*G=D1*G. A known base converts recovery into a smaller interval discrete log.

READ THE THREE STAGES

01 public point Q
02 subtract base times G
03 D1 target point

WHAT WAS ACTUALLY DONE

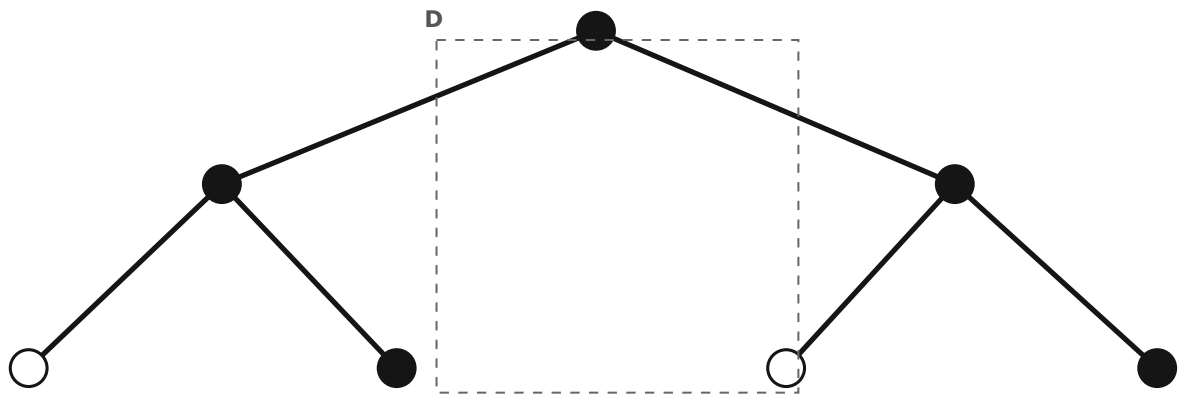
D1 solvers construct the shifted point from guessed root/coefficient coordinates. A wrong base usually makes the true key absent from that interval, so leaf cost alone is incomplete.

ORIGINAL FINDING / IMPLEMENTED

Subtract a hypothesized scalar base and search the D1 leaf; useful only if the hypothesized coordinates are correct.

M027

Wrapped CT multi-target kangaroo

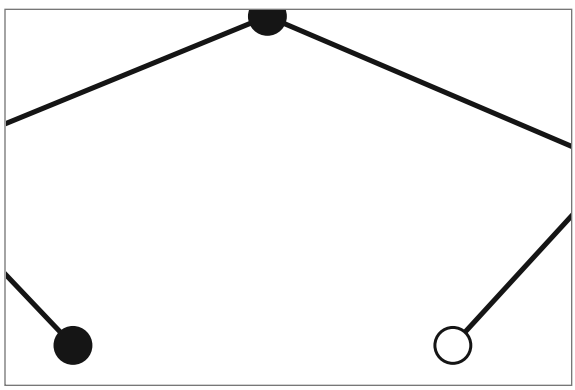


01 base hypotheses

02 shifted targets

03 shared herd work

D / OPERATION DETAIL



Vector detail / 0.98× main-view scale

HOW THE OPERATION WORKS

Share walks or targets across several related base hypotheses, attempting to reuse work instead of solving each independently.

READ THE THREE STAGES

01 base hypotheses
02 shifted targets
03 shared herd work

WHAT WAS ACTUALLY DONE

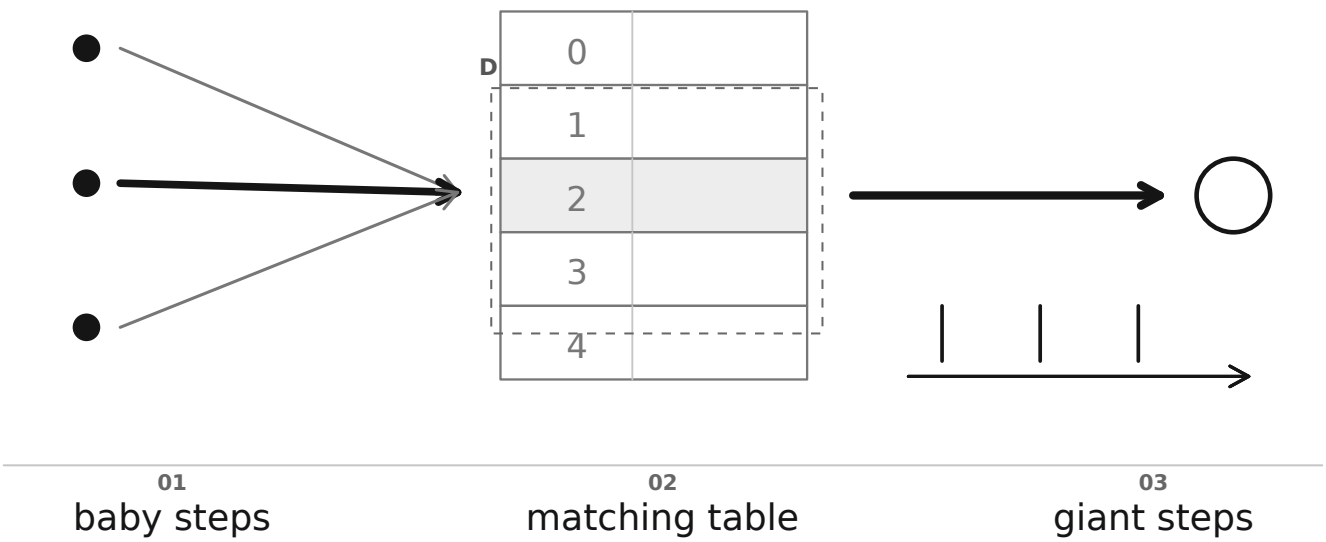
Wrapped-CT code constructs multiple shifted targets. Historical audits inspect span/setup growth and find that the advertised tiny-span benefit does not transfer unconditionally to high-index targets.

ORIGINAL FINDING / AUDITED/CORRECTED

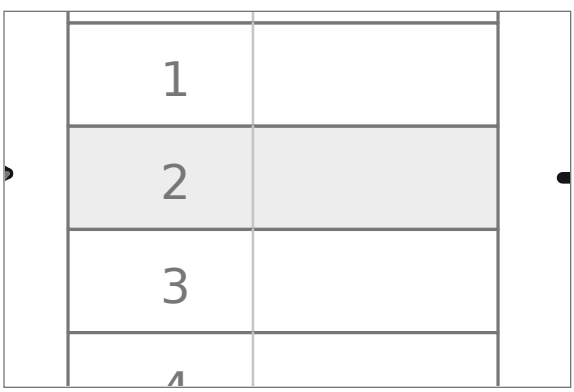
Historical small-span speed claims fail on large required spans; later audits reject unconditional recovery-cost reductions.

M028

CPU banded baby-step/giant-step D1 hunts



D / OPERATION DETAIL



Vector detail / 1.40× main-view scale

HOW THE OPERATION WORKS

BSGS stores small point multiples, then subtracts larger multiples from the target to find a match in a bounded interval.

READ THE THREE STAGES

01 baby steps
02 matching table
03 giant steps

WHAT WAS ACTUALLY DONE

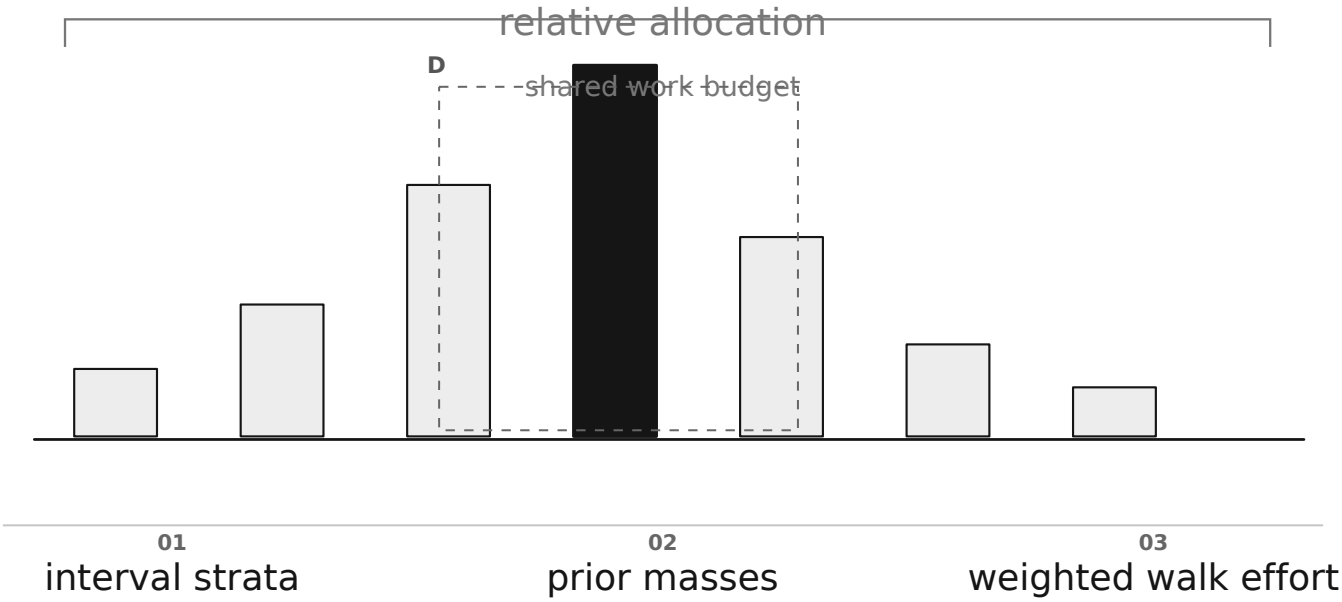
cpu_band_d1_hunt.py builds baby tables for selected D1 bands and scans giant steps. Saved deployments used CPU work even on some GPU-equipped hosts; that is not a CUDA benchmark.

ORIGINAL FINDING / EXECUTED RECORD

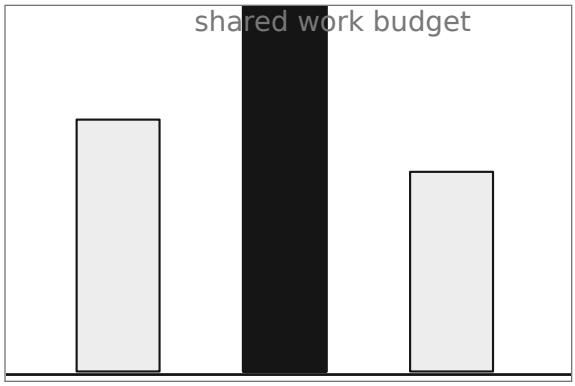
Bounded D1-band runs are documented; CPU tasks on GPU-equipped pods must not be described as GPU kernel execution.

M029

Prior-weighted stratified kangaroo



D / OPERATION DETAIL



Vector detail / 1.00× main-view scale

HOW THE OPERATION WORKS

Split a known-point interval into strata and allocate more tame-walk effort where a supplied prior assigns greater probability.

READ THE THREE STAGES

01 interval strata
02 prior masses
03 weighted walk effort

WHAT WAS ACTUALLY DONE

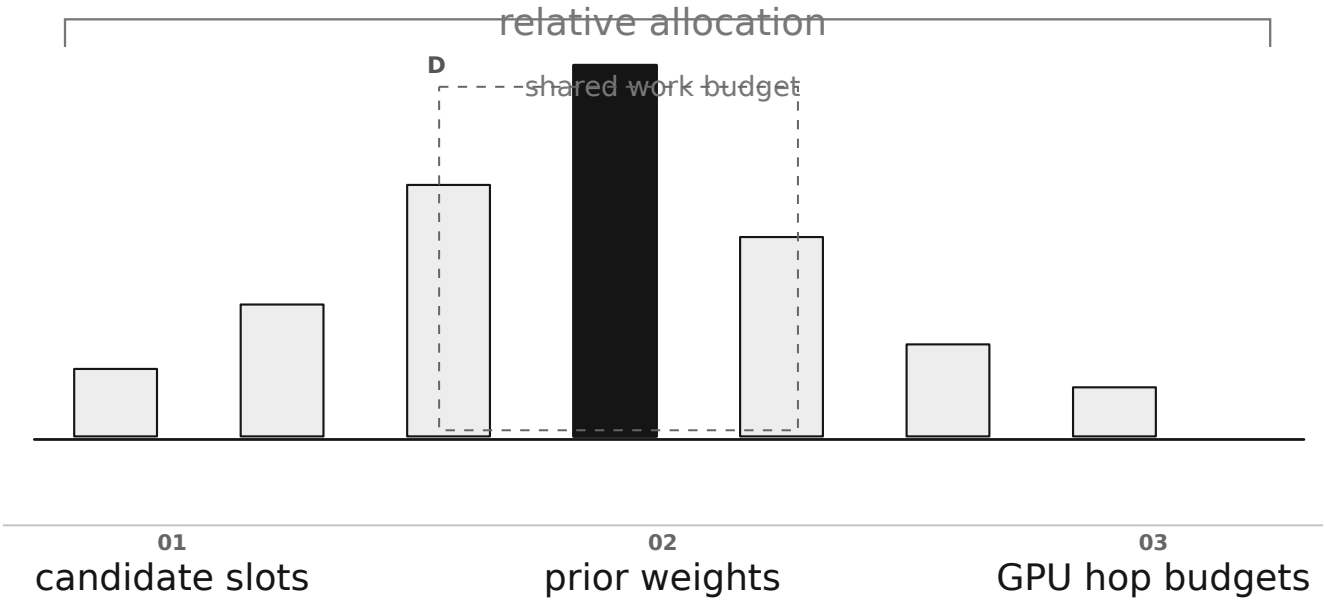
pwsk.py builds a smoothed histogram, assigns budgets using prior mass and width, and searches against a shared wild herd. Its benefit depends on the supplied prior actually predicting targets.

ORIGINAL FINDING / IMPLEMENTED

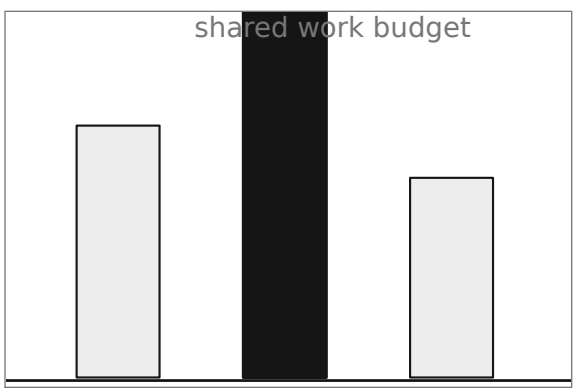
Reweights interval effort when a genuine prior exists; flat priors reduce to ordinary allocation. Prior quality is unestablished for the unknown puzzle.

M030

Prior-weighted GPU slot budgets



D / OPERATION DETAIL



Vector detail / 1.00× main-view scale

HOW THE OPERATION WORKS

Reweight work across already constructed slots without changing what each slot represents. Higher prior mass receives a larger hop budget.

READ THE THREE STAGES

01 candidate slots
02 prior weights
03 GPU hop budgets

WHAT WAS ACTUALLY DONE

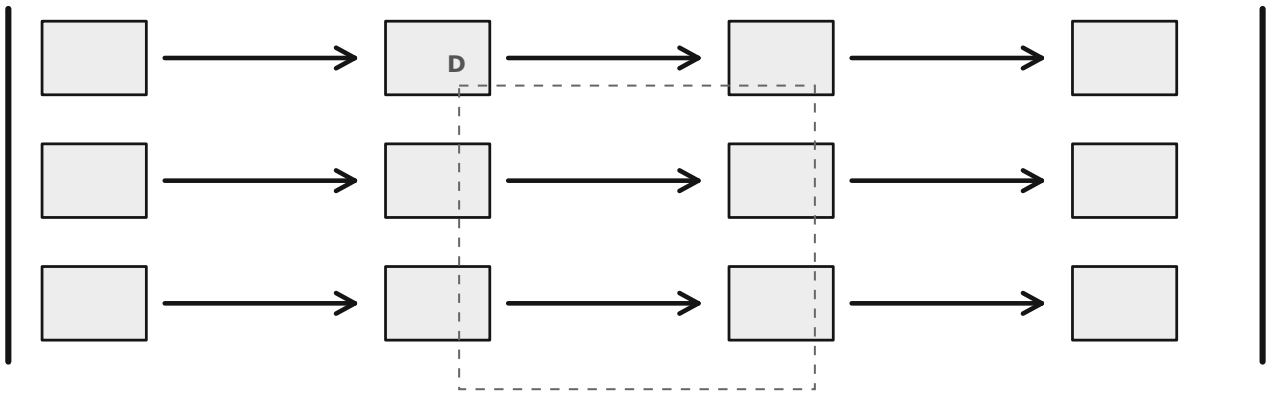
pwsk_budget.py combines FUSED-distance and optional D1 histogram weights, rounds allocations and annotates manifests. It changes scheduling, not the set of possible scalar equations.

ORIGINAL FINDING / IMPLEMENTED

Allocate existing slot budgets using FUSED distance or empirical D1 histograms without changing slot geometry.

M031

Rust native herd stepping

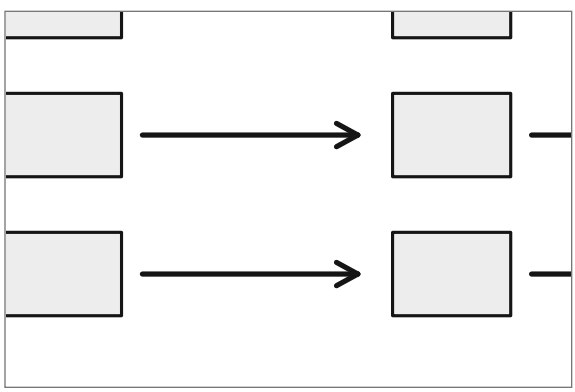


01
native walk loop

02
compiled herd steps

03
same search rule

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Move repeated curve-walk operations out of the Python interpreter into compiled native code. The mathematical search remains the same.

READ THE THREE STAGES

01 native walk loop
02 compiled herd steps
03 same search rule

WHAT WAS ACTUALLY DONE

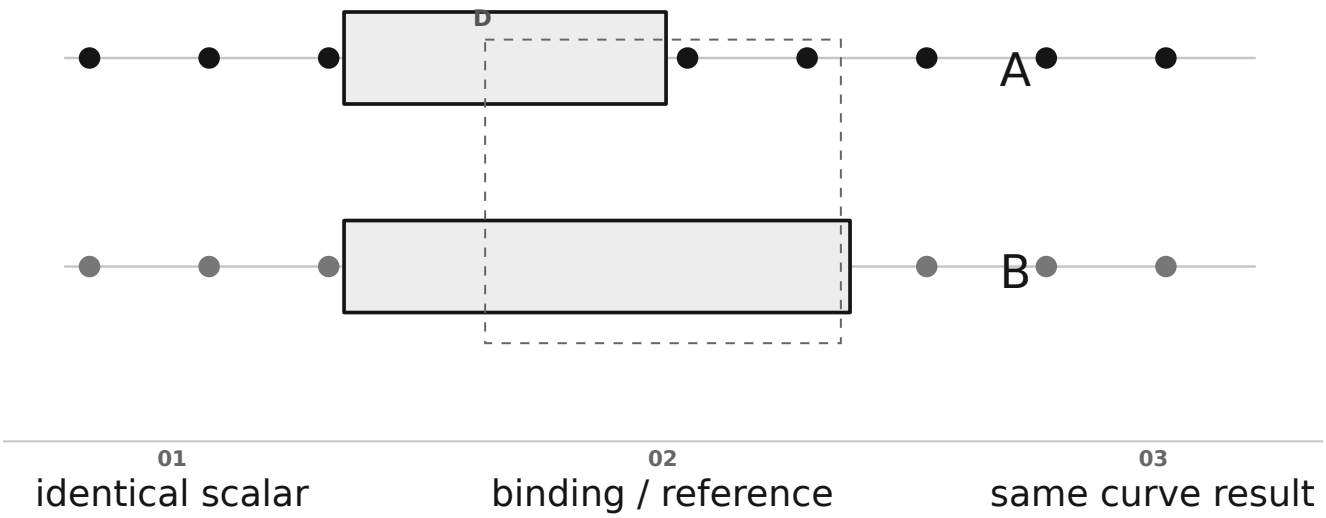
The Rust extension provides herd stepping and native lookup operations called by the Python runtime. Actual speed must come from a build-specific benchmark, not merely the presence of Rust source.

ORIGINAL FINDING / IMPLEMENTED

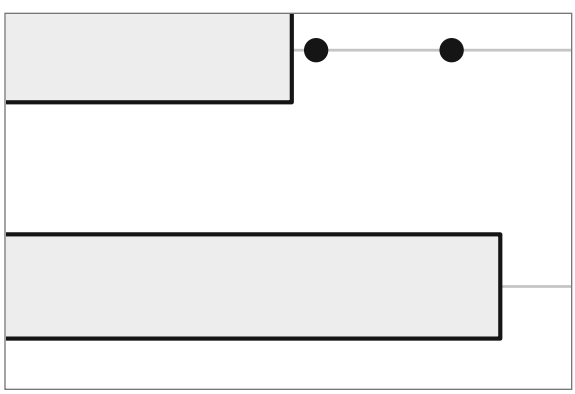
Native backend reduces interpreter overhead while preserving interval-search semantics.

M032

Coincurve/libsecp256k1 fast backend



D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Call optimized libsecp256k1 operations through a Python binding instead of performing every field operation in Python.

READ THE THREE STAGES

01 identical scalar
02 binding / reference
03 same curve result

WHAT WAS ACTUALLY DONE

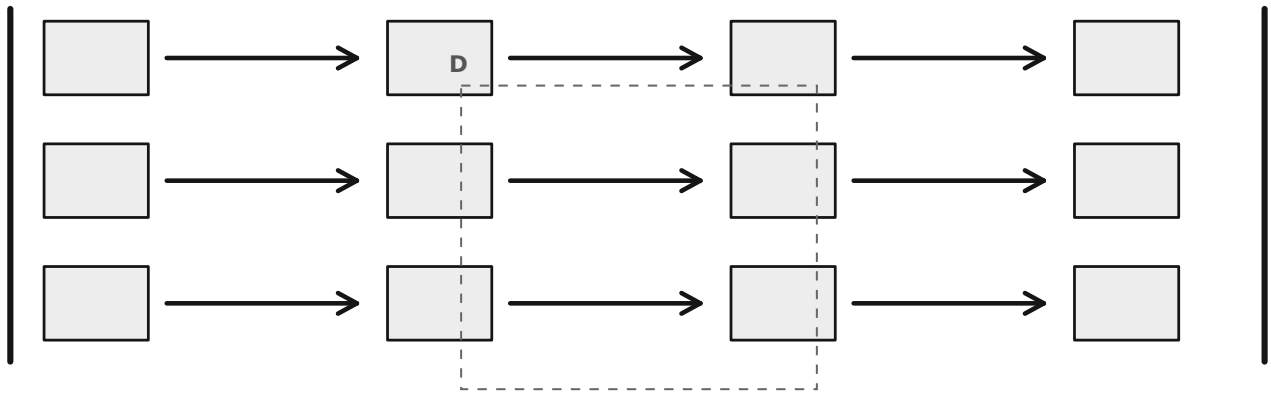
fast_ec.py selects accelerated scalar multiplication, point addition and optional native herd routines. It is a backend substitution, not an algorithm that predicts private keys.

ORIGINAL FINDING / IMPLEMENTED

Accelerated point arithmetic replaces pure Python operations; it does not reduce unknown key entropy.

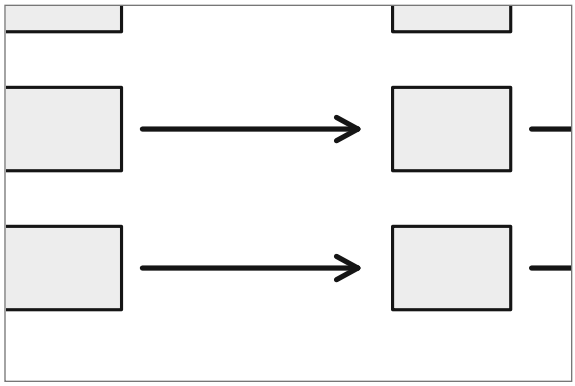
M033

GPU modular herd/D1 backends



01	02	03
device work groups	parallel walk lanes	implementation required

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

GPU kernels would execute many curve steps concurrently. A backend interface alone is insufficient; the device search must actually be implemented.

READ THE THREE STAGES

01 device work groups
02 parallel walk lanes
03 implementation required

WHAT WAS ACTUALLY DONE

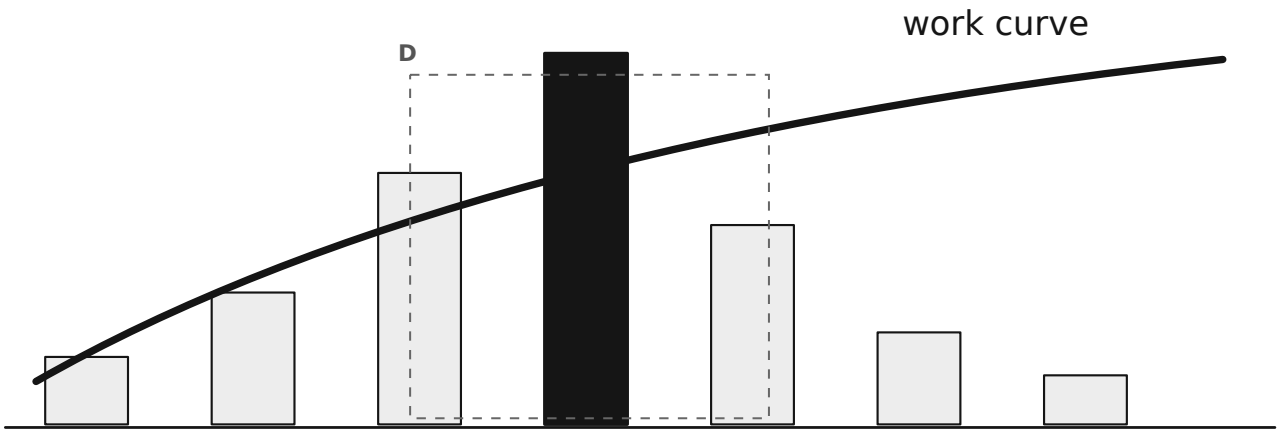
The inspected CTheory d1_search CUDA file returns unavailable/error and zeroes its output. The Python registry can report ready from markers/environment flags, which do not by themselves prove a working kernel.

ORIGINAL FINDING / IMPLEMENTATION/STUB MIX

Backend registries and CUDA files exist, but readiness and actual execution differ by module. Do not infer a complete device implementation from a registry.

M034

GPU throughput and cost calibration

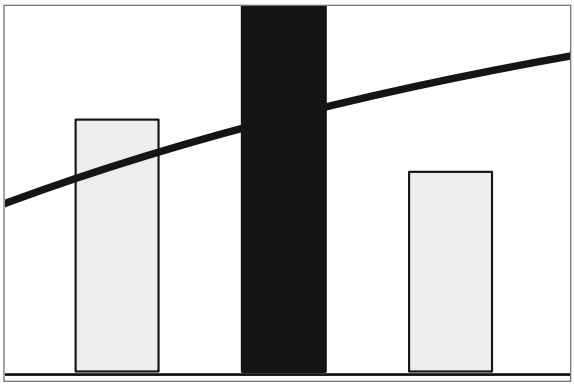


01 measured throughput

02 operation budget

03 time and cost

D / OPERATION DETAIL



Vector detail / 1.00× main-view scale

HOW THE OPERATION WORKS

Parse actual solver rates, discard warmup effects and combine measured work rates with declared operation counts.

READ THE THREE STAGES

01 measured throughput
02 operation budget
03 time and cost

WHAT WAS ACTUALLY DONE

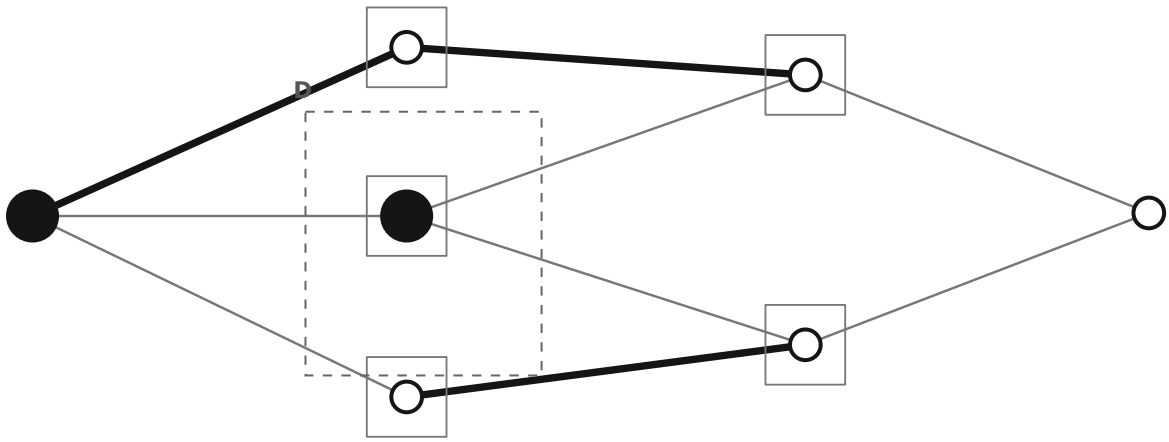
gpu_bench.py extracts throughput from logs and computes estimates. Other tables contain assumed rates/prices; those must not be confused with current hardware measurements or target success probabilities.

ORIGINAL FINDING / EXECUTED RECORD

Parse measured solver logs separately from hardcoded estimates; historical prices and throughput are not current quotations.

M035

RunPod/Vultr sharding and orchestration

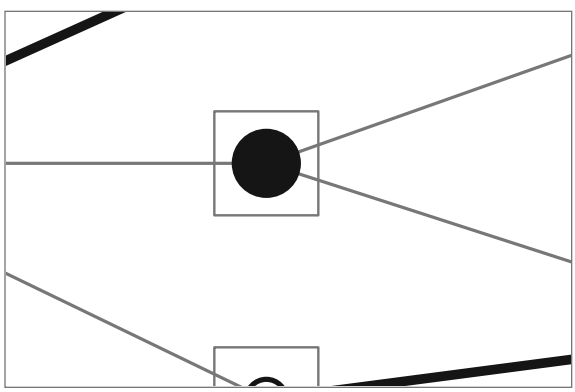


01 job manifest

02 remote shards

03 collected checkpoints

D / OPERATION DETAIL



Vector detail / 1.30× main-view scale

HOW THE OPERATION WORKS

Partition solver jobs across remote workers, transfer manifests, run executables and collect outputs/checkpoints.

READ THE THREE STAGES

01 job manifest
02 remote shards
03 collected checkpoints

WHAT WAS ACTUALLY DONE

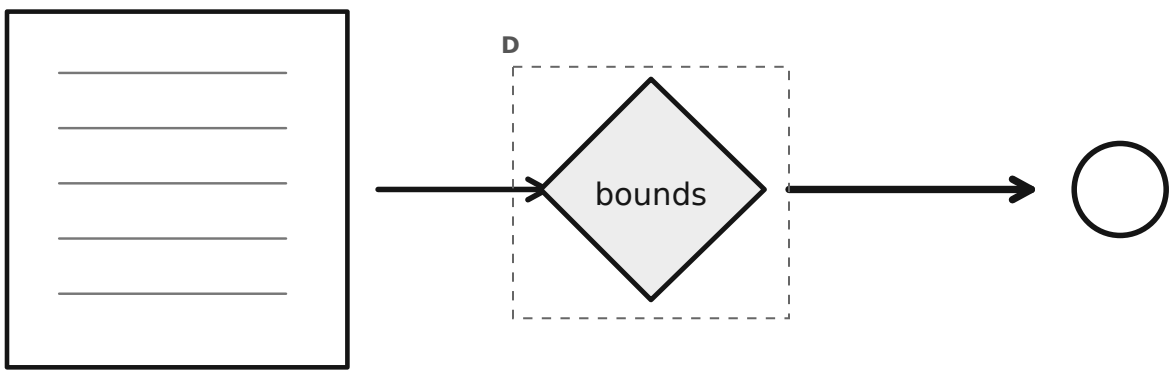
Deployment scripts and playbooks implement build/start/status procedures. This review documents those artifacts without launching instances or treating a successful cloud allocation as a successful key search.

ORIGINAL FINDING / IMPLEMENTED

Deployment runbooks, work-unit manifests and recovery scripts coordinate bounded jobs; availability is distinct from a successful search.

M036

Bounded ECDLP certificates and slot gates

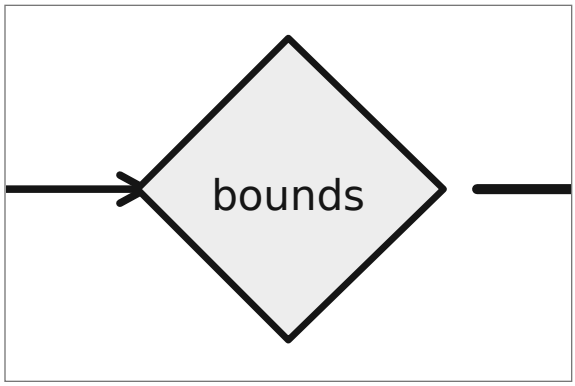


01 declared bounds

02 target / provenance

03 result verification

D / OPERATION DETAIL



Vector detail / 1.37× main-view scale

HOW THE OPERATION WORKS

Attach a machine-readable statement of bounds, target, provenance and job identity so a result can be checked against its declared task.

READ THE THREE STAGES

01 declared bounds
02 target / provenance
03 result verification

WHAT WAS ACTUALLY DONE

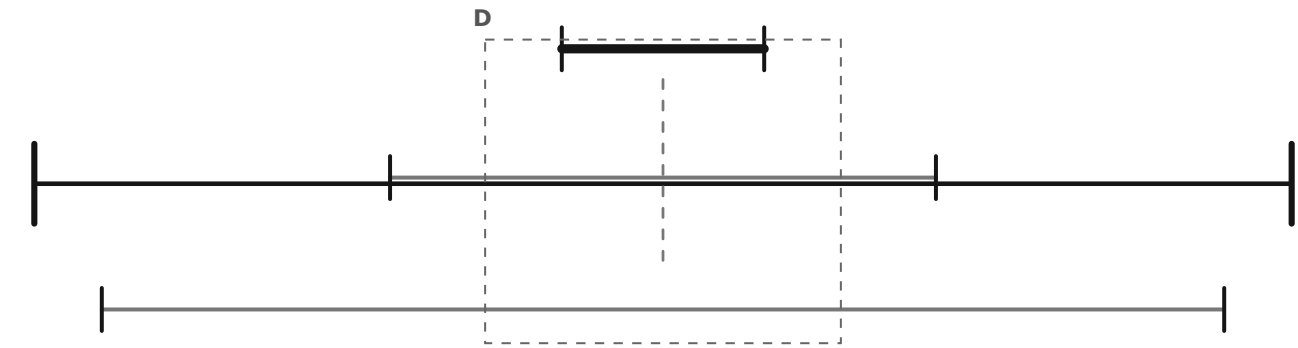
BEC and slot gates seal manifests and validate fields. They can attest what was searched, but cannot certify that a heuristic guess contains the unknown key.

ORIGINAL FINDING / IMPLEMENTED

Certifies declared bounds and evidence metadata; a certificate is not proof that the target lies in a guessed slot.

M037

Miss ladder and complement widening

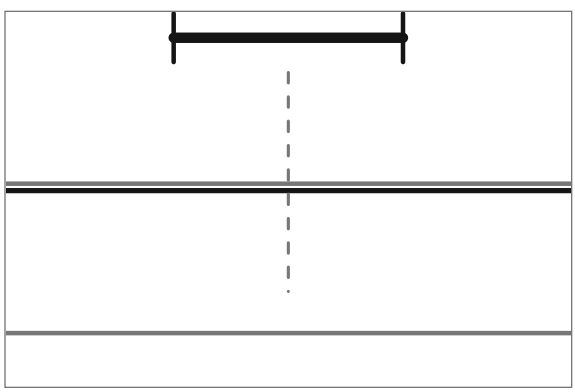


01 first search band

02 miss then enlarge

03 remaining complement

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

After a prioritized search misses, enlarge the hypothesis set in an explicit order rather than silently assuming the key was impossible.

READ THE THREE STAGES

01 first search band

02 miss then enlarge

03 remaining complement

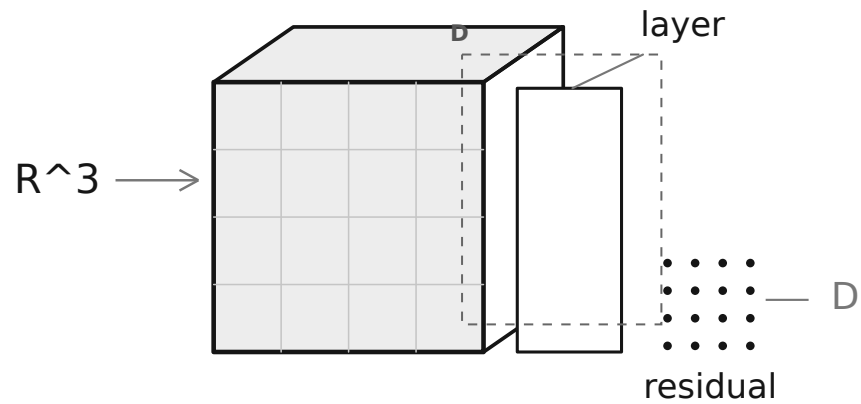
WHAT WAS ACTUALLY DONE

miss_ladder.py plans complement, lattice and classical-range fallbacks. Following the ladder spends additional work; it does not retroactively make the first narrow guess complete.

ORIGINAL FINDING / IMPLEMENTED

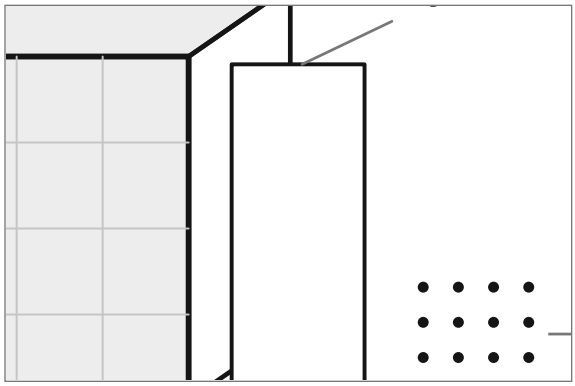
Follow a failed narrow search with complement/full-range fallbacks, rather than treating heuristic misses as exclusions.

M038
Cubic scalar decomposition



01	02	03
original scalar	cube + layer	residual

D / OPERATION DETAIL



Vector detail / 1.27× main-view scale

HOW THE OPERATION WORKS

Set R1=floor(cuberoot(k)), divide k-R1^3 by R1^2, and retain quotient C1 and remainder D1. This exactly re-encodes k.

READ THE THREE STAGES

01 original scalar
02 cube + layer
03 residual

WHAT WAS ACTUALLY DONE

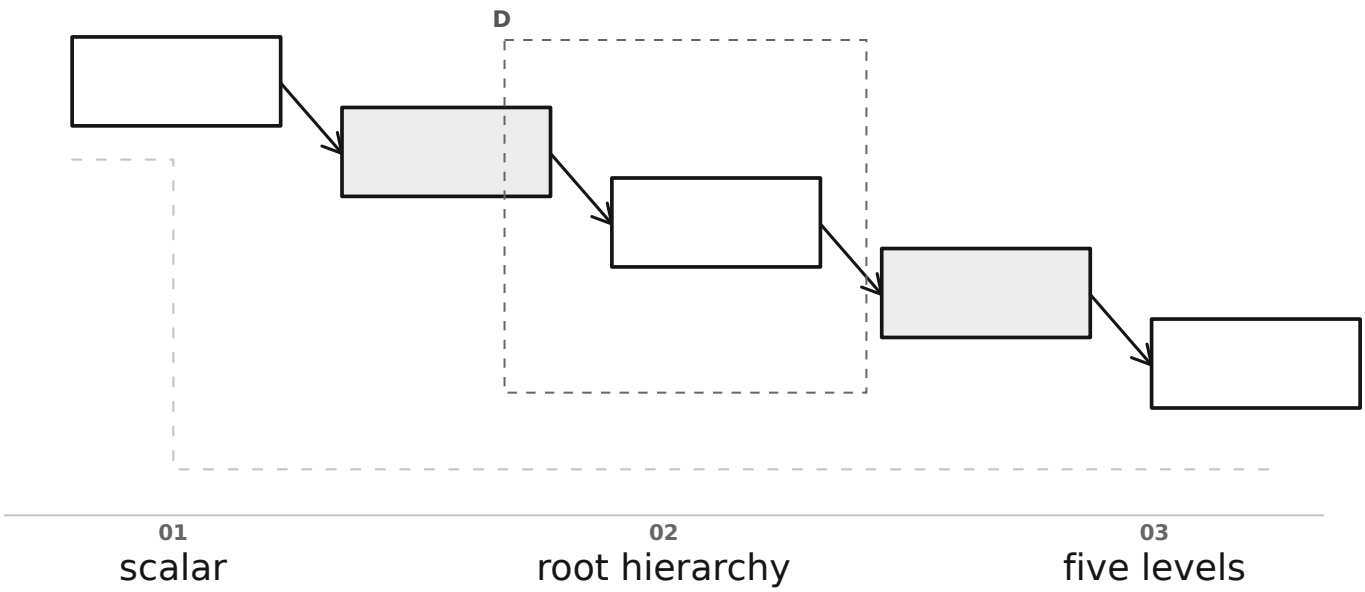
The code reconstructs known scalars from those digits and checks bounds. For an unknown key the digits are also unknown; an exact identity does not disclose them from the address.

ORIGINAL FINDING / AUDITED/CORRECTED

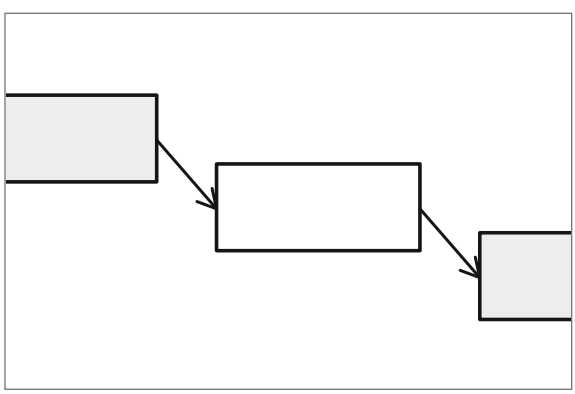
Exact reconstruction of known integers works; the representation itself supplies no unknown digits or entropy reduction.

M039

Five-level R1/R2/S3/S4/S5 tower



D / OPERATION DETAIL



Vector detail / 0.98× main-view scale

HOW THE OPERATION WORKS

Reapply root/division decompositions to R1 and later components, producing a hierarchy of smaller coordinates.

READ THE THREE STAGES

01 scalar
02 root hierarchy
03 five levels

WHAT WAS ACTUALLY DONE

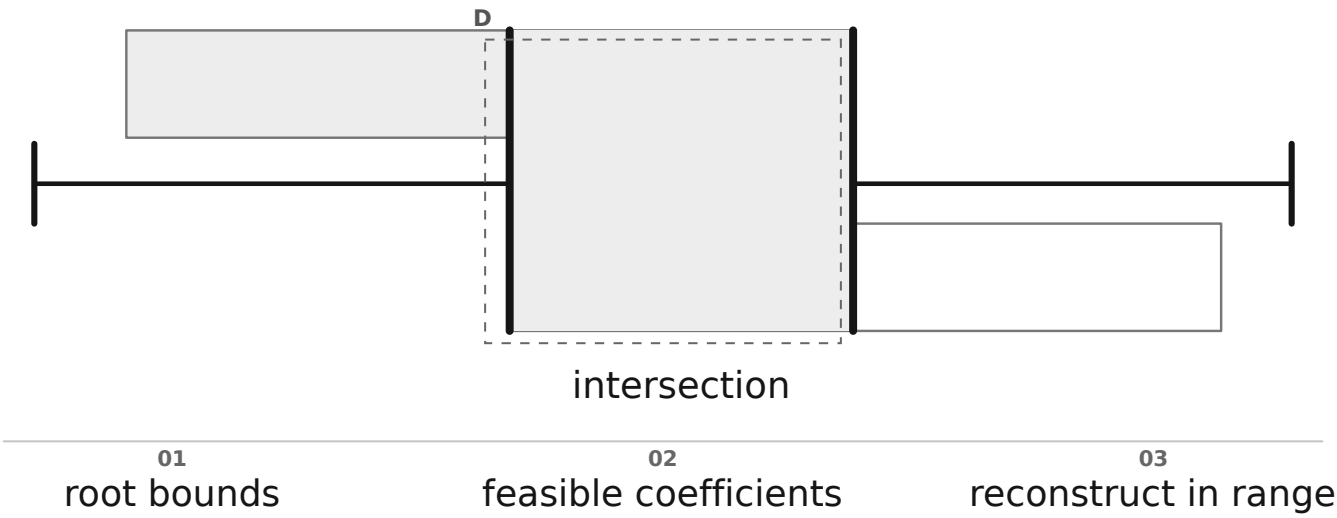
The tower code computes R2 and S-level quantities, some from bit-length formulas. Its tables describe known-key coordinates or target-size anchors, not independently observed secret digits.

ORIGINAL FINDING / IMPLEMENTED

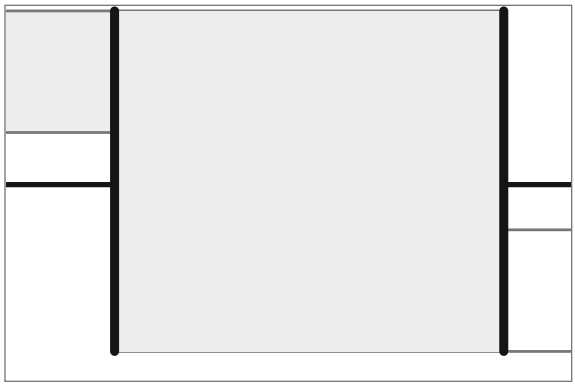
Recursive roots and coefficients organize scalar coordinates; high levels often depend only on advertised bit width.

M040

Exact root bounds and coefficient feasibility



D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Integer roots and inequality bounds determine which digit combinations could reconstruct an integer inside the advertised interval.

READ THE THREE STAGES

01 root bounds
02 feasible coefficients
03 reconstruct in range

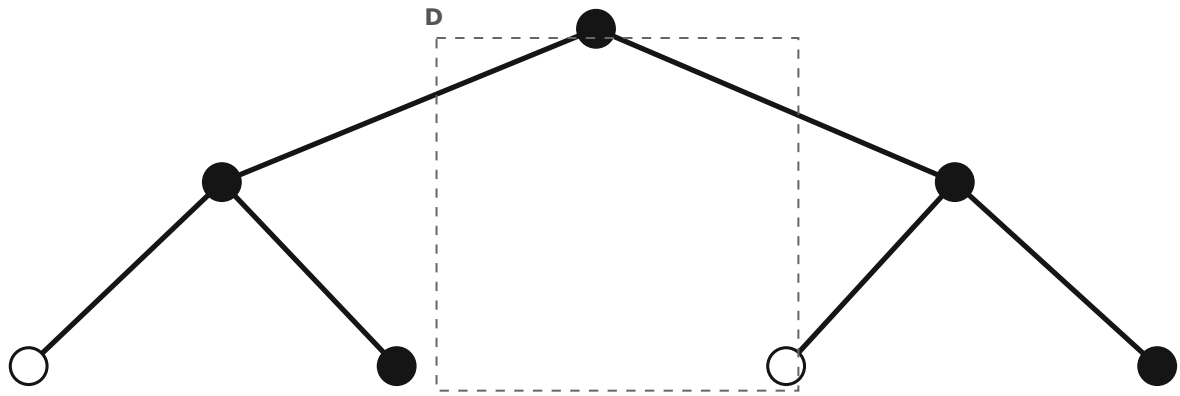
WHAT WAS ACTUALLY DONE

Exact arithmetic helpers enumerate feasible roots, clip remainders and verify reconstruction. They reject impossible combinations, while still retaining every feasible unknown scalar unless a separate heuristic cuts further.

ORIGINAL FINDING / EXECUTED RECORD

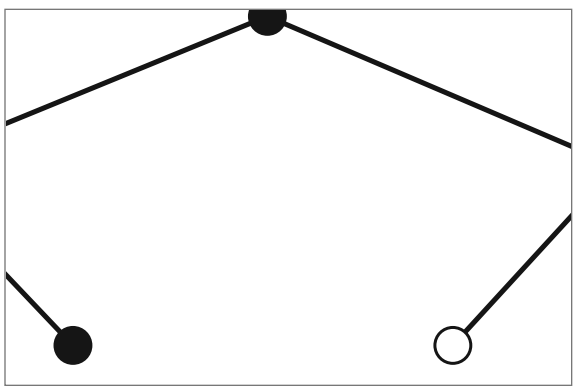
Integer roots, coefficient limits, clipping and reconstruction are tested arithmetically.

M041
Phase-2 Delta2 enumeration



01	02	03
R2 and C2	Delta2 sweep	possible R1 values

D / OPERATION DETAIL



Vector detail / 0.98× main-view scale

HOW THE OPERATION WORKS

Given R2 and C2, each Delta2 selects an R1 through R1=R2^3+C2*R2^2+Delta2.

READ THE THREE STAGES

01 R2 and C2
02 Delta2 sweep
03 possible R1 values

WHAT WAS ACTUALLY DONE

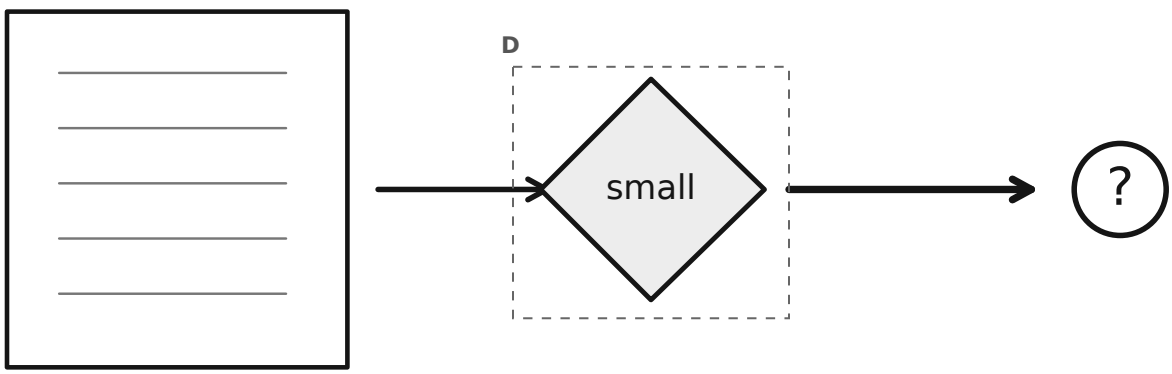
Phase-2 routines compute feasible Delta2 bounds and enumerate or stride through them. The smaller-looking parameter still represents many distinct scalar bases that may each require a D1 search.

ORIGINAL FINDING / IMPLEMENTED

Enumerate feasible R1 values within each R2/C2 hypothesis; this is a search coordinate, not an independently observed point.

M042

Phase-2 public-key-guided light probes

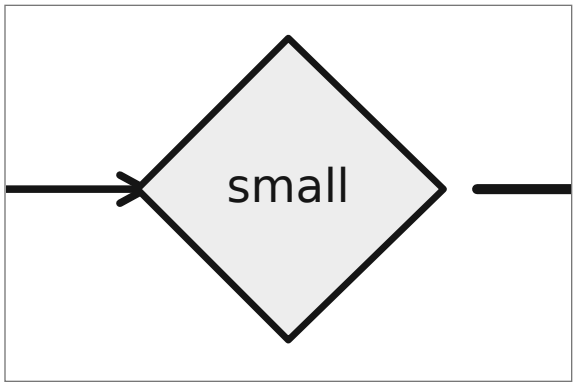


01 guessed Delta2

02 shift target point

03 short D1 probe

D / OPERATION DETAIL



Vector detail / 1.37× main-view scale

HOW THE OPERATION WORKS

For a guessed Delta2, build the corresponding shifted public point and test a small D1 budget before investing in a full leaf hunt.

READ THE THREE STAGES

01 guessed Delta2
02 shift target point
03 short D1 probe

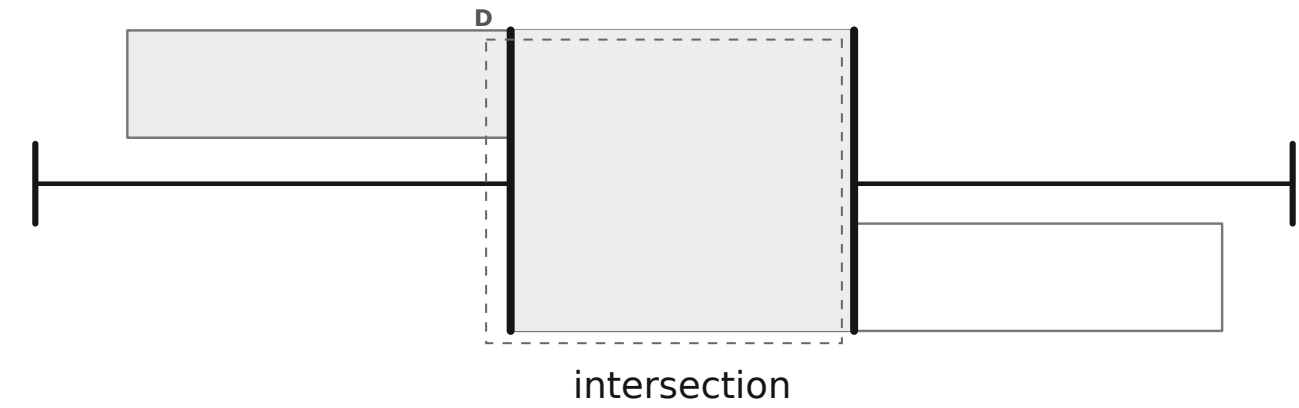
WHAT WAS ACTUALLY DONE

phase2_search.py ranks hints, constructs Phase-3 candidates and optionally probes them. It does not extract Delta2 directly from Q; every probe remains conditional on its guessed coordinates.

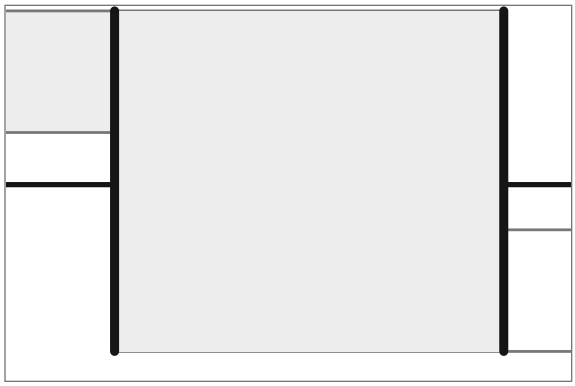
ORIGINAL FINDING / IMPLEMENTED

Construct D1 targets from Delta2 guesses and run bounded probes; no independent Delta2 oracle is obtained.

M043
Puzzle-bound clipping of D1



D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Intersect a leaf's D1 limits with the public puzzle bounds after subtracting its proposed base.

READ THE THREE STAGES

01 leaf D1 range
02 puzzle bounds
03 intersection

WHAT WAS ACTUALLY DONE

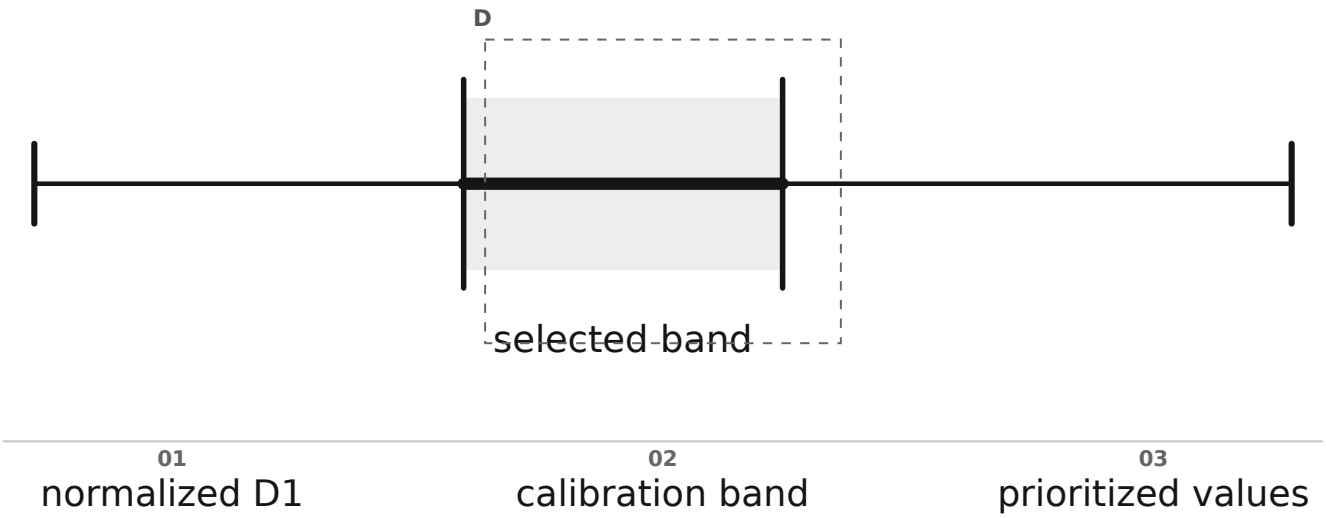
The audit measures how often clipping truly shortens D1. At high indices the public interval often contains the whole leaf, so the original D1 uncertainty remains.

ORIGINAL FINDING / EXECUTED RECORD

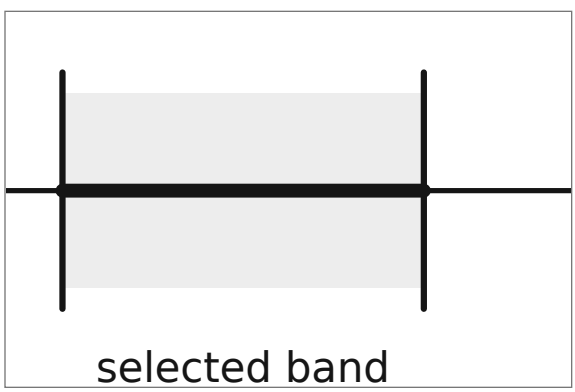
Historical audit records no useful narrowing for its high-index sample; advertised puzzle bounds are too broad at this leaf scale.

M044

D1 high-index ratio bands



D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Estimate normalized D1 positions from solved examples and prioritize a common ratio band for new slots.

READ THE THREE STAGES

01 normalized D1
02 calibration band
03 prioritized values

WHAT WAS ACTUALLY DONE

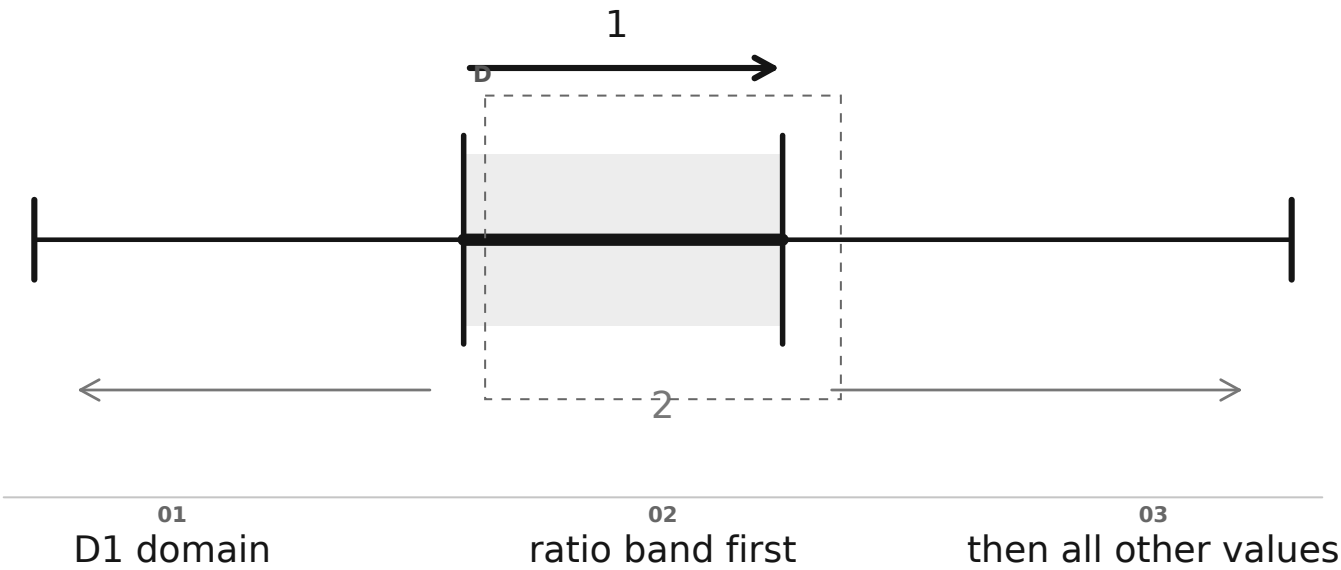
Historical high-index min/max or leave-one-out bands were exported as search presets. Their reduced width is real for the chosen band, but held-out misses show it is not guaranteed containment.

ORIGINAL FINDING / EXECUTED RECORD

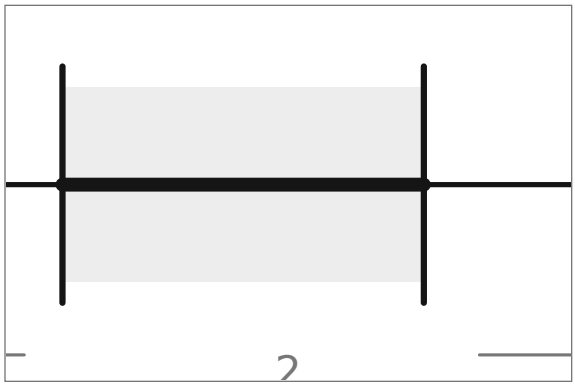
Calibrated narrow bands reduce width conditionally but miss held-out keys; their complement is required for complete support.

M045

Two-phase D1 band/complement policy



D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Search the ratio band first; if it misses, search all remaining D1 values. This preserves eventual full leaf coverage.

READ THE THREE STAGES

01 D1 domain
02 ratio band first
03 then all other values

WHAT WAS ACTUALLY DONE

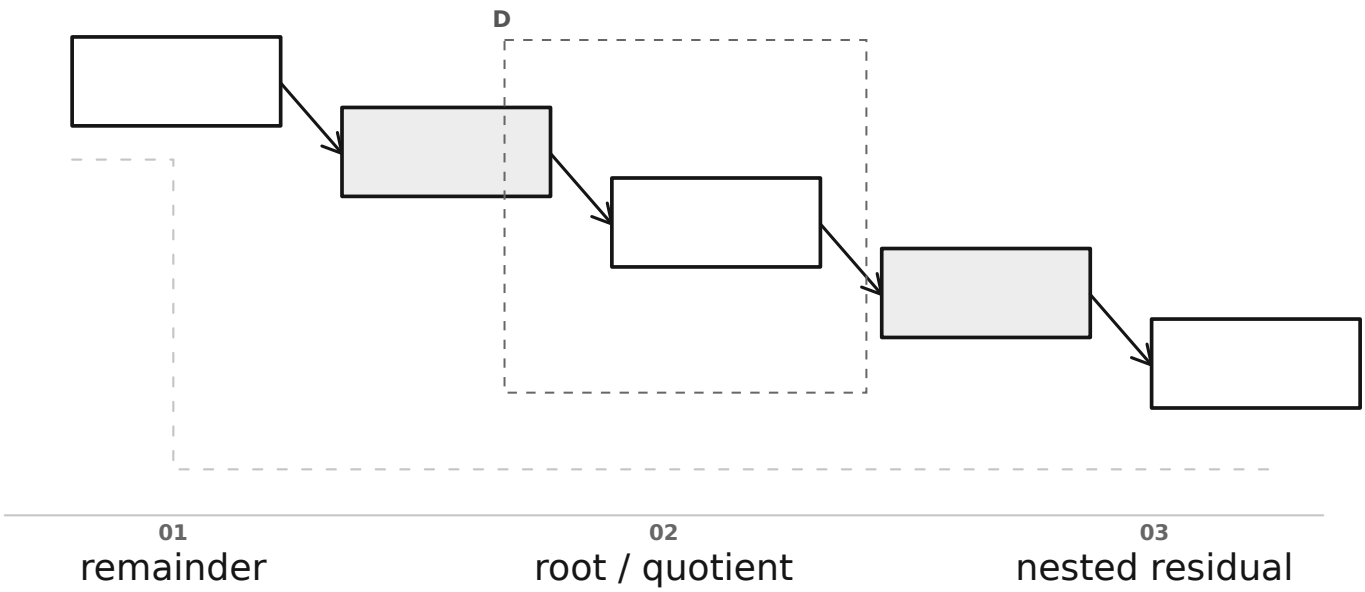
Preset builders generate phase-one and complementary phase-two jobs. The combined policy restores eligibility but can cost more than a single unpartitioned interval search.

ORIGINAL FINDING / IMPLEMENTED

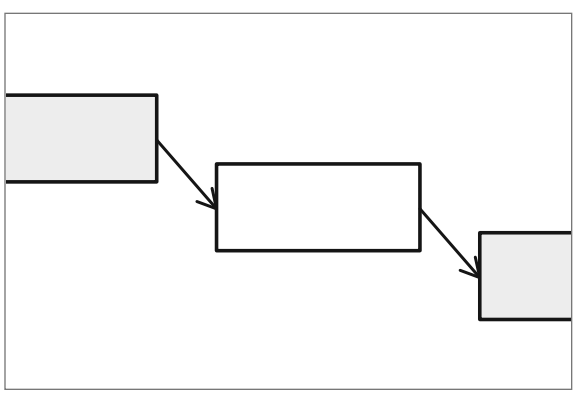
Prioritize a calibrated band and search its remainder after a miss. Union coverage restores full range rather than reducing entropy.

M046

Recursive remainder/sub-digit decomposition



D / OPERATION DETAIL



Vector detail / 0.98× main-view scale

HOW THE OPERATION WORKS

Decompose a remainder again into its own root, coefficient and residual, then repeat recursively.

READ THE THREE STAGES

01 remainder
02 root / quotient
03 nested residual

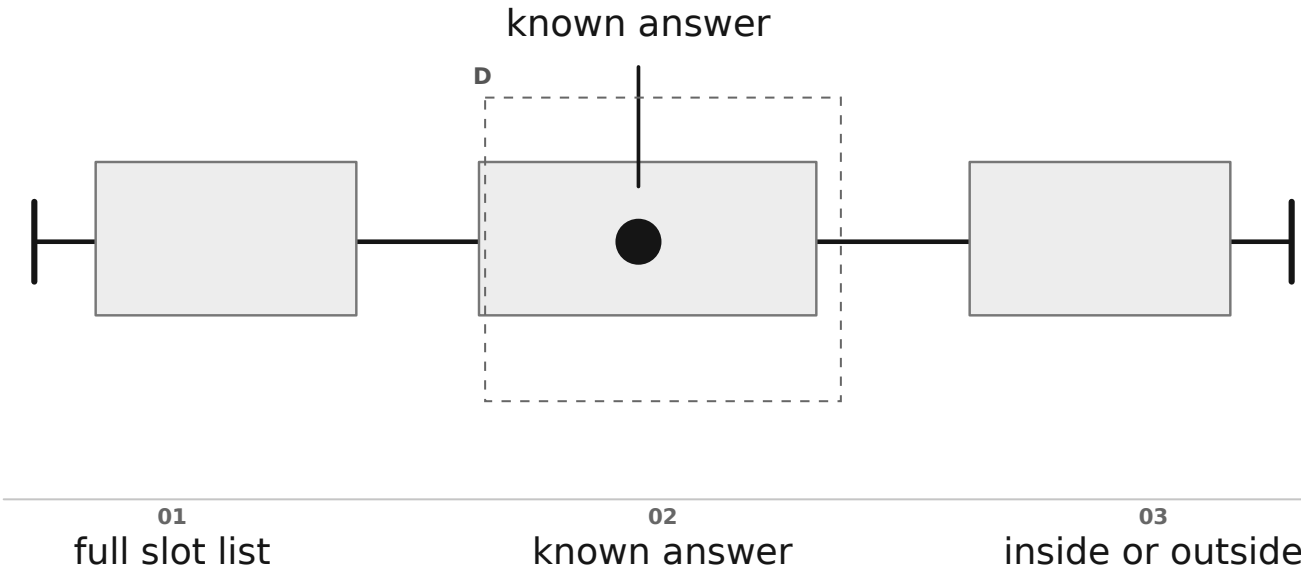
WHAT WAS ACTUALLY DONE

Backtests count subdigits and compare their distributions with equally transformed random integers. Most apparent simplification is a change of coordinates; omitted subdigits remain unknown.

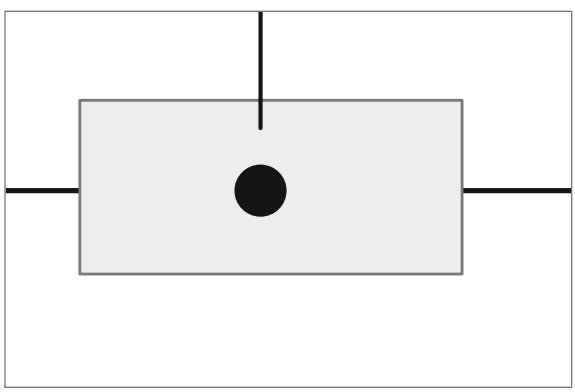
ORIGINAL FINDING / EXECUTED RECORD

Deeper splits were compared with random controls; apparent digit biases can arise from the map itself.

M047
All-known-key slot containment



D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

A complete hypothesis includes every base digit needed to define the final searched interval. Test whether the published answer lies in that interval.

READ THE THREE STAGES

01 full slot list
02 known answer
03 inside or outside

WHAT WAS ACTUALLY DONE

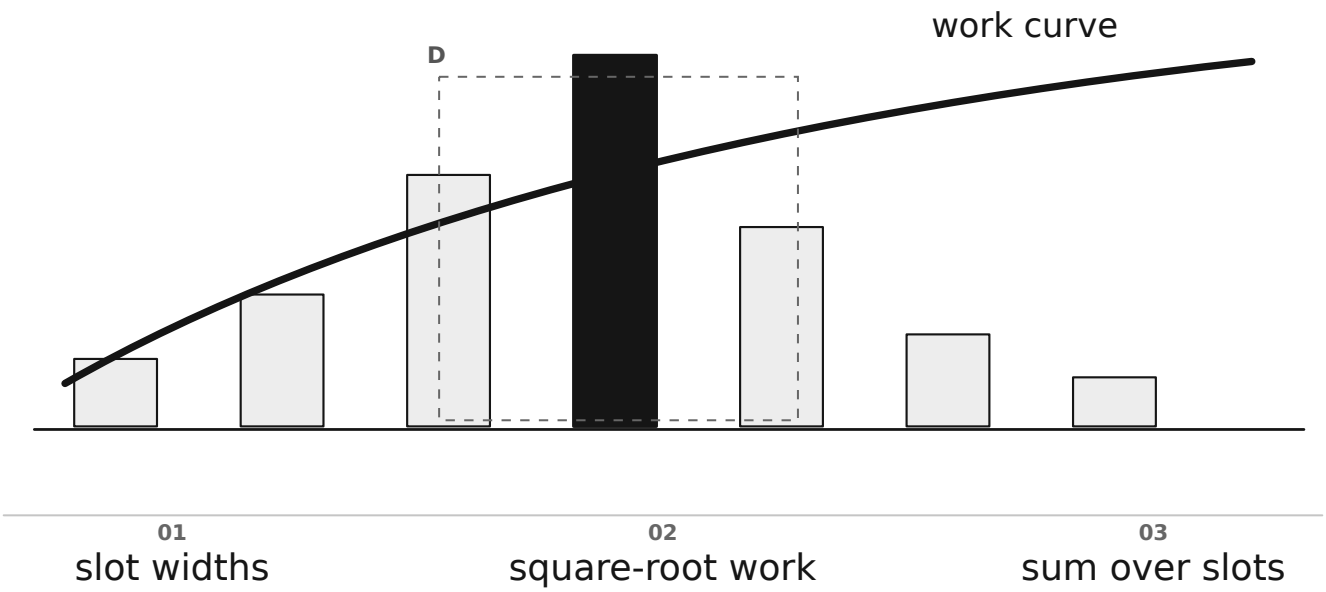
Backtest scripts evaluate known-key slot containment after candidate construction. Knowing the answer for evaluation must not leak into hint generation or be reported as an actual cryptographic recovery.

ORIGINAL FINDING / EXECUTED RECORD

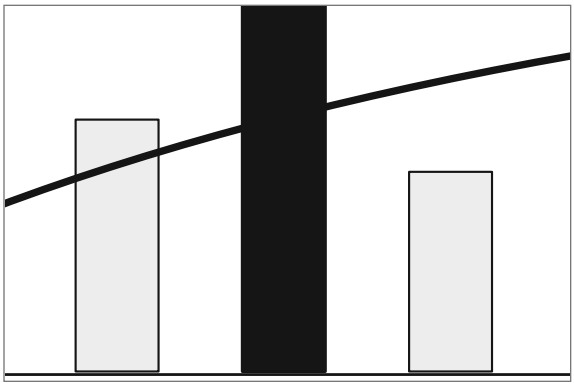
Check whether a generated complete slot includes a published answer; containment is not independent discovery of that answer.

M048

Post-slot square-root cost scaling



D / OPERATION DETAIL



Vector detail / 1.00× main-view scale

HOW THE OPERATION WORKS

Kangaroo leaf work scales roughly with the square root of its interval width, conditional on the correct target interval.

READ THE THREE STAGES

01 slot widths
02 square-root work
03 sum over slots

WHAT WAS ACTUALLY DONE

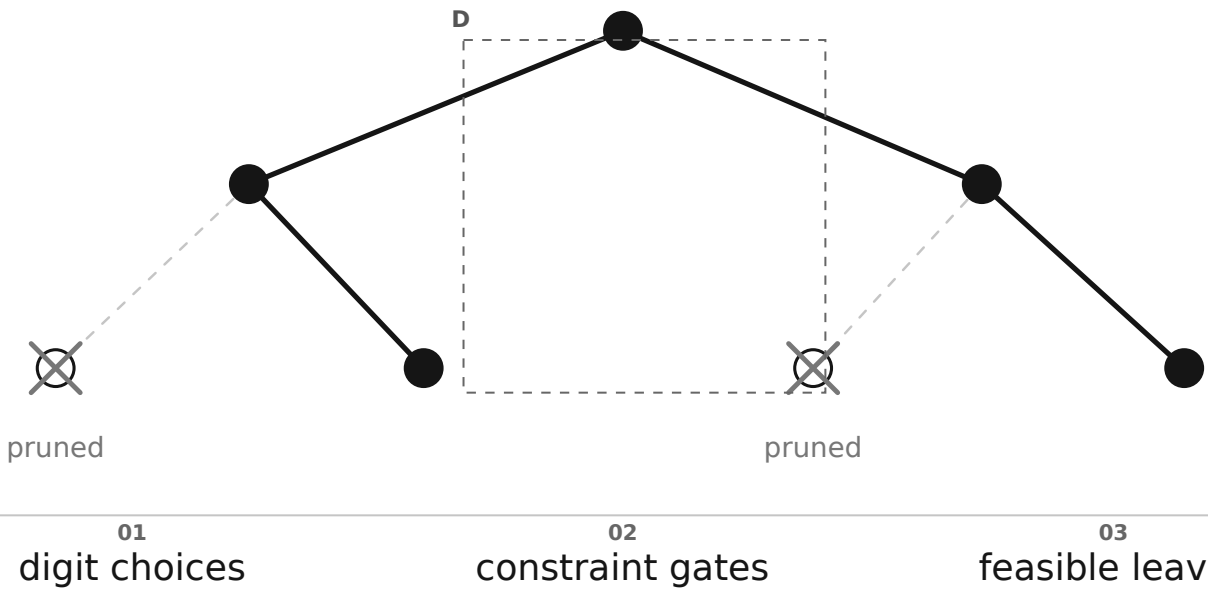
slot_scaling.py reports D1 leaf costs and related floor bounds. A real plan must additionally count all root/remainder hypotheses and its probability of including the correct leaf.

ORIGINAL FINDING / IMPLEMENTED

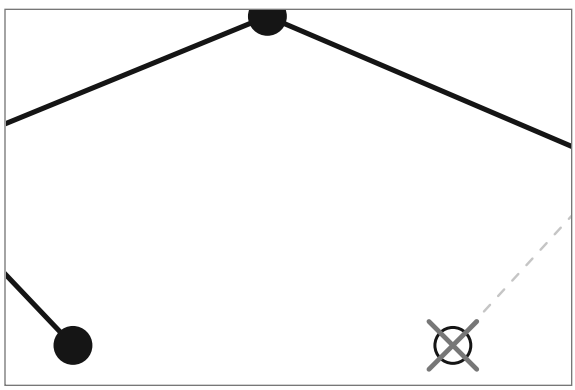
Count leaf cost conditional on a correct slot; audits warn against omitting the number of unknown slots.

M049

Constraint-satisfaction slot construction



D / OPERATION DETAIL



Vector detail / 0.98× main-view scale

HOW THE OPERATION WORKS

Express digit domains and interval inequalities as constraints, then select variable orderings to prune impossible partial assignments.

READ THE THREE STAGES

01 digit choices
02 constraint gates
03 feasible leaves

WHAT WAS ACTUALLY DONE

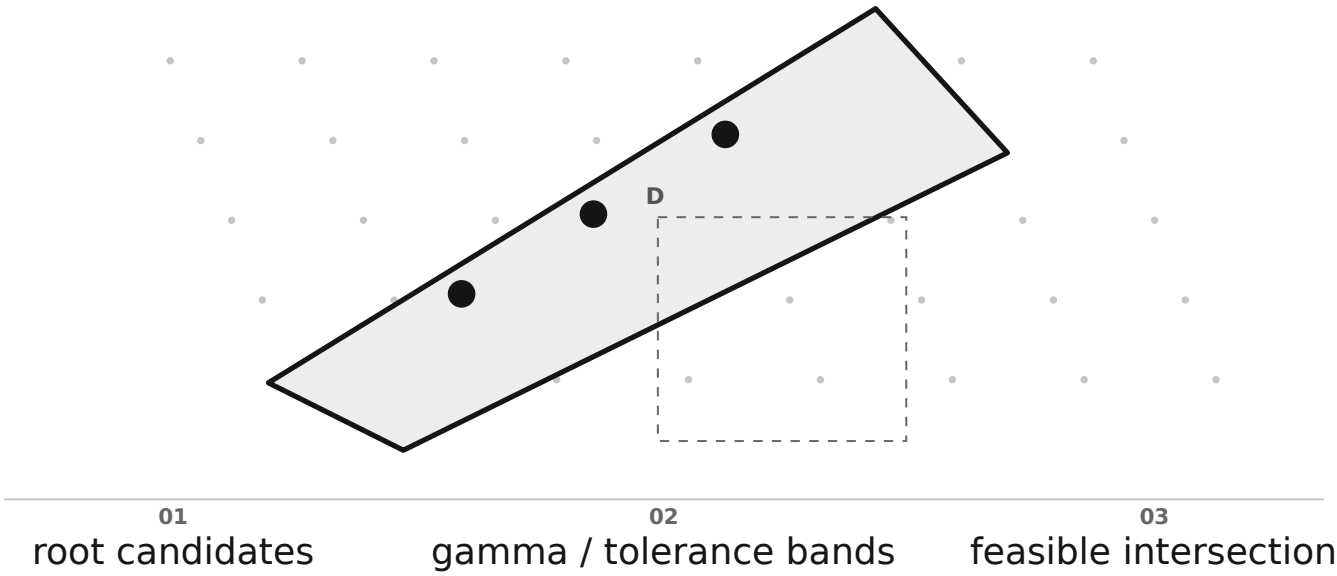
CSP modules compare fixed, minimum-remaining-values and degree strategies. They reduce wasted inconsistent tuples but do not add new information about which consistent tuple is the secret.

ORIGINAL FINDING / EXECUTED RECORD

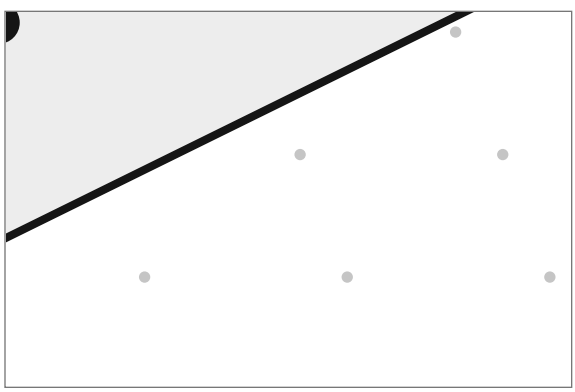
Exact feasibility graph with fixed, MRV and degree orderings; pruning invalid tuples does not predict the valid target tuple.

M050

Gamma-band and lattice narrowing presets



D / OPERATION DETAIL



Vector detail / 1.54× main-view scale

HOW THE OPERATION WORKS

Intersect feasible roots with selected gamma bands, FUSED tolerance bands or a two-stage union.

READ THE THREE STAGES

01 root candidates
02 gamma / tolerance bands
03 feasible intersection

WHAT WAS ACTUALLY DONE

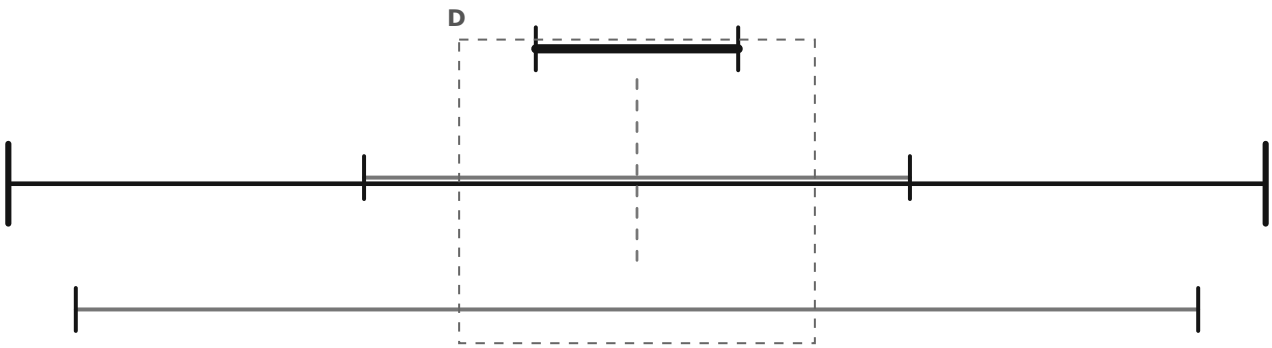
Preset reports count surviving R2 values and measure known-key coverage. A large candidate reduction can simply mean many true keys were excluded by the heuristic.

ORIGINAL FINDING / EXECUTED RECORD

Compare full lattice, gamma restrictions and FUSED intersections; stronger cuts reduce measured target coverage.

M051

Tiered minimal/narrow/focus/lattice exports

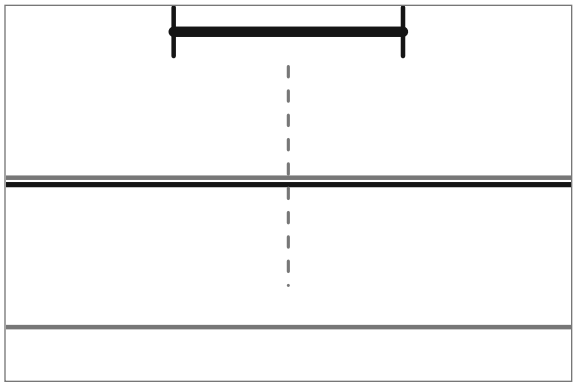


01 minimal tier

02 narrow / focus tiers

03 full lattice tier

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Package different breadths of the same hypothesis family as minimal, narrow, focus or full-lattice tiers.

READ THE THREE STAGES

01 minimal tier

02 narrow / focus tiers

03 full lattice tier

WHAT WAS ACTUALLY DONE

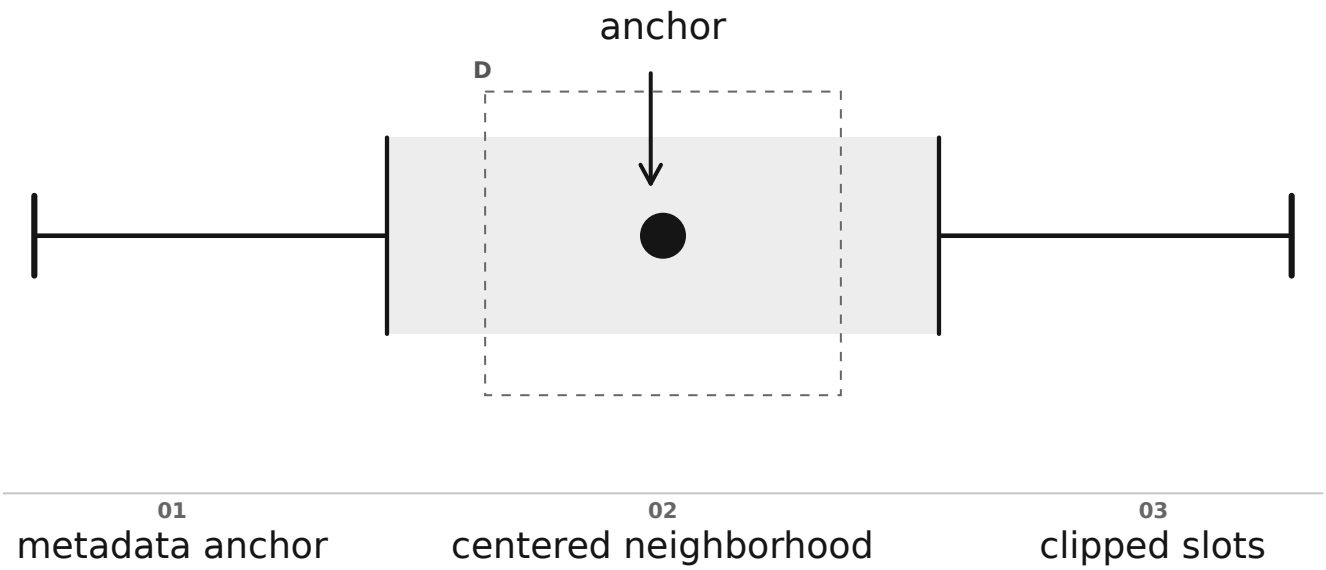
Exporters select/cap slots, sometimes with diversity rules. Saved calibration reports explicitly check whether the known true R2 and full tuple were omitted from each exported job set.

ORIGINAL FINDING / EXECUTED RECORD

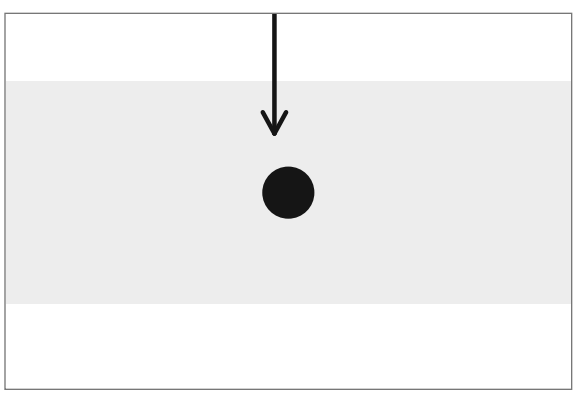
Saved manifests can exclude the known calibration target; a valid export is not proof of target inclusion.

M052

Anchor/TRAILING-centered slot construction



D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Use a supplied metadata number as a plausible center, or derive CTheory coordinates from it to define nearby slots.

READ THE THREE STAGES

01 metadata anchor
02 centered neighborhood
03 clipped slots

WHAT WAS ACTUALLY DONE

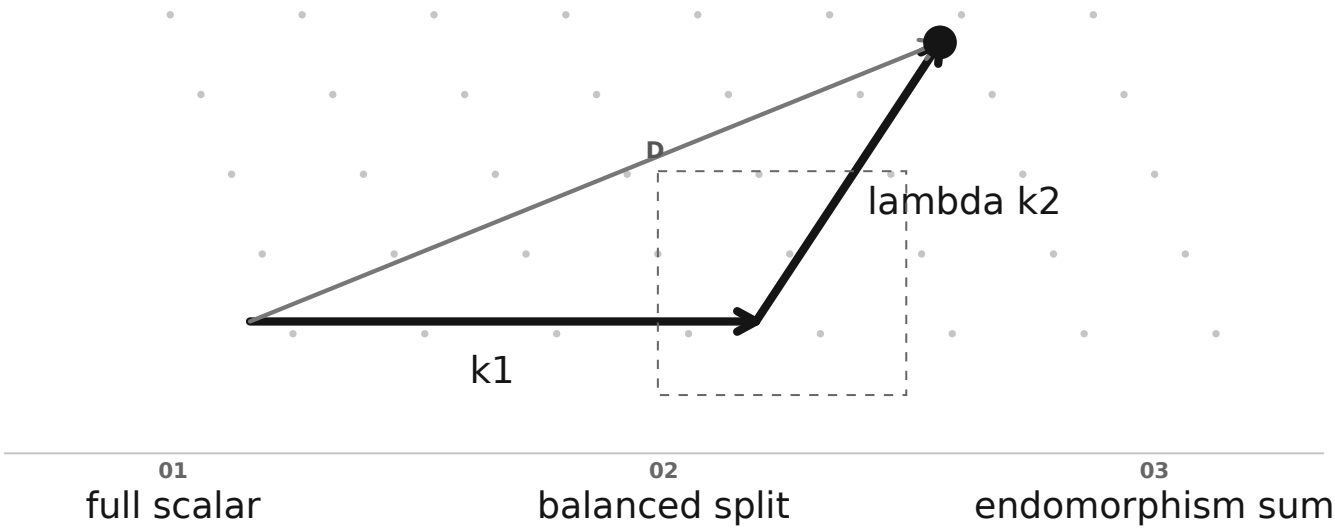
wrapped_export.py constructs anchor and TRAILING jobs. They are guesses informed by an artifact, not proven high bits or offsets of the target scalar.

ORIGINAL FINDING / IMPLEMENTED

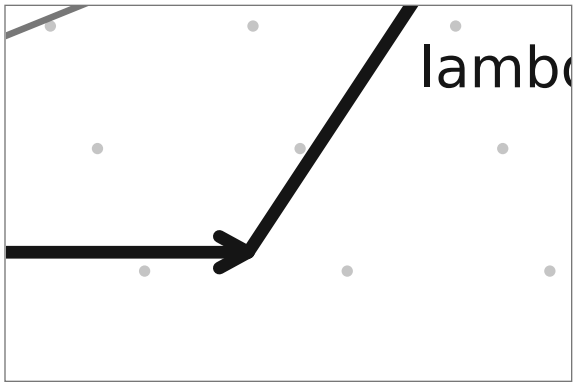
Metadata-centered guesses are explicit hypotheses and require cryptographic verification.

M053

GLV scalar decomposition and endomorphism



D / OPERATION DETAIL



Vector detail / 1.54× main-view scale

HOW THE OPERATION WORKS

The secp256k1 endomorphism permits k congruent to r1+lambda*r2 modulo group order, with a related point map.

READ THE THREE STAGES

01 full scalar
02 balanced split
03 endomorphism sum

WHAT WAS ACTUALLY DONE

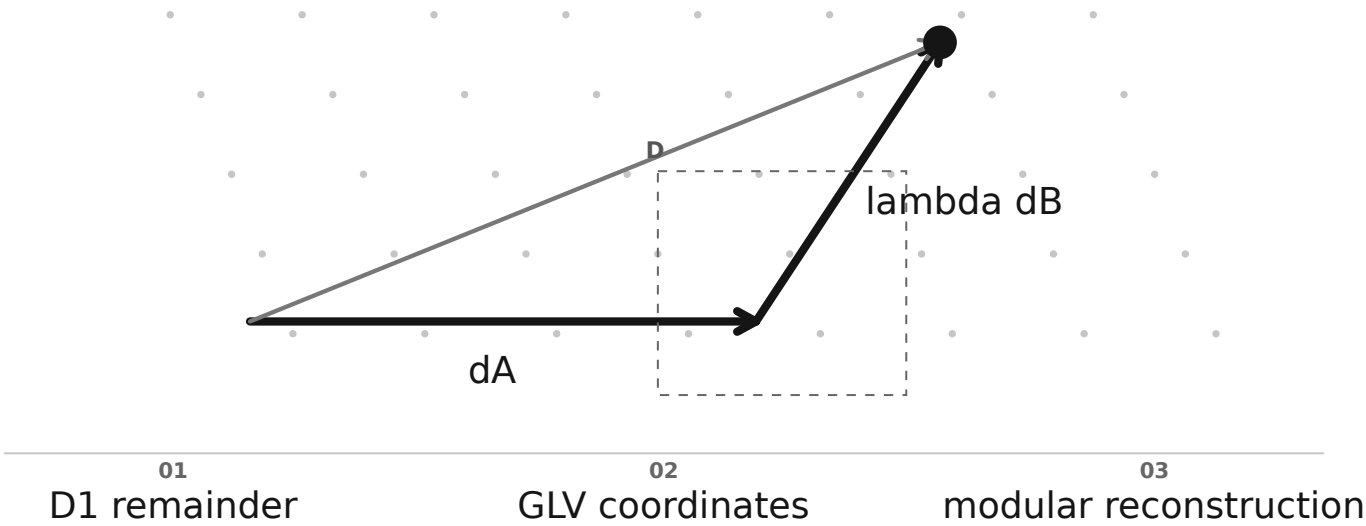
EC decomposition code verifies scalar and point identities and can accelerate multiplication. Computing a split of a known scalar does not recover that scalar from a public point.

ORIGINAL FINDING / EXECUTED RECORD

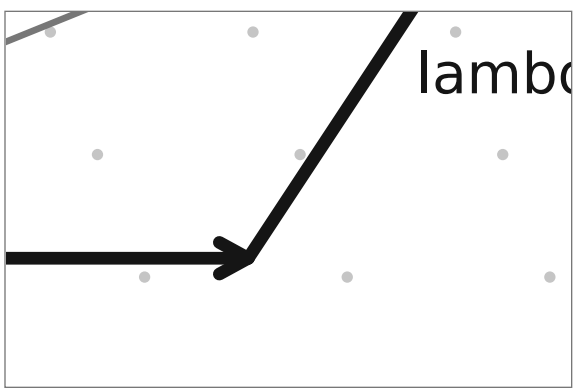
Correct arithmetic acceleration checked on known points; often degenerate for small scalars and not a generic entropy cut.

M054

GLV splitting specifically on D1



D / OPERATION DETAIL



Vector detail / 1.54× main-view scale

HOW THE OPERATION WORKS

Attempt to apply the same balanced GLV split to the much smaller D1 scalar instead of a full-width scalar.

READ THE THREE STAGES

01 D1 remainder
02 GLV coordinates
03 modular reconstruction

WHAT WAS ACTUALLY DONE

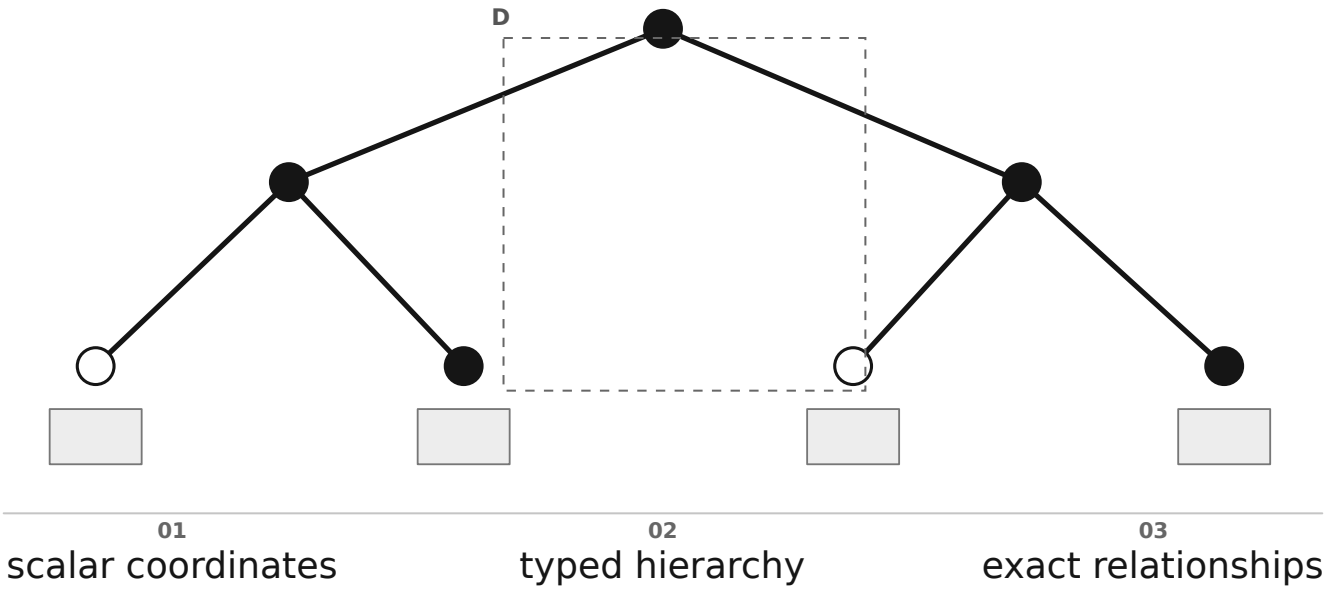
The D1 audit checks the resulting parts and finds degeneration: one component can retain effectively the original small scalar. A nominal two-part representation supplies no automatic search-width reduction.

ORIGINAL FINDING / AUDITED/CORRECTED

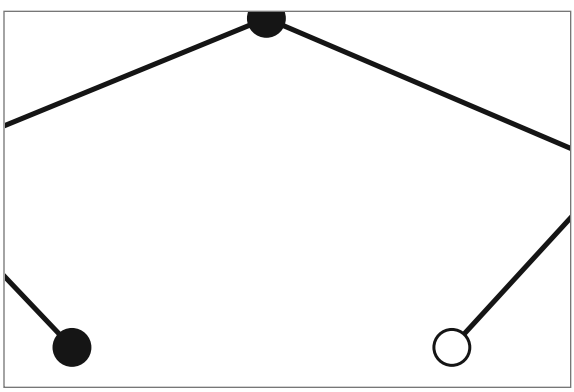
Leaf scalars far below group order do not obtain the advertised balanced split; no free narrowing follows.

M055

Exact value atlas and hierarchy visualizations



D / OPERATION DETAIL



Vector detail / 0.98× main-view scale

HOW THE OPERATION WORKS

Export hierarchy values with explicit types, sizes and relationships so arithmetic and hypothesis ranges can be inspected visually.

READ THE THREE STAGES

01 scalar coordinates
02 typed hierarchy
03 exact relationships

WHAT WAS ACTUALLY DONE

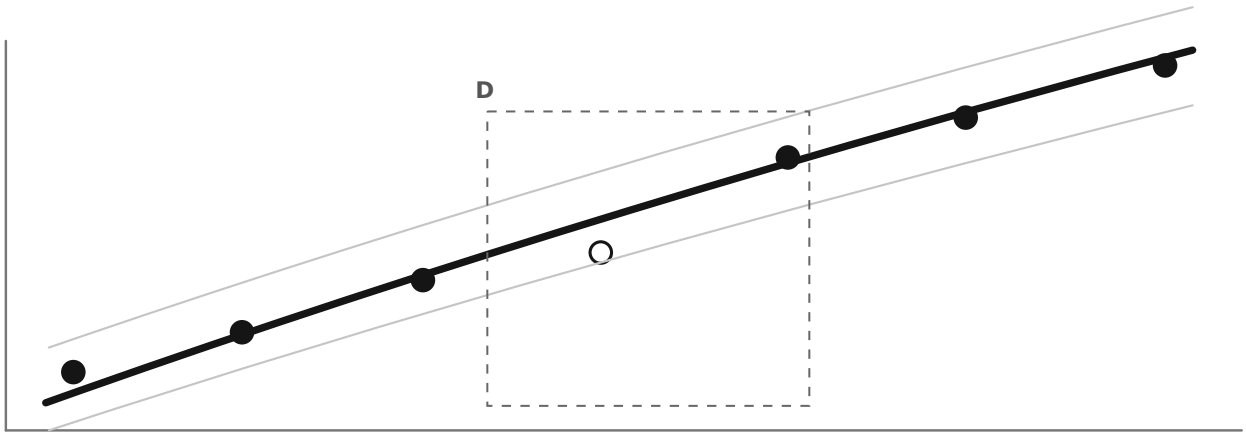
build_value_atlas writes exact data tables and separate plotting-friendly values. The atlas explains known quantities; a chart that looks clustered is not an independent predictor.

ORIGINAL FINDING / IMPLEMENTED

Typed exports, scientific plots and offline views expose values and bounds; visualization itself is not a predictor.

M056

FUSED global/local R2 regression

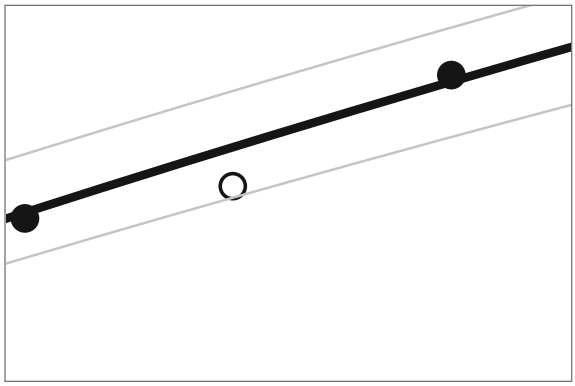


01 global and local fits

02 blend forecasts

03 R2 estimate

D / OPERATION DETAIL



Vector detail / 1.17× main-view scale

HOW THE OPERATION WORKS

Fit root growth globally and locally, then blend forecasts and sequence terms to predict R2 for a target index.

READ THE THREE STAGES

01 global and local fits
02 blend forecasts
03 R2 estimate

WHAT WAS ACTUALLY DONE

ctheory.py combines fitted constants with backward-looking anchors. Audits refit or exclude known answers to measure leakage; a forecast supplies a root guess, not Delta2 or D1.

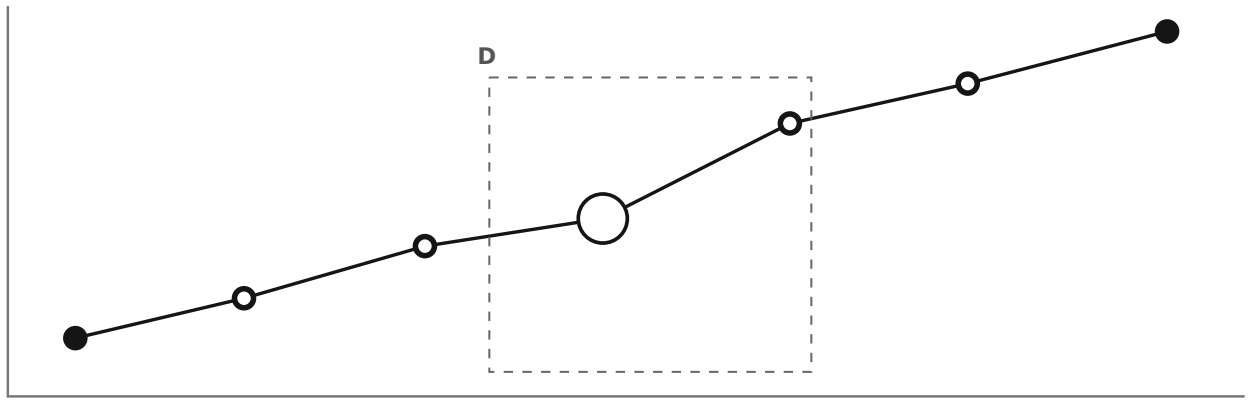
ORIGINAL FINDING / EXECUTED RECORD

Historical blends forecast ninth roots; narrow transformed error must be converted to actual scalar coverage.

M057

Gap-five sequence extrapolation



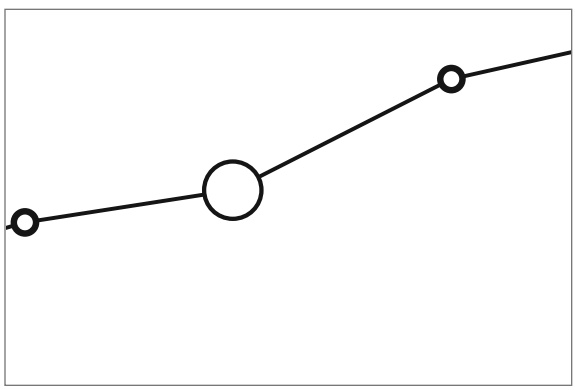


01 known roots

02 five-index steps

03 extrapolated root

D / OPERATION DETAIL



Vector detail / 1.17× main-view scale

HOW THE OPERATION WORKS

Use the regularly spaced disclosed high-index keys to estimate trends or differences five puzzle numbers apart.

READ THE THREE STAGES

01 known roots
02 five-index steps
03 extrapolated root

WHAT WAS ACTUALLY DONE

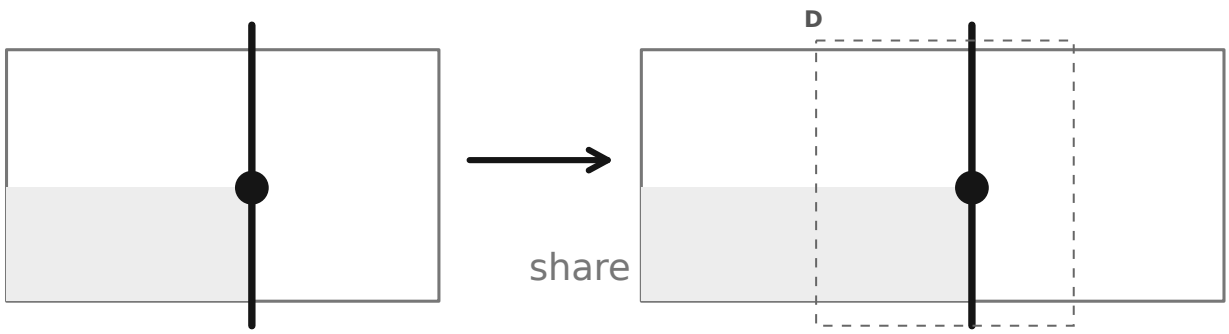
research.py extrapolates gap-five R2/remainder hints. Later controls run the same fitting on synthetic corpora to test whether apparent predictability exceeds the interval geometry alone.

ORIGINAL FINDING / EXECUTED RECORD

Forecast R2 or remainder ratios from every-fifth-puzzle sequences; tested against forward-only and uniform controls.

M058

Strategic Delta2 ratio hints

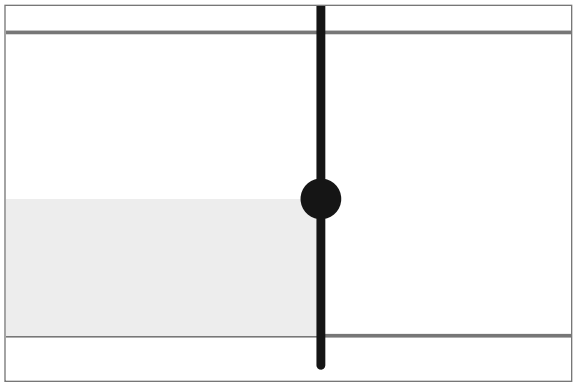


01 training Delta2 share

02 new R2 bucket

03 proposed Delta2

D / OPERATION DETAIL



Vector detail / 1.21× main-view scale

HOW THE OPERATION WORKS

Reuse normalized Delta2 ratios from calibration keys to propose a short list of exact Delta2 values.

READ THE THREE STAGES

01 training Delta2 share

02 new R2 bucket

03 proposed Delta2

WHAT WAS ACTUALLY DONE

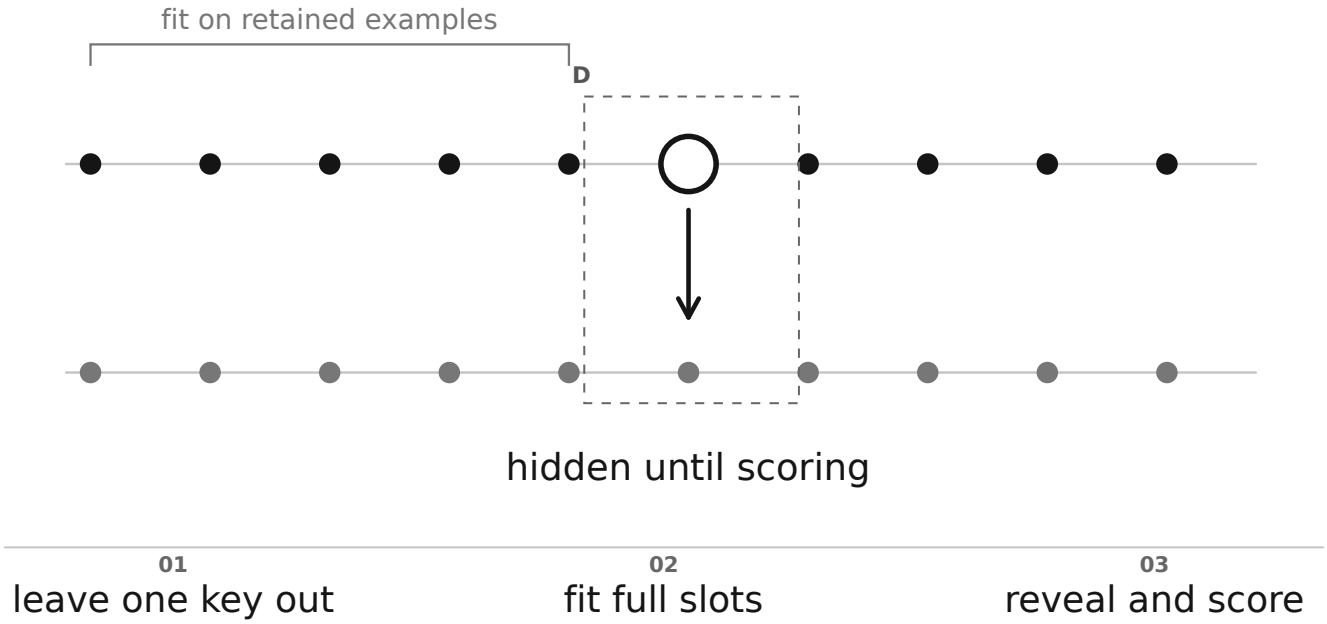
Historical code could include the evaluated key's own ratio, making recovery self-confirming. Corrected exclusion removes that shortcut and exposes the much weaker blind exact-hint performance.

ORIGINAL FINDING / AUDITED/CORRECTED

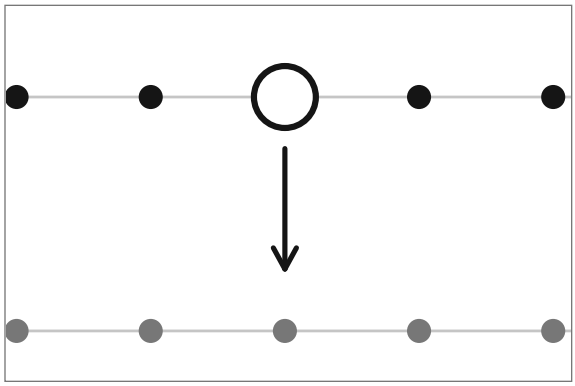
Target-in-calibration leakage explained the old 7/7 result; corrected blind high-index results were 0/7.

M059

Leave-one-out full-slot backtest



D / OPERATION DETAIL



Vector detail / 1.12× main-view scale

HOW THE OPERATION WORKS

Hide one key from every calibration source, build its full slot list, then reveal the answer only to score containment.

READ THE THREE STAGES

01 leave one key out
02 fit full slots
03 reveal and score

WHAT WAS ACTUALLY DONE

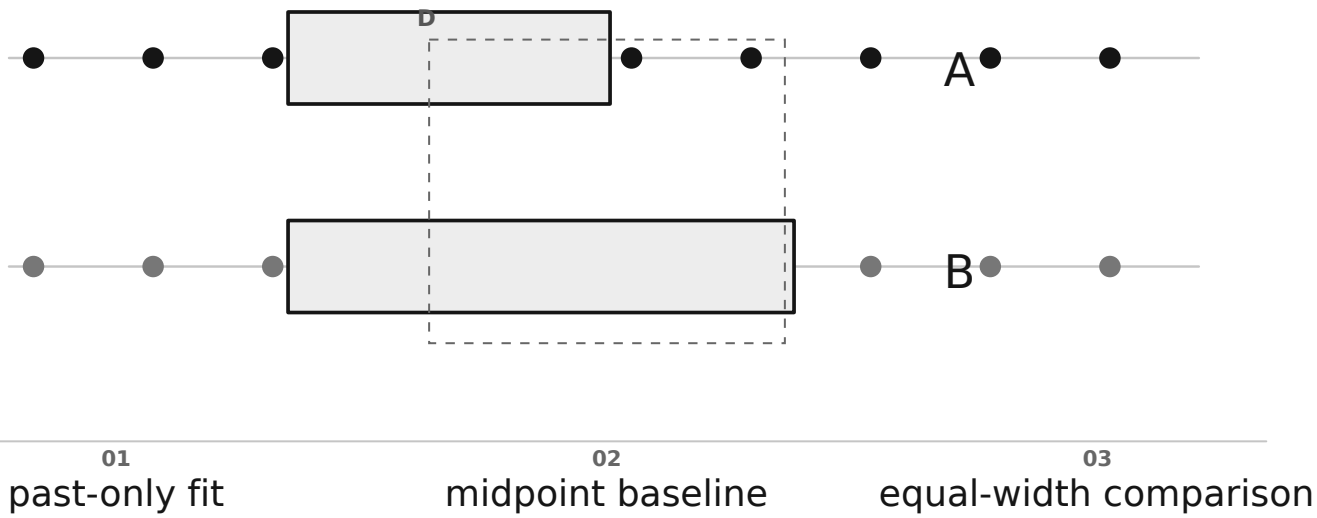
The corrected LOO report separately counts R2, C2, Delta2 and C1 success. Coarse root coverage cannot compensate for a missing exact Delta2 coordinate.

ORIGINAL FINDING / EXECUTED RECORD

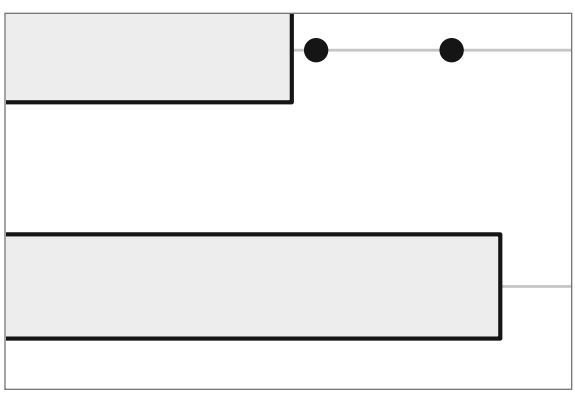
Corrected full-slot requirements fail in the documented seven high-index holdouts, despite coarse R2 coverage.

M060

Past-only FUSED versus midpoint



D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Compare a learned R2 forecast with the ninth root of the scalar interval midpoint using equal tolerances and actual scalar mass.

READ THE THREE STAGES

01 past-only fit
02 midpoint baseline
03 equal-width comparison

WHAT WAS ACTUALLY DONE

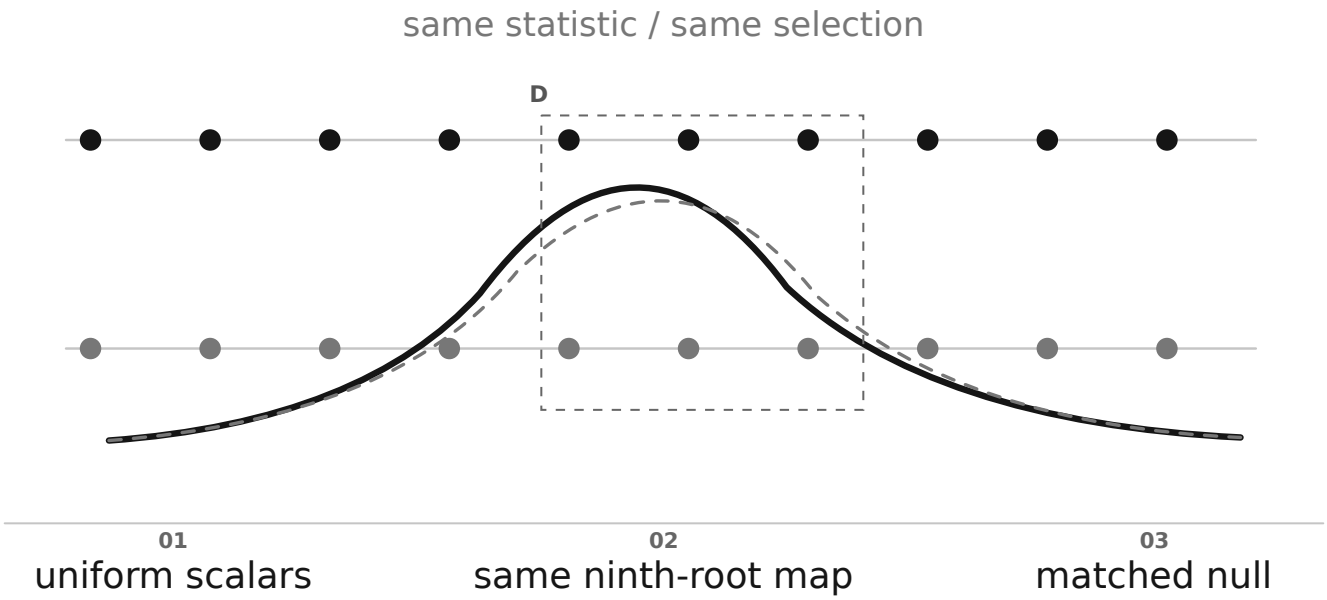
The comparison reports root error and coverage for past-only, LOO and production fits. Later solved keys or fixed full-corpus coefficients can make an ostensibly held-out fit optimistic.

ORIGINAL FINDING / EXECUTED RECORD

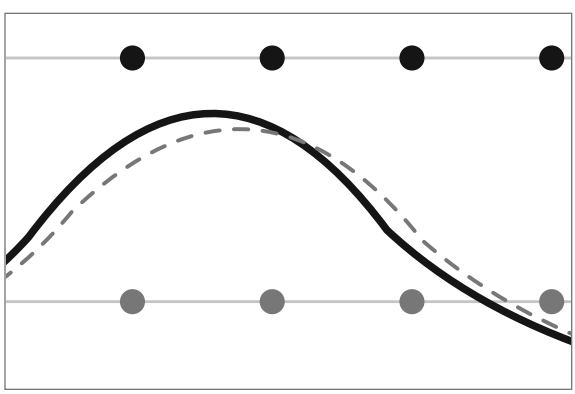
Same-band comparisons show that transformed midpoint baselines can rival or beat apparent R2 coverage.

M061

Correct uniform null for ninth-root clustering



D / OPERATION DETAIL



Vector detail / 1.17× main-view scale

HOW THE OPERATION WORKS

Transform uniform random scalars through the same ninth-root map as the real keys before comparing clustering.

READ THE THREE STAGES

01 uniform scalars
02 same ninth-root map
03 matched null

WHAT WAS ACTUALLY DONE

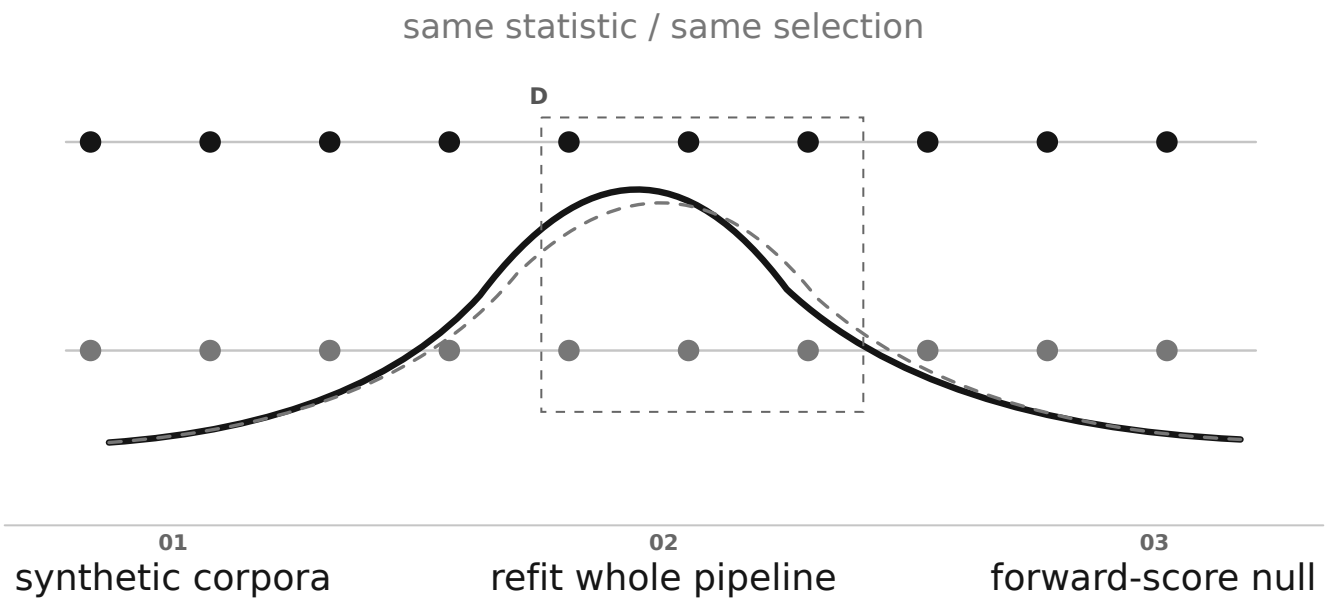
The chat-audit experiment derives the transformed baseline and simulates matched-bit-width keys. It corrects an earlier null that mistakenly treated the compressed root coordinate as nearly uniform.

ORIGINAL FINDING / AUDITED/CORRECTED

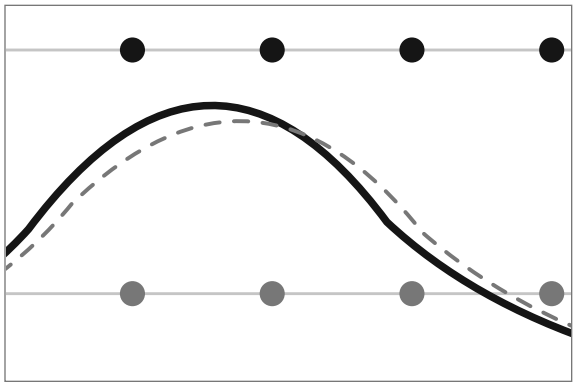
Uniform scalars naturally cluster after ninth-root transformation; an earlier overly broad null standard deviation was corrected.

M062

Forward-only synthetic-corpus null



D / OPERATION DETAIL



Vector detail / 1.17× main-view scale

HOW THE OPERATION WORKS

Run the entire forward-fitting procedure on synthetic uniform corpora, not just a single fixed prediction under a new null.

READ THE THREE STAGES

01 synthetic corpora
02 refit whole pipeline
03 forward-score null

WHAT WAS ACTUALLY DONE

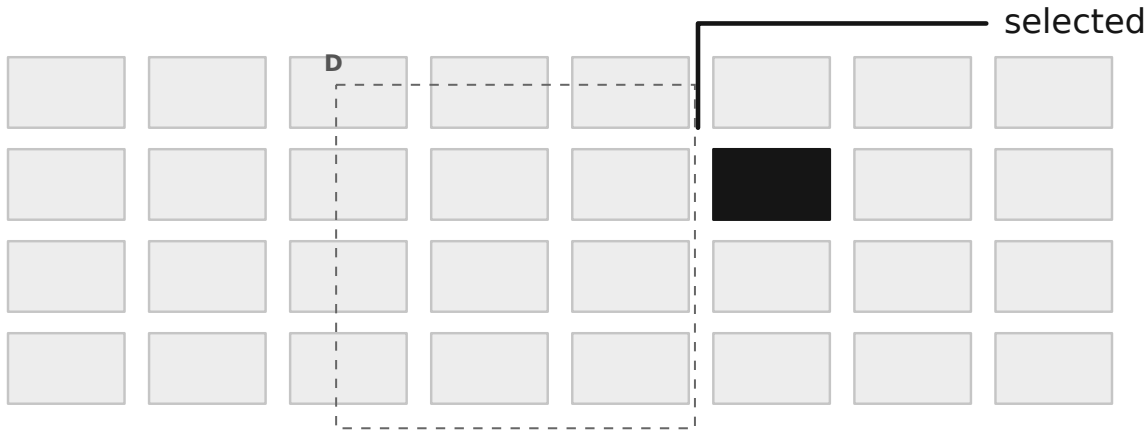
The frozen experiment counts exact covered scalar fractions and compares aggregate hits-minus-coverage with replicated null corpora. This tests that named predictor under stated historical-index conventions.

ORIGINAL FINDING / EXECUTED RECORD

Repeat the predictor-fitting process on uniform synthetic keys and count exact scalar coverage.

M063

Family-wise forward R2 grid



01
forecast grid

02
select best statistic

03
selected null comparison

D / OPERATION DETAIL



Vector detail / 1.00× main-view scale

HOW THE OPERATION WORKS

When many cutoffs, widths and forecasts are examined, compare the best-looking observed result with equally selected null results.

READ THE THREE STAGES

01 forecast grid
02 select best statistic
03 selected null comparison

WHAT WAS ACTUALLY DONE

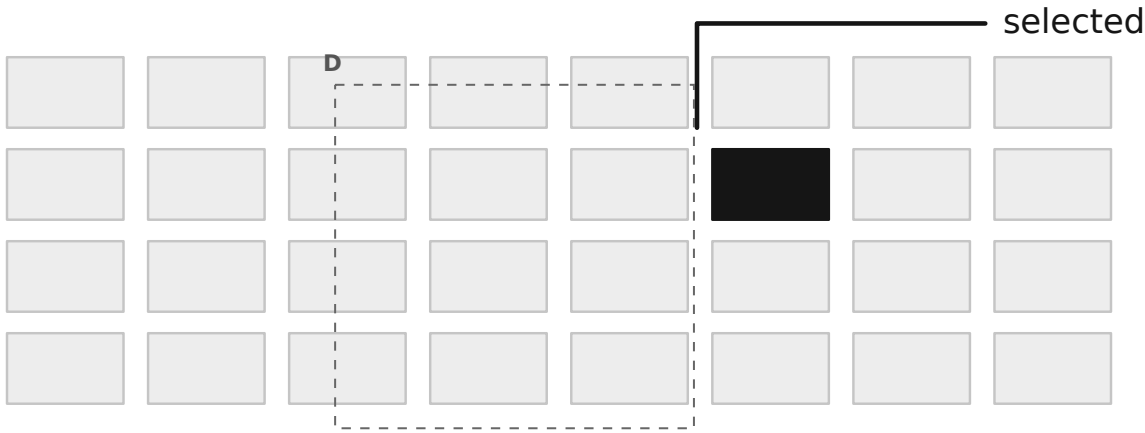
The family-wise script searches its declared model grid on both observed and synthetic keys. Adjusted evidence is weaker than quoting the smallest unadjusted p-value.

ORIGINAL FINDING / EXECUTED RECORD

Test selection across FUSED, gap-five, midpoint, cutoffs and window widths with family-wise controls.

M064

Contiguous-slice selection adjustment



01 candidate slices

02 best observed slice

03 repeat selection in null

D / OPERATION DETAIL



Vector detail / 1.00× main-view scale

HOW THE OPERATION WORKS

A favorable consecutive slice can be selected by chance. The null comparison must also search candidate slices.

READ THE THREE STAGES

01 candidate slices
02 best observed slice
03 repeat selection in null

WHAT WAS ACTUALLY DONE

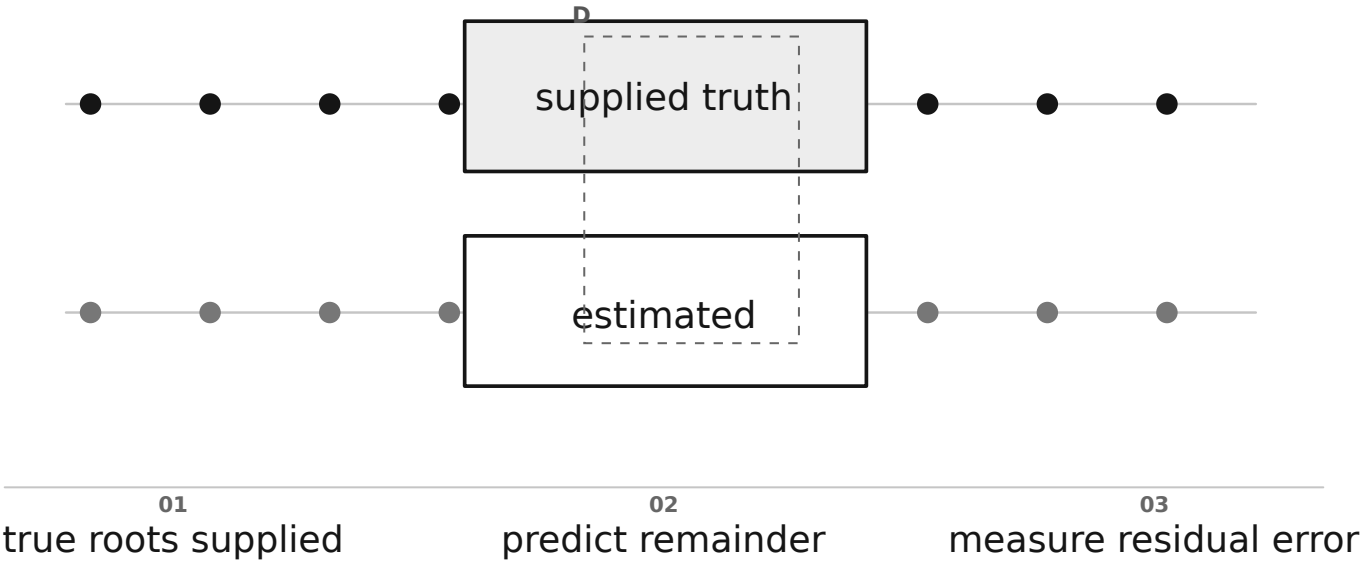
The slice-family script evaluates contiguous tier ranges and selection-adjusts their scores. It measures the effect of choosing a convenient evaluation interval after seeing outcomes.

ORIGINAL FINDING / EXECUTED RECORD

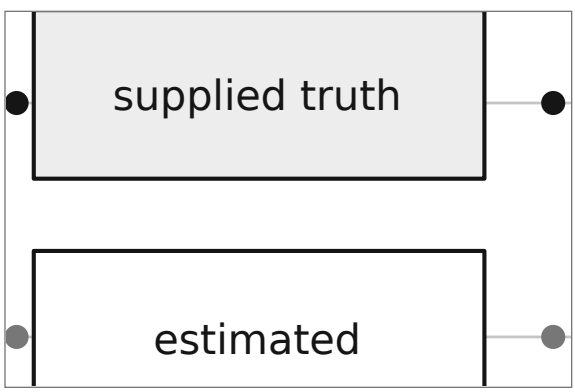
Compare best-looking index slices with similarly selected synthetic controls; guards against picking favorable subsequences.

M065

Remainder precision granted true roots



D / OPERATION DETAIL



Vector detail / 1.12× main-view scale

HOW THE OPERATION WORKS

Give a predictor the true root coordinates to isolate whether it can predict the remaining offset accurately enough.

READ THE THREE STAGES

01 true roots supplied
02 predict remainder
03 measure residual error

WHAT WAS ACTUALLY DONE

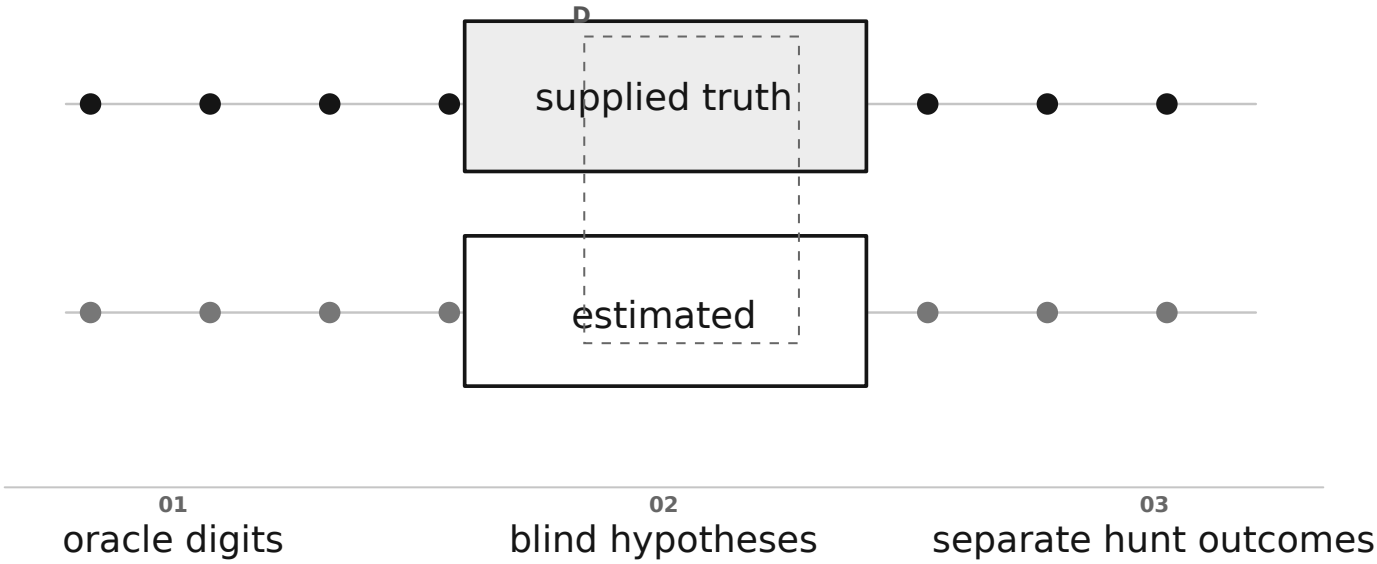
The precision audit compares predicted and true remainders in integer/key units. It is an oracle diagnostic, deliberately easier than an unknown-key attack, and still exposes large residual errors.

ORIGINAL FINDING / EXECUTED RECORD

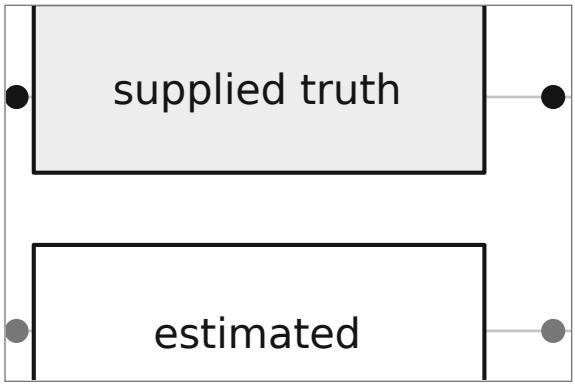
Measures exact offset-prediction errors even when root coordinates are supplied; coarse root prediction does not supply leaf precision.

M066

Oracle-versus-blind hunt comparison



D / OPERATION DETAIL



Vector detail / 1.12× main-view scale

HOW THE OPERATION WORKS

Separate three tasks: reconstruct from supplied digits, include the answer in a candidate interval, and actually discover it by target verification.

READ THE THREE STAGES

01 oracle digits
02 blind hypotheses
03 separate hunt outcomes

WHAT WAS ACTUALLY DONE

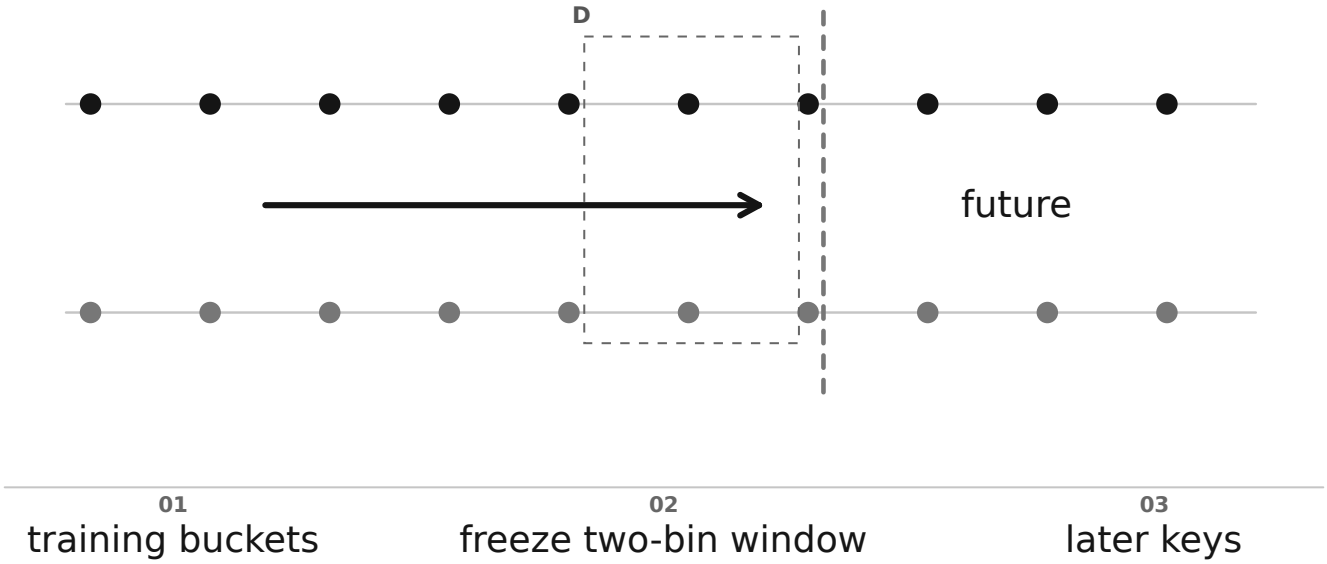
Saved audits rerun planner logic with target answers excluded and compare oracle/full-slot results. A true-key-assisted coordinate lookup is labeled oracle rather than blind recovery.

ORIGINAL FINDING / AUDITED/CORRECTED

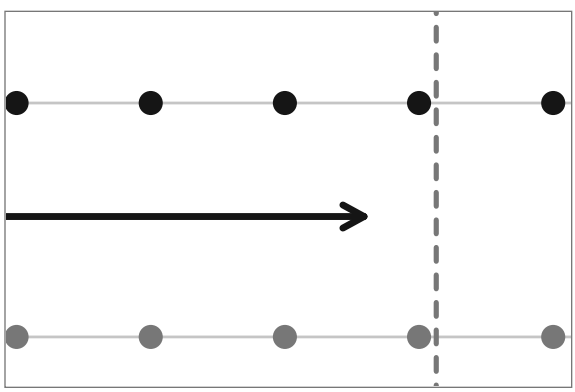
Distinguishes provided true coordinates, key containment, and actual address/public-point discovery.

M067

Frozen ninth-root remainder window



D / OPERATION DETAIL



Vector detail / 1.12× main-view scale

HOW THE OPERATION WORKS

Normalize the ninth-root bucket remainder, fit a two-bin window on training keys, then freeze it for later keys.

READ THE THREE STAGES

01 training buckets
02 freeze two-bin window
03 later keys

WHAT WAS ACTUALLY DONE

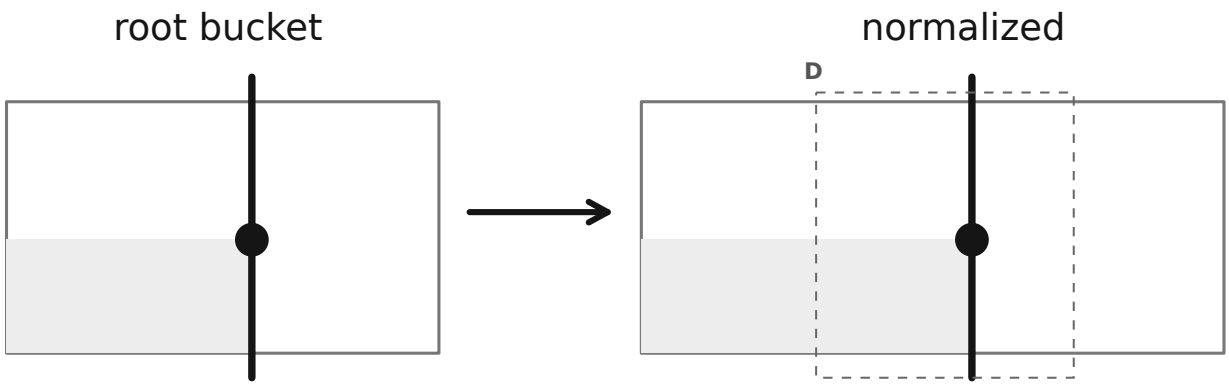
The recent exact experiment uses 16 bins and puzzles 17-32 for fitting, then tests 33-64. It records both hits and the actual fraction of scalar candidates selected.

ORIGINAL FINDING / EXECUTED RECORD

Recent test trained on 17-32 and covered 7/32 evaluation keys; uncorrected p about 0.116, not convincing evidence.

M068

Square/cube/ninth-root window family

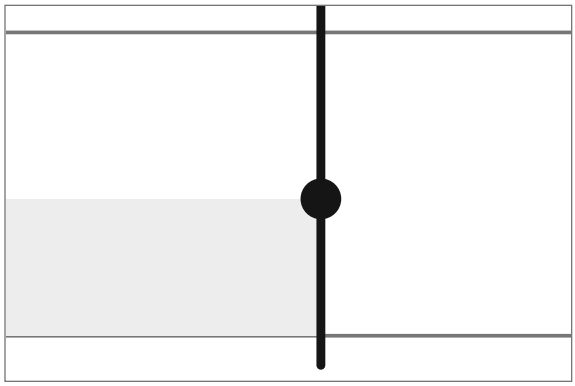


01 square / cube / ninth

02 normalized buckets

03 same window rule

D / OPERATION DETAIL



Vector detail / 1.21× main-view scale

HOW THE OPERATION WORKS

Repeat the same declared window-fitting rule for square, cube, ninth-root and ordinary range coordinates at four widths.

READ THE THREE STAGES

01 square / cube / ninth
02 normalized buckets
03 same window rule

WHAT WAS ACTUALLY DONE

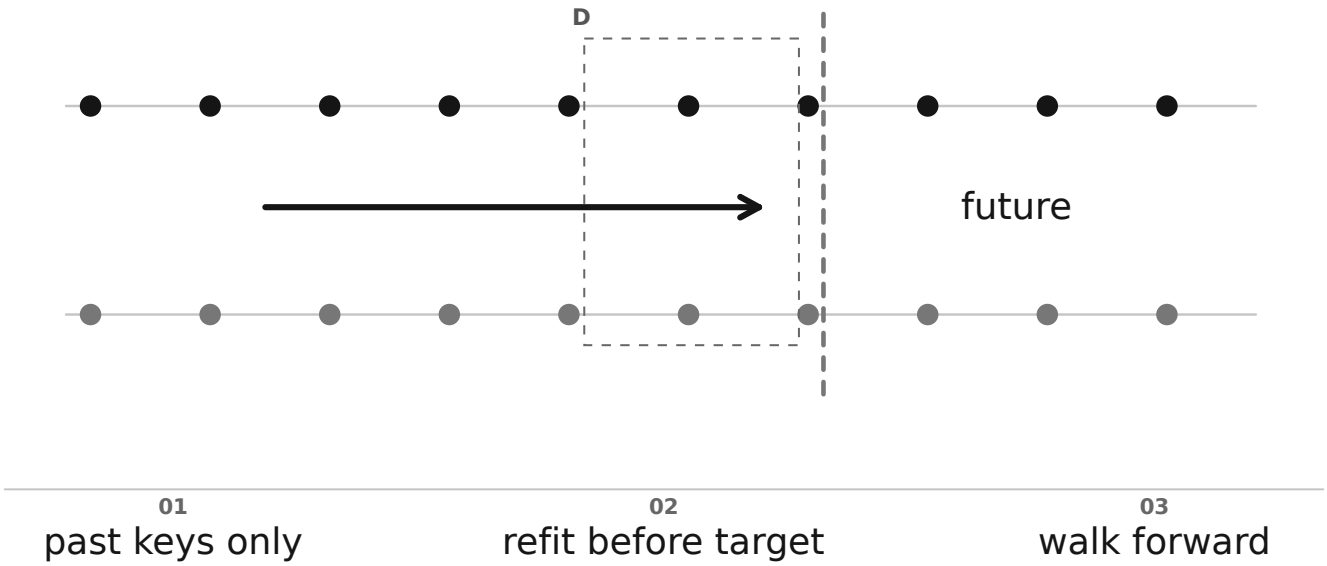
extended.py tests 16 models, computes exact candidate counts and applies a multiple-model correction. It does not keep tuning windows after examining each evaluation result.

ORIGINAL FINDING / EXECUTED RECORD

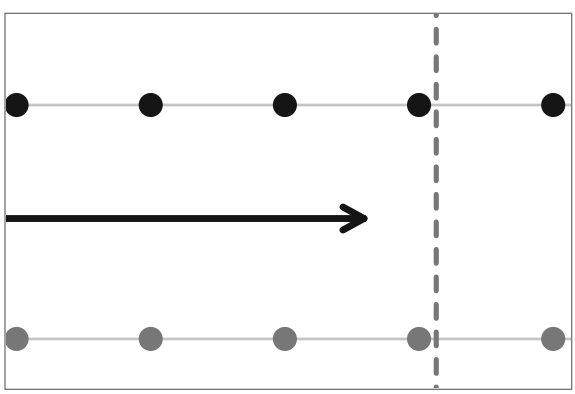
Sixteen frozen models across four nominal widths; none significant after multiple-model correction.

M069

Walk-forward adaptive remainder windows



D / OPERATION DETAIL



Vector detail / 1.12× main-view scale

HOW THE OPERATION WORKS

Before predicting each evaluation key, refit only on smaller indices, making every forecast independent of that target's stored answer.

READ THE THREE STAGES

01 past keys only
02 refit before target
03 walk forward

WHAT WAS ACTUALLY DONE

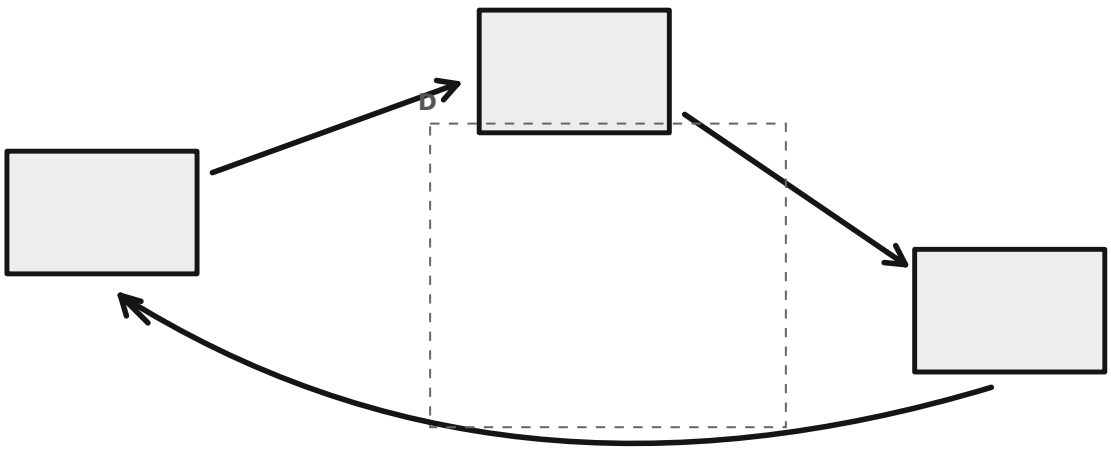
The expanded study records per-key windows, actual coverage and sequential evidence scores. Previously inspected data still make this retrospective rather than pristine prospective validation.

ORIGINAL FINDING / EXECUTED RECORD

Refit only on lower indices; final evidence scores did not exceed the family threshold.

M070

Self-improving CTheory parameter loop

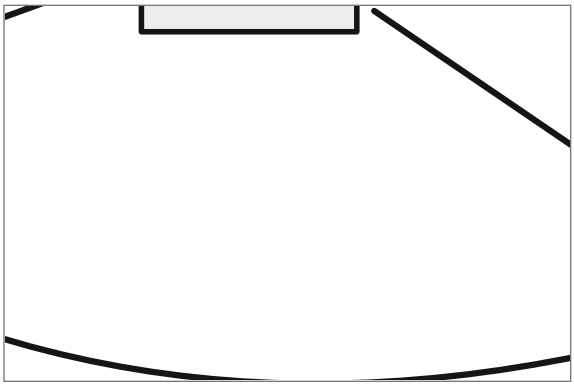


01 search planner

02 measure outcomes

03 update parameters

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Alternate measurement, statistical analysis and parameter changes to tune a search planner's hint/window budgets.

READ THE THREE STAGES

01 search planner
02 measure outcomes
03 update parameters

WHAT WAS ACTUALLY DONE

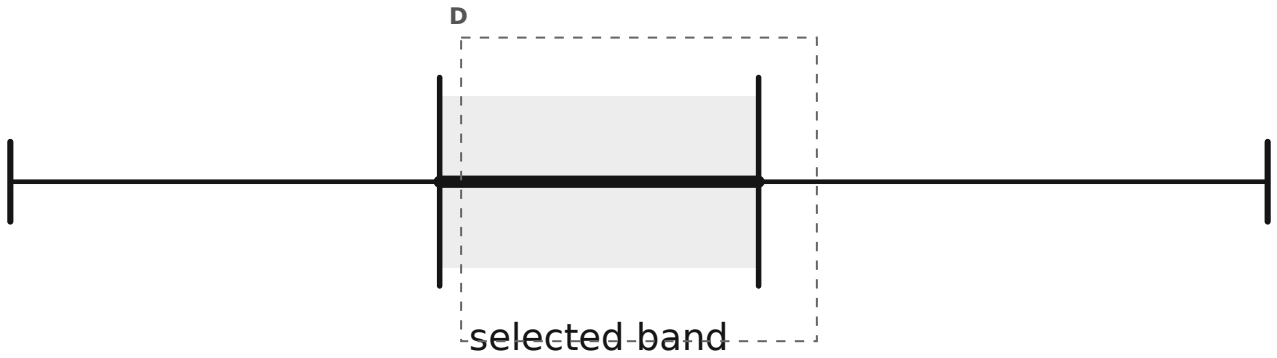
ctheory_sim adjusts r2_pct, hint counts, tier or slot limits from recorded performance. It optimizes the tested planner; biased/leaky metrics would teach it the wrong objective.

ORIGINAL FINDING / IMPLEMENTED

Measure/analyze/improve loop tunes hint budgets, R2 width and slots; usefulness is limited by honest target inclusion.

M071

R2 intervals as conditional search prioritization

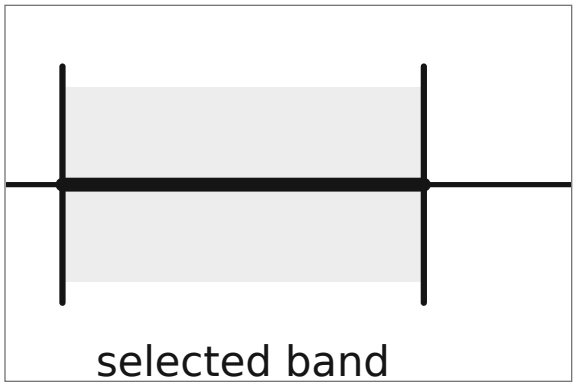


01
R2 root band

02
map bucket boundaries

03
conditional scalar range

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

A root band [a,b] corresponds exactly to scalars from a^9 through (b+1)^9-1, clipped to puzzle bounds.

READ THE THREE STAGES

01 R2 root band
02 map bucket boundaries
03 conditional scalar range

WHAT WAS ACTUALLY DONE

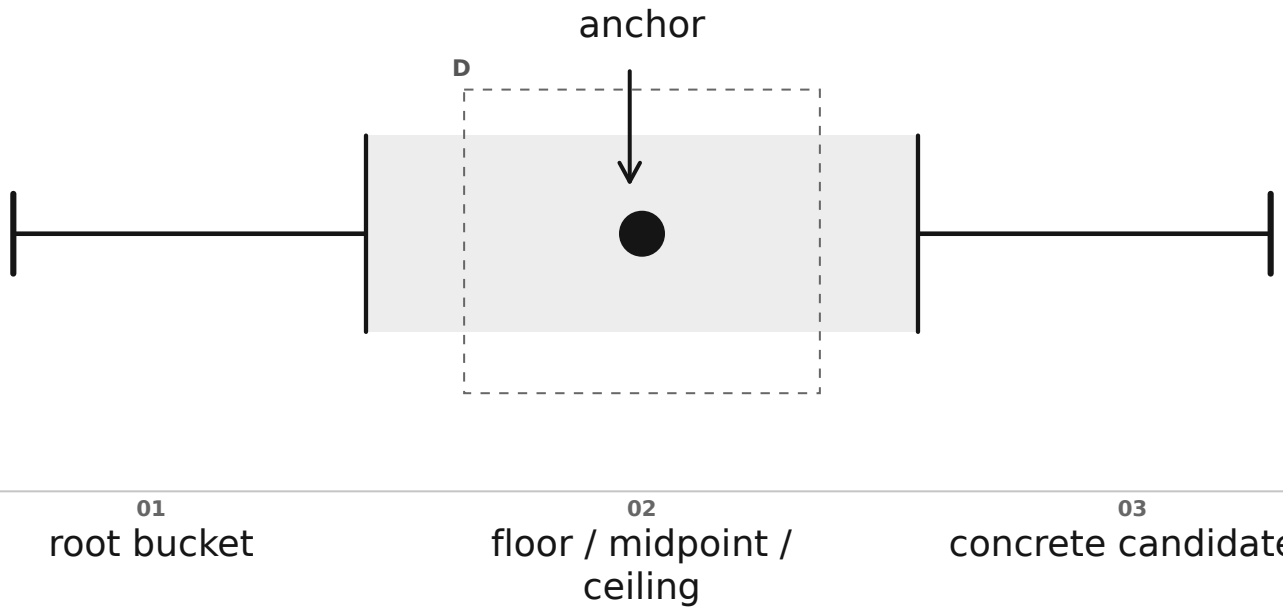
Interval-analysis scripts export these regions and report held-out coverage. The conversion is exact, but whether the unknown target belongs to the forecast band remains a probabilistic hypothesis.

ORIGINAL FINDING / EXECUTED RECORD

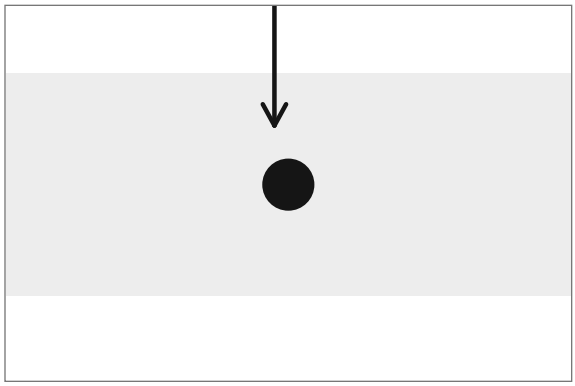
Root forecast bands map to exact scalar intervals; reports distinguish conditional narrowing from guaranteed coverage and statistical advantage.

M072

Root floor/midpoint/ceiling candidate anchors



D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Turn plausible root locations into concrete integers using floors, midpoints, ceilings or differences from disclosed roots.

READ THE THREE STAGES

01 root bucket
02 floor / midpoint / ceiling
03 concrete candidates

WHAT WAS ACTUALLY DONE

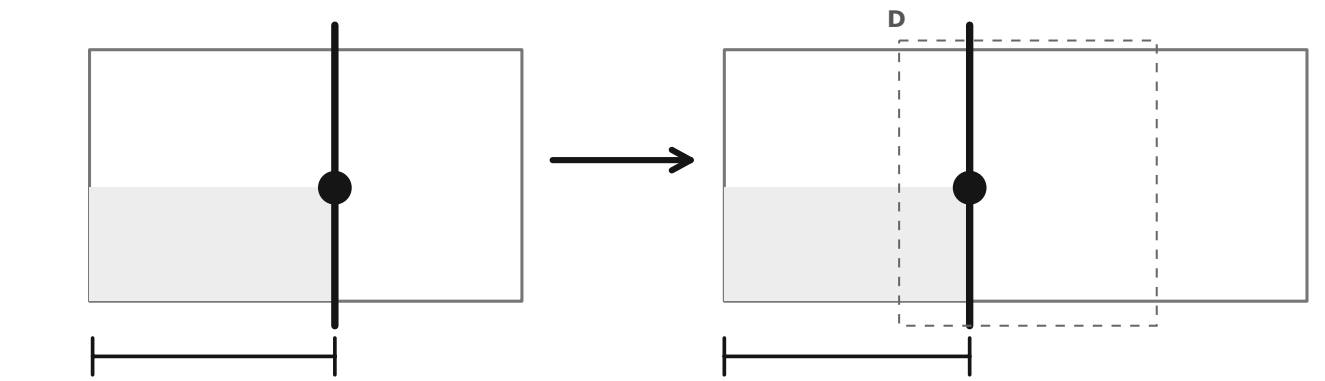
The root-test engine builds a fixed catalog of anchor candidates without reading the target key. Each resulting scalar is checked against that target's actual HASH160.

ORIGINAL FINDING / EXECUTED RECORD

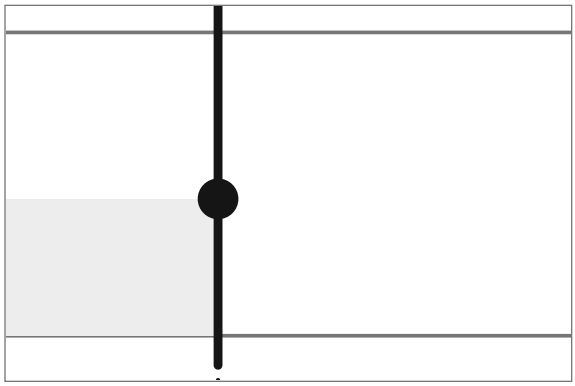
Fixed root-band and root-difference candidate families checked against known and unknown targets.

M073

Nearest-donor raw remainder carry



D / OPERATION DETAIL



Vector detail / 1.21× main-view scale

HOW THE OPERATION WORKS

Express a donor key as donor_root^9 plus an offset, then place that same offset above a legal root for the target.

READ THE THREE STAGES

01 donor bucket
02 carry raw offset
03 target bucket

WHAT WAS ACTUALLY DONE

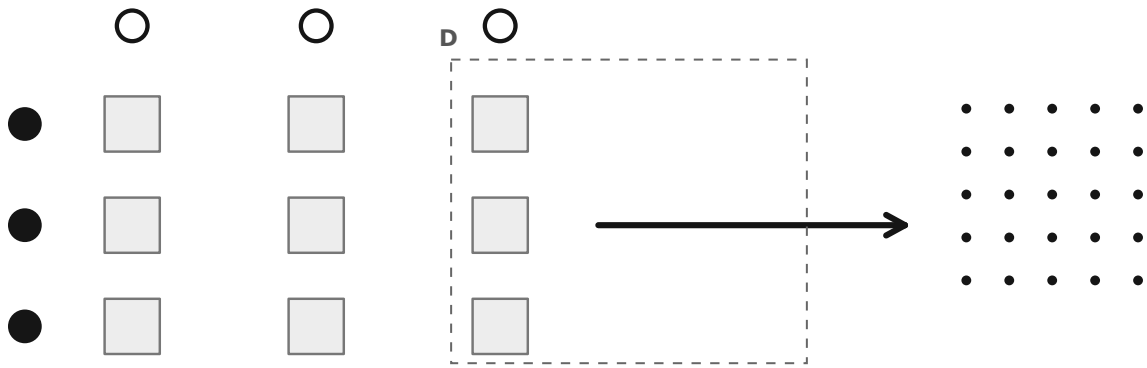
The nearest-donor candidate families transfer offsets from neighboring disclosed puzzle indices. They test an offset-inheritance hypothesis, not a consequence of ninth-root arithmetic.

ORIGINAL FINDING / EXECUTED RECORD

Transfer a disclosed key remainder to target root hypotheses; saved unknown-target candidate set contains no matches.

M074

All-donor carry over every legal ninth root

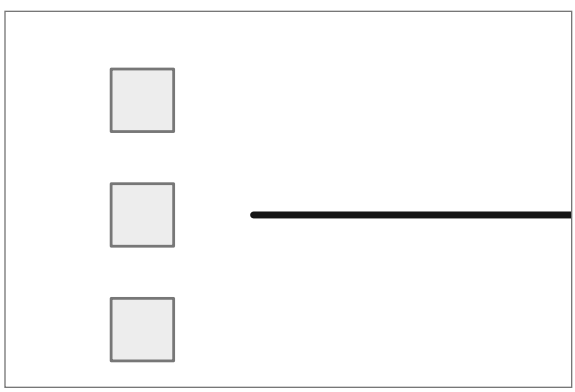


01 all donor offsets

02 all legal roots

03 candidate cross-product

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Broaden the carry test to every disclosed donor and every legal target root, eliminating the nearest-donor selection restriction.

READ THE THREE STAGES

01 all donor offsets
02 all legal roots
03 candidate cross-product

WHAT WAS ACTUALLY DONE

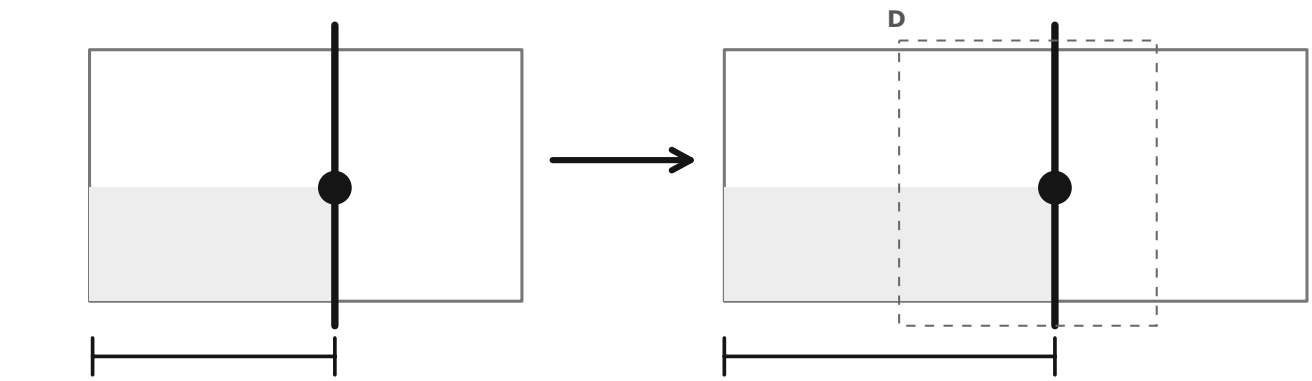
exhaustive_carry.py enumerates the entire stated donor/root family and verifies candidates. Zero hits rejects those concrete carried candidates, not all possible remainder-generation laws.

ORIGINAL FINDING / EXECUTED RECORD

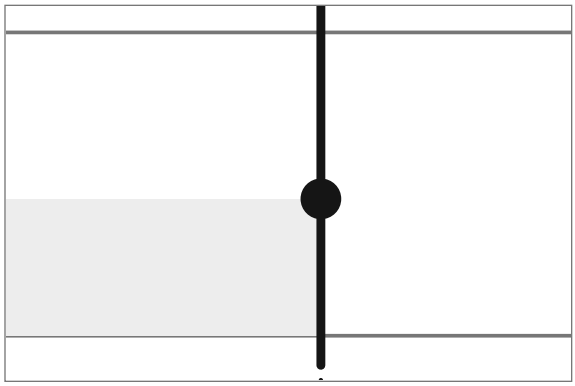
Saved exhaustive-within-family report records 35,982,885 candidate checks and zero target matches.

M075

Transferred within-bucket share



D / OPERATION DETAIL



Vector detail / 1.21× main-view scale

HOW THE OPERATION WORKS

Transfer a fraction of a donor bucket rather than its raw offset so buckets of different sizes can be compared.

READ THE THREE STAGES

01 donor fraction
02 rescale bucket share
03 target offset

WHAT WAS ACTUALLY DONE

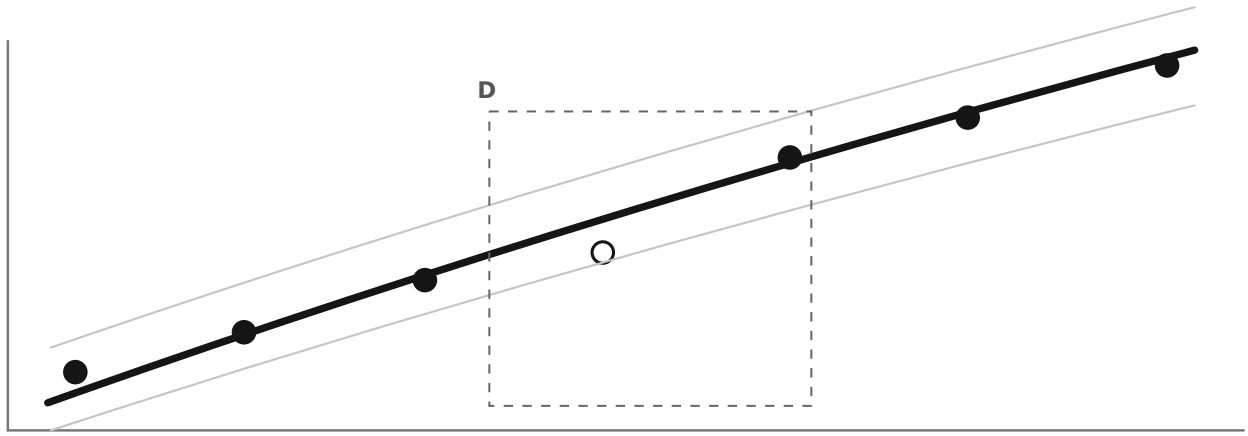
The code rescales each selected share by the target bucket width and rounds to an integer candidate. Any rounding and donor choice are part of the tested hypothesis.

ORIGINAL FINDING / EXECUTED RECORD

Reuse disclosed normalized root-bucket shares at legal target roots; tests do not cover arbitrary offset functions.

M076

Smooth polynomial share in puzzle index

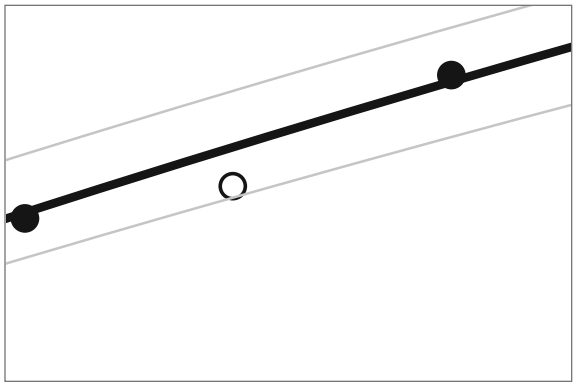


01 bucket shares

02 polynomial in index

03 predicted share

D / OPERATION DETAIL



Vector detail / 1.17× main-view scale

HOW THE OPERATION WORKS

Assume normalized bucket share is a low-degree polynomial in puzzle index and use fitted values to generate target offsets.

READ THE THREE STAGES

01 bucket shares
02 polynomial in index
03 predicted share

WHAT WAS ACTUALLY DONE

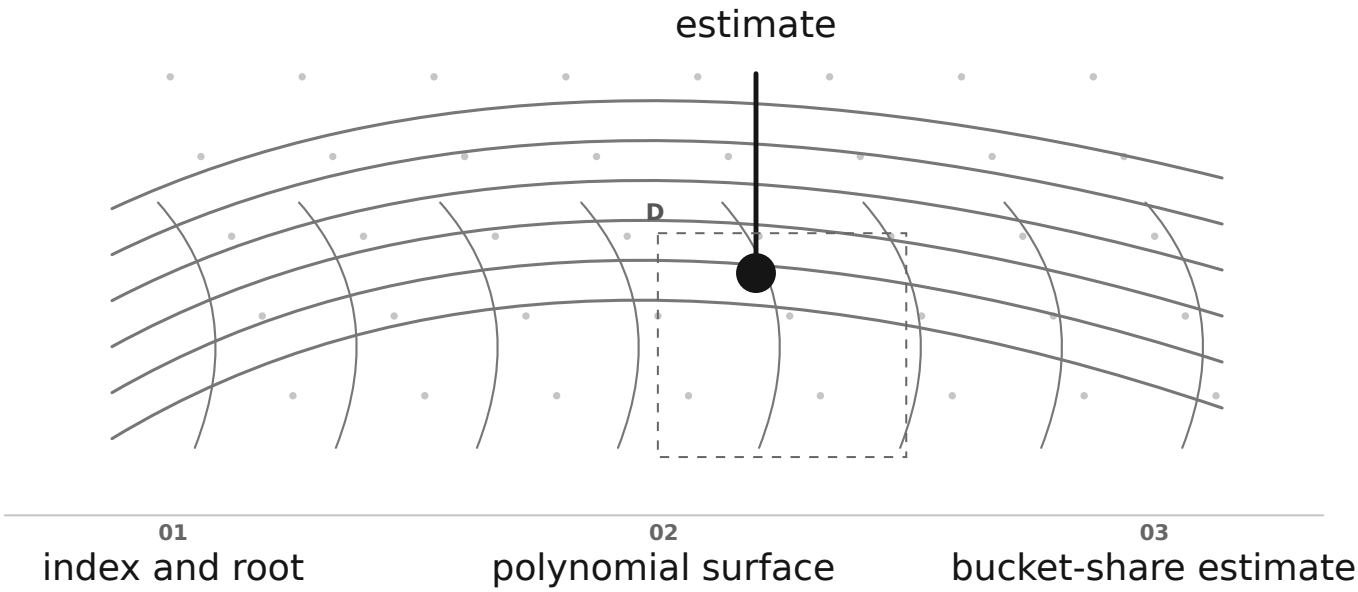
smooth_share.py fits degrees 1-4, evaluates errors against a uniform-share baseline and checks candidates over legal target roots. Numerical fit quality is reported separately from address hits.

ORIGINAL FINDING / EXECUTED RECORD

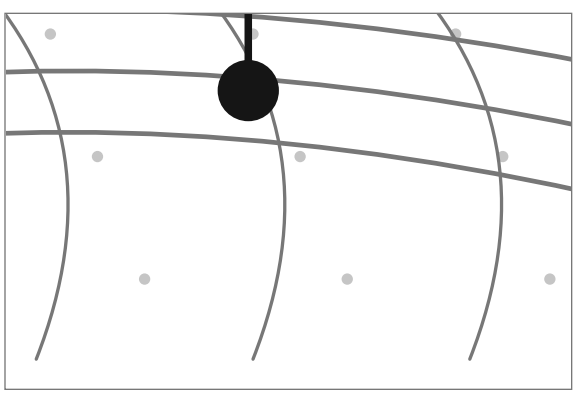
Degrees 1-4 predict bucket shares; saved per-degree unknown-target tests report no matches.

M077

Two-dimensional index/root share surface



D / OPERATION DETAIL



Vector detail / 1.54× main-view scale

HOW THE OPERATION WORKS

Include both puzzle index and legal root as inputs to a polynomial surface predicting the bucket share.

READ THE THREE STAGES

01 index and root
02 polynomial surface
03 bucket-share estimate

WHAT WAS ACTUALLY DONE

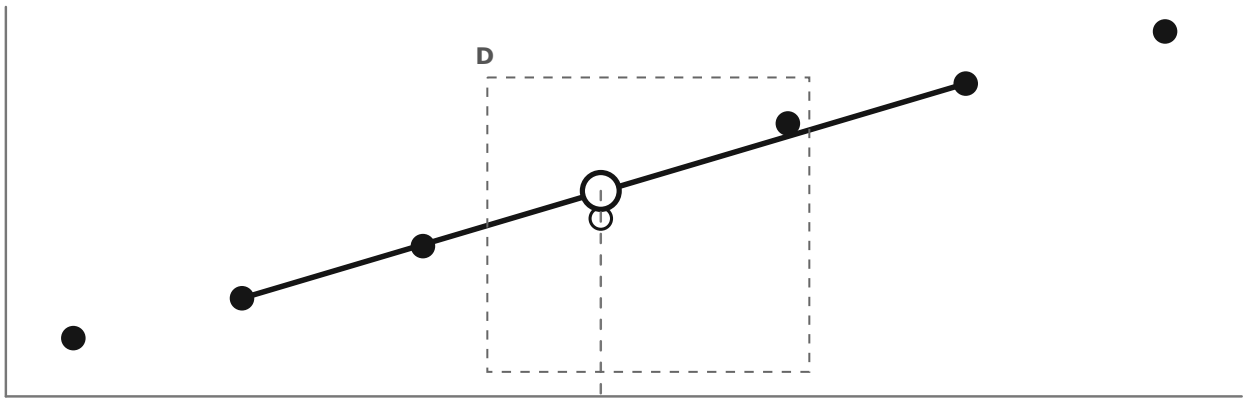
surface2d.py fits the declared low-degree surfaces on disclosed data and verifies reconstructed candidates. Additional model dimensions do not by themselves resolve an unknown remainder.

ORIGINAL FINDING / EXECUTED RECORD

Fit share as a low-degree surface of index and root, then verify resulting candidates; no saved unknown-target hits.

M078

Gap-fill normalized-position interpolation

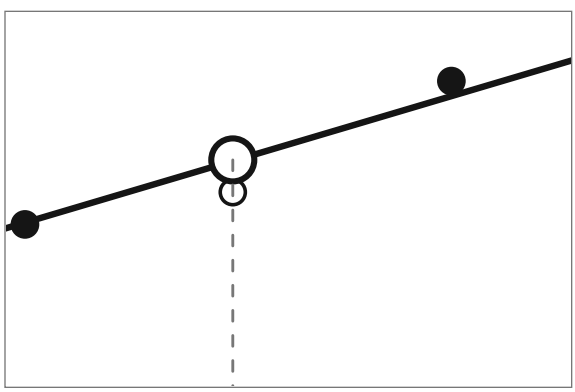


01 known neighbor

02 interpolate position

03 known neighbor

D / OPERATION DETAIL



Vector detail / 1.17× main-view scale

HOW THE OPERATION WORKS

Assume normalized interval position changes smoothly between the nearest disclosed neighbors. Interpolate that position into the target interval.

READ THE THREE STAGES

01 known neighbor
02 interpolate position
03 known neighbor

WHAT WAS ACTUALLY DONE

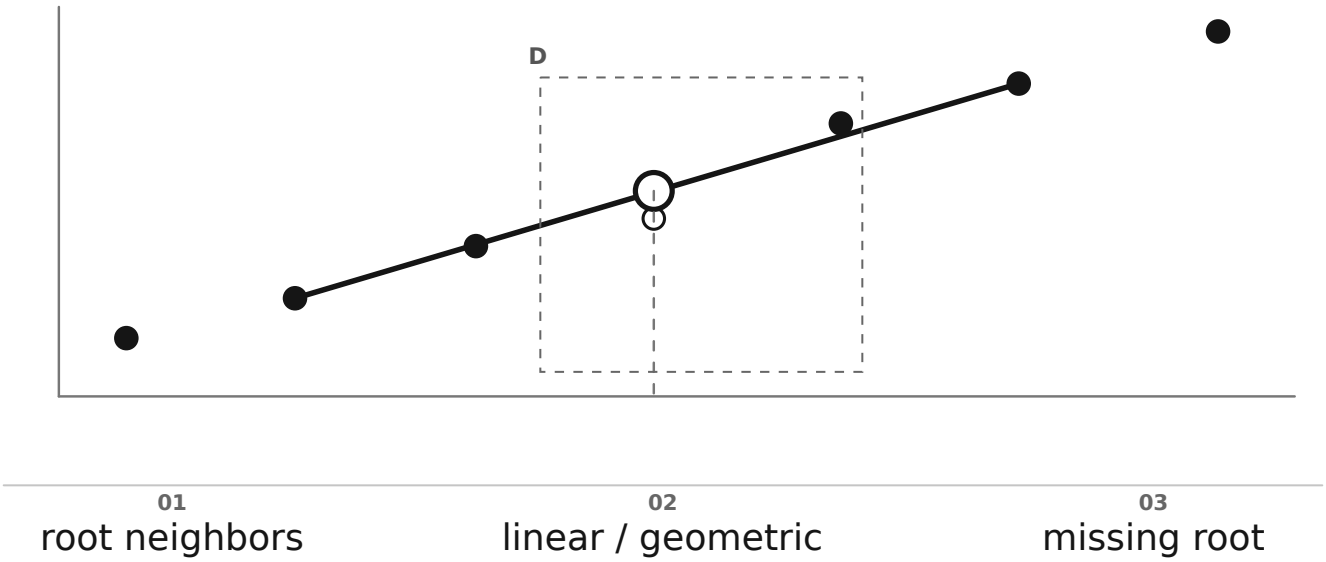
gap_fill.py creates one or more exact integer candidates from those neighboring positions. The hypothesis is smoothness; the verifier, not visual continuity, determines success.

ORIGINAL FINDING / EXECUTED RECORD

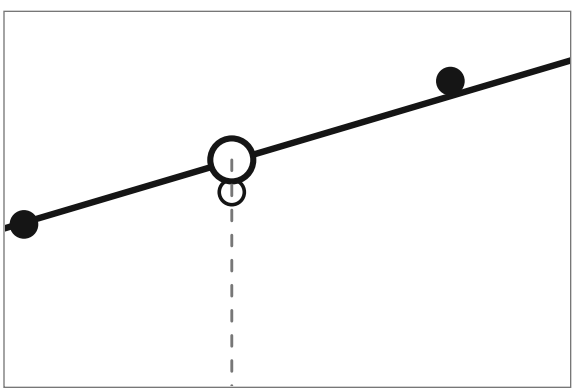
Interpolate/extrapolate between disclosed neighboring puzzles and verify candidates; no recorded matches.

M079

Gap-fill linear/geometric root interpolation



D / OPERATION DETAIL



Vector detail / 1.17× main-view scale

HOW THE OPERATION WORKS

Interpolate roots linearly or geometrically, then map the forecast root back to an integer scalar.

READ THE THREE STAGES

01 root neighbors
02 linear / geometric
03 missing root

WHAT WAS ACTUALLY DONE

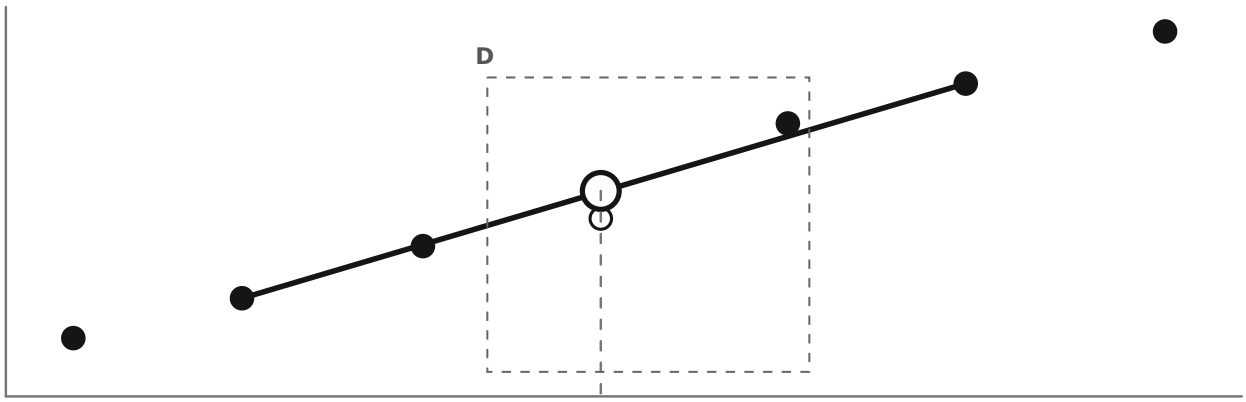
The gap-fill test generates candidates from both root interpolation rules and clips to the target range. A nearby root can still correspond to a very large scalar error.

ORIGINAL FINDING / EXECUTED RECORD

Interpolate root coordinates and reconstruct candidate powers; tested candidates did not recover unknown keys.

M080

Gap-fill log-key and raw-offset interpolation

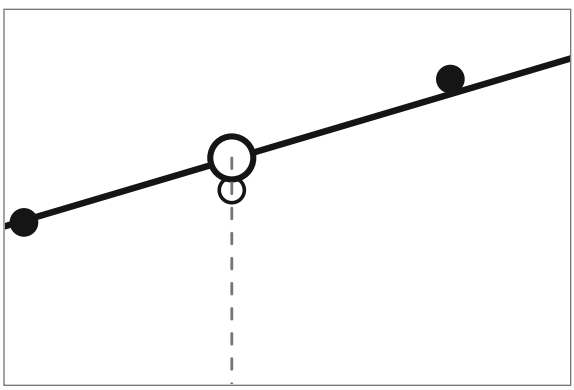


01 neighbor key values

02 log-key / raw offset

03 gap estimate

D / OPERATION DETAIL



Vector detail / 1.17× main-view scale

HOW THE OPERATION WORKS

Interpolate log2(key) or the raw ninth-root residual as an alternative notion of smooth progression.

READ THE THREE STAGES

01 neighbor key values
02 log-key / raw offset
03 gap estimate

WHAT WAS ACTUALLY DONE

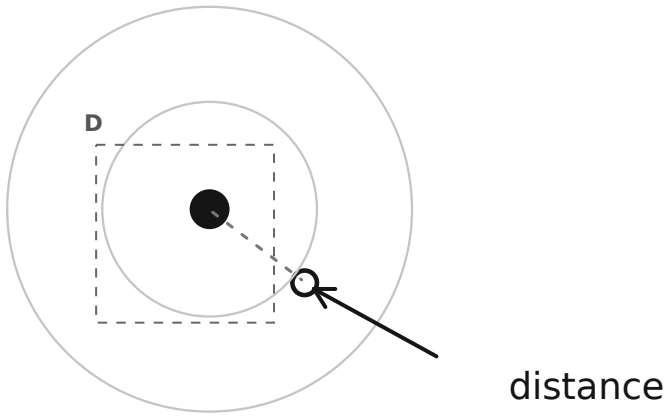
The same bounded test reconstructs scalars under these assumptions and compares addresses. Log closeness or matching early values is not accepted as a recovered key.

ORIGINAL FINDING / EXECUTED RECORD

Alternative smoothness assumptions tested against exact address hashes, not judged by numerical proximity alone.

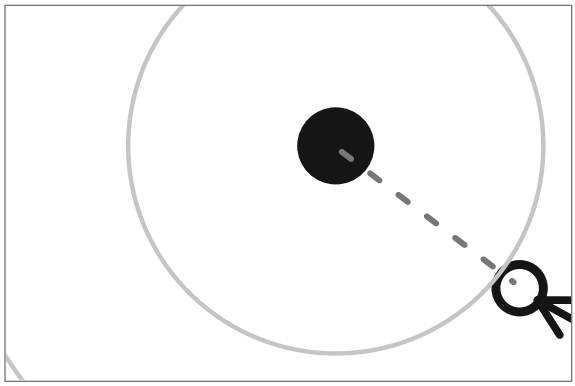
M081

Near-miss anomaly testing



01	02	03
observed near miss	distance statistic	chance comparison

D / OPERATION DETAIL



Vector detail / 1.93× main-view scale

HOW THE OPERATION WORKS

Judge proximity relative to the density and placement of the entire candidate set. Dense sets naturally create close numerical neighbors.

READ THE THREE STAGES

01 observed near miss
02 distance statistic
03 chance comparison

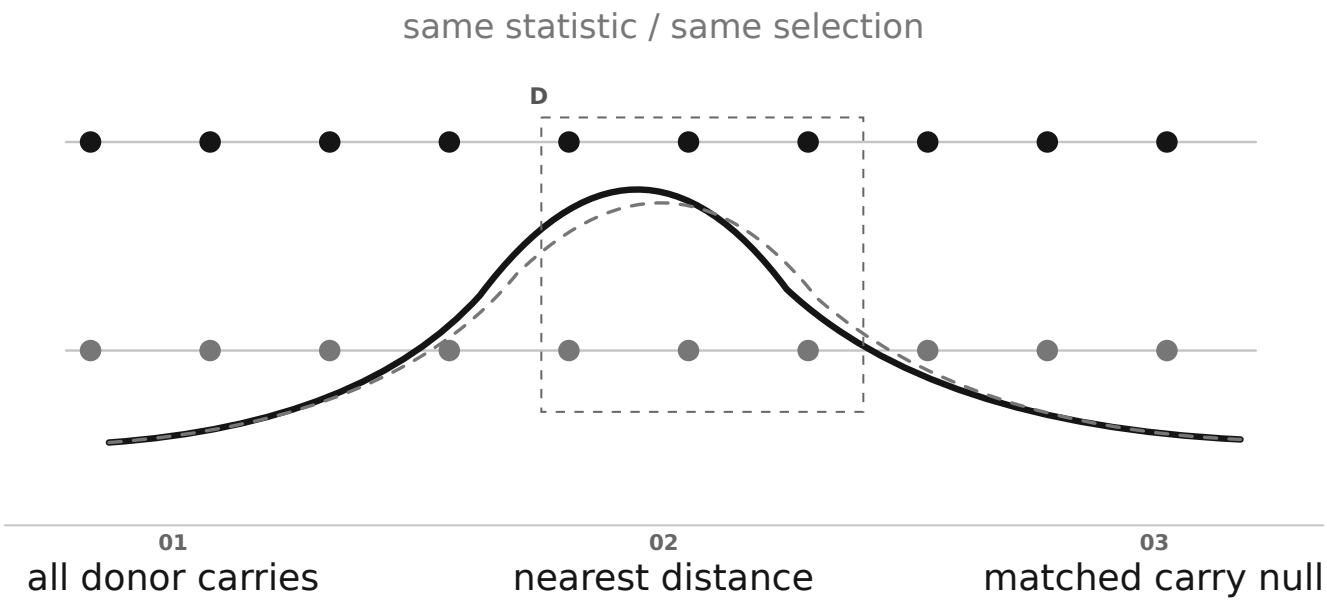
WHAT WAS ACTUALLY DONE

nearmiss_analysis.py holds the candidate geometry fixed, draws uniform targets in the same interval and compares minimum distances. This is a diagnostic of chance closeness, not a new candidate verifier.

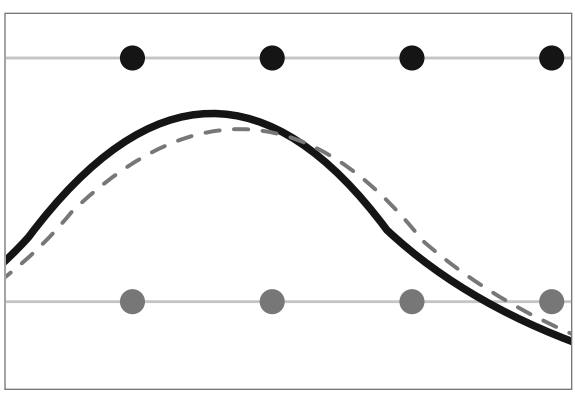
ORIGINAL FINDING / EXECUTED RECORD

Compare nearest-candidate distances with uniform targets against the same candidate geometry; closeness alone is not a verified key.

M082
All-donor carry proximity null



D / OPERATION DETAIL



Vector detail / 1.17× main-view scale

HOW THE OPERATION WORKS

Ask whether any donor-offset/root combination lands unusually near a held-out key compared with identical carry geometry under uniform targets.

READ THE THREE STAGES

01 all donor carries
02 nearest distance
03 matched carry null

WHAT WAS ACTUALLY DONE

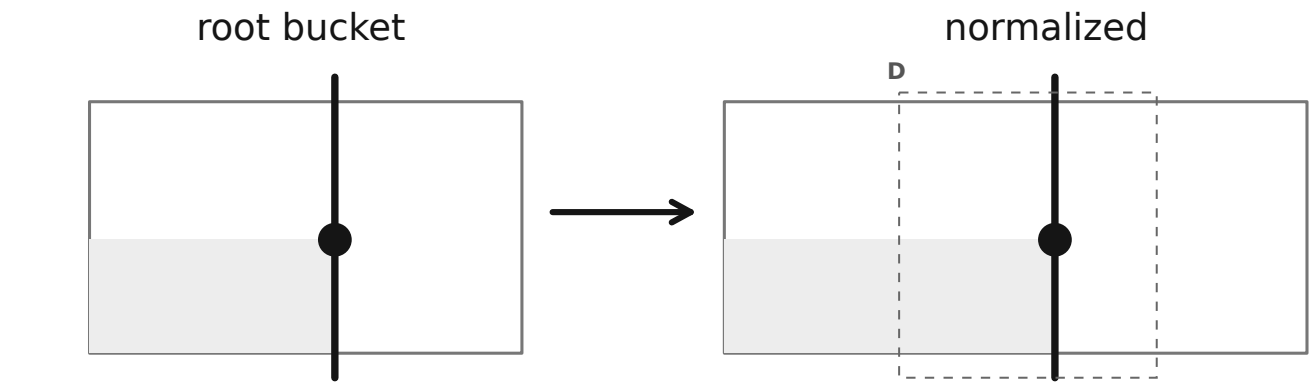
all_donor_carry.py excludes that target from donor construction and computes observed versus null proximity. A favorable minimum is not an address match and may require selection adjustment.

ORIGINAL FINDING / EXECUTED RECORD

Compare minimum carry distance to uniform controls while excluding the known target from its donor set.

M083

Blind nth-root ladder address tests

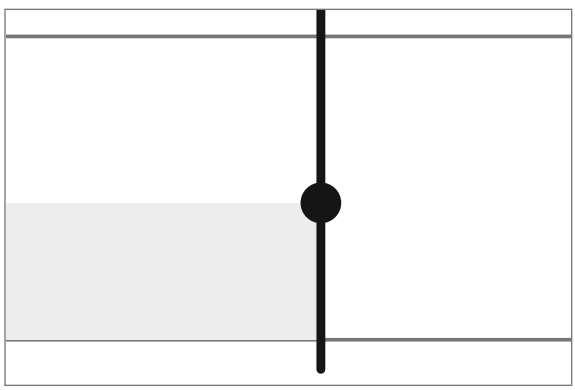


01 nth-root ladder

02 finite anchors

03 address checks

D / OPERATION DETAIL



Vector detail / 1.21× main-view scale

HOW THE OPERATION WORKS

Build nth-root scheduled anchors and difference-based forecasts without supplying the hidden scalar to the candidate generator.

READ THE THREE STAGES

01 nth-root ladder
02 finite anchors
03 address checks

WHAT WAS ACTUALLY DONE

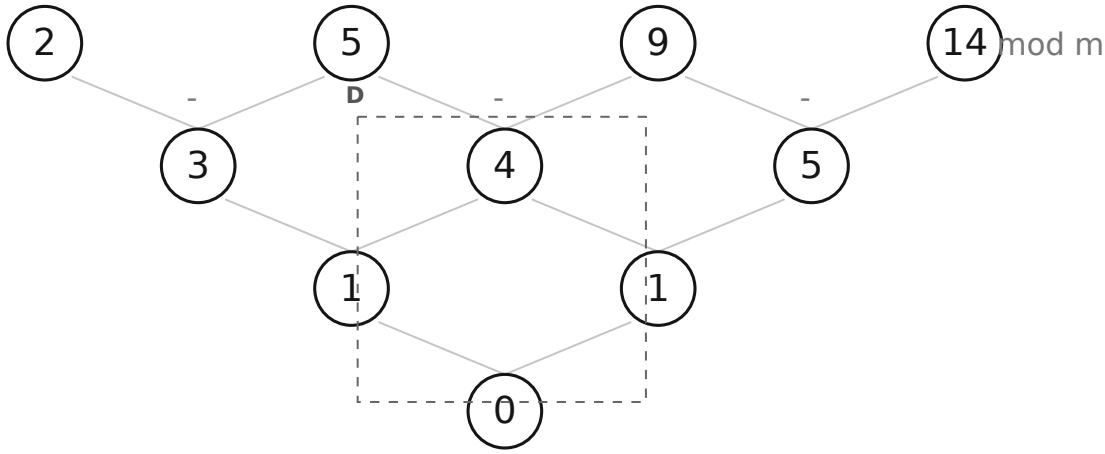
The saved PEC37 test covers index-only, leave-one-out, random-holdout and unknown-target candidates. Reported matches are confined to tiny trivial intervals rather than scalable unknown-key recovery.

ORIGINAL FINDING / EXECUTED RECORD

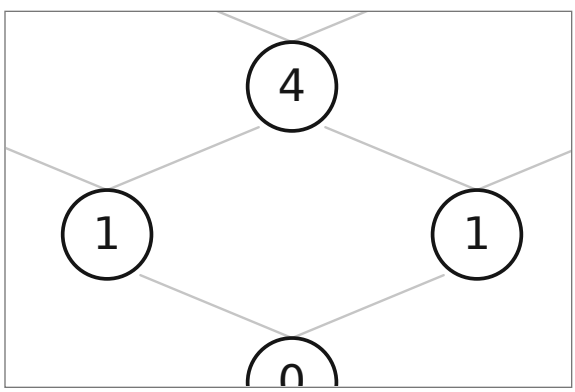
Index-only, leave-one-out and random holdout candidates mostly match only trivial tiny puzzles; no unknown-target matches recorded.

M084

Exact polynomial/difference continuation



D / OPERATION DETAIL



Vector detail / 1.21× main-view scale

HOW THE OPERATION WORKS

Fit exact rational polynomials to known scalars, roots or normalized positions, then evaluate them at held-out indices.

READ THE THREE STAGES

01 known sequence
02 exact differences
03 candidate continuation

WHAT WAS ACTUALLY DONE

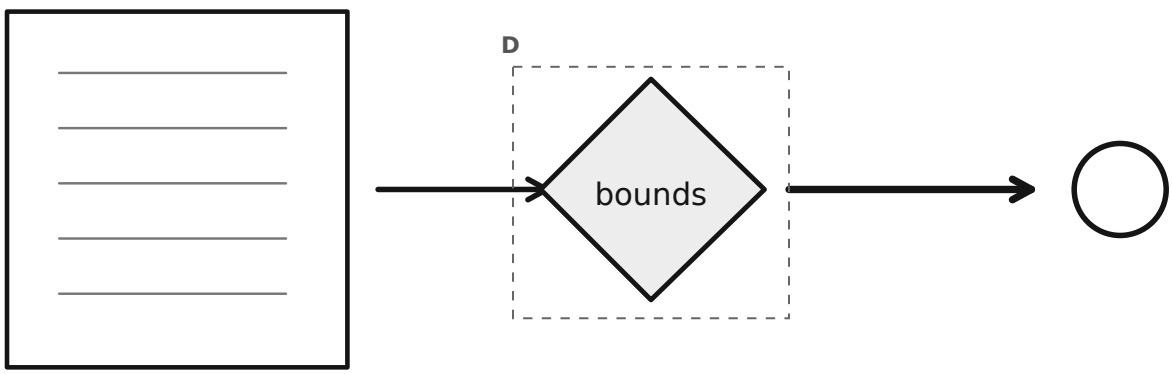
The atlas audit tests degrees 0-6 and multiple masking/encoding variants. It uses published answers afterward for scoring and checks target-derived addresses for concrete candidate attempts.

ORIGINAL FINDING / EXECUTED RECORD

Degree 0-6 models of scalars, range positions and roots failed exact higher-index recovery in the saved tests.

M085

Frozen candidate families on 77 unknown targets

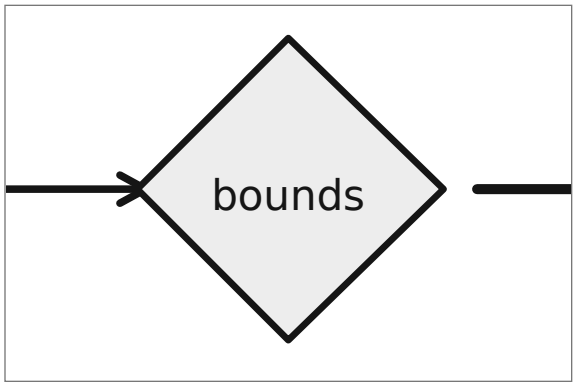


01
frozen families

02
77 unknown targets

03
bounded checks

D / OPERATION DETAIL



Vector detail / 1.37× main-view scale

HOW THE OPERATION WORKS

Freeze a finite candidate recipe, apply it to every locally undisclosed puzzle, and compare each output with that puzzle's target hash.

READ THE THREE STAGES

01 frozen families
02 77 unknown targets
03 bounded checks

WHAT WAS ACTUALLY DONE

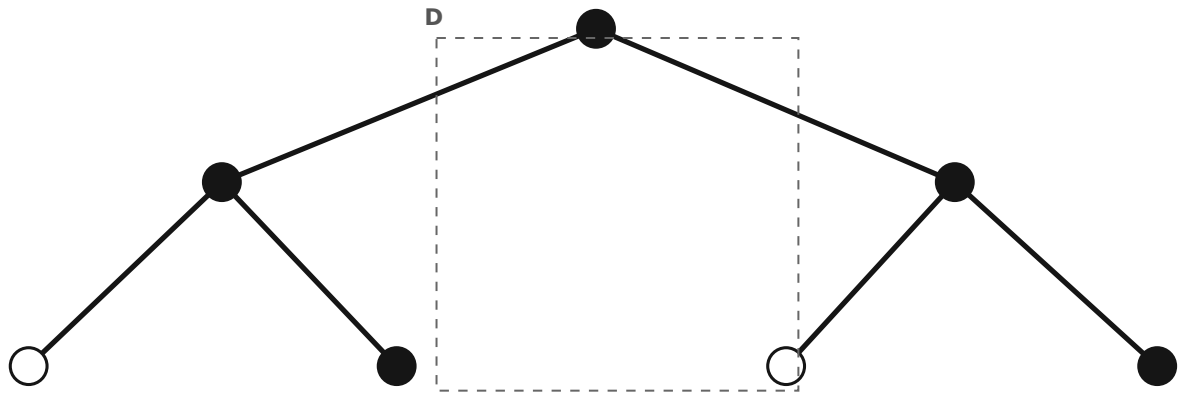
all_unknown_check.py evaluates 77 database entries and saves candidate/target counts. A completed finite recipe search is not an exhaustive search of all their intervals.

ORIGINAL FINDING / EXECUTED RECORD

Local all-unknown report records 2,744 distinct candidate/target checks and no matches.

M086

Finite continuation attempts specifically for #140

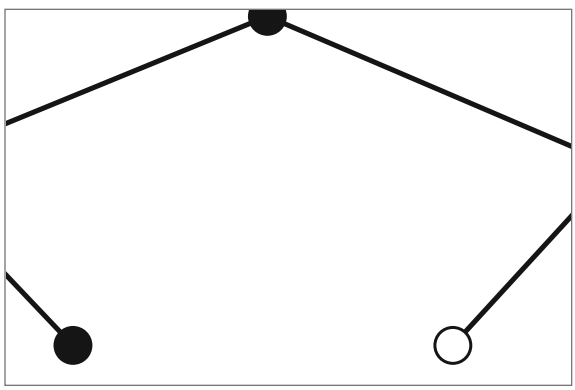


01
known neighbors

02
finite continuations

03
puzzle 140 candidates

D / OPERATION DETAIL



Vector detail / 0.98× main-view scale

HOW THE OPERATION WORKS

Collect new finite leads specifically for a public-point target, derive candidate scalars or intervals and perform exact comparisons.

READ THE THREE STAGES

01 known neighbors
02 finite continuations
03 puzzle 140 candidates

WHAT WAS ACTUALLY DONE

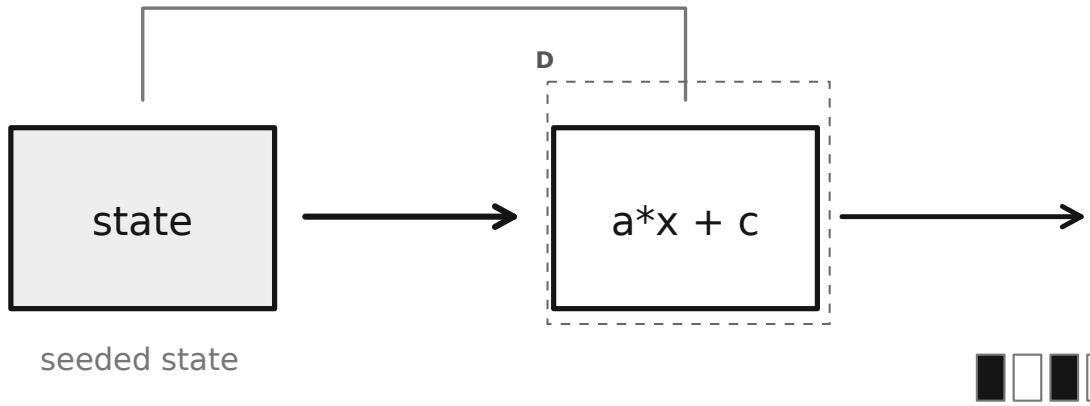
continue_140.py indexes archive clues, binary-derived numbers and signature relations. It records bounded attempts; finding a plausible story or interval does not count as discovering #140.

ORIGINAL FINDING / EXECUTED RECORD

Archive leads and binary candidates were verified as bounded hypotheses, not exhaustive interval searches.

M087

Fixed affine/LCG key recurrence

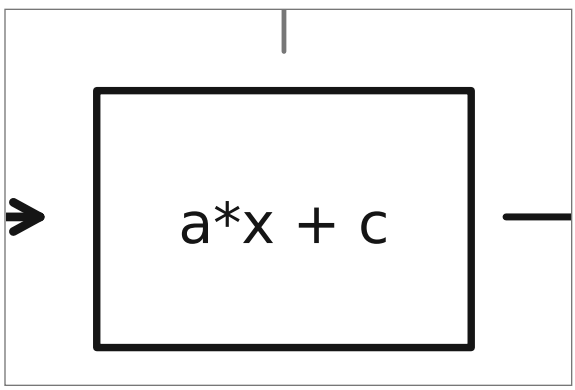


01 state x

02 multiply + increment

03 modular next state

D / OPERATION DETAIL



Vector detail / 1.42× main-view scale

HOW THE OPERATION WORKS

A fixed affine recurrence maps the same low-bit input to the same output. Contradictory observed transitions can therefore reject that recurrence family.

READ THE THREE STAGES

01 state x
02 multiply + increment
03 modular next state

WHAT WAS ACTUALLY DONE

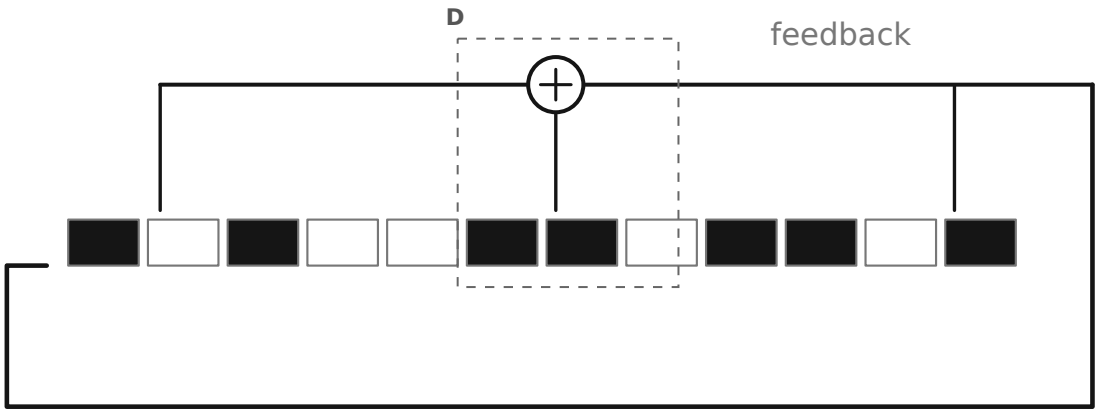
The audit exhausts small modular coefficient pairs and gives an exact parity obstruction. It does not exclude hidden state, changing coefficients, discarded outputs or unrelated deterministic wallets.

ORIGINAL FINDING / EXECUTED RECORD

Exact low-bit constraints reject the named fixed-parameter recurrence family; hidden state and different generators remain outside scope.

M088

LFSR / Berlekamp-Massey reconstruction

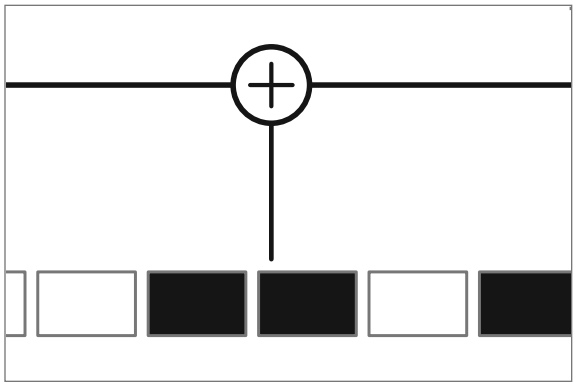


01 observed bits

02 feedback taps

03 reconstructed register

D / OPERATION DETAIL



Vector detail / 1.38× main-view scale

HOW THE OPERATION WORKS

An LFSR predicts later bits from a fixed linear combination of earlier bits. Berlekamp-Massey finds a shortest recurrence fitting a supplied bitstream.

READ THE THREE STAGES

01 observed bits
02 feedback taps
03 reconstructed register

WHAT WAS ACTUALLY DONE

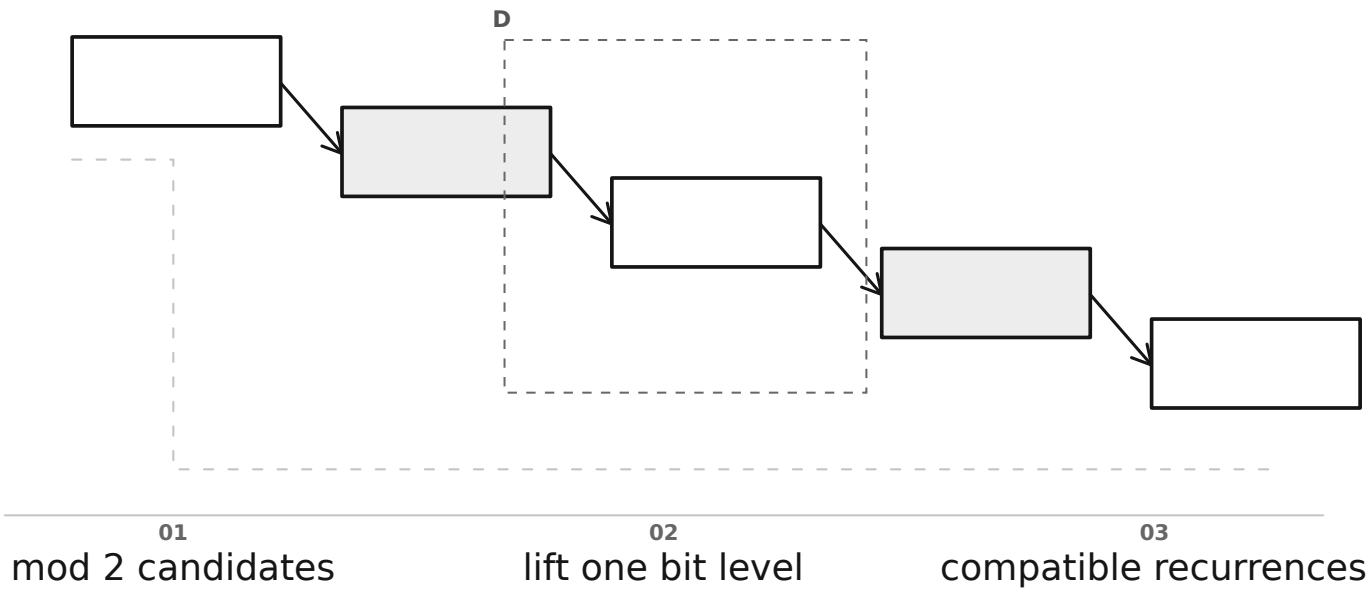
Archived generator work applies linear recurrence tests to concatenated known keys. Fitting the observed stream is insufficient unless the same state and output schedule predict unseen keys.

ORIGINAL FINDING / RECORDED RESEARCH

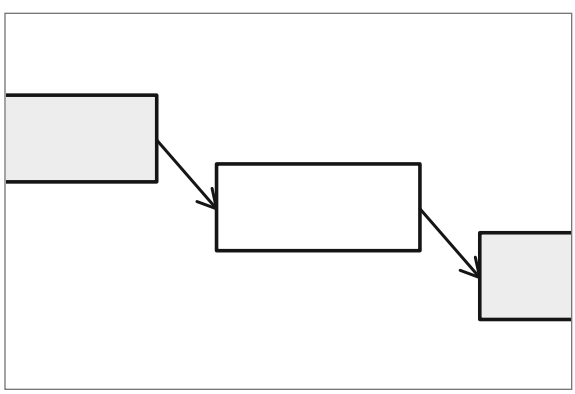
Historical generator review includes bitstream linear-recurrence tests; no transferable unknown-key generator established.

M089

Two-adic/Hensel polynomial recurrence



D / OPERATION DETAIL



Vector detail / 0.98× main-view scale

HOW THE OPERATION WORKS

A polynomial recurrence modulo powers of two can constrain parameters one bit at a time; lifting extends compatible low-bit solutions.

READ THE THREE STAGES

01 mod 2 candidates
02 lift one bit level
03 compatible recurrences

WHAT WAS ACTUALLY DONE

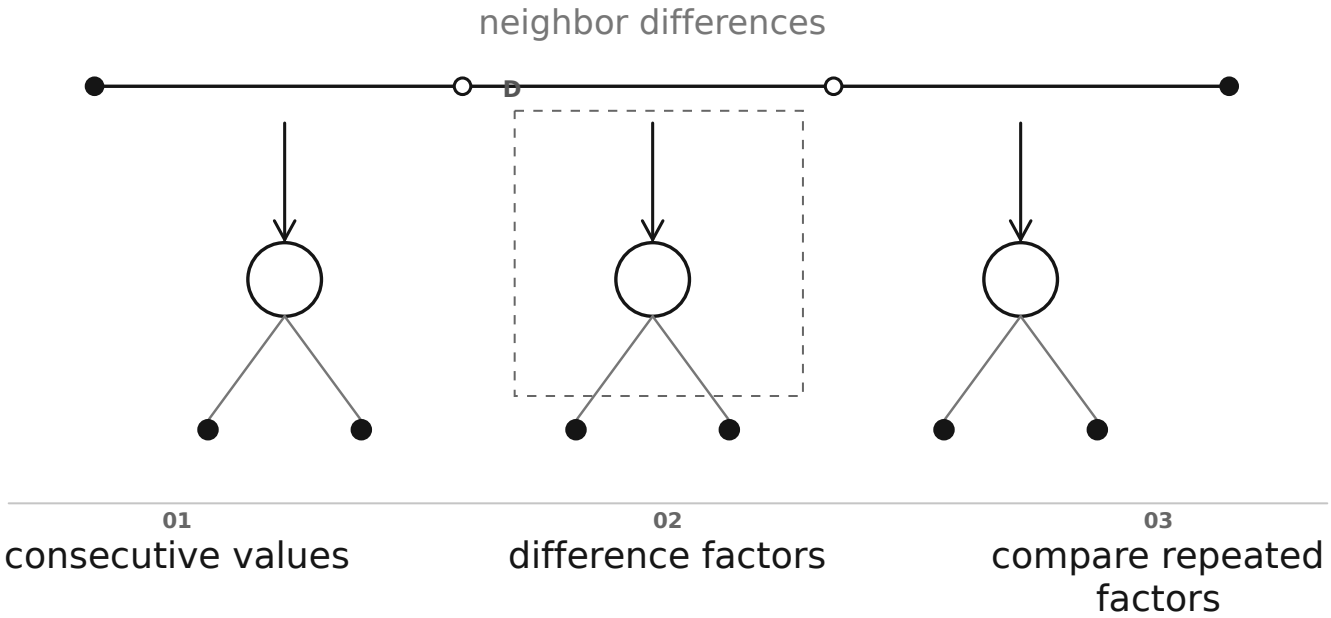
The generator catalog tests declared polynomial/2-adic forms against known puzzle transitions. It searches a restricted family, not all possible deterministic key generators.

ORIGINAL FINDING / RECORDED RESEARCH

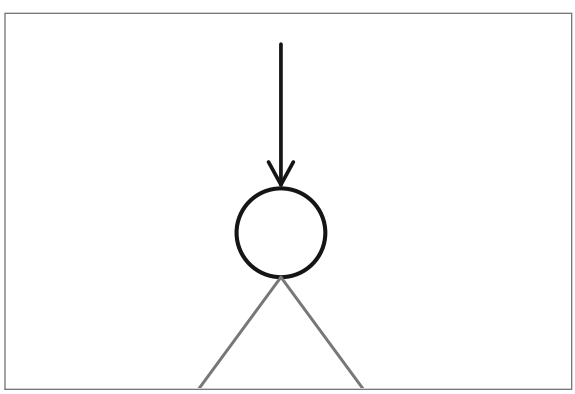
Tests of squared-state and modular recurrence hypotheses are archived; finite failures are not impossibility theorems.

M090

Consecutive-difference factor patterns



D / OPERATION DETAIL



Vector detail / 1.21× main-view scale

HOW THE OPERATION WORKS

If consecutive keys share a multiplicative or arithmetic construction, their differences may expose common factors or repeated relations.

READ THE THREE STAGES

01 consecutive values
02 difference factors
03 compare repeated factors

WHAT WAS ACTUALLY DONE

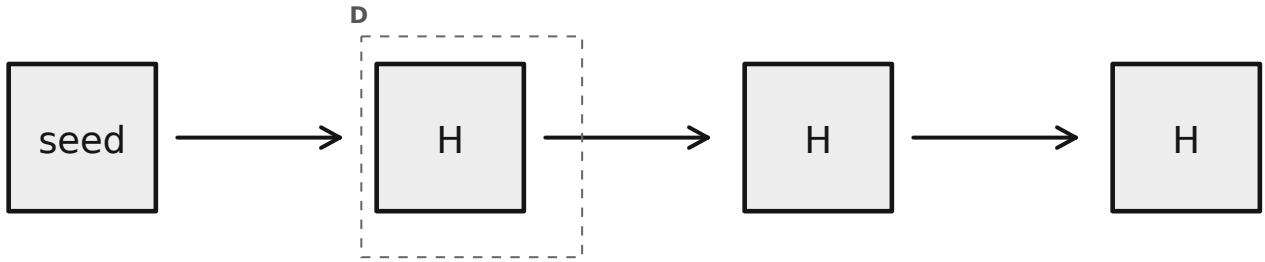
Historical reviews factor or compare disclosed differences and check proposed recurrences. The known interval sizes themselves create scale trends that must be distinguished from secret generator structure.

ORIGINAL FINDING / RECORDED RESEARCH

Search common factors or structured differences across published puzzle keys; no validated target predictor recorded.

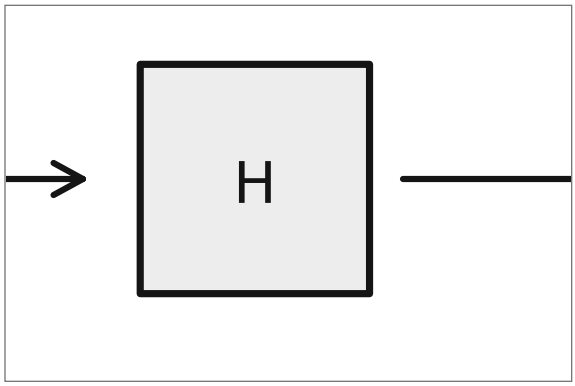
M091

Hash-chain continuation



01	02	03
starting value	repeated hash	next digest

D / OPERATION DETAIL



Vector detail / 1.56× main-view scale

HOW THE OPERATION WORKS

Hash a preceding key in a specified encoding, then mask the digest into the next puzzle's range.

READ THE THREE STAGES

01 starting value
02 repeated hash
03 next digest

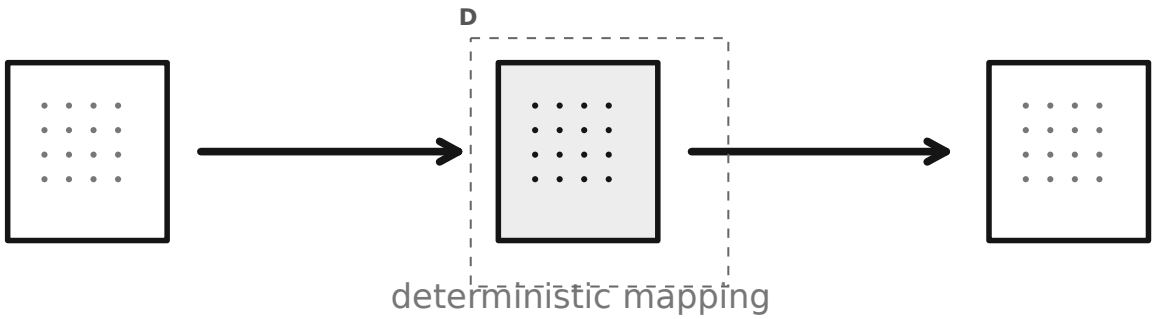
WHAT WAS ACTUALLY DONE

Candidate scripts try SHA-256/double-hash, byte orders and minimal/32-byte lengths. Every recipe is deterministic; zero target matches applies to those encodings and transitions only.

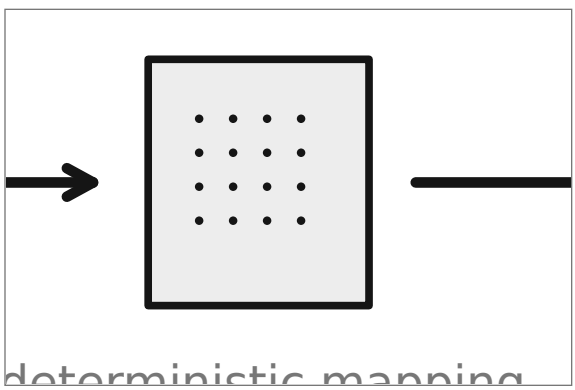
ORIGINAL FINDING / EXECUTED RECORD

SHA-256/double-hash variants of previous keys, with endian/length choices and masks, failed named holdout and target checks.

M092
Seed-plus-index hash/HMAC families



D / OPERATION DETAIL



Vector detail / 1.38× main-view scale

HOW THE OPERATION WORKS

Derive a candidate from a small seed and an index encoding using hashing or HMAC, then compare many disclosed tiers.

READ THE THREE STAGES

01 seed and index
02 hash / HMAC
03 candidate output

WHAT WAS ACTUALLY DONE

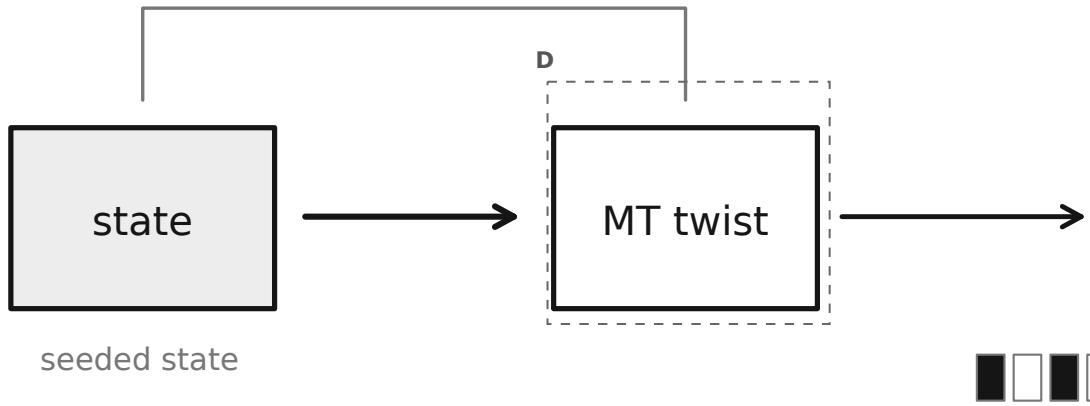
hunt_multi_significant.py sweeps named seed families and tracks nontrivial matches. Matching a one- or two-bit puzzle is expected by chance and is not a recovered creator seed.

ORIGINAL FINDING / IMPLEMENTED

Score candidate seeds over different index encodings; requiring significant later-tier matches avoids trivial small-range coincidences.

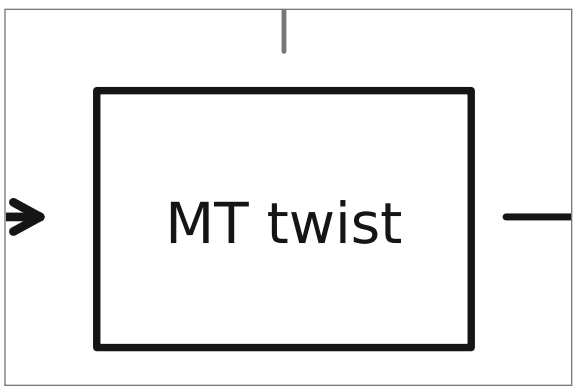
M093

Python MT19937 masked-key replay



01	02	03
Python MT seed	word stream	puzzle-width mask

D / OPERATION DETAIL



Vector detail / 1.42× main-view scale

HOW THE OPERATION WORKS

Replay Python's seeded PRNG under a specific call schedule and force outputs into each tier.

READ THE THREE STAGES

01 Python MT seed
02 word stream
03 puzzle-width mask

WHAT WAS ACTUALLY DONE

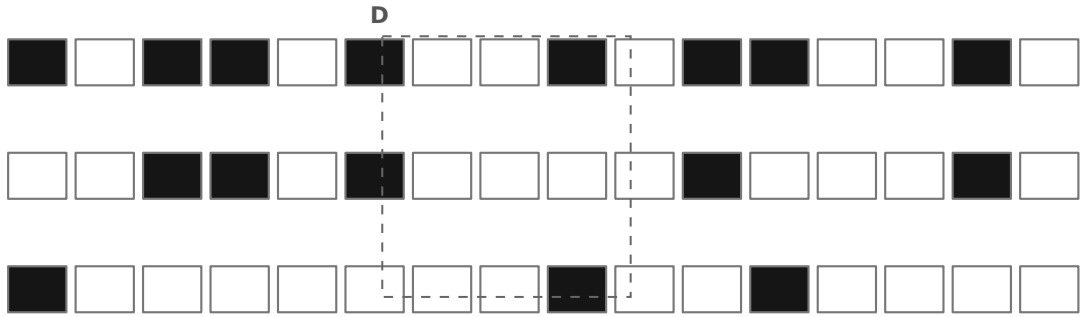
The hunter tests randrange and getrandbits variants across disclosed keys. Exact software version, seeding and draw order are part of the hypothesis, not generic consequences of MT19937.

ORIGINAL FINDING / IMPLEMENTED

Test seeded randrange/getrandbits generation schedules against published puzzle keys.

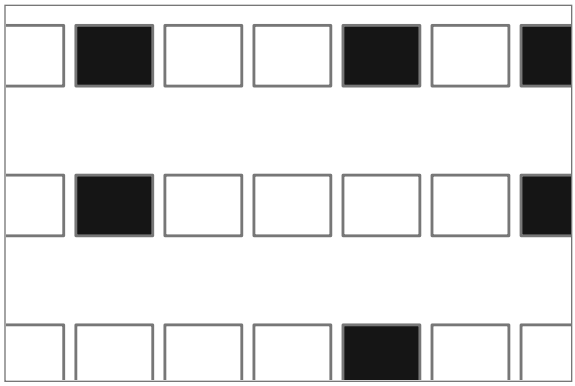
M094

Shared unmasked value, prefix and suffix tests



01	02	03
unmasked candidates	prefix / suffix overlap	shared-value test

D / OPERATION DETAIL



Vector detail / 1.32× main-view scale

HOW THE OPERATION WORKS

Re-masking one common integer would force predictable prefix/suffix agreement among tiers. Independent draws need not share those bits.

READ THE THREE STAGES

01 unmasked candidates
02 prefix / suffix overlap
03 shared-value test

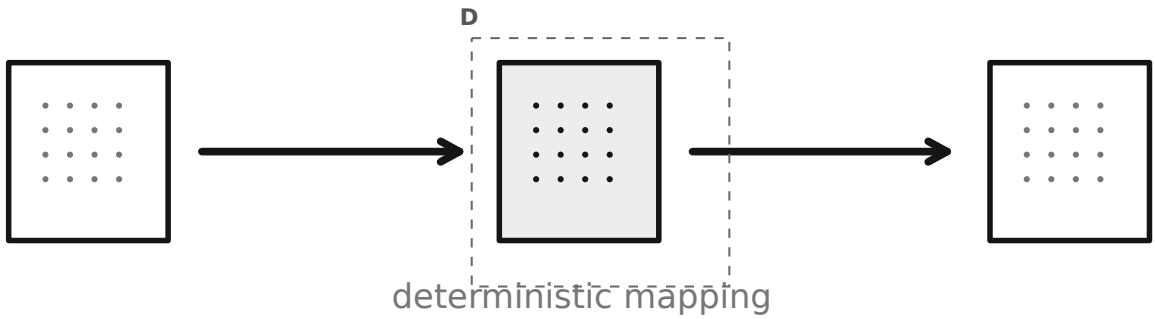
WHAT WAS ACTUALLY DONE

Known-key pair tests count overlap and compare aligned bits with random controls. They test the common-value construction; ordinary BIP32 child masking does not imply that same shared-value model.

ORIGINAL FINDING / EXECUTED RECORD

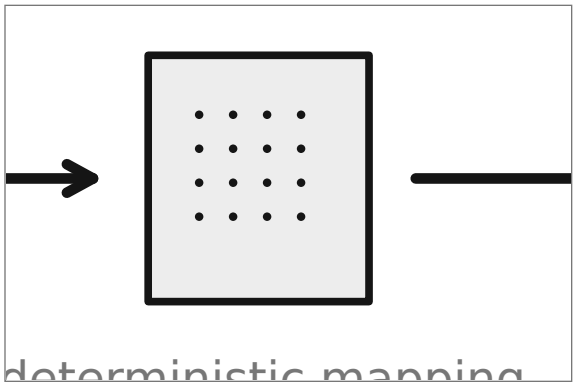
Known-key overlap and random controls reject the tested common-prefix/suffix hypotheses, not every wallet generator.

M095
CoinVault/mSIGNA BIP32 plus mask



01	02	03
wallet seed	BIP32 child	puzzle mask

D / OPERATION DETAIL



Vector detail / 1.38× main-view scale

HOW THE OPERATION WORKS

BIP32 derives a child scalar using HMAC-SHA512 and chain codes; the proposed creator mask keeps the required leading bit and lower child bits.

READ THE THREE STAGES

01 wallet seed
02 BIP32 child
03 puzzle mask

WHAT WAS ACTUALLY DONE

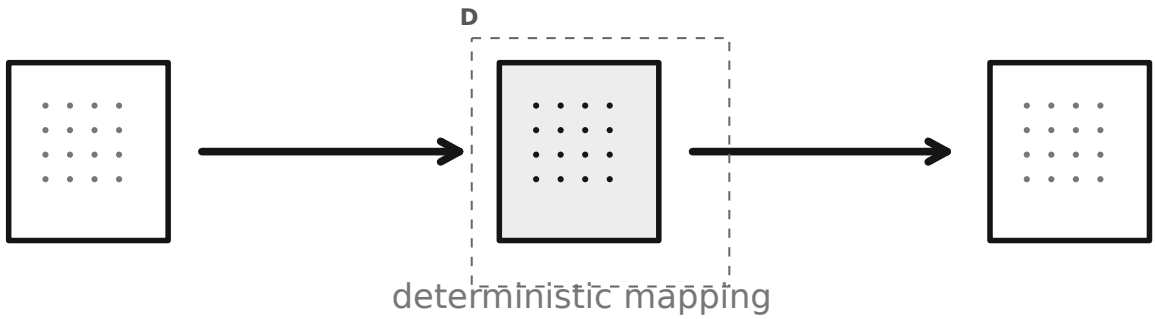
The catalog tests supplied seed materials and CoinVault/mSIGNA-style paths, with synthetic positive controls. No tested catalog entry reproduces the nontrivial public puzzle sequence.

ORIGINAL FINDING / EXECUTED RECORD

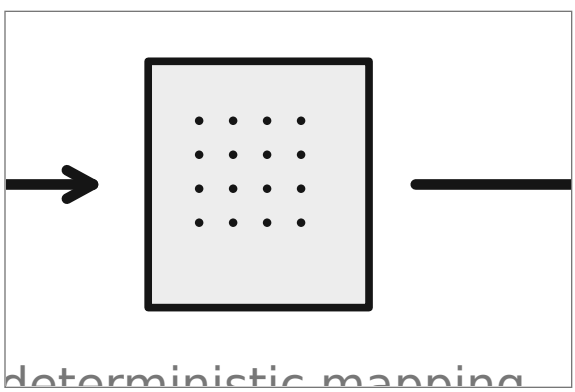
Bounded seed/path catalog did not reproduce nontrivial puzzle keys; synthetic known-wallet recovery validates the checker.

M096

Electrum-v1 and Armory masking models



D / OPERATION DETAIL



Vector detail / 1.38× main-view scale

HOW THE OPERATION WORKS

Historical deterministic wallets can derive children by algorithms different from BIP32. Apply each modeled derivation before the puzzle mask.

READ THE THREE STAGES

01 wallet state
02 Electrum / Armory
03 masked candidates

WHAT WAS ACTUALLY DONE

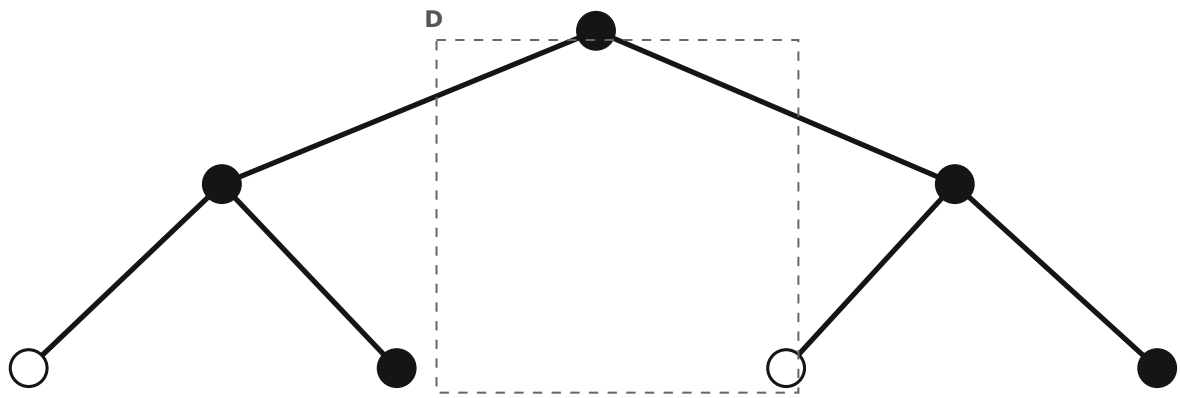
Dedicated Electrum-v1 and Armory scripts test those concrete wallet families. Their existence does not establish that the creator used either wallet or that its seed is enumerable.

ORIGINAL FINDING / IMPLEMENTED

Historical deterministic-wallet derivation families and creator-style masks have dedicated test scripts.

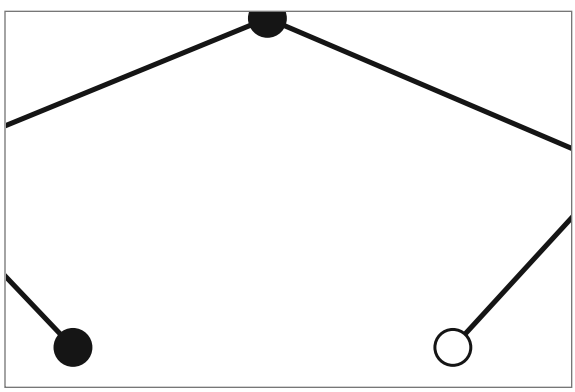
M097

BIP32 child/index interval pairing



01	02	03
HD parent	child and index choices	interval pairing

D / OPERATION DETAIL



Vector detail / 0.98× main-view scale

HOW THE OPERATION WORKS

Treat child derivation indices as a pair such as start index and interval, and test path templates built from those integers.

READ THE THREE STAGES

01 HD parent
02 child and index choices
03 interval pairing

WHAT WAS ACTUALLY DONE

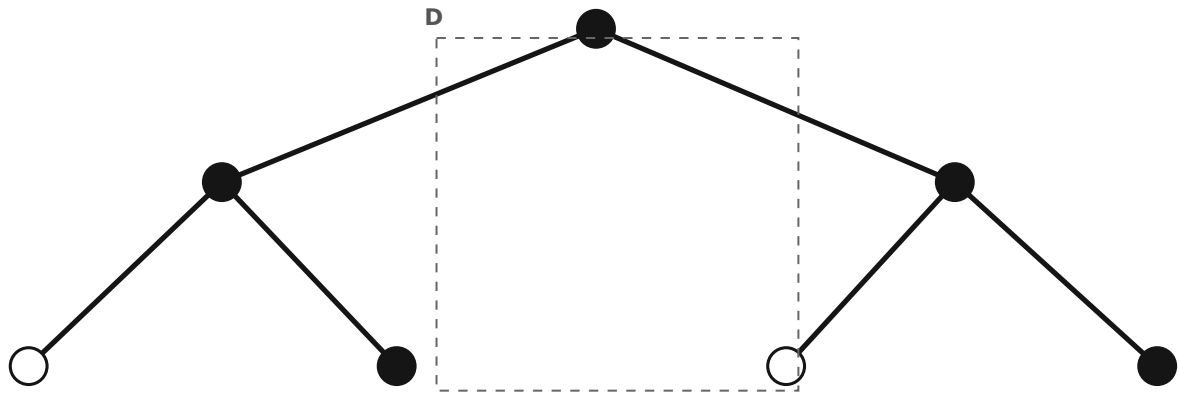
child_wallet_interval_probe.py derives masked children for supplied seeds and compares public targets. A synthetic known-template test checks the probe; unknown creator provenance remains missing.

ORIGINAL FINDING / IMPLEMENTED

Enumerate path-template index/interval pairs under an explicitly supplied seed; synthetic self-test is separate from creator identification.

M098

Supplied extended-key derivation leads

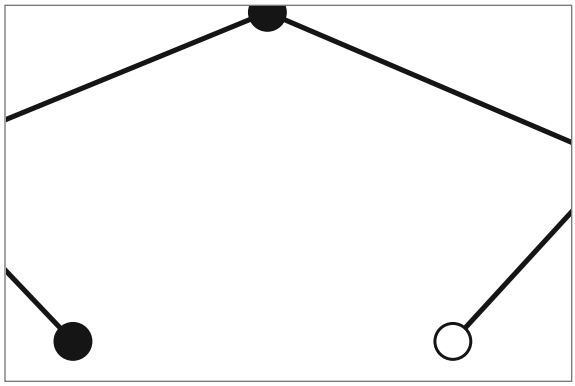


01
supplied extended key

02
declared child paths

03
derived candidates

D / OPERATION DETAIL



Vector detail / 0.98× main-view scale

HOW THE OPERATION WORKS

An explicitly supplied extended private node allows deterministic child derivation along a bounded list of relative paths.

READ THE THREE STAGES

01 supplied extended key
02 declared child paths
03 derived candidates

WHAT WAS ACTUALLY DONE

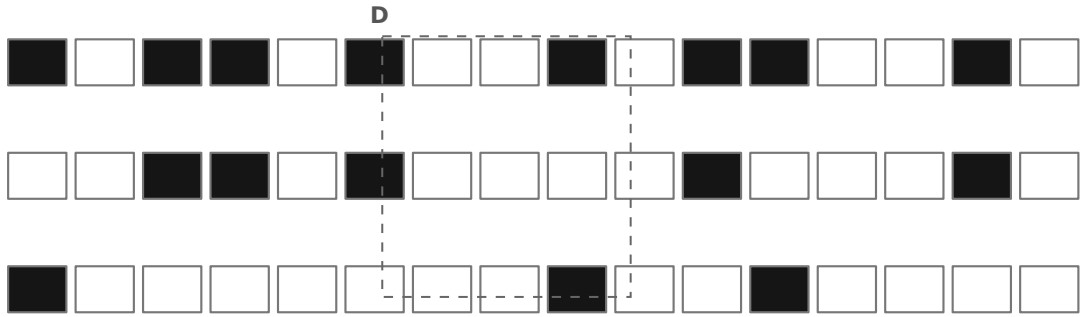
The audit validates checksums, tests seven relative path families and index shifts, then compares initial nontrivial probe keys. It never prints or republishes the supplied secret nodes.

ORIGINAL FINDING / EXECUTED RECORD

Three unique supplied leads tested over bounded relative paths/index shifts; no reproduction of nontrivial probe keys. Secrets are not included here.

M099

Entropy-prefix and full-width constraints

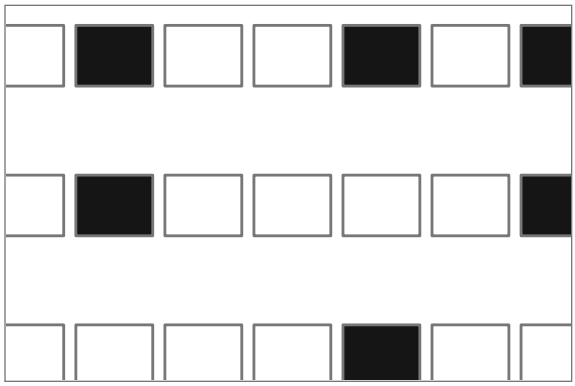


01 seed constraints

02 prefix and full width

03 allowed candidates

D / OPERATION DETAIL



Vector detail / 1.32× main-view scale

HOW THE OPERATION WORKS

A known source-entropy prefix or full-width relation could restrict which masked outputs are possible. It must be derived independently of the hidden key.

READ THE THREE STAGES

01 seed constraints
02 prefix and full width
03 allowed candidates

WHAT WAS ACTUALLY DONE

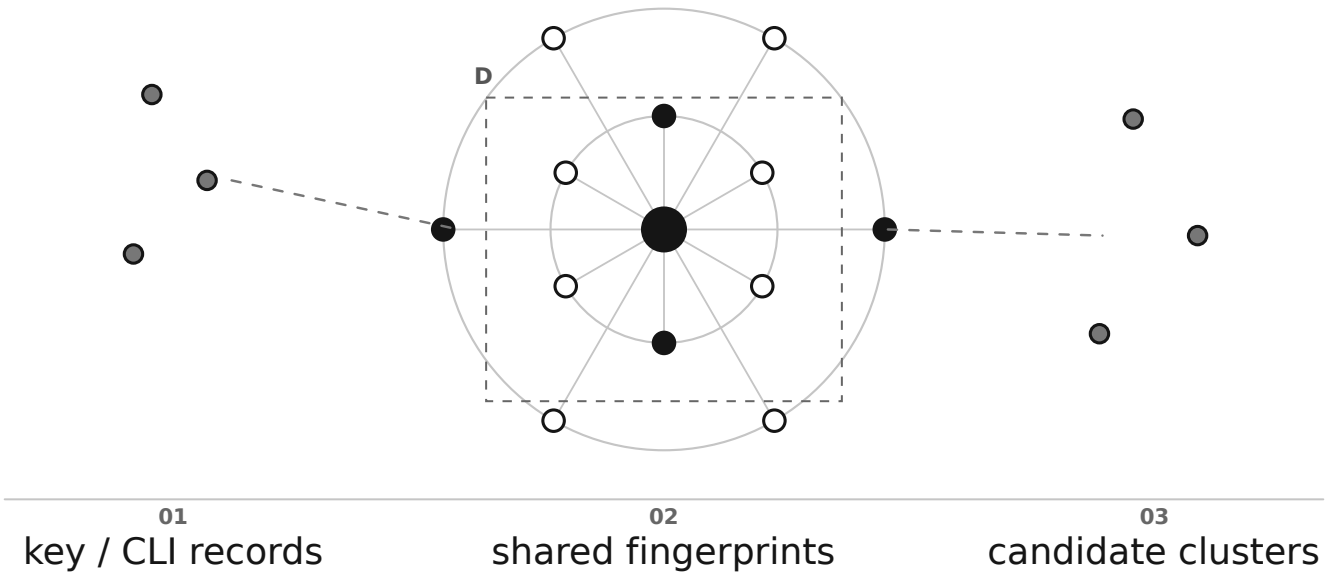
These scripts construct and score named prefix/full-width hypotheses. They do not turn a public difficulty interval into additional leaked secret bits without matching evidence.

ORIGINAL FINDING / IMPLEMENTED

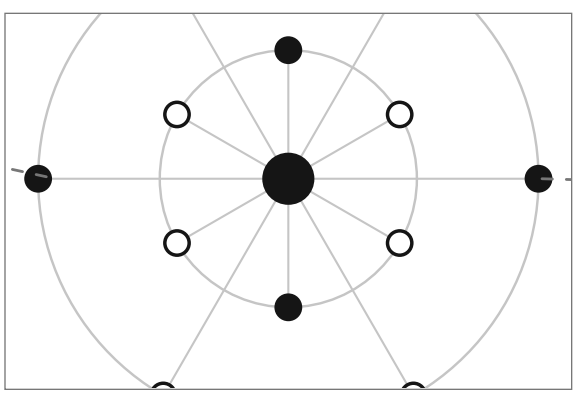
Named candidate scripts examine how scalar masking and source entropy could constrain keys; no generic recovery follows.

M100

CLI fingerprint and key clustering



D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

A short fingerprint of known components can group similar CLI outputs or candidate tuples. It helps organize evidence and detect duplicate hypotheses.

READ THE THREE STAGES

01 key / CLI records
02 shared fingerprints
03 candidate clusters

WHAT WAS ACTUALLY DONE

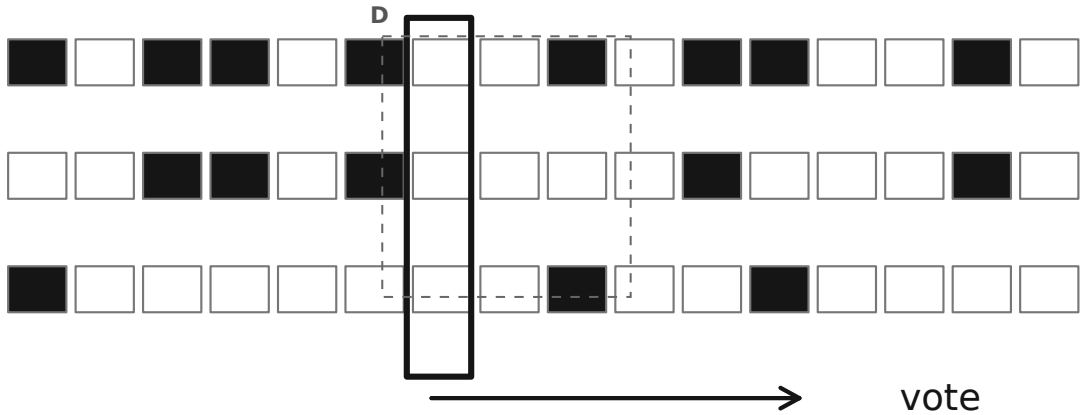
set_bits.py and clustering tools compute partial masks, Hamming statistics and groups. A matching fingerprint is a lossy summary, not proof of an exact scalar or valid search cut.

ORIGINAL FINDING / EXECUTED RECORD

Hamming, low-word masks and partial fingerprints organize known coordinates; not established target narrowing.

M101

Popcount and aligned-bit distribution tests

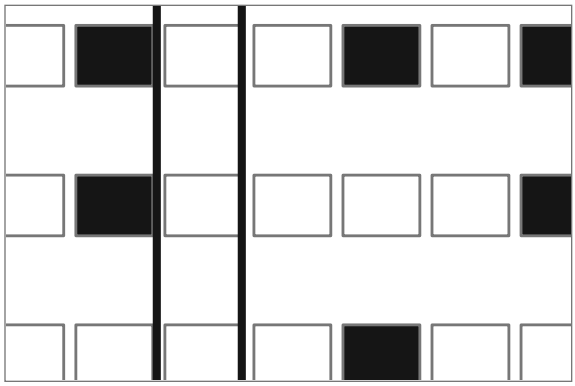


01 aligned bit rows

02 popcount / column rates

03 compare distributions

D / OPERATION DETAIL



Vector detail / 1.32× main-view scale

HOW THE OPERATION WORKS

Compare the number of set bits and aligned bit agreements with the distribution expected from uniform interval keys.

READ THE THREE STAGES

01 aligned bit rows
02 popcount / column rates
03 compare distributions

WHAT WAS ACTUALLY DONE

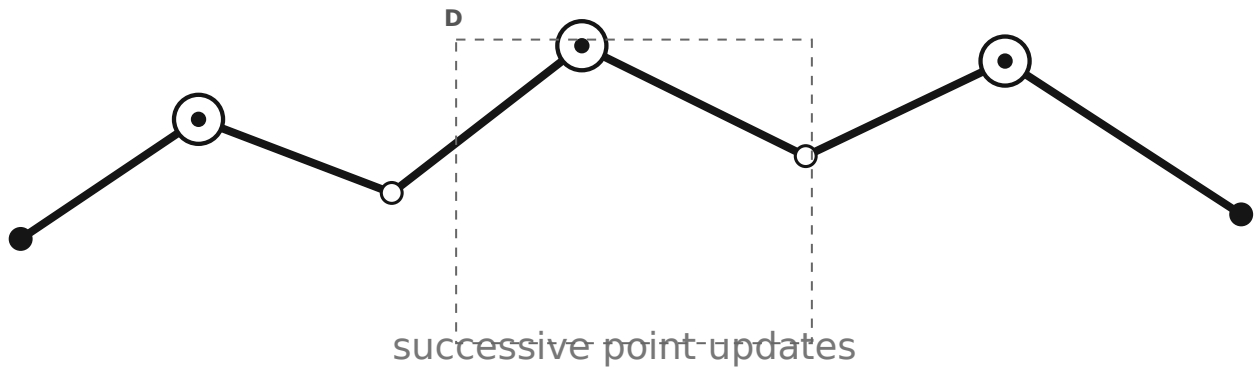
Reports measure popcounts, low-bit patterns and pairwise overlaps. Statistical similarity/difference concerns the tested corpus; it does not directly provide the unknown bits of the next key.

ORIGINAL FINDING / EXECUTED RECORD

Known-key bit densities and overlap are compared with controls; observed appearance is not a scalar equation.

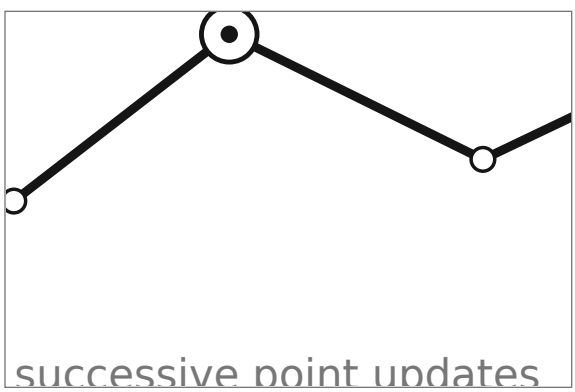
M102

Counter/small-affine early public-key relations



01	02	03
public points	counter / small offsets	test linear relations

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Small scalar relations are visible as public-point relations: alpha*P_i+beta*P_j+gamma*G equals the identity if the corresponding scalars satisfy them.

READ THE THREE STAGES

01 public points
02 counter / small offsets
03 test linear relations

WHAT WAS ACTUALLY DONE

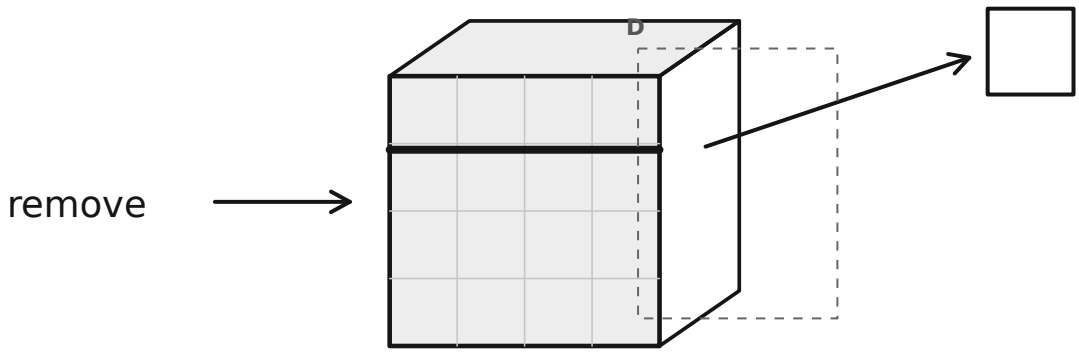
Early-key scripts sweep bounded coefficient and offset grids, including nonce-to-signer pairs. A genuine relation could help, but the saved finite sweeps report no such target recovery.

ORIGINAL FINDING / EXECUTED RECORD

Recorded all-pairs and focused early-key coefficient sweeps found no relation in their finite coefficient/offset bounds.

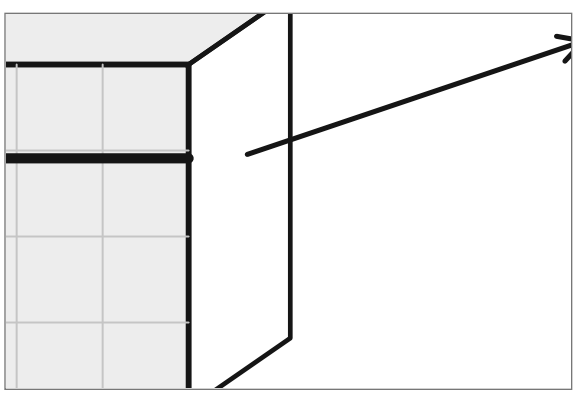
M103

Sparse ORIG decomposition and peel chain



01	02	03
sparse scalar	remove high terms	residual tail

D / OPERATION DETAIL



Vector detail / 1.27× main-view scale

HOW THE OPERATION WORKS

Describe a supplied sparse integer as a few shifted terms plus a tail, then reverse those terms to expose nested smaller values.

READ THE THREE STAGES

01 sparse scalar
02 remove high terms
03 residual tail

WHAT WAS ACTUALLY DONE

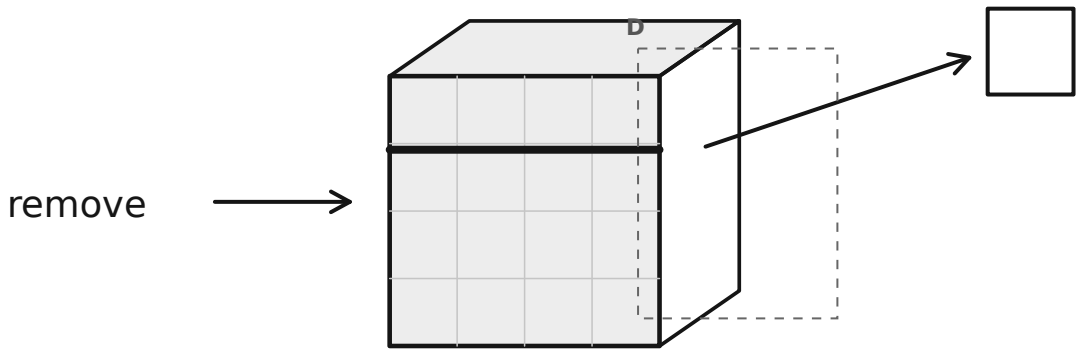
The ORIG investigation verifies its exact decomposition and peel chain to tiny public keys. Constructing an integer around a known tail does not explain generation of later independent puzzles.

ORIGINAL FINDING / EXECUTED RECORD

A constructed sparse scalar peels to tiny known puzzle values; this does not generate the numbered puzzle sequence.

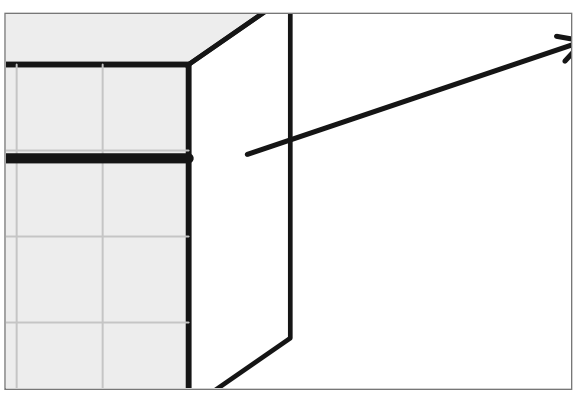
M104

Nested previous-key tail grow/peel



01	02	03
previous key tail	grow or peel bits	nested candidate

D / OPERATION DETAIL



Vector detail / 1.27× main-view scale

HOW THE OPERATION WORKS

Build the next sparse value by embedding the previous entire value under a new frame, and reverse that operation by peeling the frame.

READ THE THREE STAGES

01 previous key tail
02 grow or peel bits
03 nested candidate

WHAT WAS ACTUALLY DONE

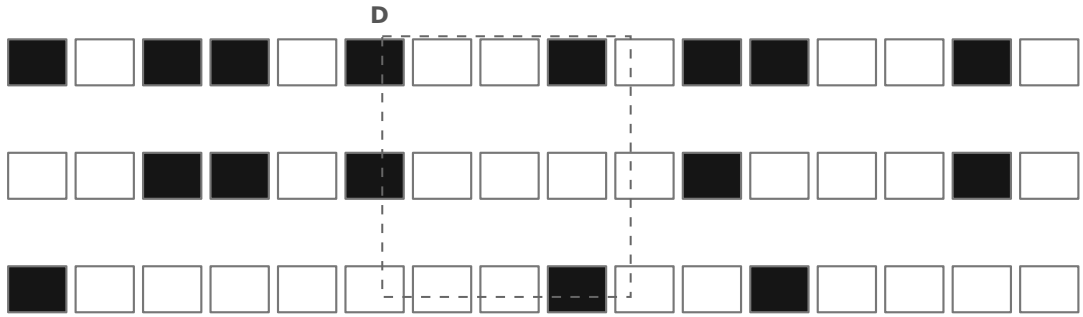
The audit compares this recursive rule with the actual fixed-tail expression. They coincide at one chosen step but produce different subsequent integers, exposing a conflated model.

ORIGINAL FINDING / AUDITED/CORRECTED

Distinguishes nested-tail updates from fixed-tail co-shifts; the two agree at one construction step, not generally.

M105

Fixed-tail co-shift family

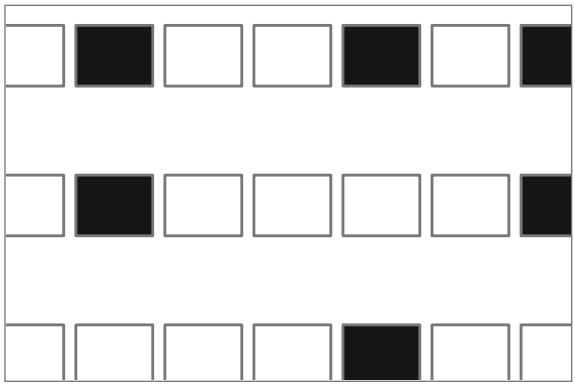


01 fixed tail

02 shift both coordinates

03 candidate family

D / OPERATION DETAIL



Vector detail / 1.32× main-view scale

HOW THE OPERATION WORKS

Keep a small tail constant while shifting selected high and middle terms together. This defines a precise sparse family.

READ THE THREE STAGES

01 fixed tail
02 shift both coordinates
03 candidate family

WHAT WAS ACTUALLY DONE

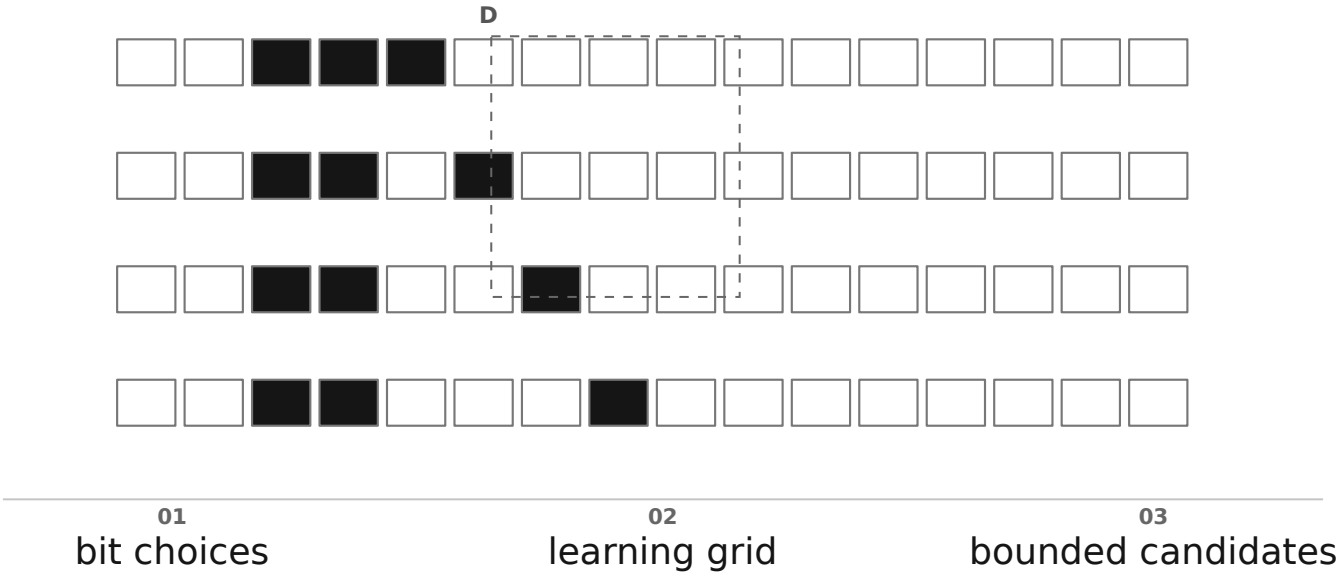
sparse_key_coshift.py generates that family's members. Reversibility within the constructed family is a program property, not evidence that numbered Bitcoin puzzles lie on its orbit.

ORIGINAL FINDING / IMPLEMENTED

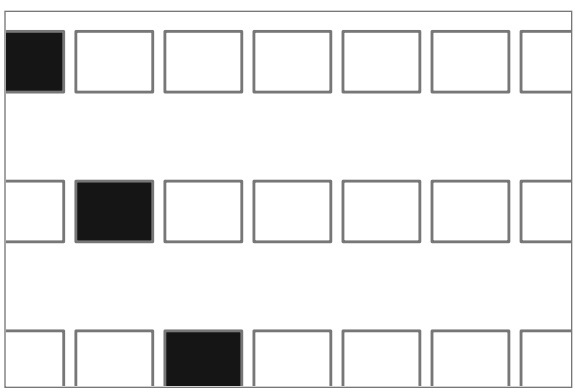
Exact sparse-key program shifts its frame while preserving the tail; valid program structure is not a Bitcoin predictor.

M106

Sparse bit-learning and dial-in grids



D / OPERATION DETAIL



Vector detail / 1.32× main-view scale

HOW THE OPERATION WORKS

Fit simple frame positions, bit rules or mid/gap parameters to known sparse values and test whether they extend across puzzle keys.

READ THE THREE STAGES

01 bit choices
02 learning grid
03 bounded candidates

WHAT WAS ACTUALLY DONE

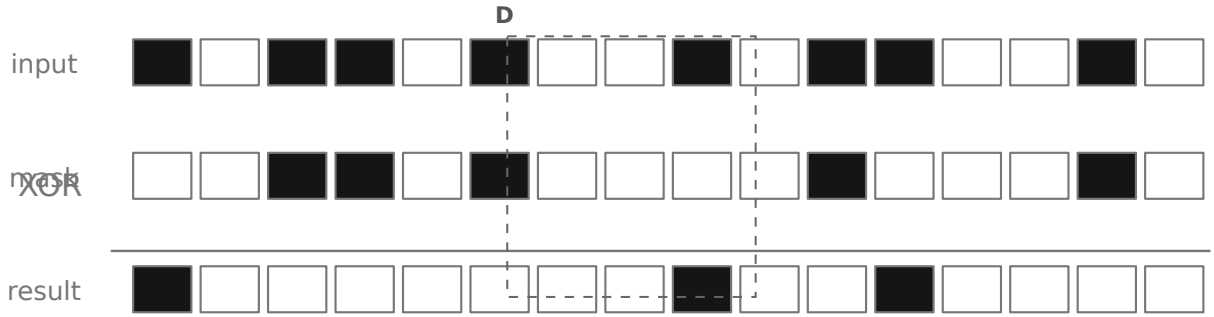
Bit-learning and dial-in scripts sweep bounded templates and report coverage. The later investigation separates fitted tiny-key coincidences from actual unseen-address hits.

ORIGINAL FINDING / EXECUTED RECORD

Learn/test mid, gap, bit and frame variants; archived investigation reports poor known-key recall and no unknown address hits.

M107

Sparse XOR-tail / delta / Bloom variants

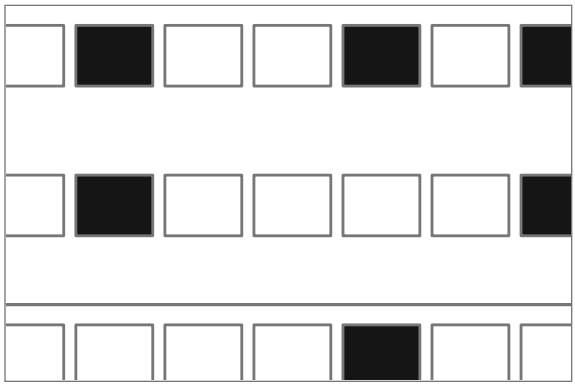


01 seed and tail

02 XOR / delta masks

03 candidate checks

D / OPERATION DETAIL



Vector detail / 1.32× main-view scale

HOW THE OPERATION WORKS

Change the recurrence using XOR tails, deltas or Bloom-filter-assisted membership screening and compare its generated values with targets.

READ THE THREE STAGES

01 seed and tail
02 XOR / delta masks
03 candidate checks

WHAT WAS ACTUALLY DONE

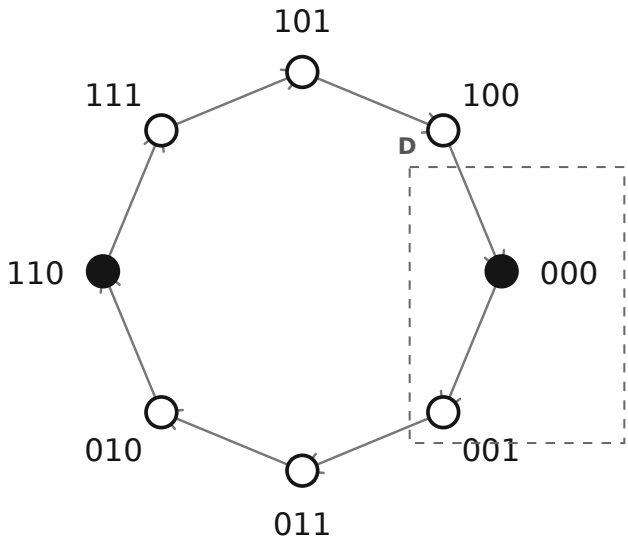
These are alternative sparse candidate recipes. A Bloom hit is only a screening event; the scalar still needs exact address verification, and saved summaries do not establish a transferable generator.

ORIGINAL FINDING / EXECUTED RECORD

Alternate sparse generative families were tested without establishing a transferable key-generation rule.

M108

Gray-code range and anchor walks

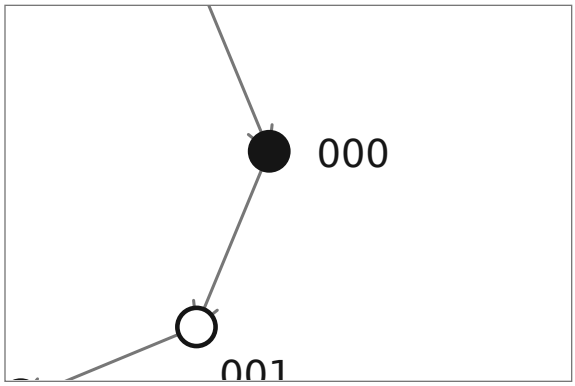


01 anchor bits

02 one bit changes

03 Gray-code walk

D / OPERATION DETAIL



Vector detail / 1.25× main-view scale

HOW THE OPERATION WORKS

Gray code orders bitstrings so adjacent offsets differ in one bit; adding or XORing offsets around an anchor defines a traversal.

READ THE THREE STAGES

01 anchor bits
02 one bit changes
03 Gray-code walk

WHAT WAS ACTUALLY DONE

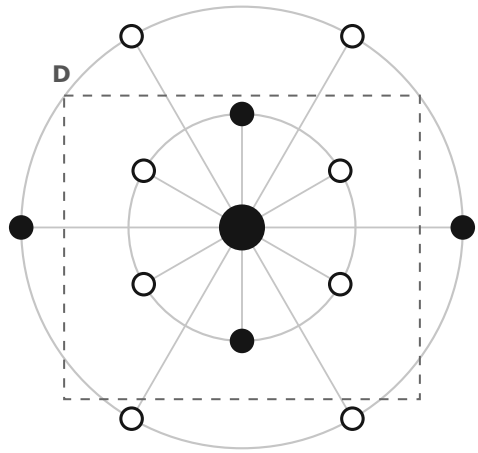
bit_flip_hunt.py provides range and anchor variants. They visit specified candidates in a different order and do not prove the key is near the selected anchor.

ORIGINAL FINDING / IMPLEMENTED

Enumerate Gray offsets around range floors or metadata anchors; this changes order rather than entropy.

M109

BFS Hamming balls and previous-key XOR

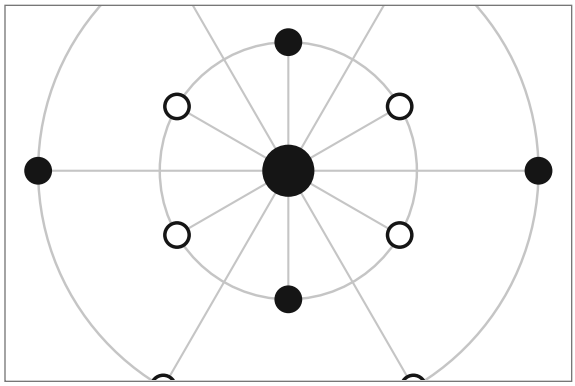


01 anchor key

02 Hamming shells

03 breadth-first neighbors

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Enumerate all masks up to a chosen Hamming weight around a seed, or XOR selected bits into a previous known key.

READ THE THREE STAGES

01 anchor key
02 Hamming shells
03 breadth-first neighbors

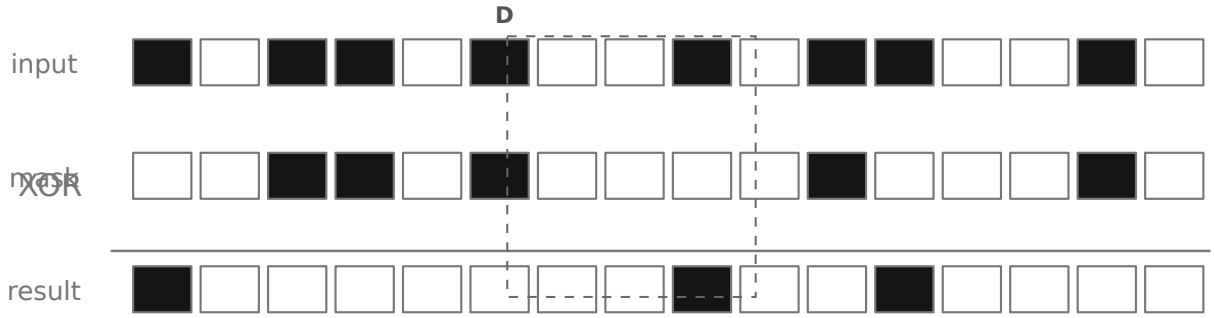
WHAT WAS ACTUALLY DONE

BFS/previous-key routines search bounded neighborhoods. A completed neighborhood search says nothing about integers outside that radius unless independent target-distance information exists.

ORIGINAL FINDING / IMPLEMENTED

Check bounded low-Hamming-distance neighborhoods around seeds or prior keys; completeness is limited to that neighborhood.

M110
TRAILING-mask and FUSED-D1 bit flips

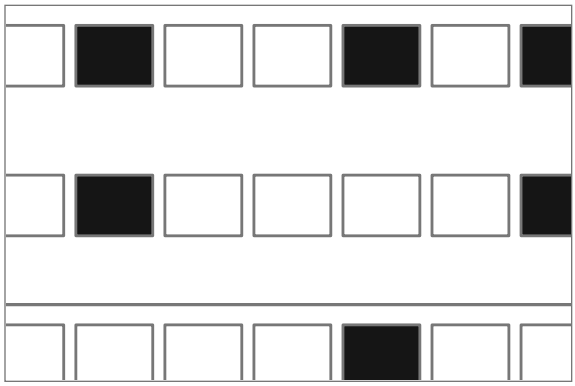


01 TRAILING / D1 anchor

02 flip selected bits

03 candidate neighborhood

D / OPERATION DETAIL



Vector detail / 1.32× main-view scale

HOW THE OPERATION WORKS

Choose flip positions from TRAILING bits or apply flips within a D1 coordinate derived from a FUSED anchor.

READ THE THREE STAGES

01 TRAILING / D1 anchor
02 flip selected bits
03 candidate neighborhood

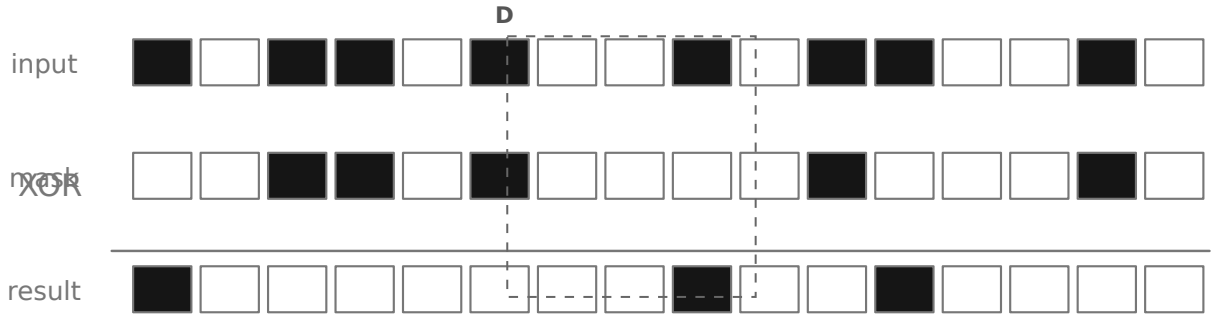
WHAT WAS ACTUALLY DONE

These methods impose a candidate-selection hypothesis on where the unknown bits differ. Their bounds are computational choices, not leaked bit positions from the target.

ORIGINAL FINDING / IMPLEMENTED

Restrict flipped positions or anchor the D1 coordinate; target inclusion is unproven.

M111
Dual-mask byte/bit and Gray-PDEP walks

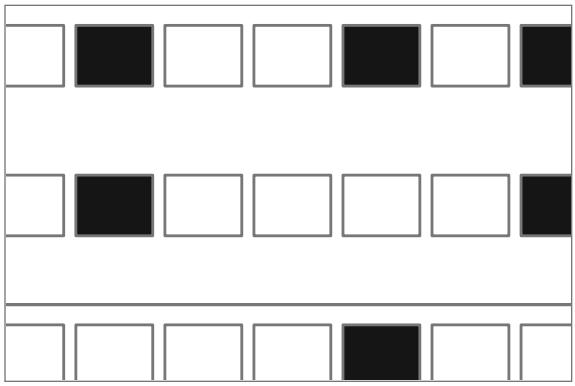


01 dual masks

02 Gray bit placement

03 byte / bit candidates

D / OPERATION DETAIL



Vector detail / 1.32× main-view scale

HOW THE OPERATION WORKS

Deposit counter bits into chosen mask positions; optionally Gray-code the counter so consecutive deposits change sparsely.

READ THE THREE STAGES

01 dual masks
02 Gray bit placement
03 byte / bit candidates

WHAT WAS ACTUALLY DONE

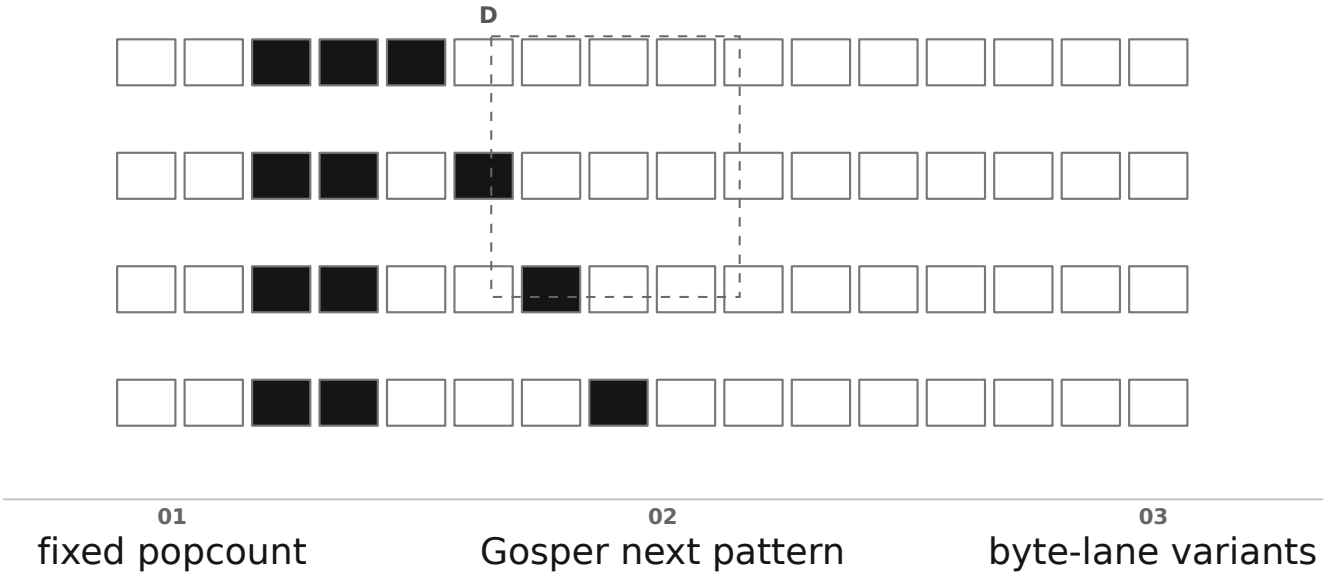
dual_mask_seq.py implements byte- and bit-index complementary-mask schedules. The masks describe allowed moves; their symbolic organization does not create missing target information.

ORIGINAL FINDING / IMPLEMENTED

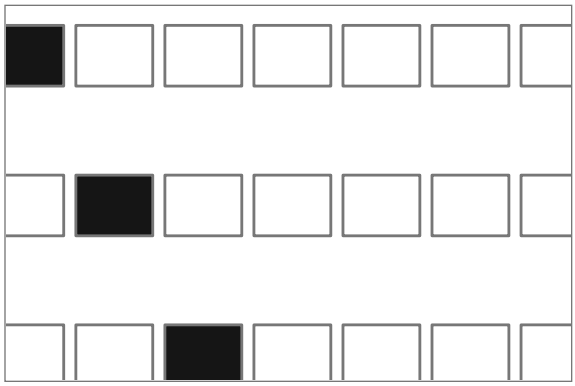
Deposit counter bits into complementary masks, including Gray ordering; these are deterministic traversal families.

M112

Gosper constant-weight and byte-lane variants



D / OPERATION DETAIL



Vector detail / 1.32× main-view scale

HOW THE OPERATION WORKS

Gosper's rule enumerates masks of fixed Hamming weight. A byte-lane walk substitutes every byte value at one position.

READ THE THREE STAGES

01 fixed popcount
02 Gosper next pattern
03 byte-lane variants

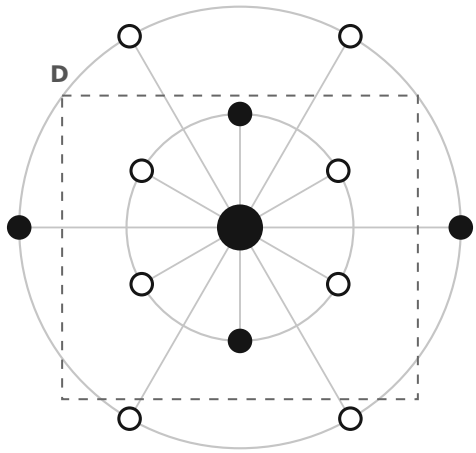
WHAT WAS ACTUALLY DONE

The bit-flip engine generates these candidate families around anchors and verifies addresses. Their coverage is explicitly restricted to the selected mask weights or byte substitutions.

ORIGINAL FINDING / IMPLEMENTED

Visit fixed-weight XOR masks or replace individual bytes; no claim that these neighborhoods contain the target.

M113
Shift-shell and suffix Hamming hunts

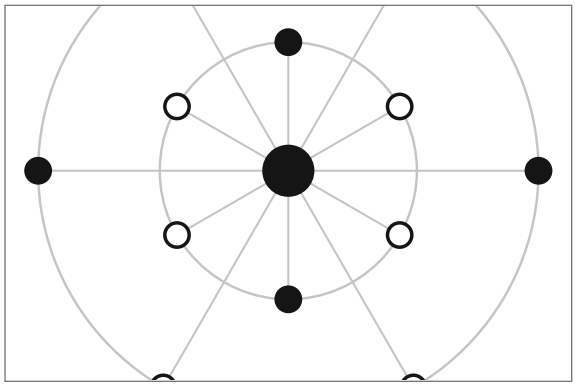


01
shifted anchor

02
suffix bit shells

03
bounded hunts

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Search literal scalar shells around a metadata distance, or flip a limited number of bits in a chosen suffix.

READ THE THREE STAGES

01 shifted anchor
02 suffix bit shells
03 bounded hunts

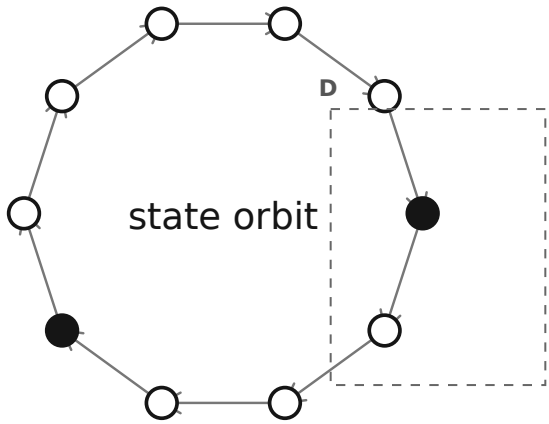
WHAT WAS ACTUALLY DONE

hunt_shift_windows.py tests distance shells, a 106-bit suffix neighborhood and small TRAILING gaps. Each is a bounded anchor hypothesis, not an exhaustive private-key interval search.

ORIGINAL FINDING / IMPLEMENTED

Search metadata-distance shells, suffix flips and literal TRAILING gaps around selected anchors.

M114
Synthesis transformation orbit engine

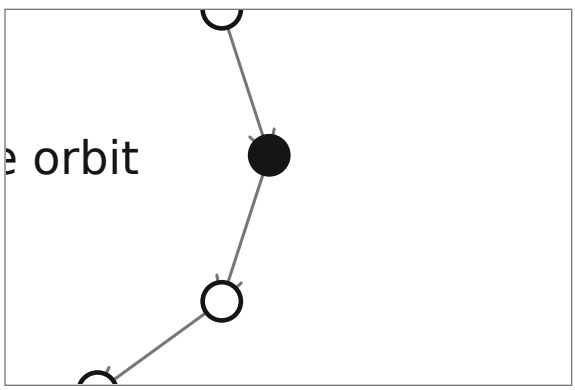


01
seed state

02
declared transformations

03
reachable orbit

D / OPERATION DETAIL



Vector detail / 1.25× main-view scale

HOW THE OPERATION WORKS

Apply a registry of deterministic mixing transformations repeatedly to produce candidate orbits, then rank the resulting integers.

READ THE THREE STAGES

01 seed state
02 declared transformations
03 reachable orbit

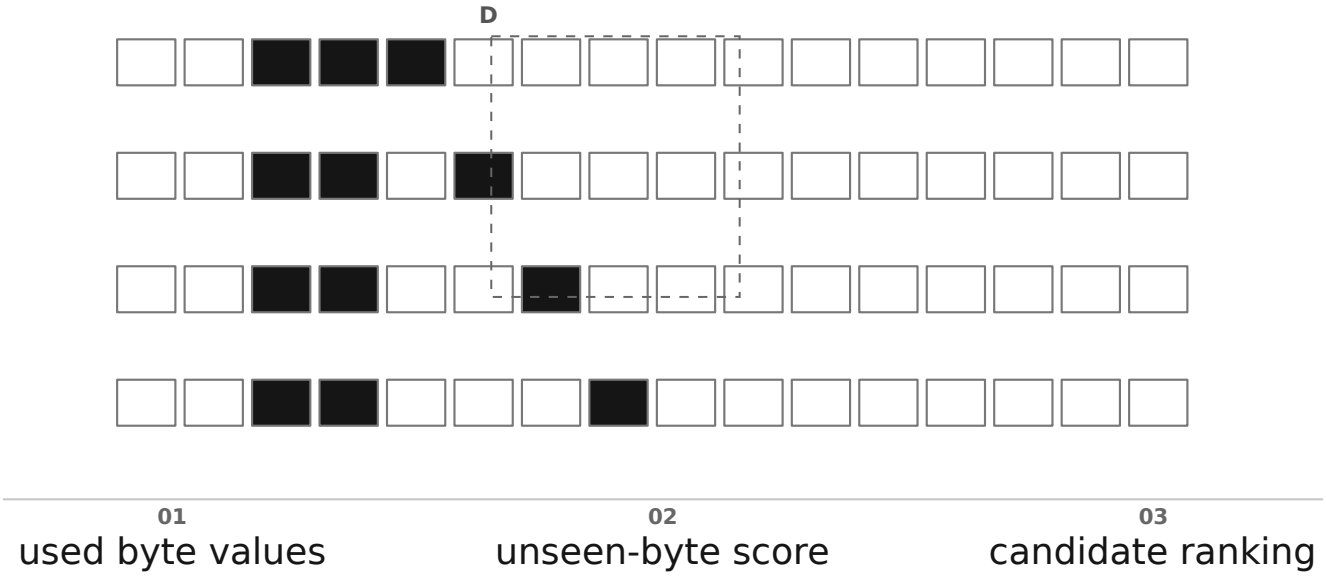
WHAT WAS ACTUALLY DONE

The synthesis script's capture/decay/catalyst names denote ordinary bit/arithmetic operations. It generates and scores candidate scalars; the names do not represent physical or quantum processes.

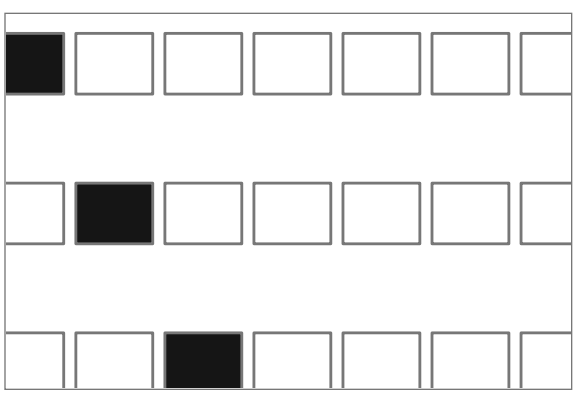
ORIGINAL FINDING / IMPLEMENTED

Hybrid orbit, capture/decay/catalyst/moderator transformations rank unused-byte novelty; terminology does not imply quantum physics.

M115
Unused-byte novelty scoring



D / OPERATION DETAIL



Vector detail / 1.32× main-view scale

HOW THE OPERATION WORKS

Count byte positions whose values have not appeared in the solved-key corpus, favoring combinations labeled novel.

READ THE THREE STAGES

01 used byte values
02 unseen-byte score
03 candidate ranking

WHAT WAS ACTUALLY DONE

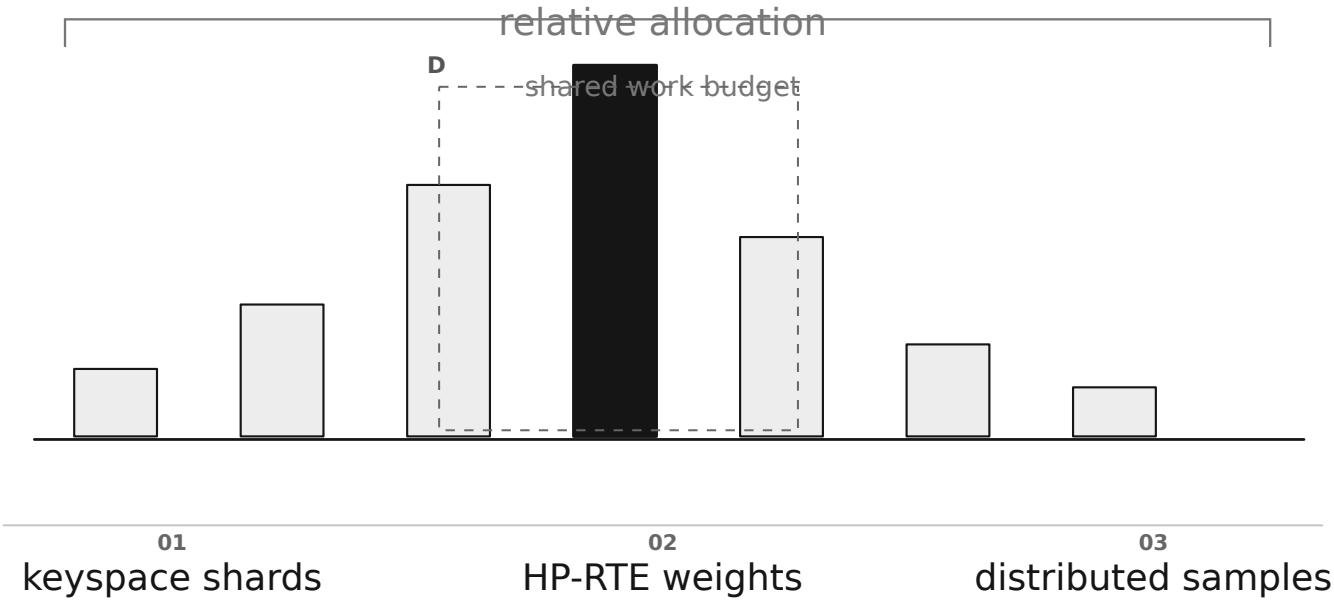
The orbit batch verifier tests the actual scalar against the address. High novelty scores do not measure target agreement; some archived candidates were even below the valid puzzle interval.

ORIGINAL FINDING / EXECUTED RECORD

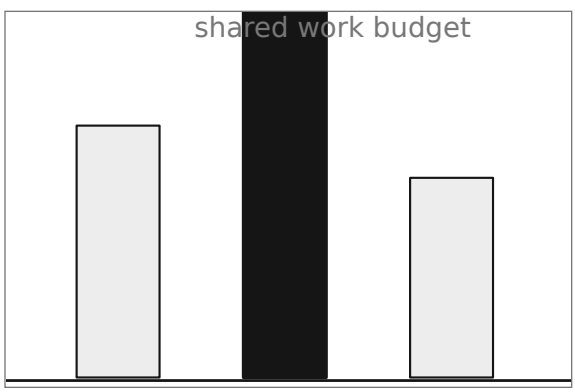
Score 8/9 or 9/9 measures byte novelty, not target agreement. Archived 1,800-candidate verification had zero hits.

M116

Original HP-RTE weighted shard sampling



D / OPERATION DETAIL



Vector detail / 1.00× main-view scale

HOW THE OPERATION WORKS

The original recipe samples a key, derives a byte/nibble label and uses its assigned weight to move the output into a biased subrange.

READ THE THREE STAGES

01 keyspace shards
02 HP-RTE weights
03 distributed samples

WHAT WAS ACTUALLY DONE

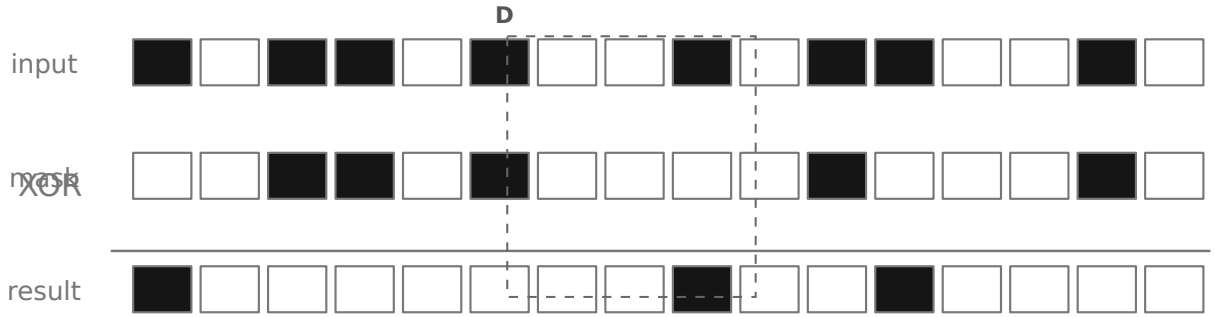
Historical code uses floating-point offsets, and successive versions exclude different parts of the interval. The repaired generator removes that unsupported exclusion and uses exact integer sampling.

ORIGINAL FINDING / AUDITED/CORRECTED

Historical float-based biased generator excludes part of the advertised interval; corrected pipeline uses full-support integer partitions.

M117

Two-byte rotation and four-hex-tree claims

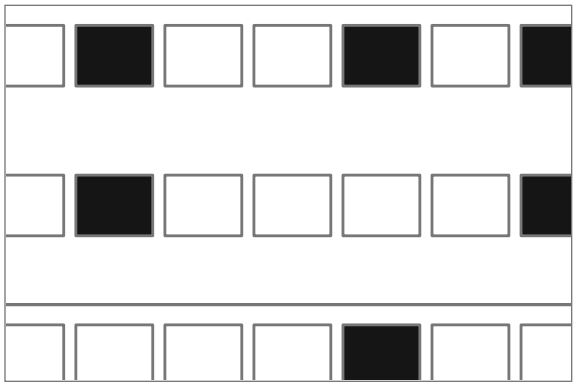


01
two-byte state

02
rotate bytes

03
claimed four-hex tree

D / OPERATION DETAIL



Vector detail / 1.32× main-view scale

HOW THE OPERATION WORKS

Rotate a byte string to change which byte is inspected. A genuine four-hex tree would also define branching and traversal over four hex digits.

READ THE THREE STAGES

01 two-byte state
02 rotate bytes
03 claimed four-hex tree

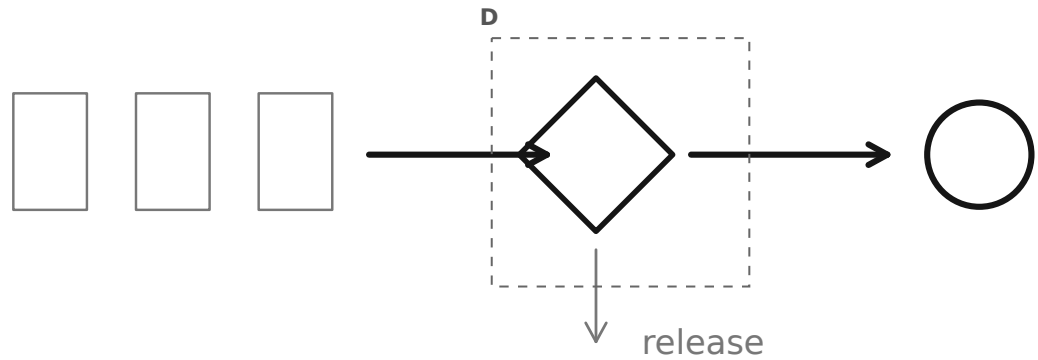
WHAT WAS ACTUALLY DONE

The supplied generator rotates by two bytes, not two hex characters. Its tree-depth name is not used to implement the claimed tree partitioning.

ORIGINAL FINDING / AUDITED/CORRECTED

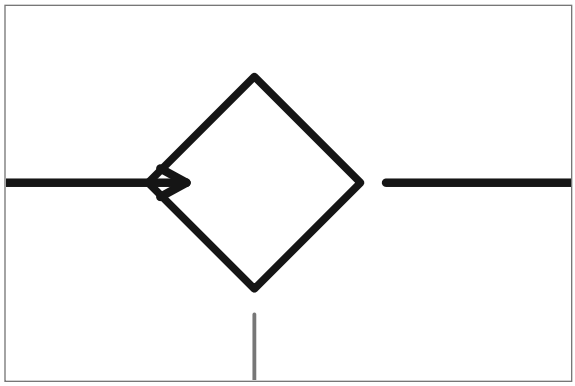
Rotation exists, but original tree-depth configuration did not implement the claimed tree search.

M118
Million-record JSON/JSONL generation



01	02	03
sample generator	JSON / JSONL records	million-record output

D / OPERATION DETAIL



Vector detail / 1.38× main-view scale

HOW THE OPERATION WORKS

Serialize one record per generated candidate or interval. JSONL allows records to be written one at a time.

READ THE THREE STAGES

01 sample generator
02 JSON / JSONL records
03 million-record output

WHAT WAS ACTUALLY DONE

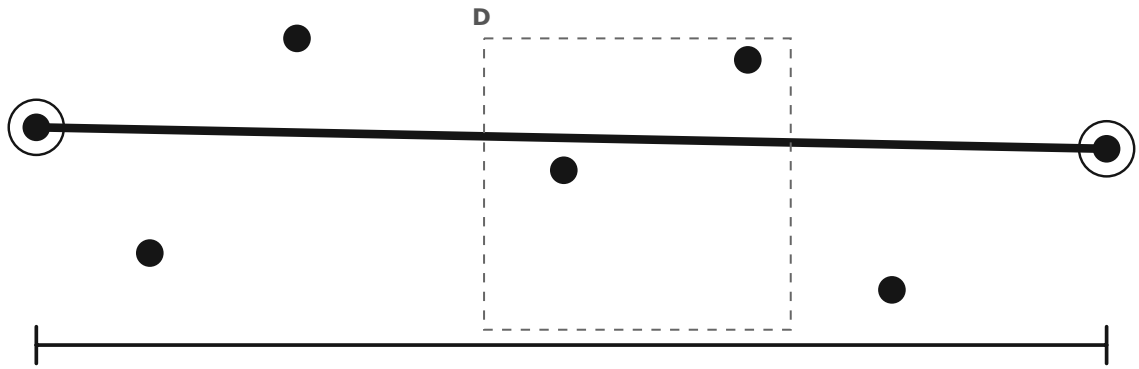
The corrected run wrote a million descriptive records, then separately verified their sample keys and boundaries. File size and record count measure storage and generation, not exhaustive cryptographic coverage.

ORIGINAL FINDING / EXECUTED RECORD

Large record counts are data-generation work, not cryptographic search coverage; streaming JSONL avoids all-record memory use.

M119

Weighted modular-diameter diversity

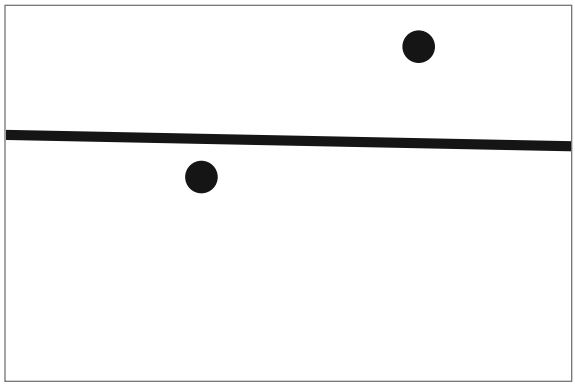


01 modular samples

02 weighted distance

03 diversity score

D / OPERATION DETAIL



Vector detail / 1.18× main-view scale

HOW THE OPERATION WORKS

The old statistic computes pairwise numeric spread and multiplies it by a subset-dependent average bias weight.

READ THE THREE STAGES

01 modular samples
02 weighted distance
03 diversity score

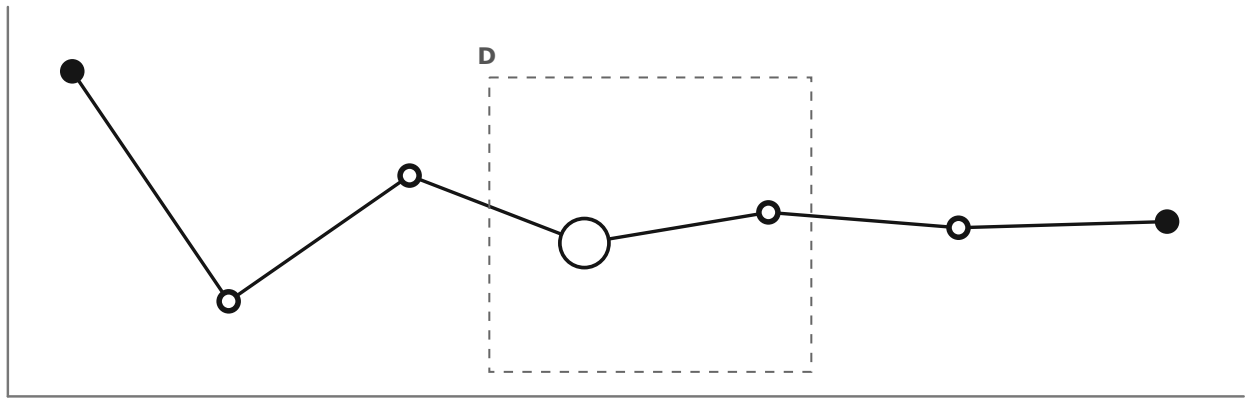
WHAT WAS ACTUALLY DONE

The original uint64 conversion cannot hold 71-bit scalars; unsigned subtraction also wraps. Fixing integer overflow alone does not prove that the weighted expression satisfies diversity axioms.

ORIGINAL FINDING / AUDITED/CORRECTED

uint64 overflow, unsigned differences and unproved subset weights invalidate original diversity claims.

M120
Finite-sample Cauchy/convergence claims

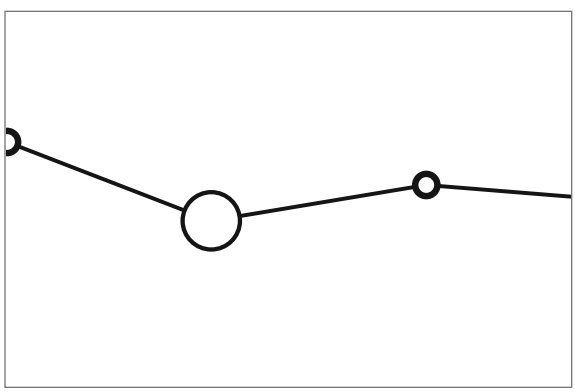


01 finite sample sequence

02 pairwise closeness

03 convergence claim check

D / OPERATION DETAIL



Vector detail / 1.17× main-view scale

HOW THE OPERATION WORKS

True Cauchyness and convergence require bounds over arbitrary sufficiently late terms of an infinite sequence, with the proposed limit included for convergence.

READ THE THREE STAGES

01 finite sample sequence
02 pairwise closeness
03 convergence claim check

WHAT WAS ACTUALLY DONE

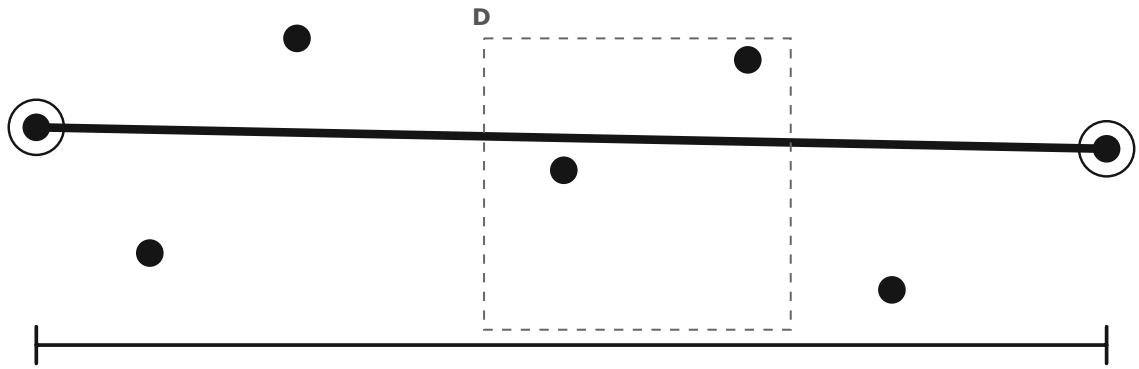
The old functions sample subsets of a finite whole history and compare an arbitrary threshold. They are finite clustering tests mislabeled as asymptotic guarantees and cannot locate an unknown private key.

ORIGINAL FINDING / AUDITED/CORRECTED

Sampling a finite history is not an asymptotic tail criterion; clustering cannot identify the unknown scalar.

M121

Exact ordinary diameter diagnostic

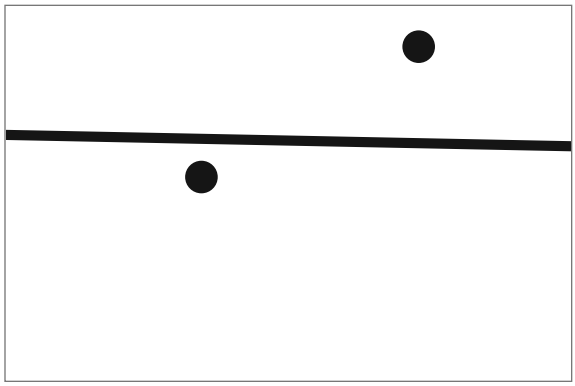


01 ordinary point set

02 farthest pair

03 exact diameter

D / OPERATION DETAIL



Vector detail / 1.18× main-view scale

HOW THE OPERATION WORKS

Diameter is the largest value minus the smallest. With fixed normalization it describes a finite sample's coordinate spread exactly.

READ THE THREE STAGES

01 ordinary point set

02 farthest pair

03 exact diameter

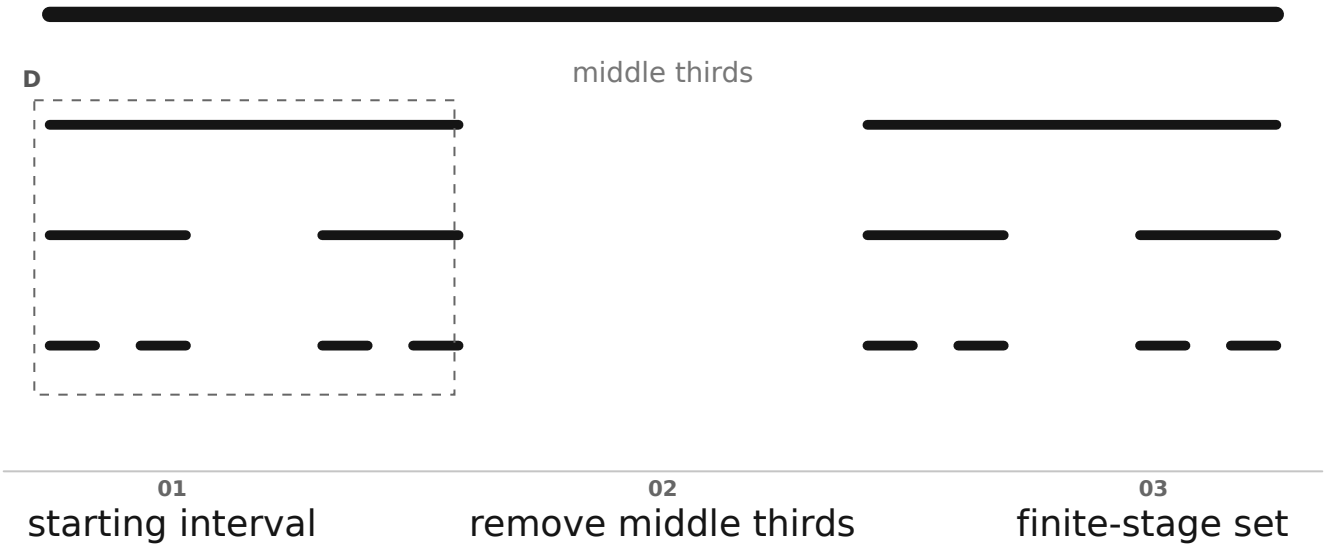
WHAT WAS ACTUALLY DONE

The corrected script computes integer min/max without an all-pairs matrix. It labels the result descriptively and makes no claim of convergence toward the cryptographic target.

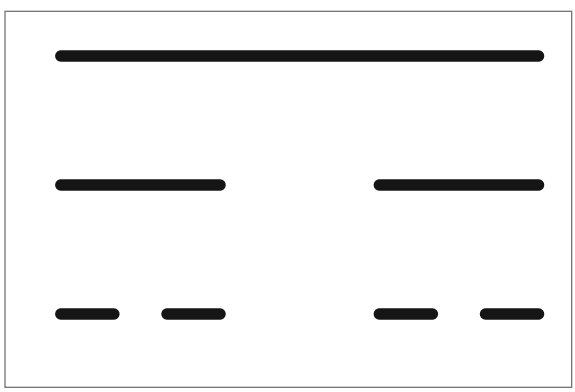
ORIGINAL FINDING / EXECUTED RECORD

Integer max-minus-min measures sample spread without claiming convergence or cryptographic proximity.

M122
Cantor-like finite-stage constructions



D / OPERATION DETAIL



Vector detail / 1.17× main-view scale

HOW THE OPERATION WORKS

Repeatedly remove a middle interval from each remaining closed interval, tracking exact rational endpoints and total surviving length.

READ THE THREE STAGES

01 starting interval
02 remove middle thirds
03 finite-stage set

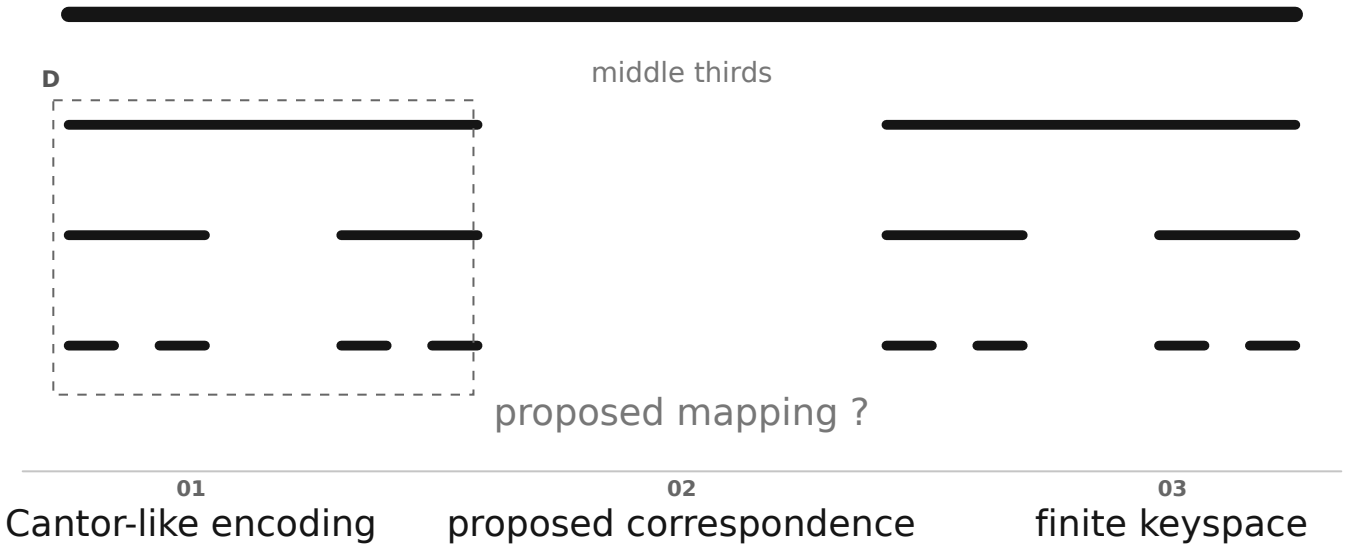
WHAT WAS ACTUALLY DONE

The corrected example removes half at every stage; after eight stages length is 1/256. The infinite limit has zero measure, unlike the earlier claimed fat-Cantor interpretation.

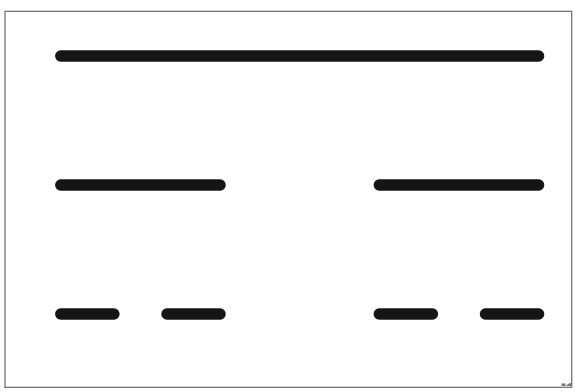
ORIGINAL FINDING / AUDITED/CORRECTED

Exact rational construction corrects historical measure calculations; constant half-removal tends to zero measure.

M123
Cantor-to-keyspace homeomorphism claim



D / OPERATION DETAIL



Vector detail / 1.17× main-view scale

HOW THE OPERATION WORKS

A homeomorphism must be a continuous bijection with a continuous inverse. An infinite Cantor set cannot be bijective with a finite integer keyspace.

READ THE THREE STAGES

01 Cantor-like encoding
02 proposed correspondence
03 finite keyspace

WHAT WAS ACTUALLY DONE

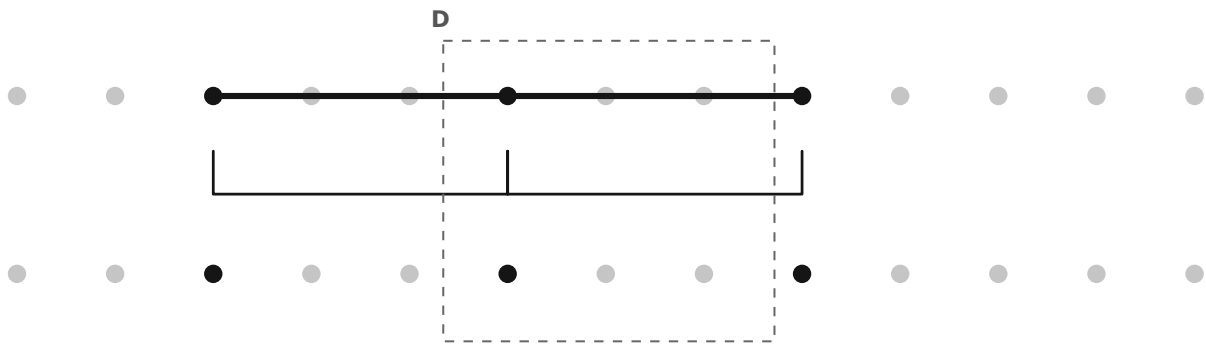
The earlier function merely scales floating coordinates into sample min/max bounds and rounds to integers. It is a lossy candidate map, not the source paper's homeomorphism construction.

ORIGINAL FINDING / AUDITED/CORRECTED

Rounding real samples to finite keys is not a homeomorphism from an infinite Cantor set.

M124

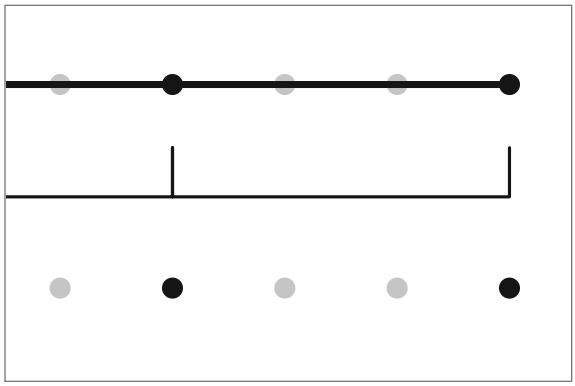
Byte-value arithmetic-progression avoidance



equal gaps / selected labels

01	02	03
byte values	equal-step triples	avoid progressions

D / OPERATION DETAIL



Vector detail / 1.14× main-view scale

HOW THE OPERATION WORKS

Search the set of distinct byte values for a nonconstant arithmetic progression such as 10,20,30. Every requested term must be present.

READ THE THREE STAGES

01 byte values
02 equal-step triples
03 avoid progressions

WHAT WAS ACTUALLY DONE

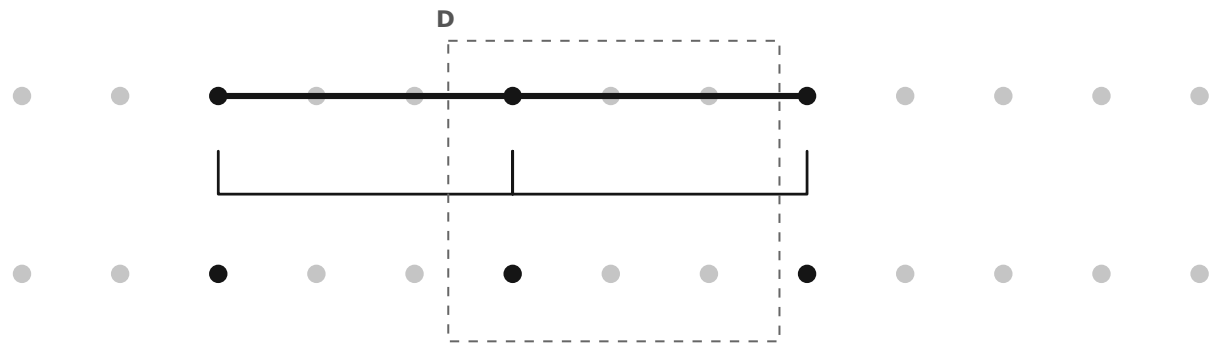
Corrected code extracts nine actual bytes, ignores order/multiplicity for this test, and records a Boolean label. It does not treat pattern avoidance as evidence that the key is more likely.

ORIGINAL FINDING / EXECUTED RECORD

Corrected diagnostic detects nonconstant APs among byte values; it is not used to discard search candidates.

M125

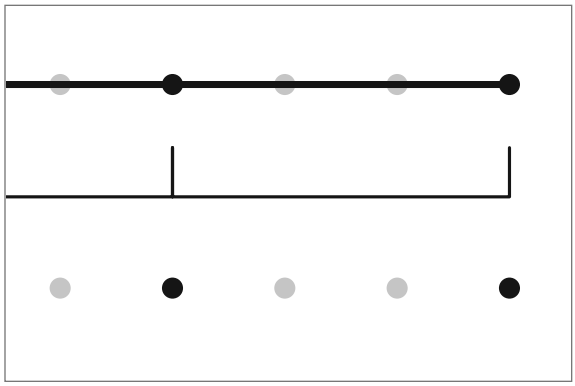
Monochromatic byte-position progression checks



equal gaps / selected labels

01	02	03
byte positions	monochromatic triples	position checks

D / OPERATION DETAIL



Vector detail / 1.14× main-view scale

HOW THE OPERATION WORKS

Treat byte positions as the arithmetic progression and ask whether equally spaced positions carry the same byte value.

READ THE THREE STAGES

01 byte positions
02 monochromatic triples
03 position checks

WHAT WAS ACTUALLY DONE

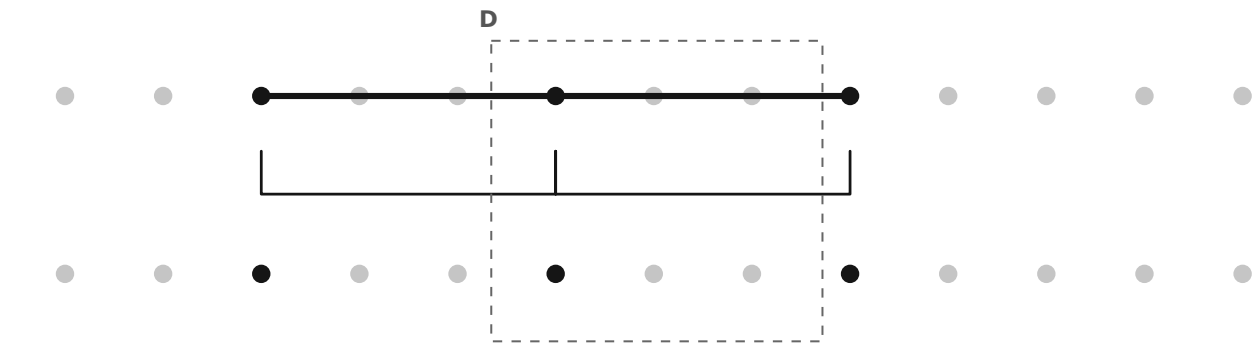
The corrected predicate differs deliberately from value-set APs. Repeated byte values can trigger a positional pattern even when the value set has no nonconstant arithmetic progression.

ORIGINAL FINDING / EXECUTED RECORD

Equal bytes at equally spaced positions are distinguished from APs in the values themselves.

M126

Double-AP and RamseyScript subset claims



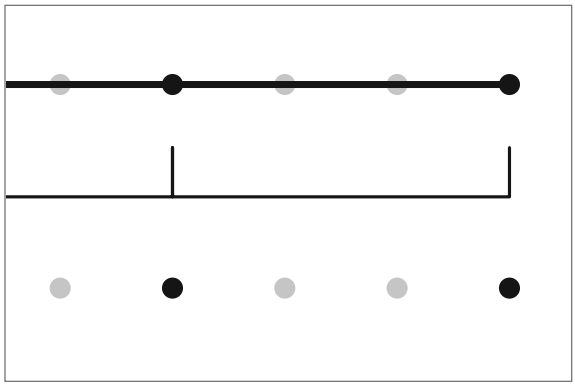
equal gaps / selected labels

01 subsets

02 double progressions

03 declared Ramsey tests

D / OPERATION DETAIL



Vector detail / 1.14× main-view scale

HOW THE OPERATION WORKS

Ramsey search recursively extends combinatorial objects while rejecting forbidden structures; a faithful interpreter must implement the specified objects and filter semantics.

READ THE THREE STAGES

01 subsets
02 double progressions
03 declared Ramsey tests

WHAT WAS ACTUALLY DONE

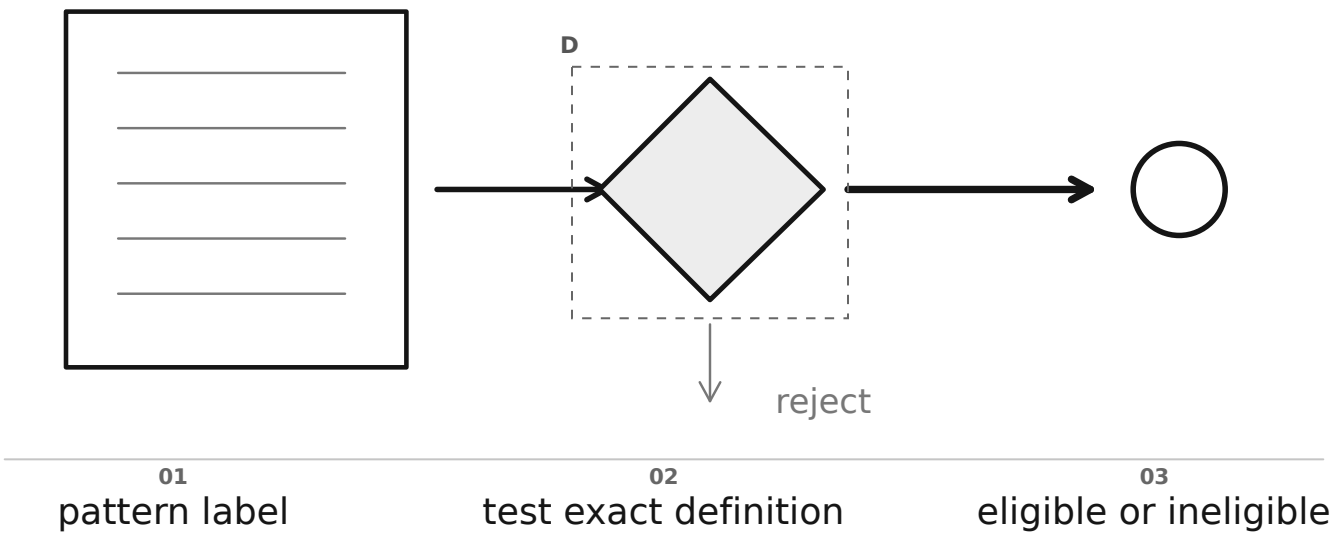
The historical class lacks a real parser and uses an unrelated parity placeholder for double APs. Its general k-AP detector can accept an incomplete progression; later versions repair only some defects.

ORIGINAL FINDING / AUDITED/CORRECTED

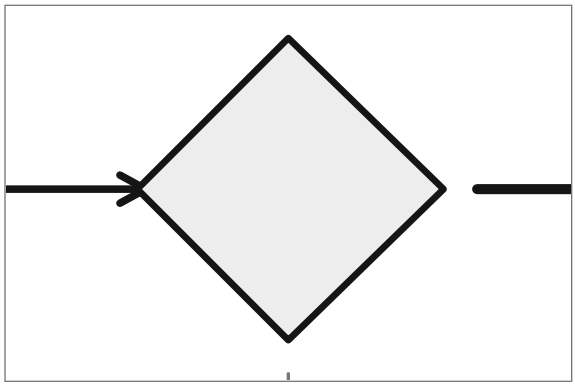
Historical code uses placeholders and incomplete language semantics; original filter claims do not establish a cryptographic advantage.

M127

Pattern-label eligibility correction



D / OPERATION DETAIL



Vector detail / 1.37× main-view scale

HOW THE OPERATION WORKS

Keep pattern classification as metadata while preserving eligibility of every integer in the advertised interval.

READ THE THREE STAGES

01 pattern label
02 test exact definition
03 eligible or ineligible

WHAT WAS ACTUALLY DONE

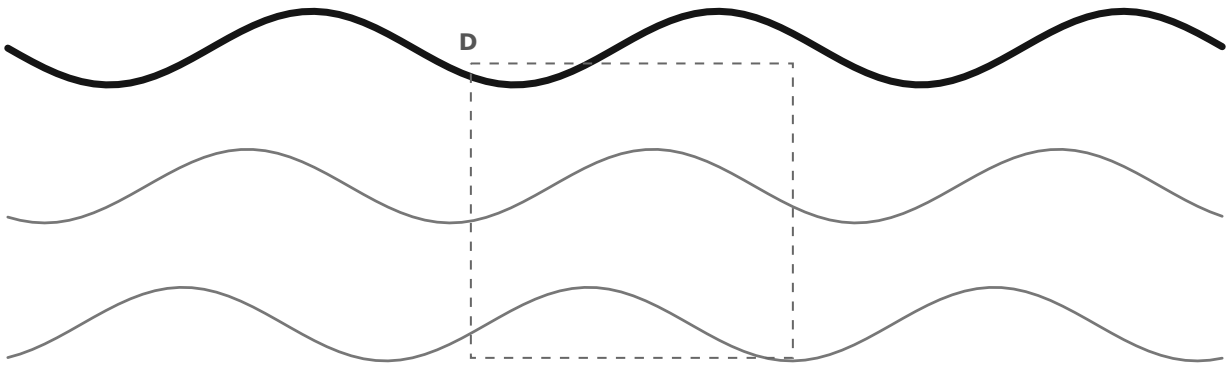
Educational and corrected pipelines do not discard a candidate for an AP label. Known-key tests separately measure label preferences and full-range eligibility, which must not be conflated.

ORIGINAL FINDING / EXECUTED RECORD

Backtests retain all candidates while reporting pattern labels; eligibility is not evidence that a pattern predicts keys.

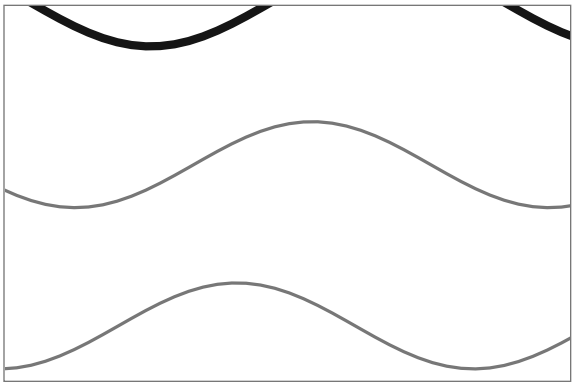
M128

Classical hyper-phase Q1-Q9 formulas



01	02	03
classical inputs	Q1 through Q9 formulas	numeric outputs

D / OPERATION DETAIL



Vector detail / 1.17× main-view scale

HOW THE OPERATION WORKS

Evaluate trigonometric/exponential recurrence formulas indexed by state and bit position, then compare a magnitude with a chosen threshold.

READ THE THREE STAGES

01 classical inputs
02 Q1 through Q9 formulas
03 numeric outputs

WHAT WAS ACTUALLY DONE

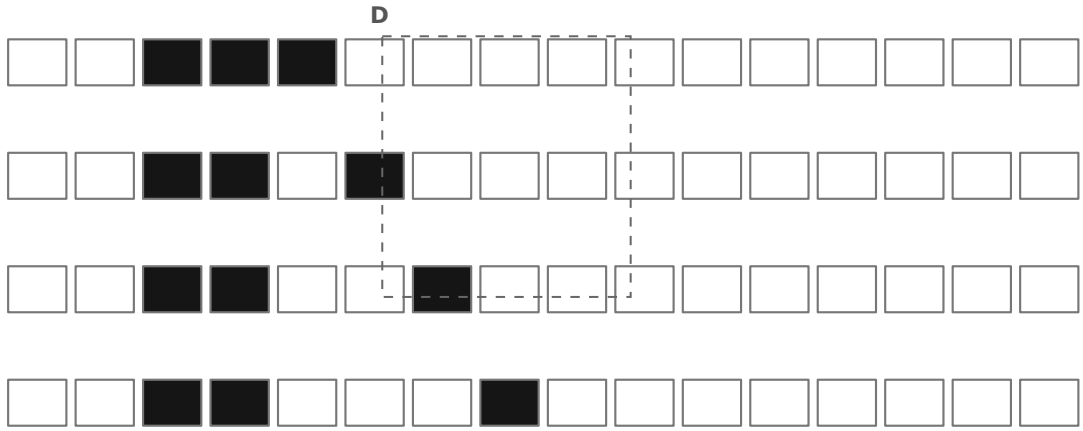
These are classical numerical calculations. Earlier versions omit states; later formulas fill them but still neither ingest the target hash nor construct a quantum circuit or valid key-extraction equation.

ORIGINAL FINDING / AUDITED/CORRECTED

Numerical toy formulas have no demonstrated target relation; no quantum circuit or recovery is implemented.

M129

Bit-position periodic table / CFSG labels

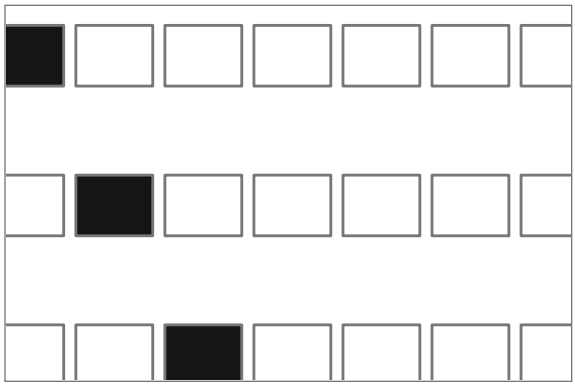


01 bit positions

02 periodic / CFSG labels

03 classification scheme

D / OPERATION DETAIL



Vector detail / 1.32× main-view scale

HOW THE OPERATION WORKS

Organize bit indices into a table with powers of two, symbolic families and heuristic scores. Such a table can be an interface for diagnostics.

READ THE THREE STAGES

01 bit positions
02 periodic / CFSG labels
03 classification scheme

WHAT WAS ACTUALLY DONE

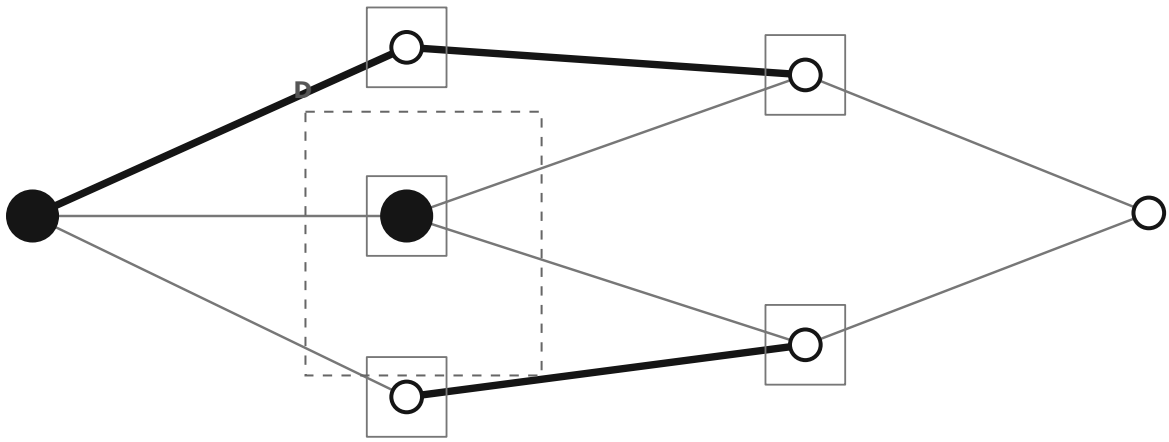
The supplied transcript assigns random simulated biases and CFSG-inspired labels. For bits below 71, reducing 2^b modulo the large group order changes nothing; no hidden algebraic advantage follows.

ORIGINAL FINDING / CONVERSATION PROPOSAL

Supplied transcript uses simulated random biases and organizational labels; no local validated cryptanalytic result was established.

M130

BPK-TSP and million-process architecture

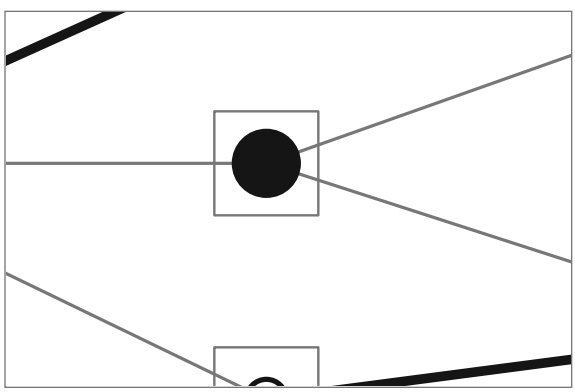


01 BPK-TSP jobs

02 many process lanes

03 architecture proposal

D / OPERATION DETAIL



Vector detail / 1.30× main-view scale

HOW THE OPERATION WORKS

A scheduler could distribute generation, verification and artifact collection across workers, with hardware-specific executors.

READ THE THREE STAGES

01 BPK-TSP jobs
02 many process lanes
03 architecture proposal

WHAT WAS ACTUALLY DONE

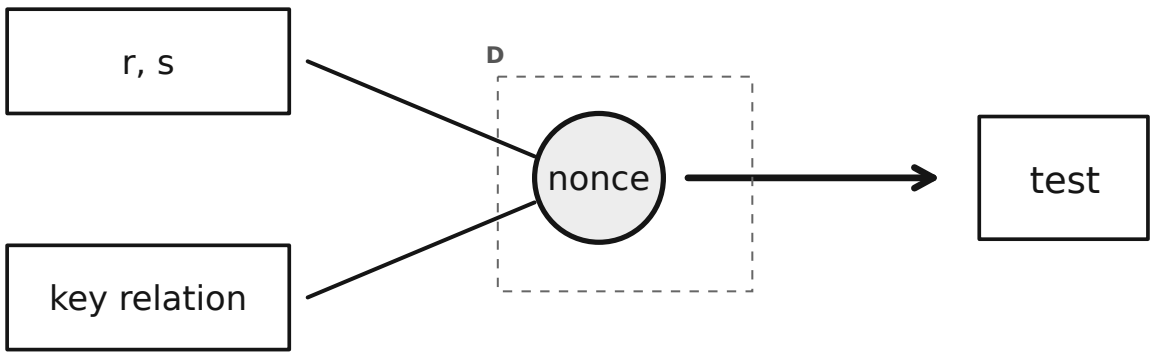
BPK-TSP, million-process and FPGA claims are architectural narrative in the supplied material. Concrete implementations elsewhere are documented separately; this entry has no corroborated complete large-scale execution.

ORIGINAL FINDING / CONVERSATION PROPOSAL

Scheduling and FPGA/massive-process claims in supplied reports are proposals without corroborated execution evidence in this inventory.

M131

Aranyn Block-1 manual-nonce binding

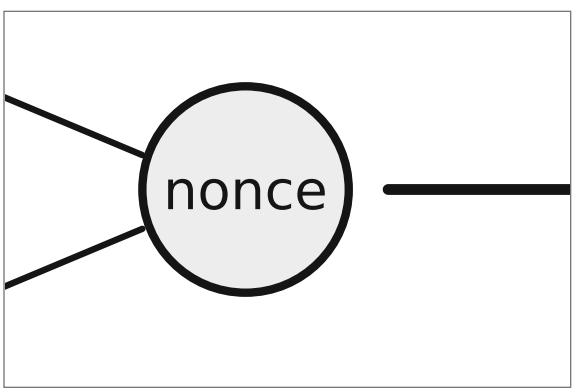


01 Block-1 signature

02 manual nonce hypothesis

03 binding equation check

D / OPERATION DETAIL



Vector detail / 1.60× main-view scale

HOW THE OPERATION WORKS

ECDSA gives s*w=z+r*d modulo the group order when the signing nonce is chosen as the Block-1 scalar w.

READ THE THREE STAGES

01 Block-1 signature
02 manual nonce hypothesis
03 binding equation check

WHAT WAS ACTUALLY DONE

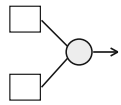
The code verifies the signature and derives w=A+B*d. Substituting that relation back into the same signature adds no independent equation; one unknown scalar remains behind public-point multiplication.

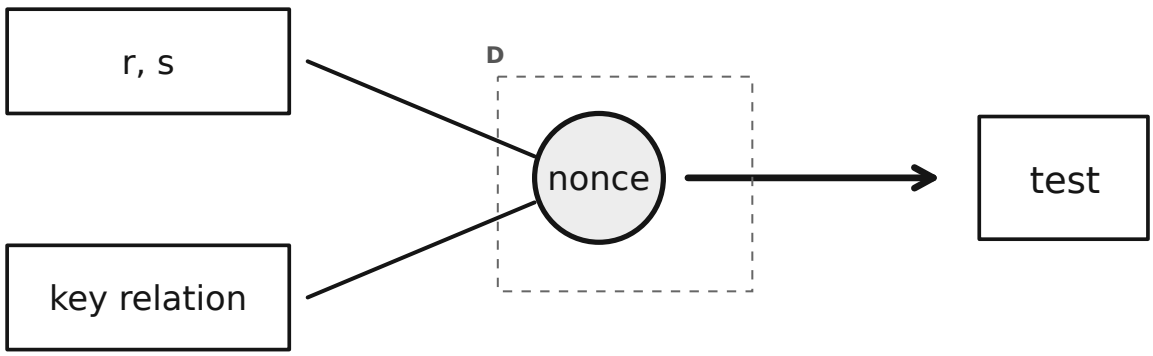
ORIGINAL FINDING / EXECUTED RECORD

One signature gives an affine relation between two scalar names but leaves an ECDLP-equivalent unknown.

M132

Puzzle-160 nonce/signing-key relation



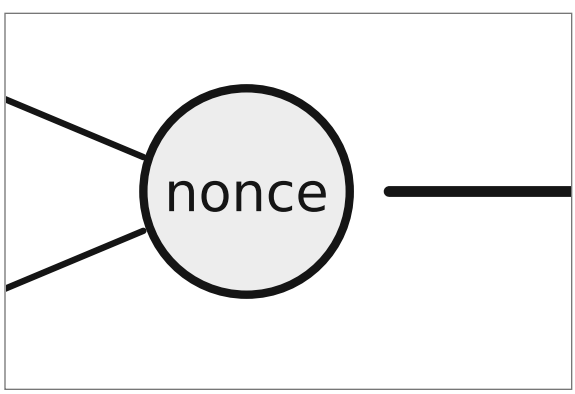


01 puzzle-160 signature

02 nonce / signing-key relation

03 public-point checks

D / OPERATION DETAIL



Vector detail / 1.60× main-view scale

HOW THE OPERATION WORKS

If an ECDSA nonce is the Puzzle-160 key, knowledge of either that nonce or the signer scalar determines the other algebraically.

READ THE THREE STAGES

01 puzzle-160 signature
02 nonce / signing-key relation
03 public-point checks

WHAT WAS ACTUALLY DONE

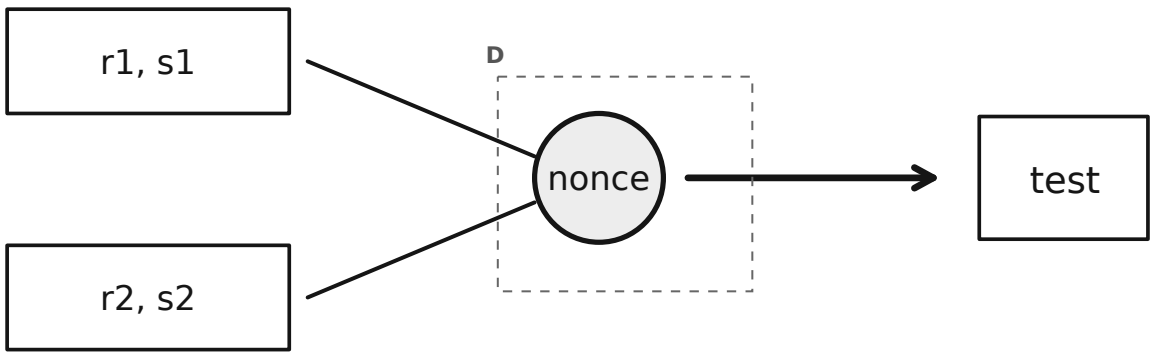
user_signature.py supplies the forward/reverse formulas and known public-point checks. Neither published relation alone reveals an undisclosed nonce; solving for it still requires new information or ECDLP work.

ORIGINAL FINDING / IMPLEMENTED

Recover one scalar if the other is supplied; the published relation does not independently reveal either unknown key.

M133

Cross-signature #160 / Block-1 linear relations

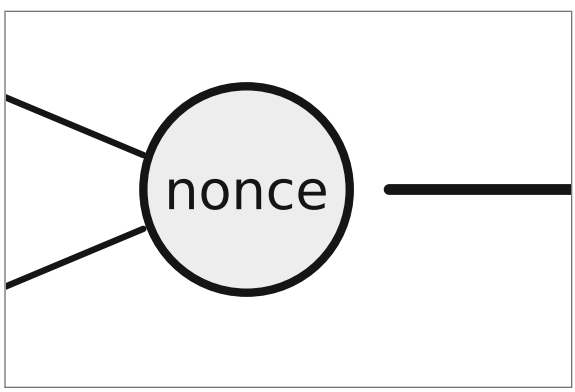


01
two artifact signatures

02
proposed linear relation

03
compatibility test

D / OPERATION DETAIL



Vector detail / 1.60× main-view scale

HOW THE OPERATION WORKS

Combine the two affine signature relations and test small coefficients directly on their public points.

READ THE THREE STAGES

01 two artifact signatures
02 proposed linear relation
03 compatibility test

WHAT WAS ACTUALLY DONE

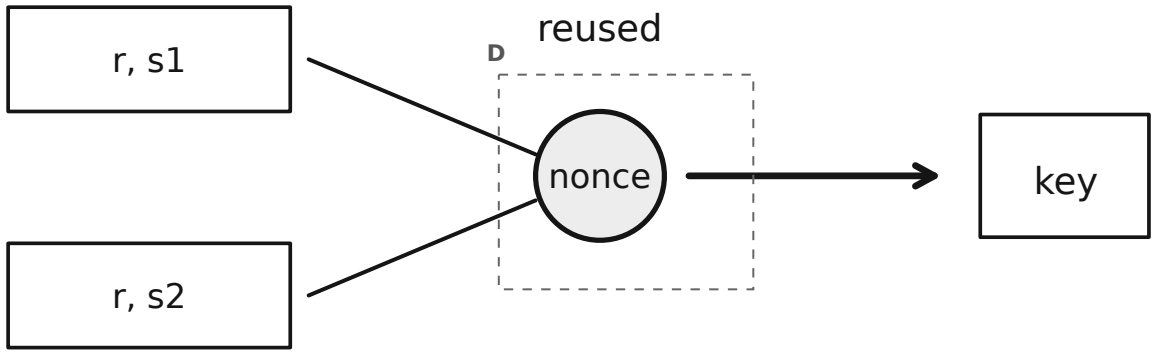
The relation search sweeps specified alpha/beta/gamma families and verifies candidate equations. This can detect a genuine small relation without knowing either scalar, but cannot assume one exists.

ORIGINAL FINDING / IMPLEMENTED

Search bounded small coefficients visible on public points; any result would need independent target verification.

M134

ECDSA nonce-reuse recovery

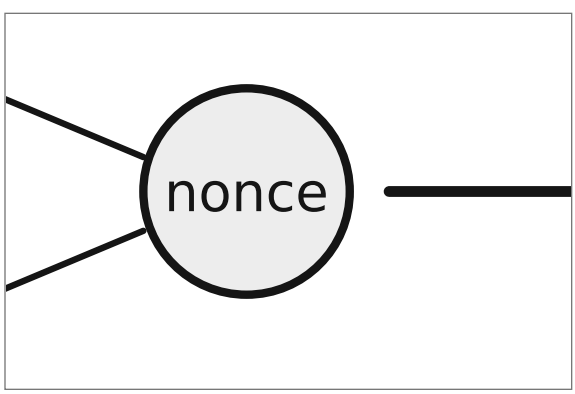


01 two signatures

02 same nonce / matching r

03 algebraic key recovery

D / OPERATION DETAIL



Vector detail / 1.60× main-view scale

HOW THE OPERATION WORKS

Reusing the same nonce for different messages creates two equations with the same unknown nonce, allowing elimination and scalar recovery.

READ THE THREE STAGES

01 two signatures
02 same nonce / matching r
03 algebraic key recovery

WHAT WAS ACTUALLY DONE

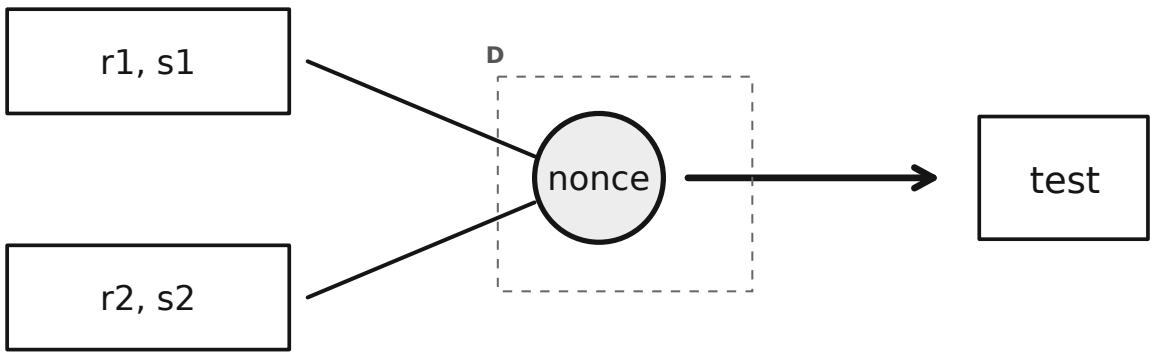
Synthetic sign/recover tests validate the algebra. Real harvested data are grouped by signer and r; the archived samples did not provide the required exploitable reuse pair.

ORIGINAL FINDING / SYNTHETIC/RECORDED TESTS

Algebraic recovery works in planted examples; harvested real signatures showed no matching reuse groups in recorded samples.

M135

Known affine-related nonce recovery

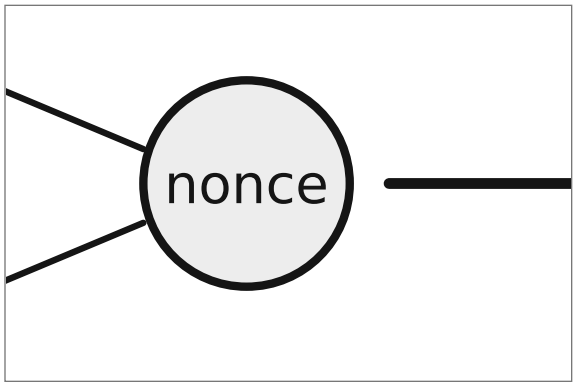


01 nonce pair

02 affine relation

03 linear-system recovery

D / OPERATION DETAIL



Vector detail / 1.60× main-view scale

HOW THE OPERATION WORKS

A known relation k2=a*k1+b between two ECDSA nonces supplies an extra equation that makes the signer scalar solvable.

READ THE THREE STAGES

01 nonce pair
02 affine relation
03 linear-system recovery

WHAT WAS ACTUALLY DONE

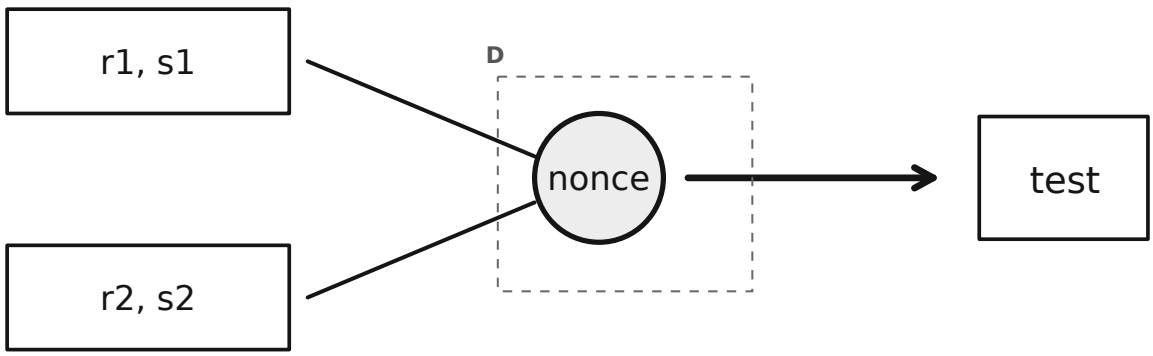
ecdsa_affine_nonce.py creates controlled affine signature pairs and recovers the planted key. The puzzle target does not gain that relation merely because the implementation works on planted examples.

ORIGINAL FINDING / SYNTHETIC TESTS

Synthetic signature pairs validate the formula when the nonce relation is actually known. Applicability requires such a pair.

M136

Counter nonces and small offset relation tests

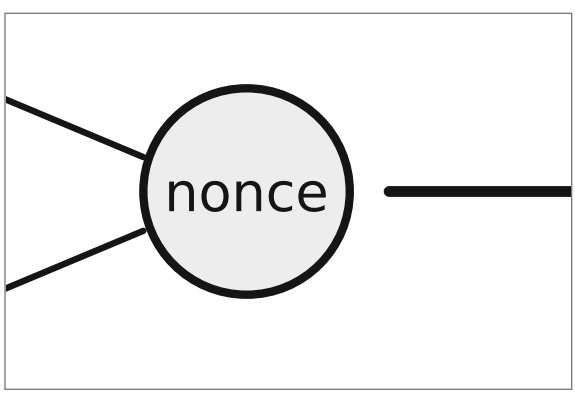


01 counter nonces

02 small offset choices

03 relation checks

D / OPERATION DETAIL



Vector detail / 1.60× main-view scale

HOW THE OPERATION WORKS

Guess tiny nonce values or small differences between two nonce values, then use ECDSA equations to derive a signer candidate.

READ THE THREE STAGES

01 counter nonces
02 small offset choices
03 relation checks

WHAT WAS ACTUALLY DONE

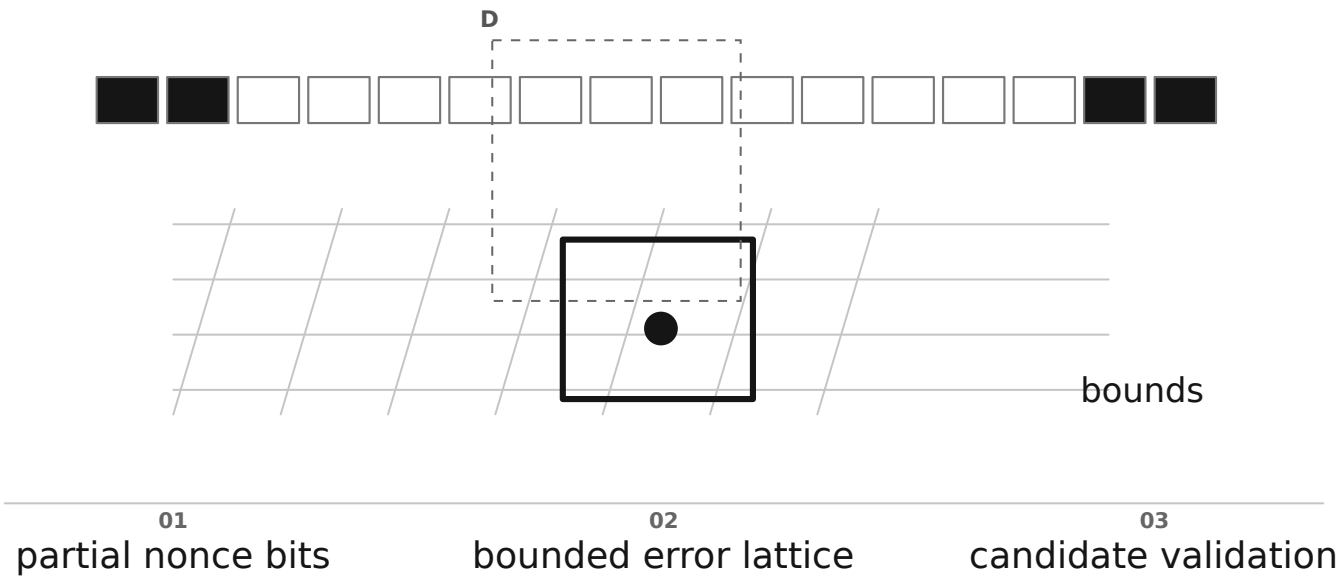
Recorded scans test declared nonce and offset ranges against harvested signatures and verify candidates. A null result covers those guesses, not arbitrary unknown nonce relations.

ORIGINAL FINDING / EXECUTED RECORD

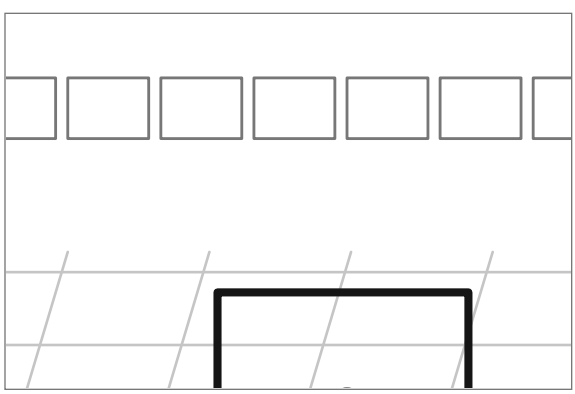
Archived harvested-signature scans tested tiny nonces and bounded offsets; no verified wallet-key recovery reported.

M137

Hidden-number/partial-MSB nonce leakage models



D / OPERATION DETAIL



Vector detail / 1.32× main-view scale

HOW THE OPERATION WORKS

Multiple signatures with known nonce bits create inequalities that can support Hidden Number Problem/lattice recovery or smaller interval searches.

READ THE THREE STAGES

01 partial nonce bits
02 bounded error lattice
03 candidate validation

WHAT WAS ACTUALLY DONE

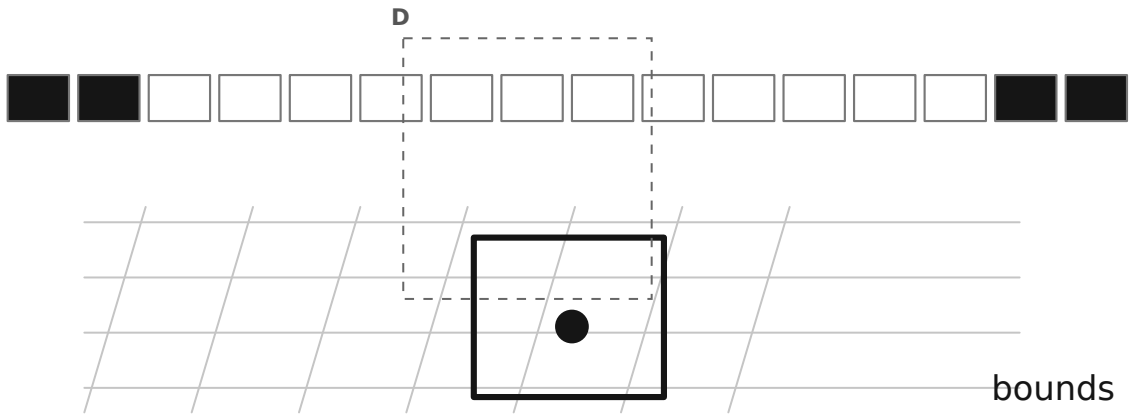
The registry models synthetic MSB leaks and required bit budgets. The available target data do not include the corresponding independent bit leaks or adequate applicable signatures.

ORIGINAL FINDING / SYNTHETIC/APPLICABILITY

Synthetic bounds and applicability checks exist; the target signatures do not supply the required leakage measurements.

M138

Polynonce/half-half/side-channel possibilities

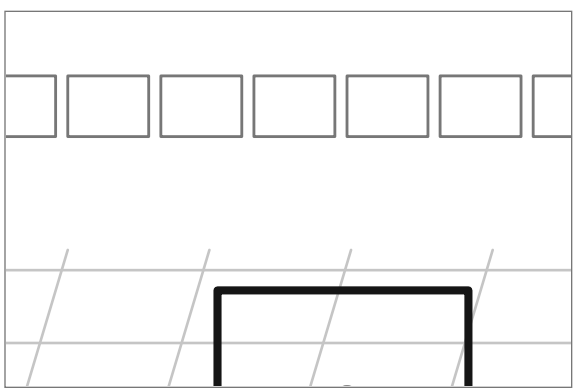


01 structured nonce model

02 additional leakage needed

03 recovery hypothesis

D / OPERATION DETAIL



Vector detail / 1.32× main-view scale

HOW THE OPERATION WORKS

Polynomial nonce recurrences, concatenated half-nonces or measured side-channel leakage provide special extra structure beyond ordinary ECDSA.

READ THE THREE STAGES

01 structured nonce model
02 additional leakage needed
03 recovery hypothesis

WHAT WAS ACTUALLY DONE

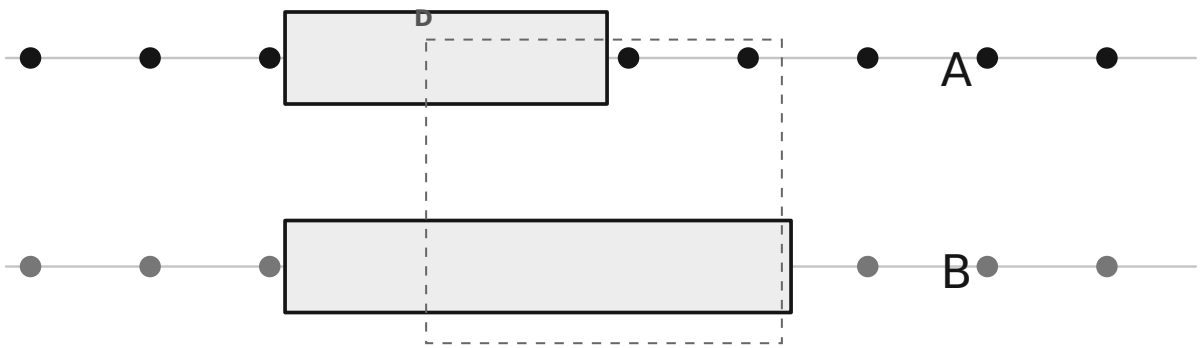
The literature mapping checks whether those prerequisites exist. This is an applicability review; it does not execute a successful lattice/side-channel attack on a single ordinary signature.

ORIGINAL FINDING / APPLICABILITY REVIEW

Reviewed as conditional attacks; missing signatures, special nonce structure or measurements block their use on the cited target.

M139

ECDSA versus Schnorr/BIP340 checks

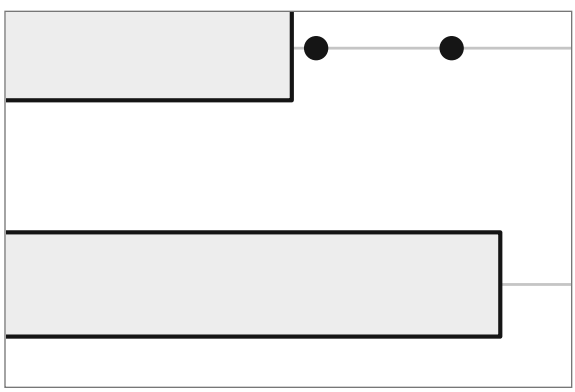


01 ECDSA equations

02 Schnorr equations

03 scheme compatibility

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Schnorr and ECDSA use different equations and message challenges. A signature must satisfy the correct scheme's verification relation.

READ THE THREE STAGES

01 ECDSA equations
02 Schnorr equations
03 scheme compatibility

WHAT WAS ACTUALLY DONE

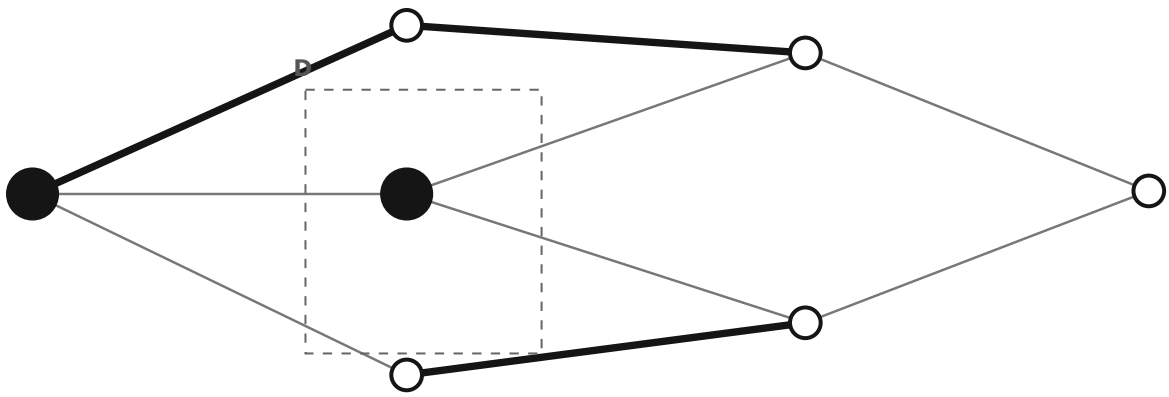
The comparison derives Schnorr/BIP340 challenges and tests forced interpretations of the ECDSA constants. Reinterpreting bytes under another scheme does not provide a second valid signature equation.

ORIGINAL FINDING / EXECUTED RECORD

Forced reinterpretations and challenge consistency are compared; changing signature labels does not create a new independent equation.

M140

Early coinbase signature harvesting

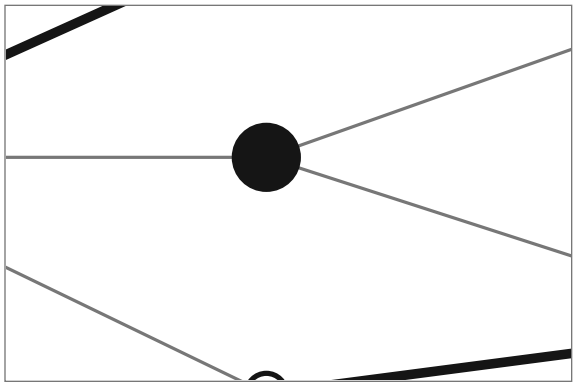


01 early outputs

02 spending signatures

03 collect records

D / OPERATION DETAIL



Vector detail / 1.30× main-view scale

HOW THE OPERATION WORKS

Extract signature data from spent early coinbase outputs and identify the signer/public point needed for nonce analysis.

READ THE THREE STAGES

01 early outputs
02 spending signatures
03 collect records

WHAT WAS ACTUALLY DONE

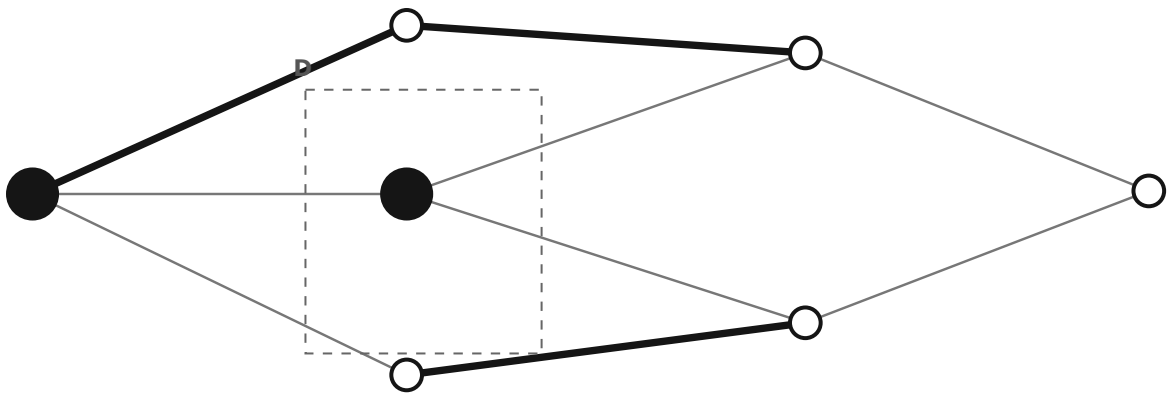
Harvesters follow actual spends, parse signatures and group them for reuse/target-r tests. The saved 51-signature sample provides data, not previously unknown recovered wallet keys.

ORIGINAL FINDING / EXECUTED RECORD

Saved report records 51 signatures and no nonce-reuse recovery; the graph is a data source, not a solve.

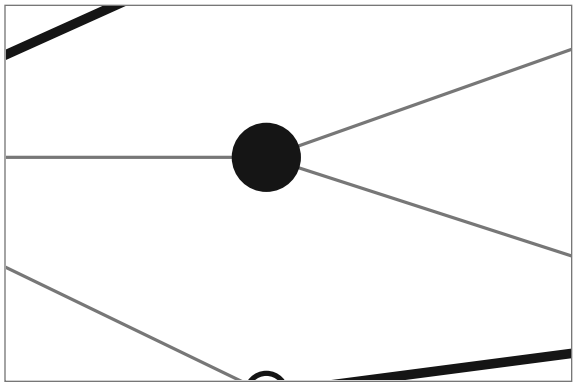
M141

Descendant/graph signature expansion



01	02	03
seed transactions	descendant traversal	expand signature corpus

D / OPERATION DETAIL



Vector detail / 1.30× main-view scale

HOW THE OPERATION WORKS

Follow descendants and consolidation transactions to collect more signatures associated with the original outputs.

READ THE THREE STAGES

01 seed transactions
02 descendant traversal
03 expand signature corpus

WHAT WAS ACTUALLY DONE

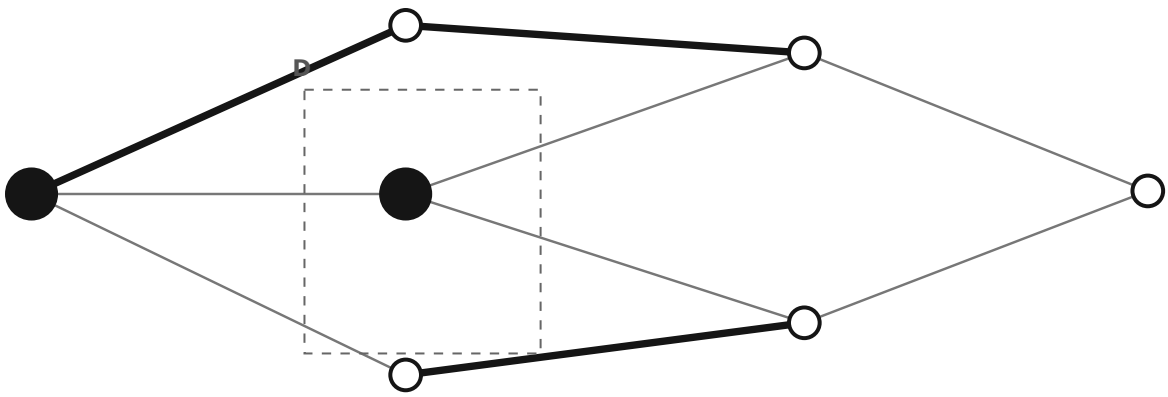
Graph/descendant tools expand a bounded transaction frontier. Additional transaction relationships can increase the sample but do not prove that every downstream signer shares a private scalar.

ORIGINAL FINDING / IMPLEMENTED

Expand UTXO descendants and consolidation transactions to obtain additional signature samples.

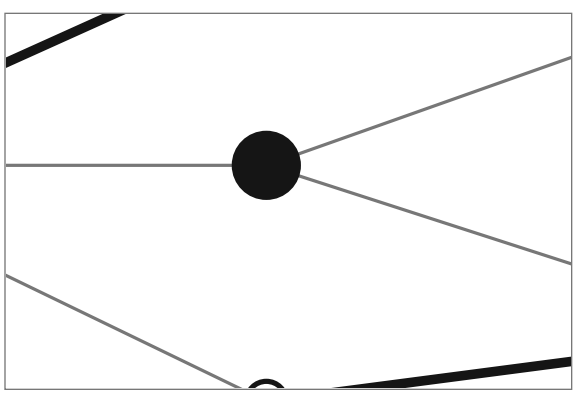
M142

BTCparser sleeping/whale sweep signatures



01	02	03
old / whale sweeps	transaction graph	extract signatures

D / OPERATION DETAIL



Vector detail / 1.30× main-view scale

HOW THE OPERATION WORKS

Use sleeping-address or whale-sweep indexes to select transactions likely to contain useful signature collections.

READ THE THREE STAGES

01 old / whale sweeps
02 transaction graph
03 extract signatures

WHAT WAS ACTUALLY DONE

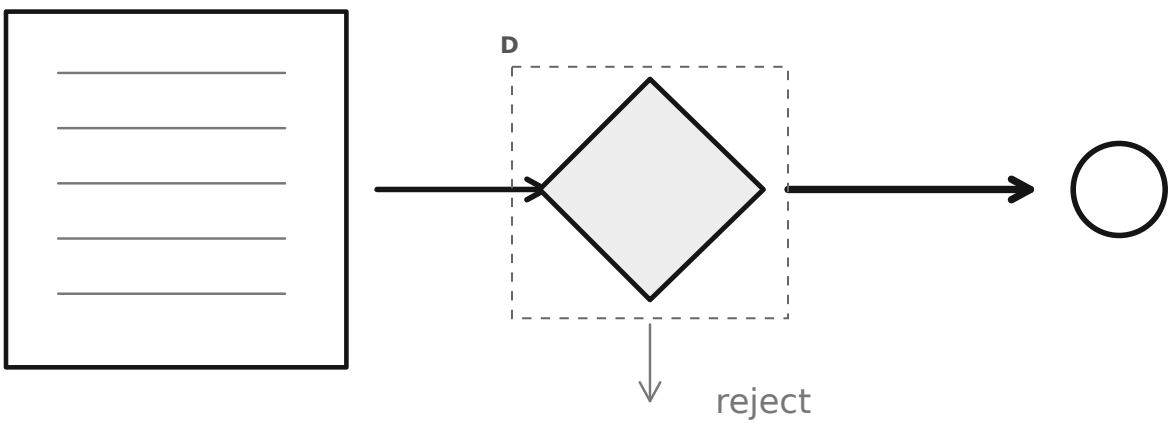
The BTCparser sweep parses indexed blocks and selected spend transactions, then tests collected r values. It is a bounded data-gathering strategy, not evidence of biased nonces.

ORIGINAL FINDING / EXECUTED RECORD

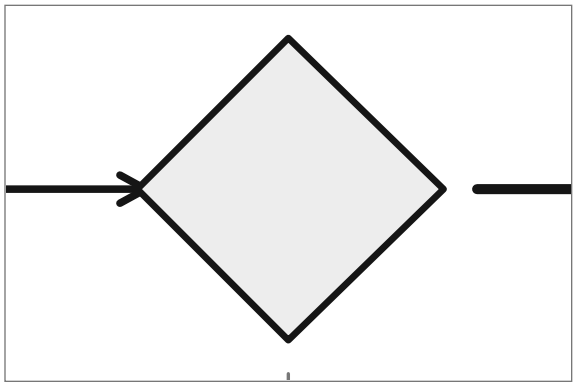
Saved sweep harvested 151 signatures with no reuse groups or target-r hits in that sample.

M143

Lattice-candidate sufficiency scan



D / OPERATION DETAIL



Vector detail / 1.37× main-view scale

HOW THE OPERATION WORKS

Count usable signatures per signer before attempting attacks that require repeated nonce information or leakage.

READ THE THREE STAGES

01 per-public-key corpus
02 nonce-data sufficiency
03 lattice eligibility

WHAT WAS ACTUALLY DONE

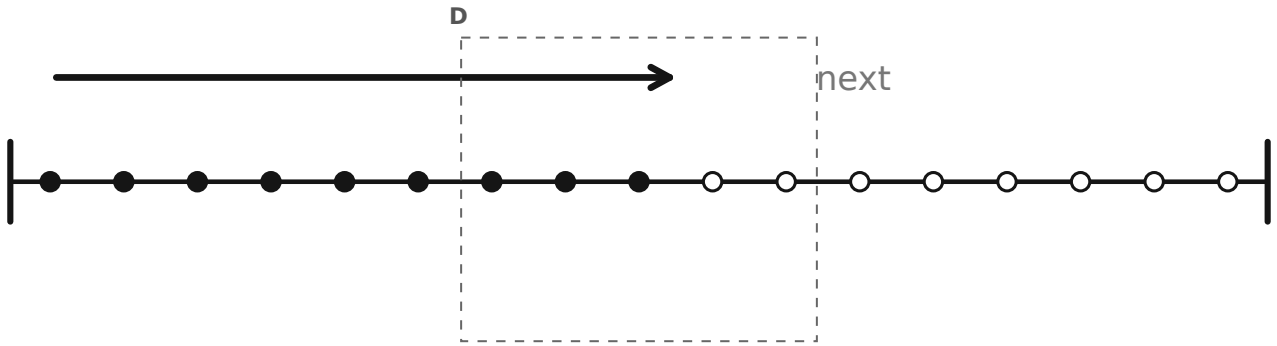
The lattice-candidate report groups 184 signatures into wallets and finds none meeting its multi-signature threshold. No target lattice recovery is performed simply by detecting that data are insufficient.

ORIGINAL FINDING / EXECUTED RECORD

Saved report has 184 signatures but no wallet with the required multi-signature count; no lattice recovery executed.

M144

SegWit/testnet target-r scans

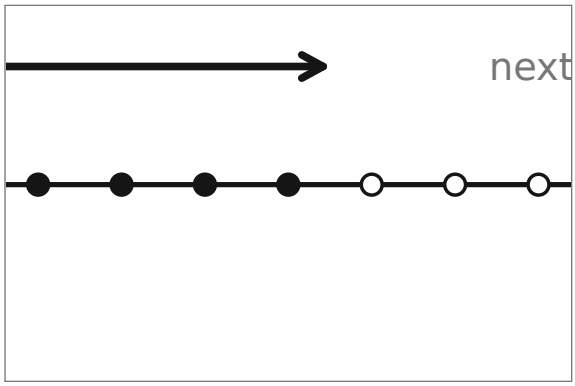


01 witness / testnet data

02 target-r comparison

03 matching records

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Search additional witness and testnet signature data for the same nonce x-coordinate or related target public material.

READ THE THREE STAGES

01 witness / testnet data
02 target-r comparison
03 matching records

WHAT WAS ACTUALLY DONE

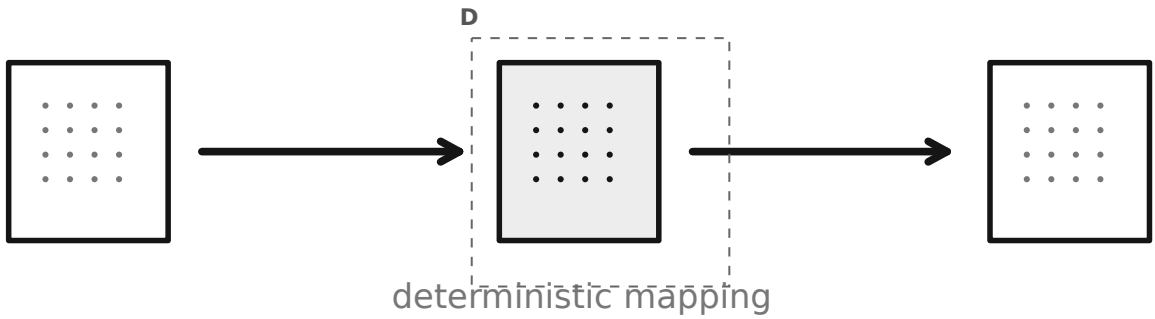
Dedicated scans compare parsed r values in their fetched samples. Matching a signature field would need scheme and signer verification; absence in a sample is not a global chain proof.

ORIGINAL FINDING / EXECUTED RECORD

Search archived alternate-chain and witness signatures for the manual-nonce x-coordinate; finite scan scope only.

M145

Signed-message and fork public-key leaks

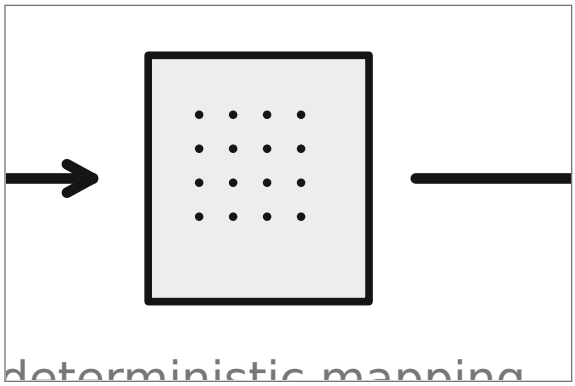


01 message / fork spend

02 signature verification

03 revealed public point

D / OPERATION DETAIL



Vector detail / 1.38× main-view scale

HOW THE OPERATION WORKS

Extra signed messages or fork spends may reveal a public point or an independently usable signature absent from the current target record.

READ THE THREE STAGES

01 message / fork spend
02 signature verification
03 revealed public point

WHAT WAS ACTUALLY DONE

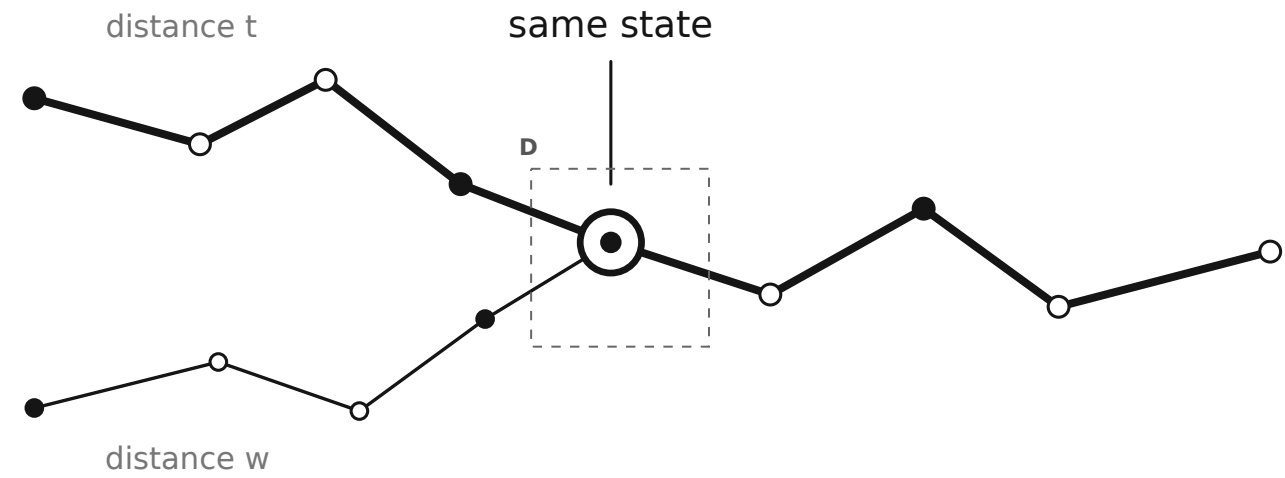
Search scripts look for that public material. They do not obtain private keys from a public-point leak alone; they may only enable a different, still expensive recovery method.

ORIGINAL FINDING / IMPLEMENTED

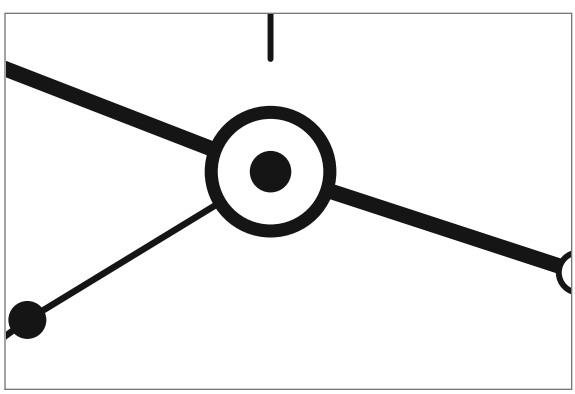
Search additional public signature/pubkey material; utility depends on finding independently applicable data.

M146

DP metadata distances and signed collision variants



D / OPERATION DETAIL



Vector detail / 1.93× main-view scale

HOW THE OPERATION WORKS

Interpret candidate metadata slices as walk distances and test whether they satisfy the claimed tame/wild public-point equations.

READ THE THREE STAGES

01 parsed DP distances
02 signed walk equation
03 candidate scalar check

WHAT WAS ACTUALLY DONE

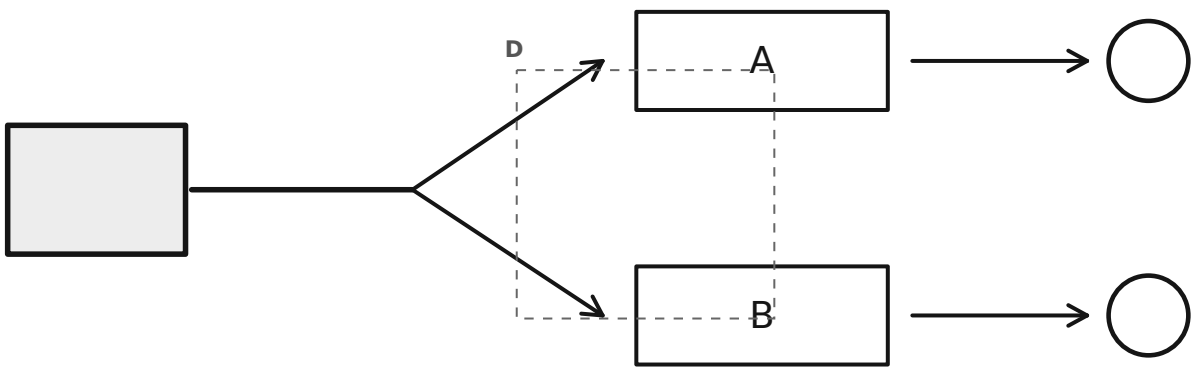
Archived scans vary bit widths, signs, starts and collision formulas. A distance-looking field is rejected unless its derived point or scalar matches the target.

ORIGINAL FINDING / EXECUTED RECORD

Archived metadata-width, start-point, sign and collision-formula tests found no verified puzzle scalar.

M147

DP x-coordinates as candidate public keys

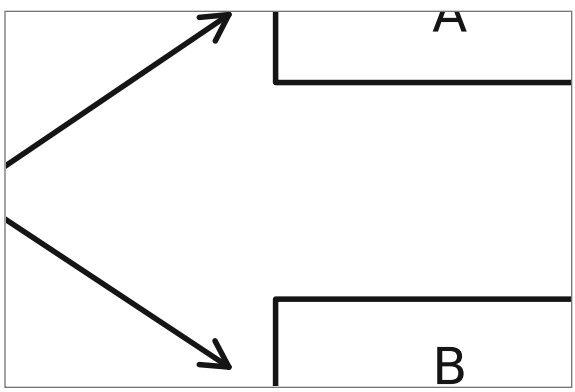


01 candidate x value

02 two possible y lifts

03 compressed public keys

D / OPERATION DETAIL



Vector detail / 1.38× main-view scale

HOW THE OPERATION WORKS

A valid x-coordinate has up to two curve-point lifts, each yielding a possible compressed public key and address hash.

READ THE THREE STAGES

01 candidate x value
02 two possible y lifts
03 compressed public keys

WHAT WAS ACTUALLY DONE

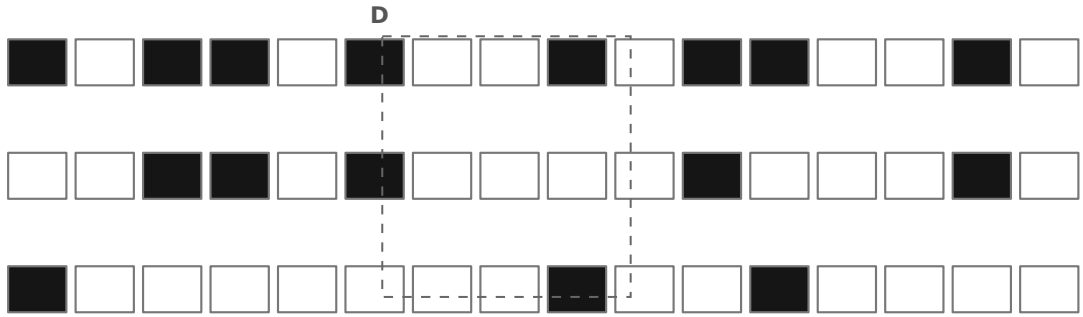
The binary audit checks both parities against puzzle targets. This tests whether the dump literally contains the target public key, not whether arbitrary points can be inverted into scalars.

ORIGINAL FINDING / EXECUTED RECORD

Both point parities were checked against puzzle addresses; no match in the recorded binary sample.

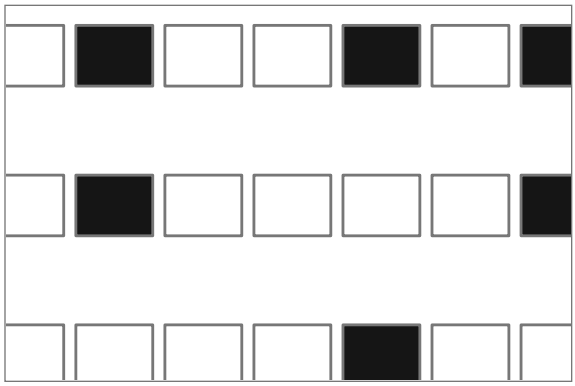
M148

TRAILING / LOW71 / bit-slice anchors



01	02	03
metadata integer	low bits / slices	interval anchor

D / OPERATION DETAIL



Vector detail / 1.32× main-view scale

HOW THE OPERATION WORKS

Slice or reinterpret a trailing metadata integer to produce anchor scalars for selected puzzle widths.

READ THE THREE STAGES

01 metadata integer
02 low bits / slices
03 interval anchor

WHAT WAS ACTUALLY DONE

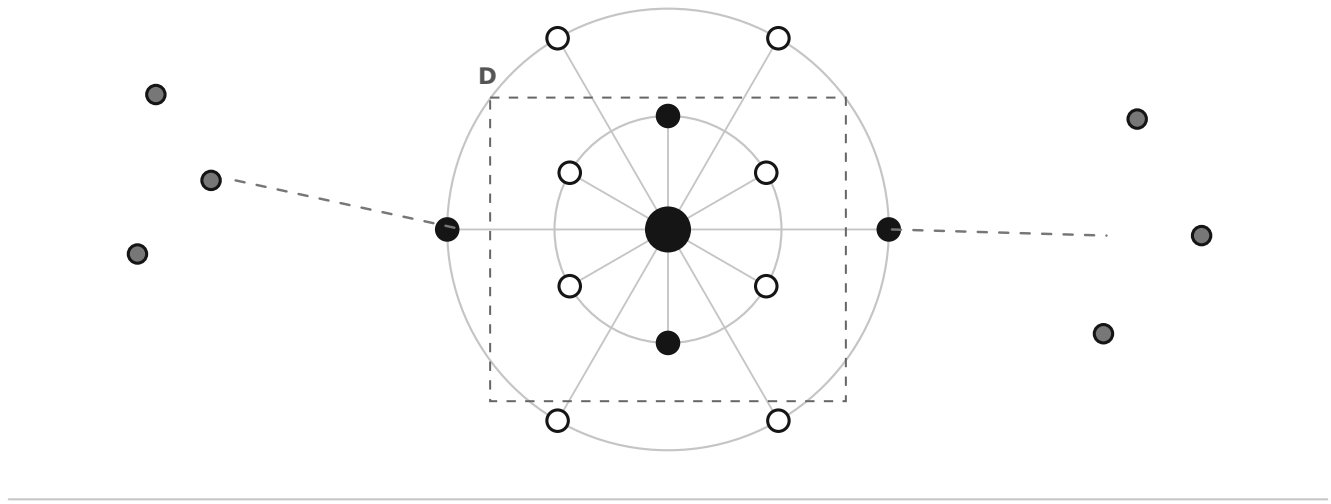
LOW71, low/high pieces and literal TRAILING values are checked as concrete candidates. Their bit length or range membership alone is not evidence of target identity.

ORIGINAL FINDING / EXECUTED RECORD

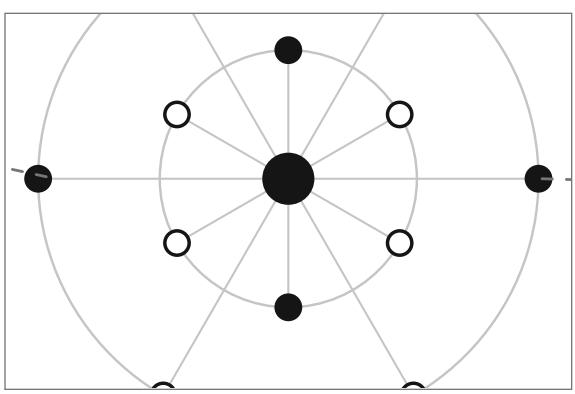
Numerical range coincidence is separate from address correctness; recorded anchor checks failed target verification.

M149

Leading-zero public-key collection analysis



D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Analyze a collection selected for leading-zero x-coordinate masks, then search membership, pairwise offsets and small public-point relations.

READ THE THREE STAGES

01 leading-zero points
02 pairwise point offsets
03 relation tests

WHAT WAS ACTUALLY DONE

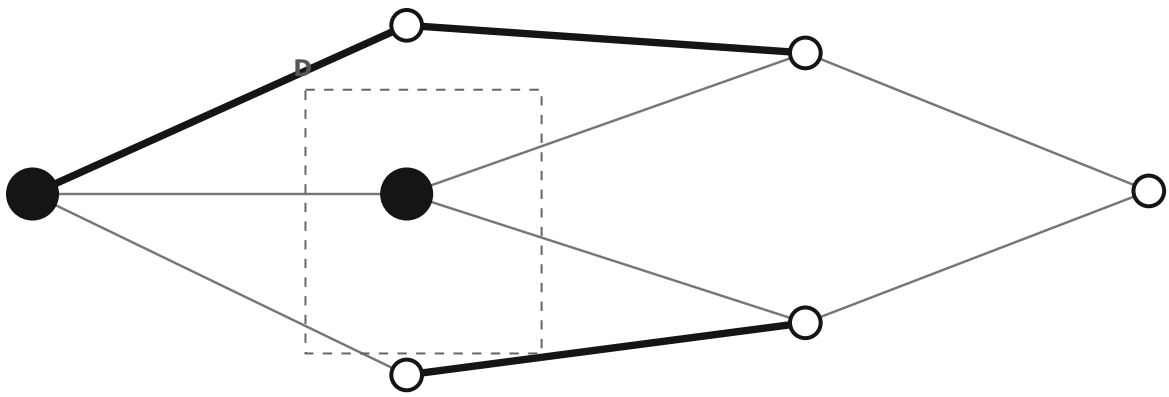
The script compares known corpora, GLV transforms, linear combinations, bounded BSGS differences and generator recipes. Mask rarity must be evaluated conditional on the deliberate point-selection rule.

ORIGINAL FINDING / IMPLEMENTED

Tests mask statistics, membership, pairwise differences, GLV relations, linear combinations and bounded BSGS offsets.

M150

Cross-artifact relation bundles

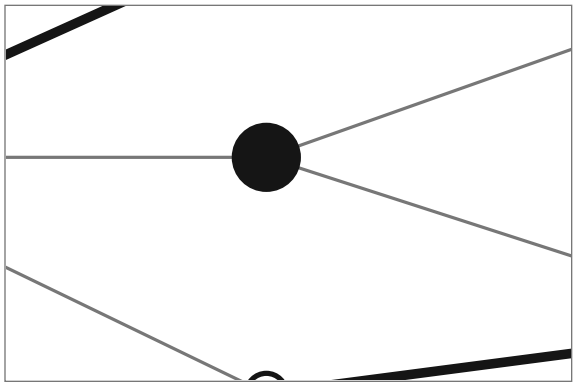


01 hashed artifacts

02 named evidence edges

03 traceable relation bundle

D / OPERATION DETAIL



Vector detail / 1.30× main-view scale

HOW THE OPERATION WORKS

Bundle source hashes, parsed values and named relationships so later investigations can trace every clue to its artifact.

READ THE THREE STAGES

01 hashed artifacts
02 named evidence edges
03 traceable relation bundle

WHAT WAS ACTUALLY DONE

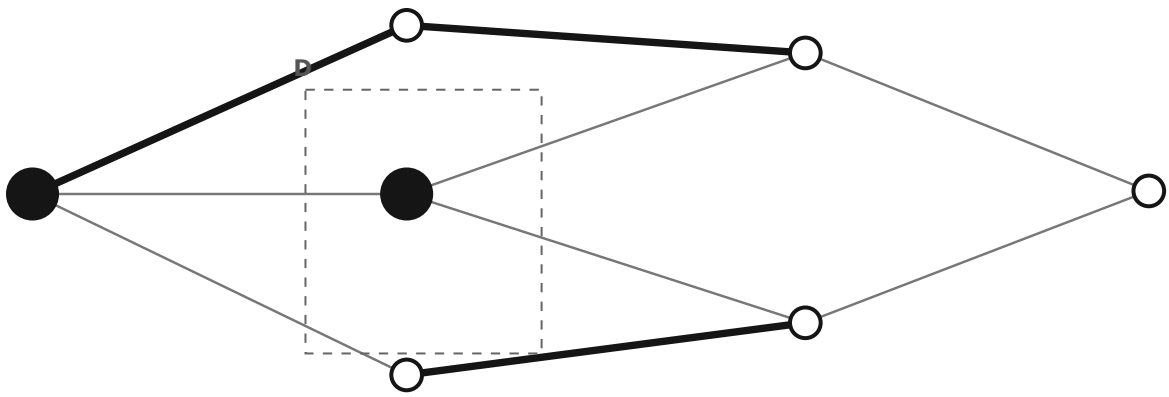
relation_bundle.py packages DP/TRAILING/CTheory hypotheses and metadata. It preserves evidence lineage without claiming that colocated numbers imply a key equation.

ORIGINAL FINDING / IMPLEMENTED

Package DP, metadata and CTheory evidence without claiming that correlated artifacts reveal a key.

M151

Genesis/early-coinbase relationship graph

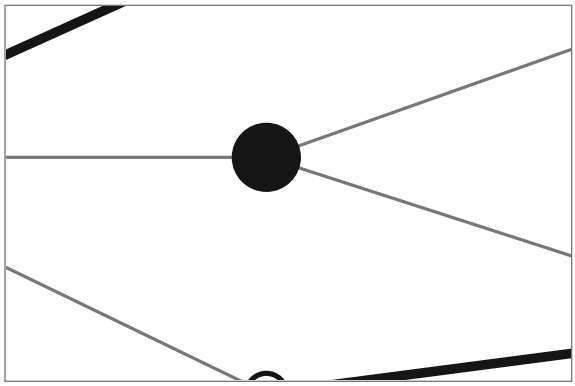


01 early blocks

02 outputs and spends

03 puzzle-linked graph

D / OPERATION DETAIL



Vector detail / 1.30× main-view scale

HOW THE OPERATION WORKS

Build a graph linking early blocks, output public keys, spends and known puzzle entities.

READ THE THREE STAGES

01 early blocks
02 outputs and spends
03 puzzle-linked graph

WHAT WAS ACTUALLY DONE

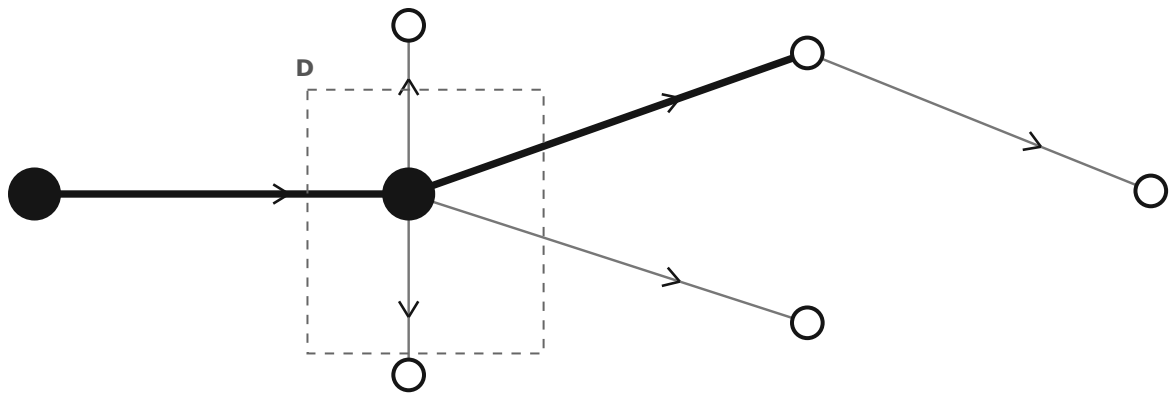
The relationship mapper collects public points and transaction edges. A shared transaction or funder is a graph link; recovering a shared scalar requires separate cryptographic evidence.

ORIGINAL FINDING / EXECUTED RECORD

Map published public points and spending relationships; shared transactions need not imply shared private keys.

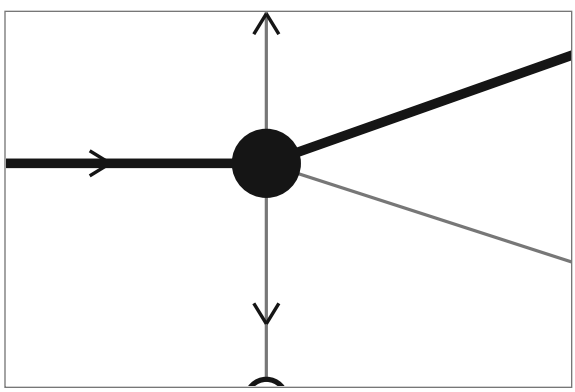
M152

Puzzle-135 sweep/fanout tracing



01	02	03
puzzle-135 spend	forward / reverse edges	related destinations

D / OPERATION DETAIL



Vector detail / 1.30× main-view scale

HOW THE OPERATION WORKS

Follow the known Puzzle-135 spending destination forward and backward to identify related addresses or messages.

READ THE THREE STAGES

01 puzzle-135 spend
02 forward / reverse edges
03 related destinations

WHAT WAS ACTUALLY DONE

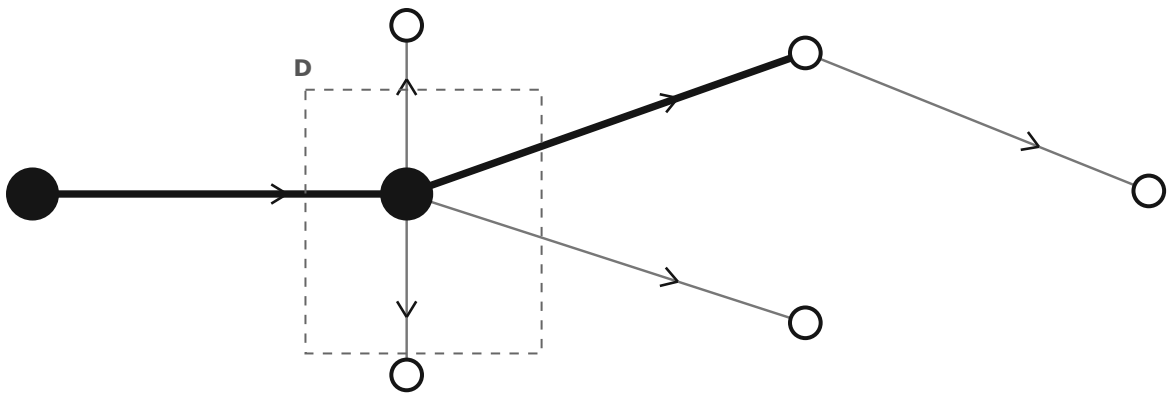
The fanout script tags puzzle neighbors and summarizes transaction flows. It tracks an observed sweep; it does not establish that this local code discovered the spent key.

ORIGINAL FINDING / EXECUTED RECORD

Follow sweep destinations, funding and tagged puzzle neighbors; movement of funds is not local proof of solving the puzzle.

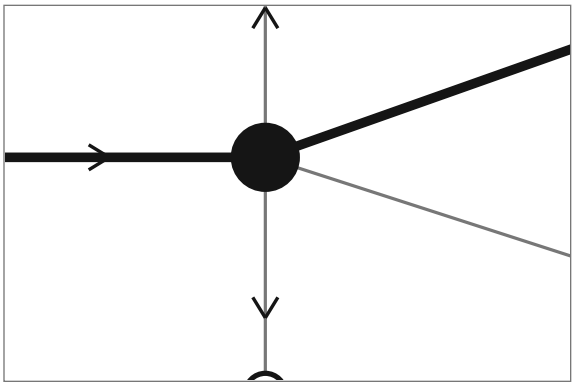
M153

0.99-BTC fanout cluster tracing



01	02	03
funding inputs	0.99-BTC outputs	neighboring cluster

D / OPERATION DETAIL



Vector detail / 1.30× main-view scale

HOW THE OPERATION WORKS

Detect transactions with repeated 0.99-BTC outputs and traverse their funding/fanout neighbors.

READ THE THREE STAGES

01 funding inputs
02 0.99-BTC outputs
03 neighboring cluster

WHAT WAS ACTUALLY DONE

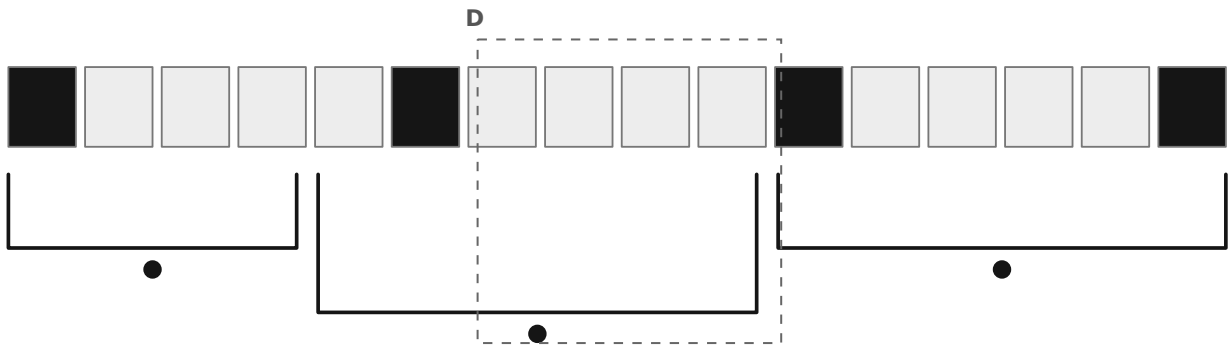
The crawler uses explicit output-count/value thresholds and bounded BFS expansion. This identifies a transaction pattern or cluster, not an algebraic relation among its private keys.

ORIGINAL FINDING / EXECUTED RECORD

Breadth-first funding/output graph searches and message scans are archived.

M154

Sender relay / OP_RETURN / extra-data clues

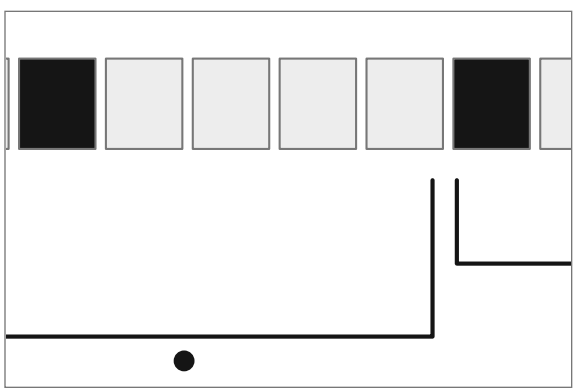


01 transaction payloads

02 relay / OP_RETURN

03 decoded clues

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Decode message-bearing transaction fields and follow a relay chain for possible puzzle instructions or artifacts.

READ THE THREE STAGES

01 transaction payloads
02 relay / OP_RETURN
03 decoded clues

WHAT WAS ACTUALLY DONE

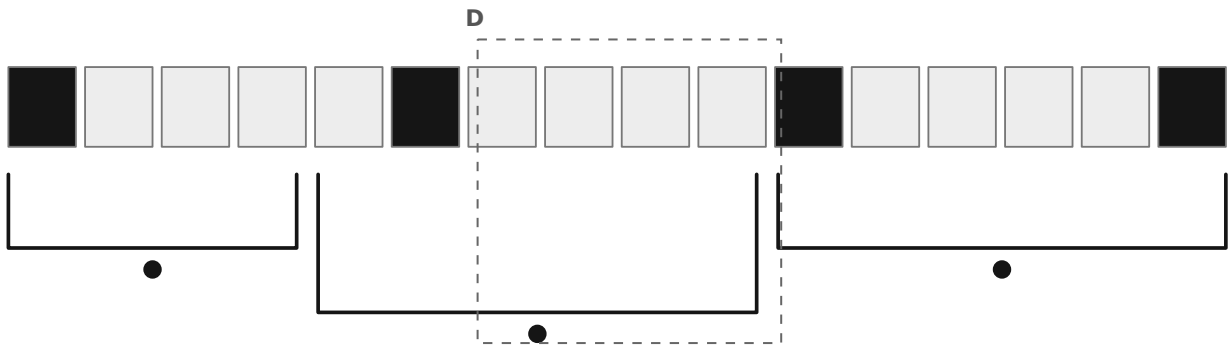
Sender tools parse OP_RETURN, inspect extra data and search references. Public text is treated as a lead requiring verification, not executable instructions or proven secret leakage.

ORIGINAL FINDING / EXECUTED RECORD

Decode message relays and scan transaction fields for key fragments or target relations.

M155

Early spent-block number encodings

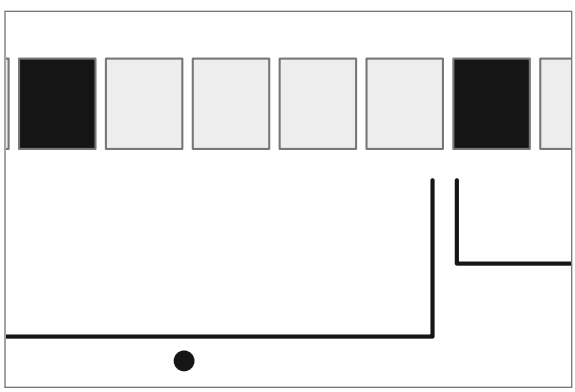


01 spent block numbers

02 encoding choices

03 candidate clues

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Convert lists of spent block numbers or transaction fields into candidate encodings and compare with known puzzle clues.

READ THE THREE STAGES

01 spent block numbers
02 encoding choices
03 candidate clues

WHAT WAS ACTUALLY DONE

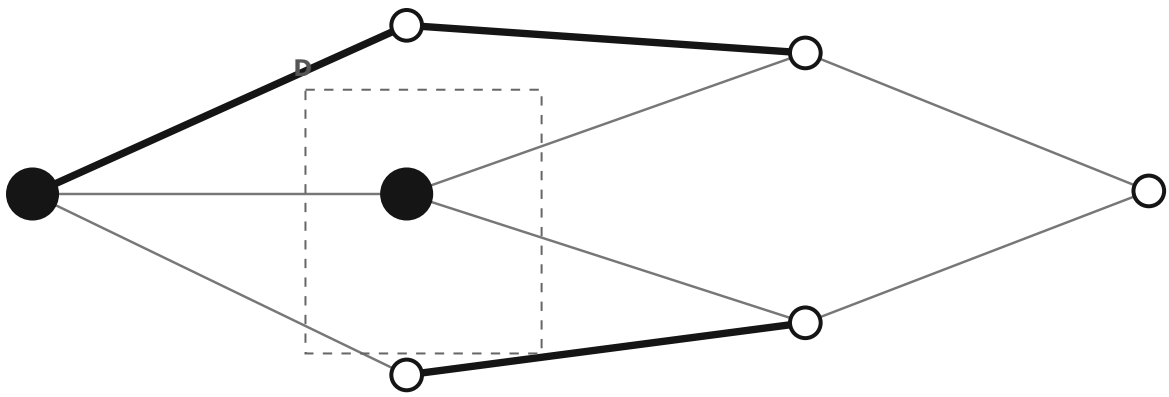
The analyzer tests stated integer/string representations and inspects spends. Finding readable text or a similar hex fragment is exploratory evidence, not a verified key.

ORIGINAL FINDING / IMPLEMENTED

Test block-index encodings and inspect spends for puzzle-like hexadecimal data.

M156

Hub-address and recoverability scans

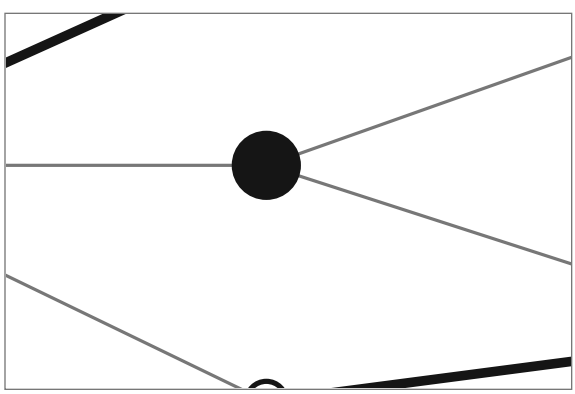


01 hub addresses

02 spends and outputs

03 recoverability evidence

D / OPERATION DETAIL



Vector detail / 1.30× main-view scale

HOW THE OPERATION WORKS

Query selected addresses and outputs to find spends, public keys, old funds or other data relevant to recoverability hypotheses.

READ THE THREE STAGES

01 hub addresses
02 spends and outputs
03 recoverability evidence

WHAT WAS ACTUALLY DONE

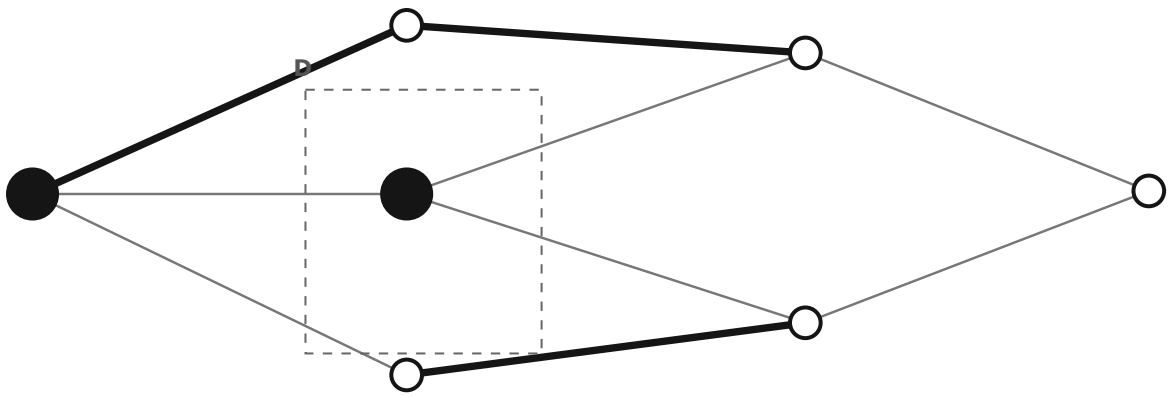
Saved scans summarize activity and candidate opportunities. They do not freshly establish present balances, authority over wallets or a successful cryptographic recovery.

ORIGINAL FINDING / EXECUTED RECORD

Address/UTXO activity scans identify data leads; no present ownership or balance conclusions are newly verified here.

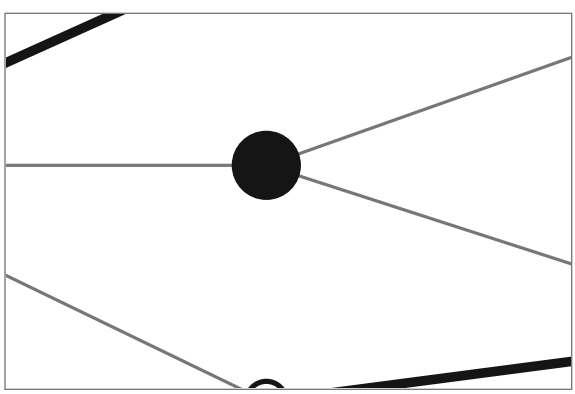
M157

Puzzle reflection and unlock relationship scans



01	02	03
puzzle labels	reflection / unlock links	relation hypotheses

D / OPERATION DETAIL



Vector detail / 1.30× main-view scale

HOW THE OPERATION WORKS

Look for reflected labels, addresses or tier relationships suggesting that one puzzle might encode information useful for another.

READ THE THREE STAGES

01 puzzle labels
02 reflection / unlock links
03 relation hypotheses

WHAT WAS ACTUALLY DONE

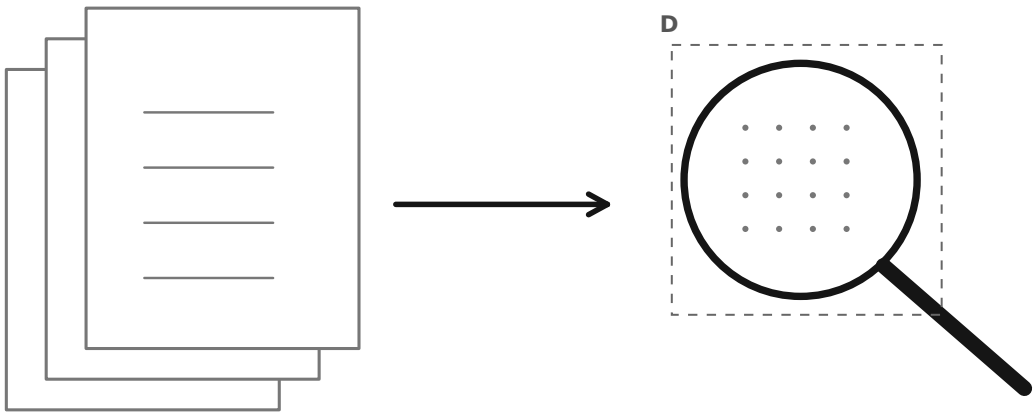
Reports enumerate tested correspondences and unlocking ideas. A relation between labels is distinct from a demonstrated scalar transformation that reproduces both targets.

ORIGINAL FINDING / EXECUTED RECORD

Compare puzzle labels, addresses and transaction relationships; no validated universal unlocking rule established.

M158

Historical website/archive and directory searches

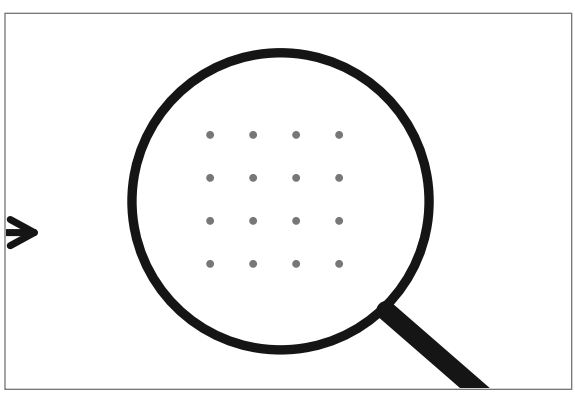


01
archive queries

02
historical pages

03
challenge artifacts

D / OPERATION DETAIL



Vector detail / 1.27× main-view scale

HOW THE OPERATION WORKS

Search historical sites and directory/archive listings for original challenge files, creator software or missing public parameters.

READ THE THREE STAGES

01 archive queries
02 historical pages
03 challenge artifacts

WHAT WAS ACTUALLY DONE

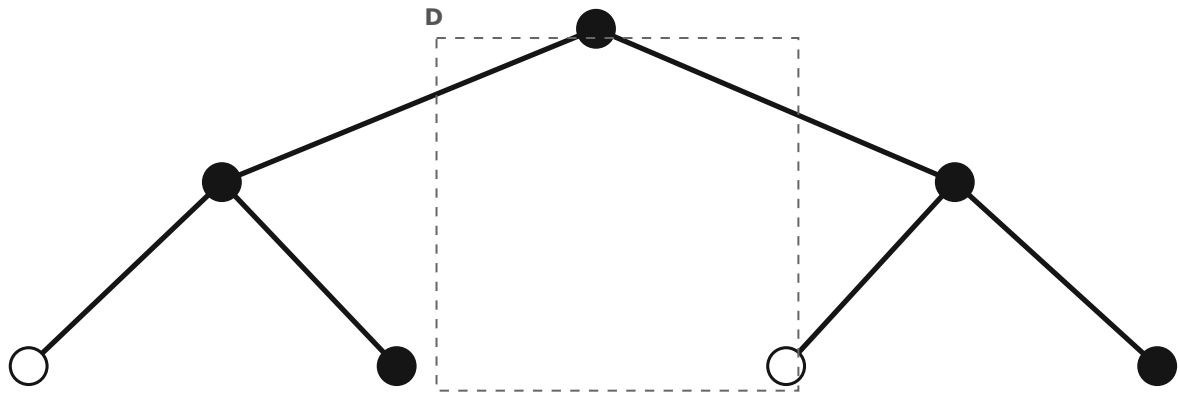
Triage scripts save candidate links and matches. The inventory records those searches without treating every archived webpage as authenticated creator provenance or a successful lead.

ORIGINAL FINDING / EXECUTED RECORD

Old Bitcoin sites, indexes and Drive leads were triaged for relevant artifacts.

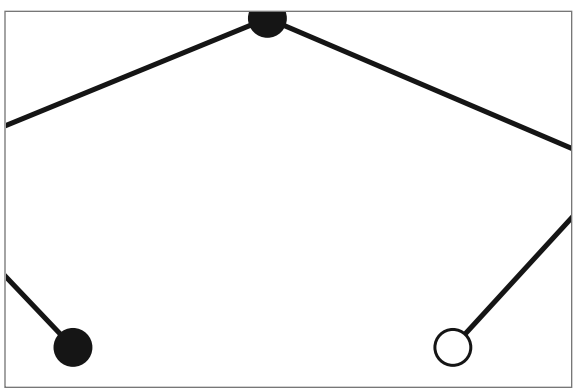
M159

Time-Will-Tell phrase/path hypotheses



01	02	03
supplied phrase	passphrase / path choices	bounded derivations

D / OPERATION DETAIL



Vector detail / 0.98× main-view scale

HOW THE OPERATION WORKS

Try bounded interpretations of an explicitly supplied recovery phrase, including word-subset and passphrase/path choices.

READ THE THREE STAGES

01 supplied phrase
02 passphrase / path choices
03 bounded derivations

WHAT WAS ACTUALLY DONE

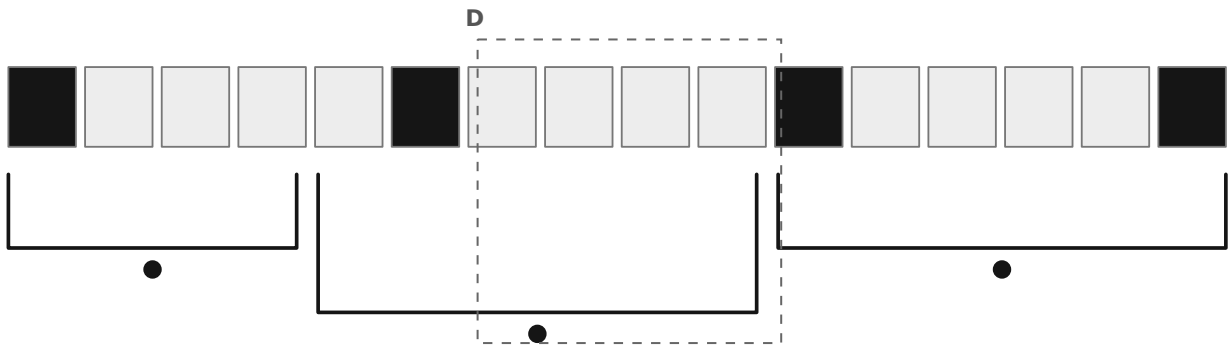
The branch implementation checks selected mnemonic and HD-wallet variants against named targets. Private phrases are omitted from the report, and unrun full search coverage is not inferred.

ORIGINAL FINDING / IMPLEMENTED/BRANCH RECORD

Bounded mnemonic subset, passphrase and derivation-path tests are documented; private phrases are excluded from this report.

M160

Complete chat-lead indexing and candidate extraction

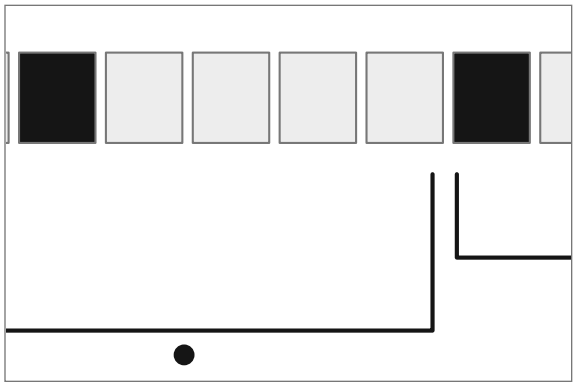


01 complete chat records

02 speaker and source tags

03 indexed candidate leads

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Index complete available user/assistant messages, distinguish speakers and extract only relevant candidate claims or supplied leads.

READ THE THREE STAGES

01 complete chat records
02 speaker and source tags
03 indexed candidate leads

WHAT WAS ACTUALLY DONE

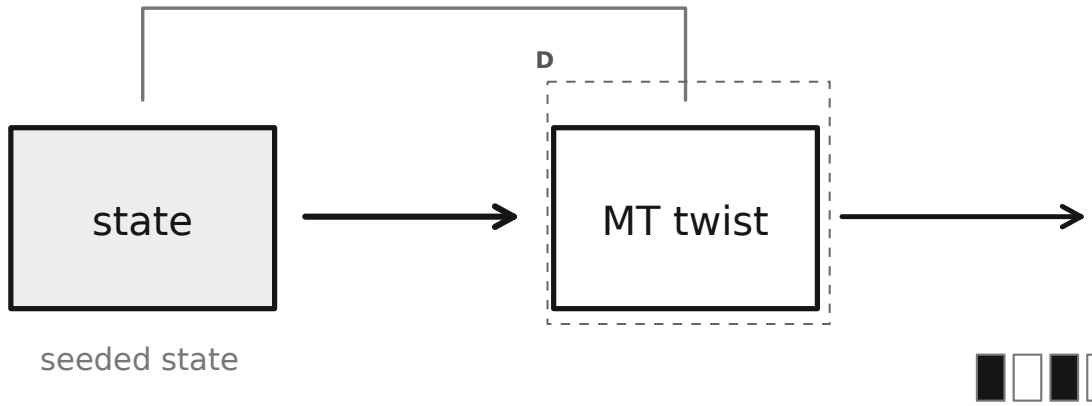
Existing audit scripts preserve source hashes, locators and redacted witnesses. Chat assertions can guide tests, but a model's claim of running code is not substituted for execution artifacts.

ORIGINAL FINDING / EXECUTED RECORD

Existing speaker-aware source index covers 685 matching conversations; assistant assertions are not execution evidence.

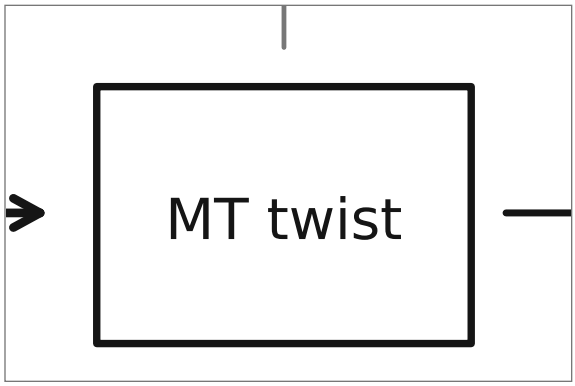
M161

Milk Sad raw MT19937 seed expansion



01	02	03
32-bit seed	MT19937 entropy	derived scalar

D / OPERATION DETAIL



Vector detail / 1.42× main-view scale

HOW THE OPERATION WORKS

Reproduce the exact weak MT19937 entropy stream produced from a 32-bit libbitcoin seed, then derive the corresponding scalar/public point.

READ THE THREE STAGES

01 32-bit seed
02 MT19937 entropy
03 derived scalar

WHAT WAS ACTUALLY DONE

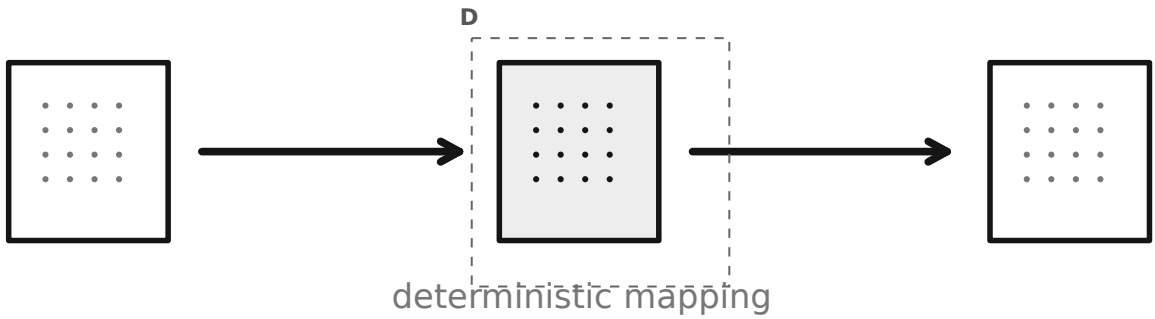
Native and CUDA implementations mirror the modeled software path. The saved raw-entropy sweep compares supported address hashes with a dated funded index, rather than attacking arbitrary secure seeds.

ORIGINAL FINDING / RECORDED GPU SWEEP

Archived report claims full 32-bit seed sweep with zero funded-index hits; this inventory does not rerun or independently certify that sweep.

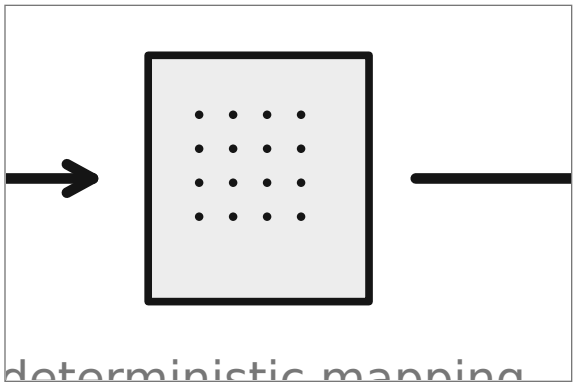
M162

Milk Sad BIP39/BIP44/49/84 derivations



01	02	03
weak entropy	BIP39 and HD paths	address types

D / OPERATION DETAIL



Vector detail / 1.38× main-view scale

HOW THE OPERATION WORKS

Turn the weak entropy into a BIP39 mnemonic seed, derive specified HD paths and compare supported address types.

READ THE THREE STAGES

01 weak entropy
02 BIP39 and HD paths
03 address types

WHAT WAS ACTUALLY DONE

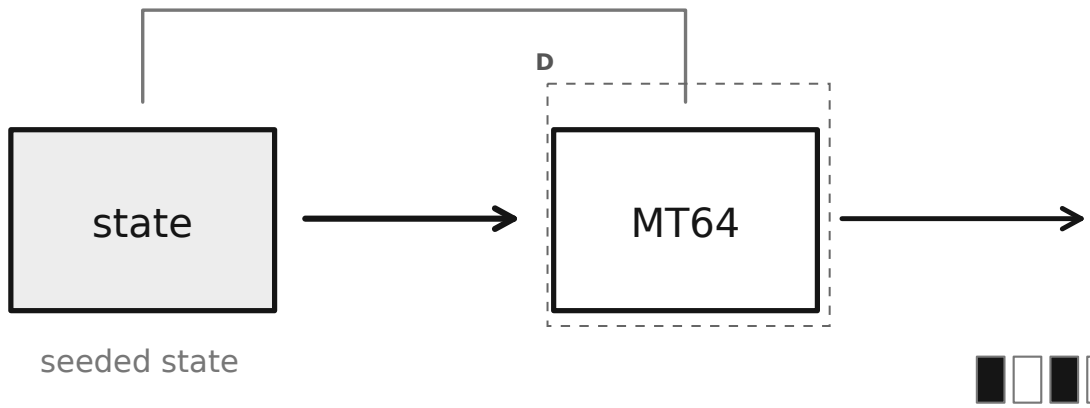
The archived sweep covers declared BIP44/49/84 account paths and serializations. PBKDF2 and child derivation make this substantially different from checking the raw entropy as a scalar.

ORIGINAL FINDING / RECORDED GPU SWEEP

Archived mnemonic and derivation-path sweep reports zero funded-index hits; bounded historical index and path coverage only.

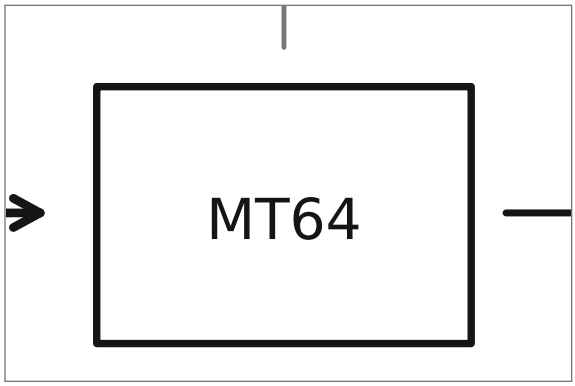
M163

Profanity MT19937-64 base-key replay



01	02	03
Profanity seed	MT19937-64 words	packed base scalar

D / OPERATION DETAIL



Vector detail / 1.42× main-view scale

HOW THE OPERATION WORKS

Replay Profanity's small seed expansion to reconstruct its base scalar under a specified output word order.

READ THE THREE STAGES

01 Profanity seed
02 MT19937-64 words
03 packed base scalar

WHAT WAS ACTUALLY DONE

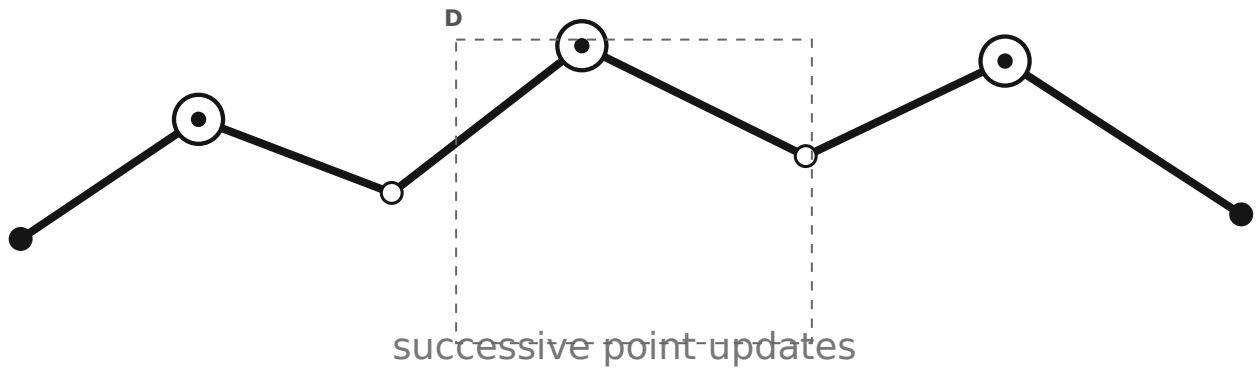
Saved CUDA results enumerate base seeds in both orders and check three hash variants. Offset-zero coverage does not automatically cover keys reached by later vanity-search increments.

ORIGINAL FINDING / RECORDED GPU SWEEP

Archived report enumerates both word orders over 32-bit seeds with zero funded-index hits at offset zero.

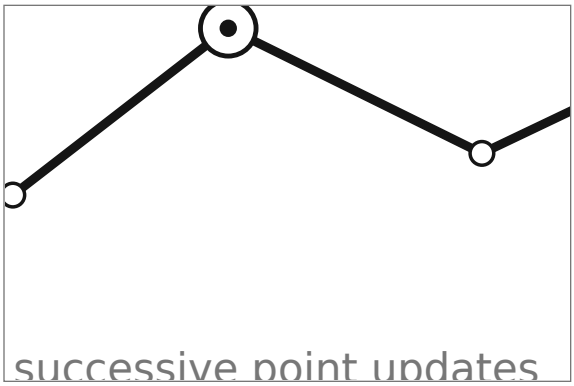
M164

Profanity incremental offset grinding



01	02	03
weak base point	add G repeatedly	incremental candidates

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Starting from a reconstructed weak base key, add small scalar offsets and reuse point additions to check nearby vanity-search states efficiently.

READ THE THREE STAGES

01 weak base point
02 add G repeatedly
03 incremental candidates

WHAT WAS ACTUALLY DONE

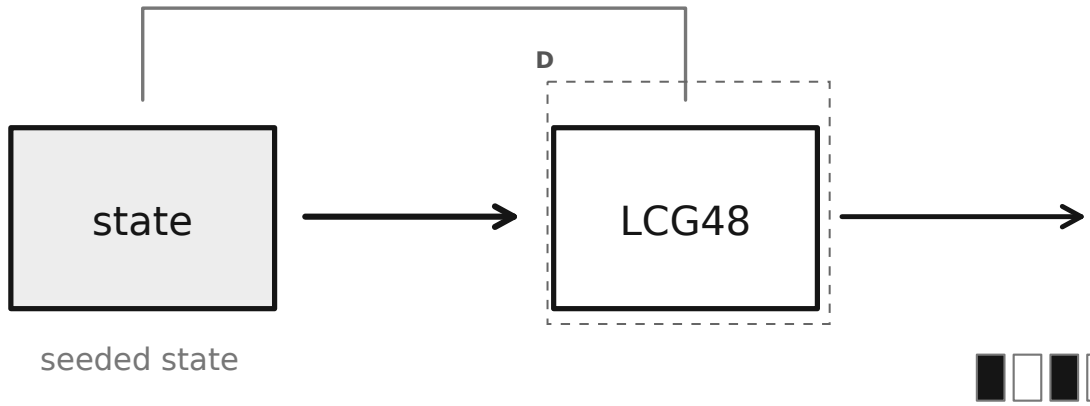
The branch adds offset-window flags and remote sharding scripts. Its documentation explicitly separates Python-side tests from real-GPU compile/verification that was still outstanding there.

ORIGINAL FINDING / BRANCH/IMPLEMENTATION

Extends base keys with small scalar offsets. Branch notes explicitly say real-GPU compile/verification was not yet done there.

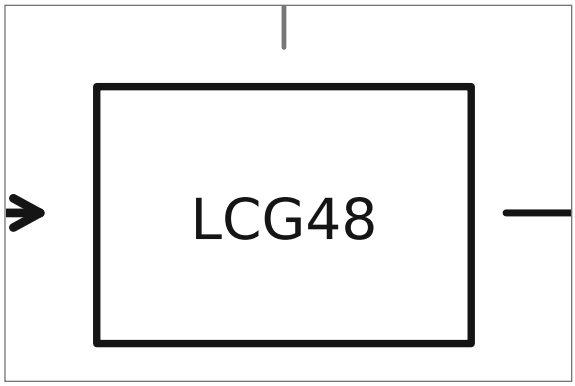
M165

Java util.Random replay



01	02	03
48-bit Java state	LCG updates	nextBytes stream

D / OPERATION DETAIL



Vector detail / 1.42× main-view scale

HOW THE OPERATION WORKS

Java Random updates a 48-bit linear-congruential state; nextBytes exposes a particular sequence of output bytes.

READ THE THREE STAGES

01 48-bit Java state
02 LCG updates
03 nextBytes stream

WHAT WAS ACTUALLY DONE

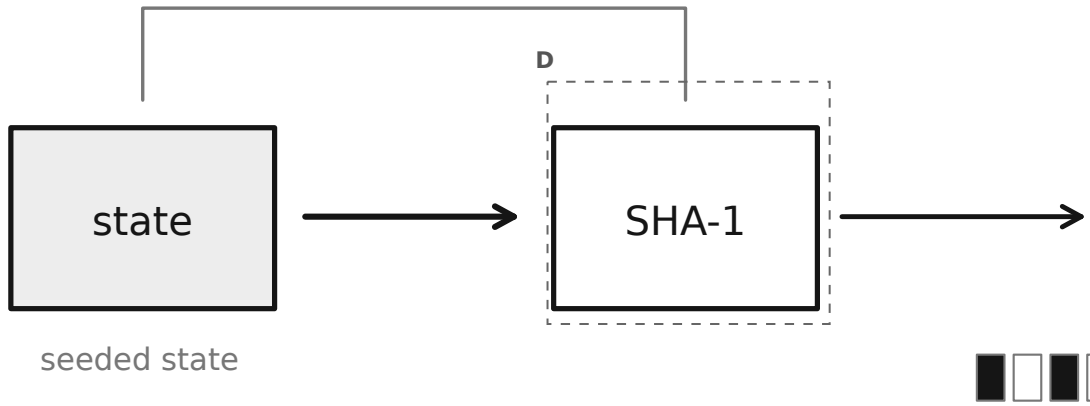
The model reproduces those bytes from a candidate explicit seed. Timestamp-window enumeration tests only programs that really seeded and consumed that generator in the modeled way.

ORIGINAL FINDING / IMPLEMENTED/REGISTRY

Models the 48-bit LCG and timestamp-seeded nextBytes path; registered runtime vectors distinguish model fidelity from a recovery.

M166

Java SUN SHA1PRNG low-entropy seed replay

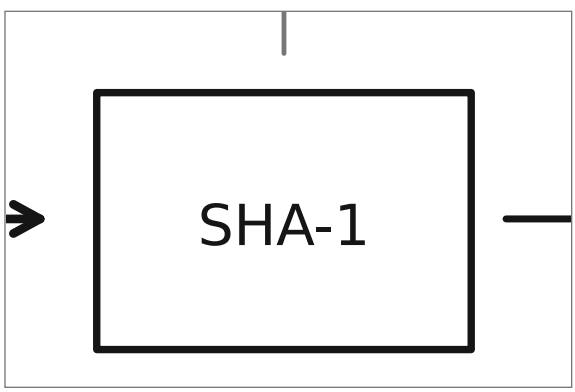


01 low-entropy seed

02 SUN SHA1PRNG state

03 deterministic bytes

D / OPERATION DETAIL



Vector detail / 1.42× main-view scale

HOW THE OPERATION WORKS

SUN SHA1PRNG with an explicit low-entropy seed evolves deterministic SHA-1-derived state and emits predictable bytes.

READ THE THREE STAGES

01 low-entropy seed
02 SUN SHA1PRNG state
03 deterministic bytes

WHAT WAS ACTUALLY DONE

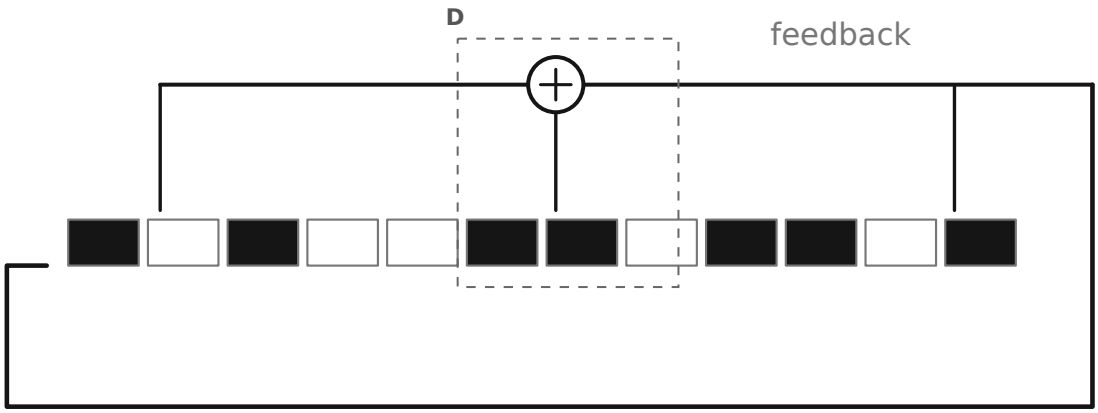
The implementation reproduces setSeed byte order and signed state updates. It does not assume every SecureRandom call is weak; normal unobserved platform entropy changes the problem.

ORIGINAL FINDING / IMPLEMENTED/REGISTRY

Deterministic setSeed model is present; not every Java/Android wallet follows that path.

M167

glibc rand/random byte-fill replay

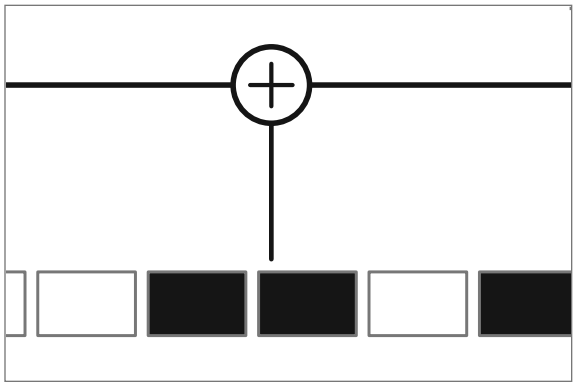


01 glibc state words

02 add feedback taps

03 truncated output bytes

D / OPERATION DETAIL



Vector detail / 1.38× main-view scale

HOW THE OPERATION WORKS

glibc rand/random uses additive-feedback state; truncating its outputs to bytes can make keys reproducible from a small seed.

READ THE THREE STAGES

01 glibc state words
02 add feedback taps
03 truncated output bytes

WHAT WAS ACTUALLY DONE

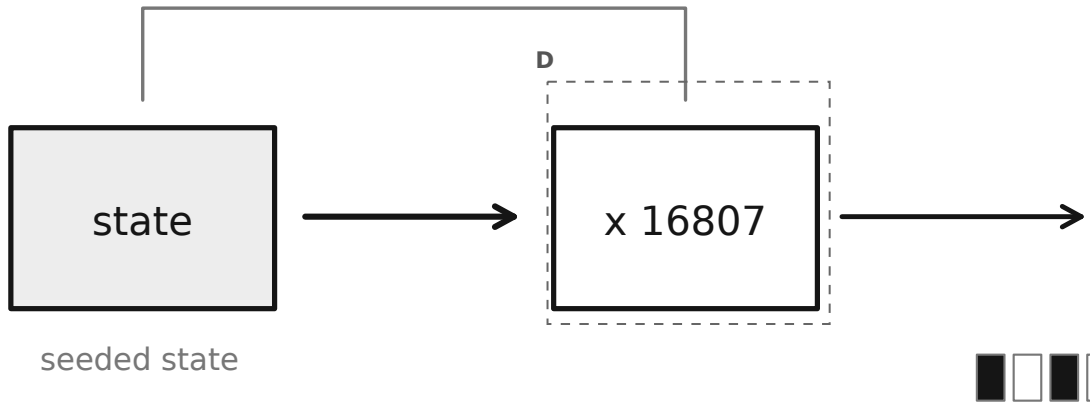
The family model initializes and warms the state, then emits low bytes under the catalog's byte-fill recipe. Generator fidelity is tested separately from applicability to a target wallet.

ORIGINAL FINDING / IMPLEMENTED/REGISTRY

Models clock-seeded additive-feedback output truncated into key bytes.

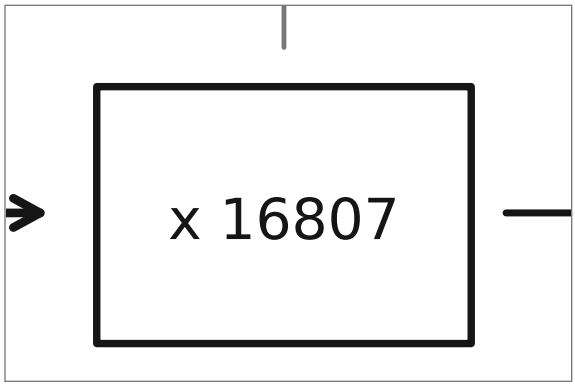
M168

Lehmer minstd_rand0 replay



01	02	03
Lehmer state	times 16807 mod p	byte stream

D / OPERATION DETAIL



Vector detail / 1.42× main-view scale

HOW THE OPERATION WORKS

minstd_rand0 multiplies state by 16807 modulo 2^31-1. An enumerable state determines the byte stream.

READ THE THREE STAGES

01 Lehmer state
02 times 16807 mod p
03 byte stream

WHAT WAS ACTUALLY DONE

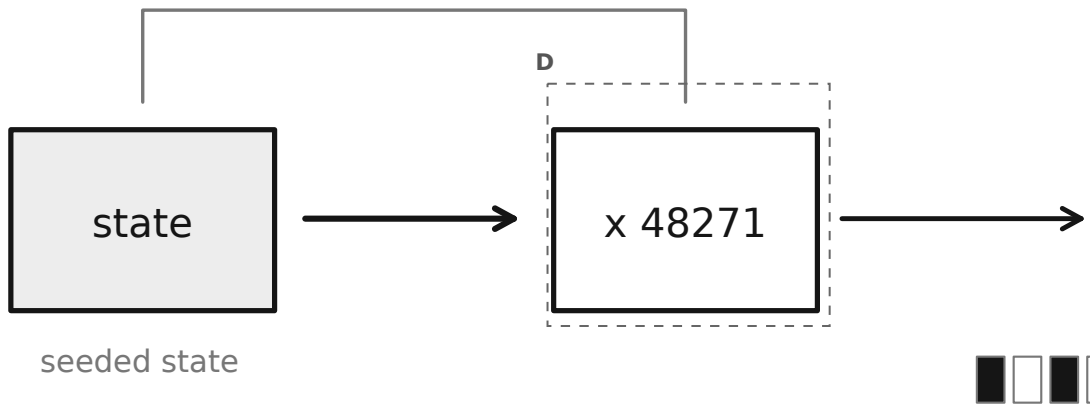
The registry wraps this Lehmer generator into candidate key bytes and runtime vectors. The attack premise is misuse for wallet entropy, not a weakness of secp256k1 itself.

ORIGINAL FINDING / IMPLEMENTED/REGISTRY

Models multiplier 16807 weak-generator key bytes.

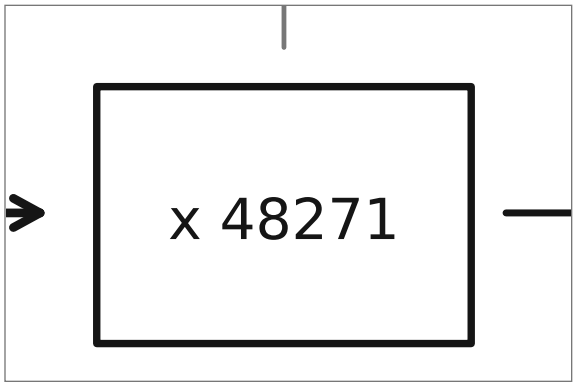
M169

Lehmer minstd_rand replay



01	02	03
Lehmer state	times 48271 mod p	byte stream

D / OPERATION DETAIL



Vector detail / 1.42× main-view scale

HOW THE OPERATION WORKS

minstd_rand uses multiplier 48271 with the same prime modulus, producing a different deterministic stream.

READ THE THREE STAGES

01 Lehmer state
02 times 48271 mod p
03 byte stream

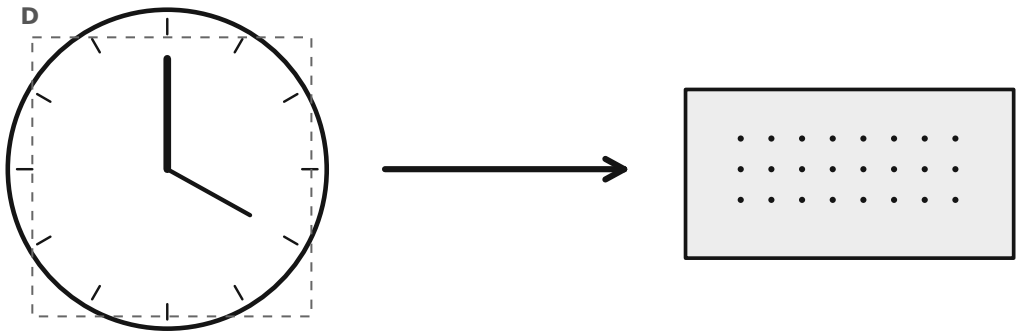
WHAT WAS ACTUALLY DONE

The model keeps this family separate because seed replay with the wrong multiplier produces different keys. A sweep must identify the exact generator and byte extraction recipe it covers.

ORIGINAL FINDING / IMPLEMENTED/REGISTRY

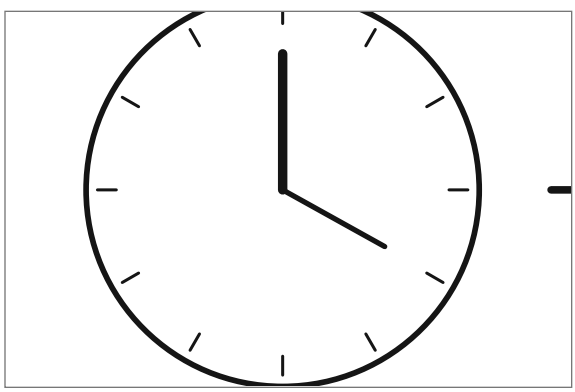
Models multiplier 48271 key-byte generation, separately from minstd_rand0.

M170
AVR/Arduino rand replay



01	02	03
millis / sensor seed	AVR rand replay	key bytes

D / OPERATION DETAIL



Vector detail / 1.23× main-view scale

HOW THE OPERATION WORKS

Embedded sketches may derive key bytes from a deterministic rand stream seeded by millis or limited sensor readings.

READ THE THREE STAGES

01 millis / sensor seed
02 AVR rand replay
03 key bytes

WHAT WAS ACTUALLY DONE

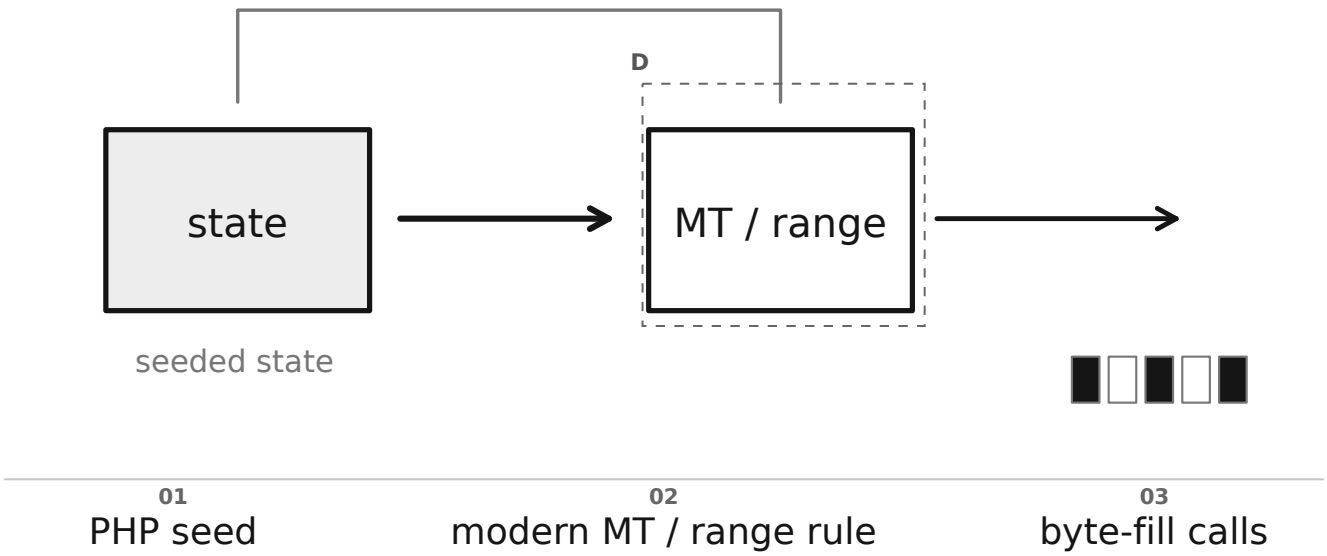
The AVR/Arduino model implements the stated Lehmer-style stream under candidate seeds. It does not model unknown hardware noise measurements or every embedded random library.

ORIGINAL FINDING / IMPLEMENTED/REGISTRY

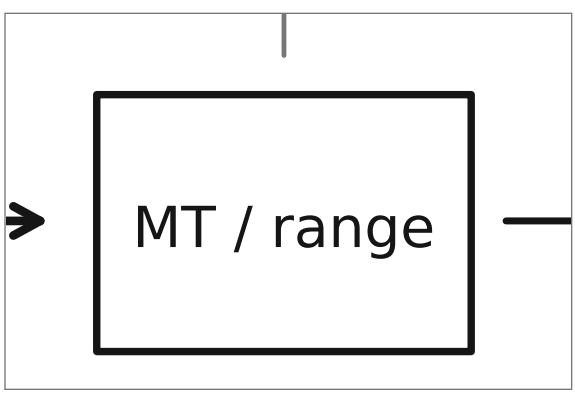
Models weak embedded Lehmer-style output under enumerable seeds.

M171

Modern PHP MT19937 byte-fill replay



D / OPERATION DETAIL



Vector detail / 1.42× main-view scale

HOW THE OPERATION WORKS

Modern PHP mt_rand uses MT19937 and particular range-output rules. Repeated byte requests determine a key from its seed.

READ THE THREE STAGES

01 PHP seed
02 modern MT / range rule
03 byte-fill calls

WHAT WAS ACTUALLY DONE

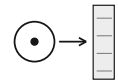
The implementation emits bytes under modern mode and compares with runtime vectors. A generic MT19937 sequence with different range scaling is not the same candidate family.

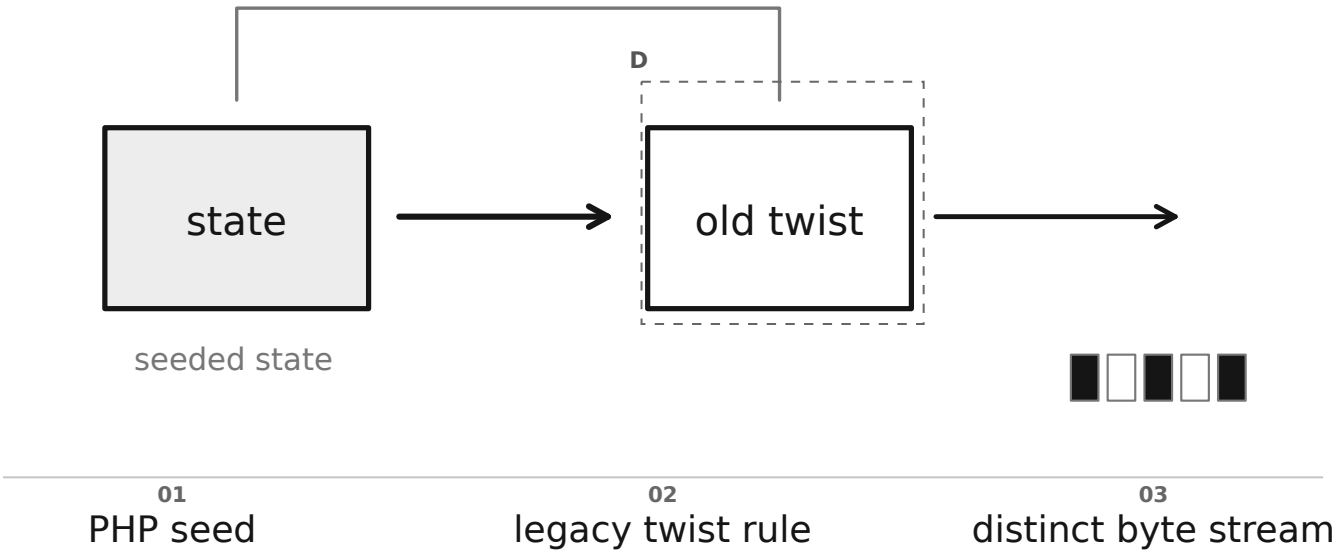
ORIGINAL FINDING / IMPLEMENTED/REGISTRY

PHP 7.1+ generator/range semantics are modeled separately from legacy PHP.

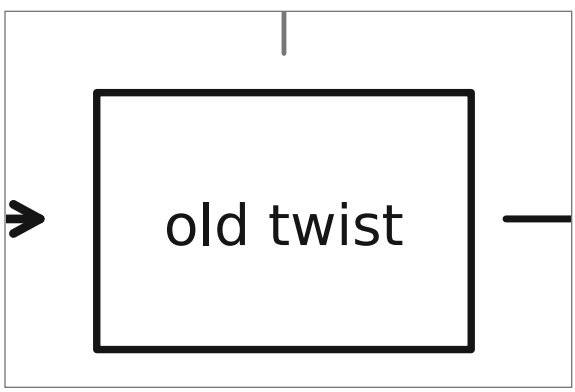
M172

Legacy PHP broken-twist replay





D / OPERATION DETAIL



Vector detail / 1.42× main-view scale

HOW THE OPERATION WORKS

Older PHP used a different twist/range behavior, so a small seed produces a distinct key stream from modern PHP.

READ THE THREE STAGES

01 PHP seed
02 legacy twist rule
03 distinct byte stream

WHAT WAS ACTUALLY DONE

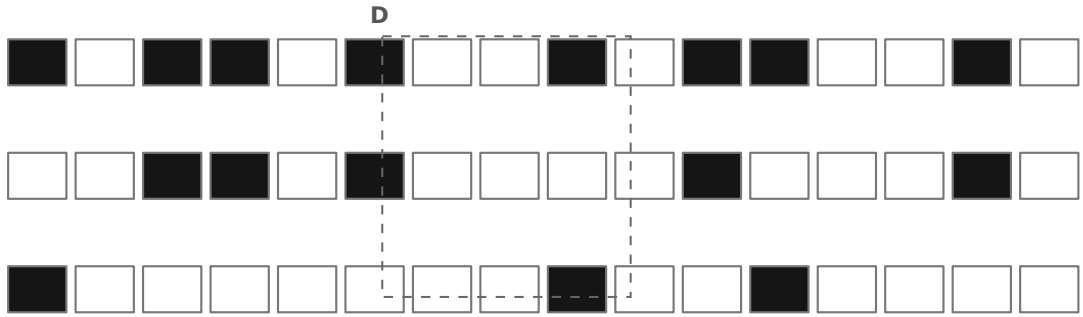
A separate legacy mode reproduces that behavior. Historical weakness labels alone are insufficient without matching the actual PHP version and key-generation recipe.

ORIGINAL FINDING / IMPLEMENTED/REGISTRY

Models the older mt_rand twist and range-scaling behavior, not covered by ordinary MT19937 assumptions.

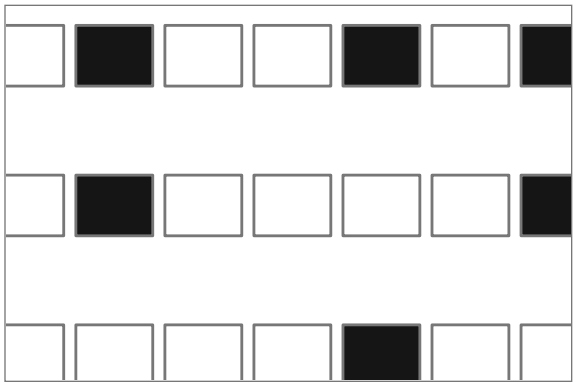
M173

PHPCoinAddress legacy and modern keygen



01	02	03
first-byte constraint	wallet-specific calls	key byte string

D / OPERATION DETAIL



Vector detail / 1.32× main-view scale

HOW THE OPERATION WORKS

PHPCoinAddress applies its own bounded byte calls, including a special first-byte range, around PHP's generator.

READ THE THREE STAGES

01 first-byte constraint
02 wallet-specific calls
03 key byte string

WHAT WAS ACTUALLY DONE

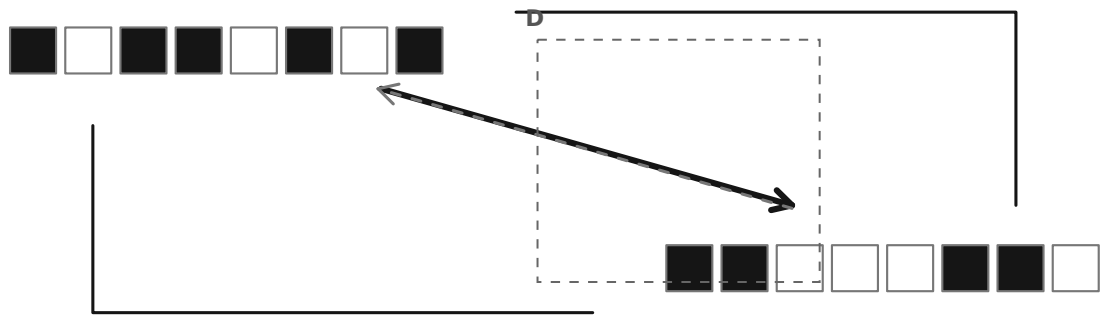
Dedicated wrappers reproduce those calls for both old and modern modes. Replaying plain mt_rand bytes would omit this wallet-specific transformation and could miss its generated keys.

ORIGINAL FINDING / IMPLEMENTED/REGISTRY

Dedicated first-byte/range rules are modeled under both PHP generator modes.

M174

V8 MWC1616 / Randstorm-era byte fill

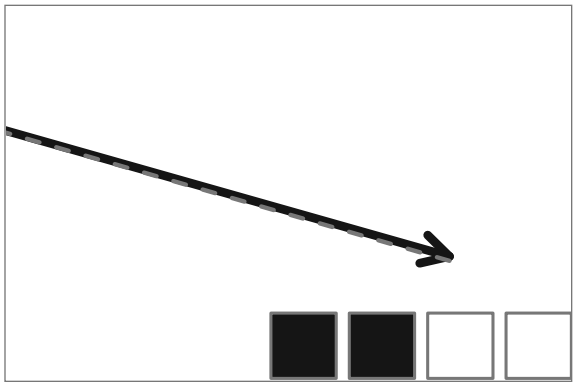


01 two V8 states

02 multiply-with-carry

03 combined outputs

D / OPERATION DETAIL



Vector detail / 1.42× main-view scale

HOW THE OPERATION WORKS

Old V8 Math.random combines multiply-with-carry state; weakly seeded outputs can be reproduced and packed into key bytes.

READ THE THREE STAGES

01 two V8 states
02 multiply-with-carry
03 combined outputs

WHAT WAS ACTUALLY DONE

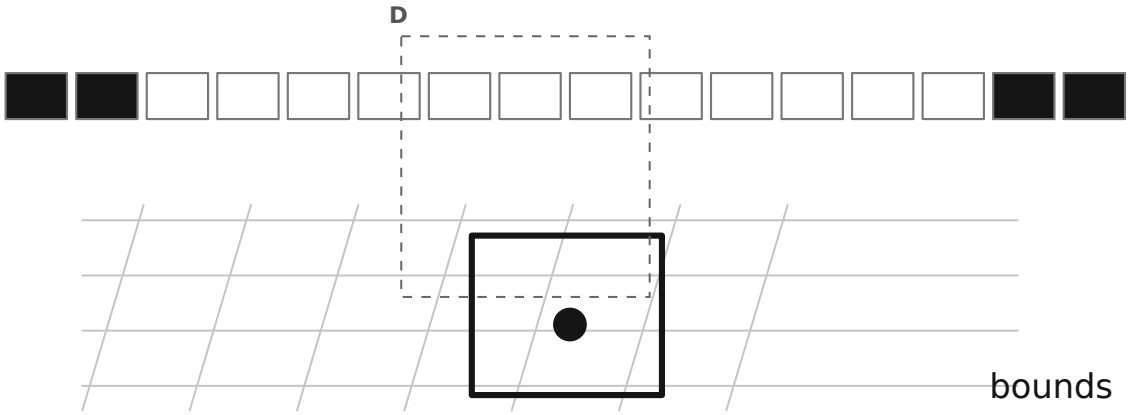
The registry supplies the historical MWC1616 byte-fill model. A full Randstorm wallet route may include additional entropy mixing and consumption order not implied by that one modeled stream.

ORIGINAL FINDING / IMPLEMENTED/REGISTRY

Historical JavaScript Math.random byte generation model; a vulnerable-family model is not attribution to a target wallet.

M175

V8 xorshift128 state recovery

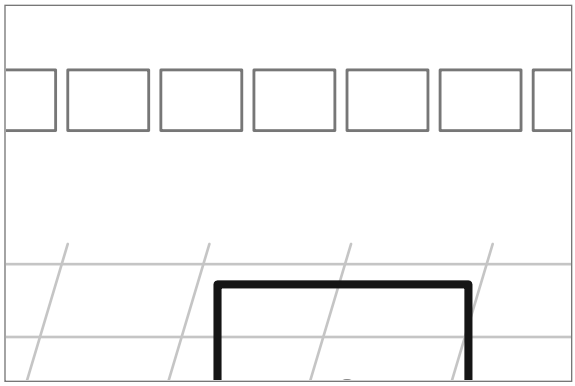


01 observed random outputs

02 xorshift constraints

03 compatible states

D / OPERATION DETAIL



Vector detail / 1.32× main-view scale

HOW THE OPERATION WORKS

Observed floating Math.random outputs expose some generator bits. Constraint solving can recover compatible xorshift state under the modeled engine order.

READ THE THREE STAGES

01 observed random outputs
02 xorshift constraints
03 compatible states

WHAT WAS ACTUALLY DONE

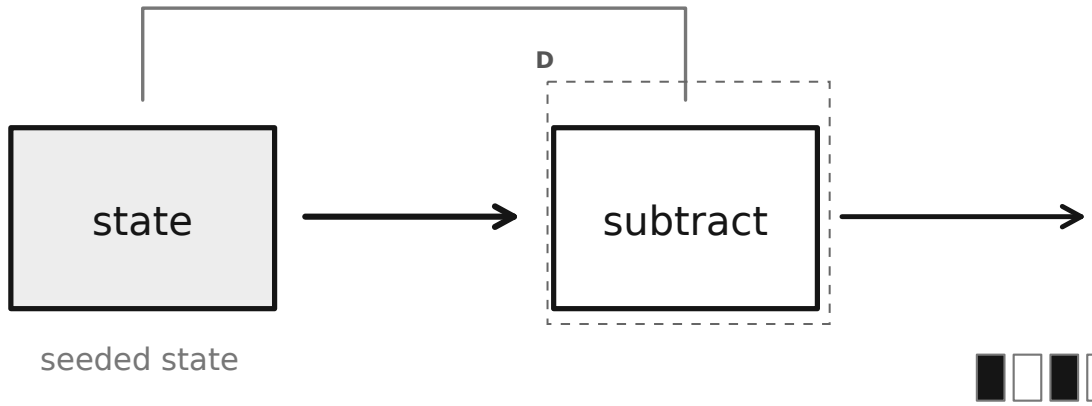
The source contains forward generation and state-recovery helpers. It requires actual usable observations; no such target observations are created by merely knowing a Bitcoin address.

ORIGINAL FINDING / IMPLEMENTED

Recovers generator state from observed Math.random outputs under the modeled engine semantics; no target observations established here.

M176

.NET System.Random replay

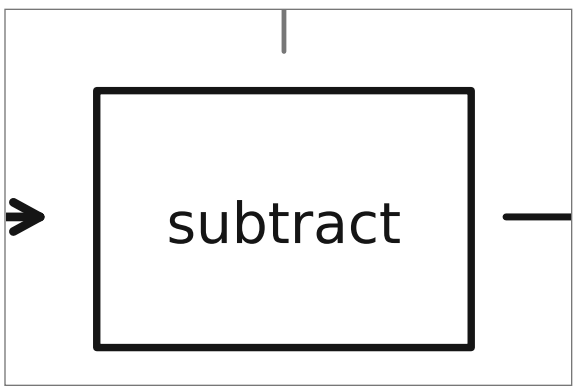


01 TickCount-style seed

02 .NET subtractive state

03 replayed outputs

D / OPERATION DETAIL



Vector detail / 1.42× main-view scale

HOW THE OPERATION WORKS

Explicitly seeded .NET System.Random evolves subtractive state; low-entropy TickCount-style seeds allow replay.

READ THE THREE STAGES

01 TickCount-style seed
02 .NET subtractive state
03 replayed outputs

WHAT WAS ACTUALLY DONE

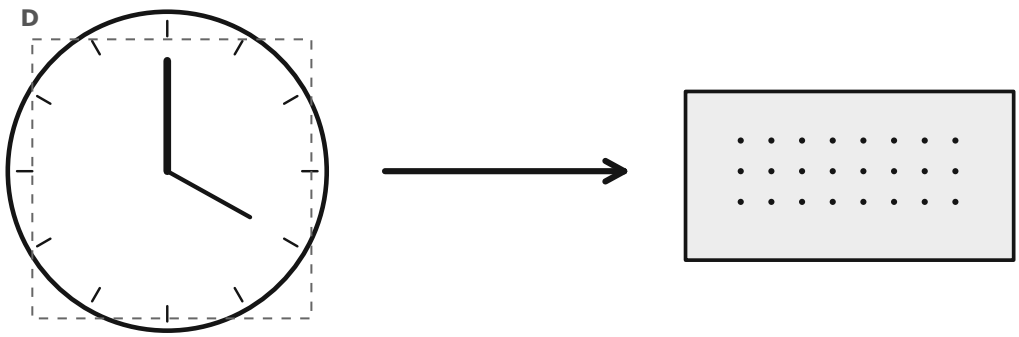
The model reproduces the framework-compatible byte stream and supports runtime vectors. Newer runtime semantics or additional entropy require a different model.

ORIGINAL FINDING / IMPLEMENTED/REGISTRY

Models explicitly seeded subtractive generator and TickCount-style seed scope.

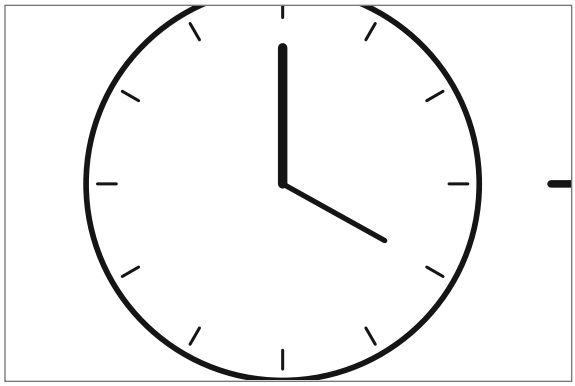
M177

Timestamp/UUID hash-derived keys



01	02	03
timestamp / UUID	encode and hash	full-width output

D / OPERATION DETAIL



Vector detail / 1.23× main-view scale

HOW THE OPERATION WORKS

Hashing a predictable timestamp or weak UUID produces a full-length string but retains the small input uncertainty.

READ THE THREE STAGES

01 timestamp / UUID
02 encode and hash
03 full-width output

WHAT WAS ACTUALLY DONE

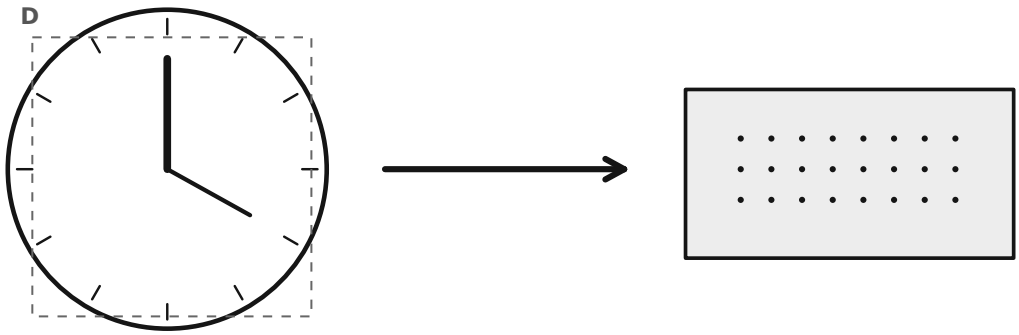
The catalog hashes selected second/millisecond and weak-UUID encodings into candidate keys. It searches their input choices, not the full 256-bit digest space.

ORIGINAL FINDING / IMPLEMENTED

SHA-256 clock, millisecond and weak-UUID key recipes are modeled as bounded generator hypotheses.

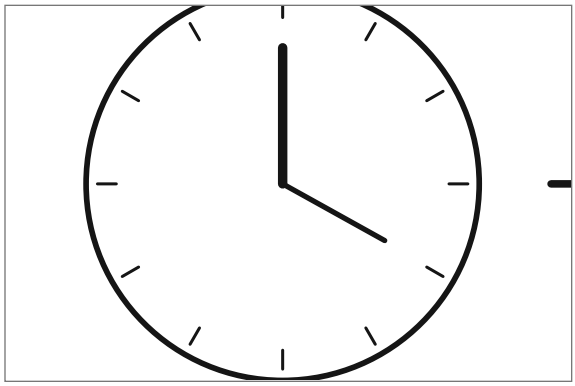
M178

Early-2009 clock-window seed replay



01	02	03
bounded 2009 times	generator families	coinbase comparison

D / OPERATION DETAIL



Vector detail / 1.23× main-view scale

HOW THE OPERATION WORKS

Guess creation-time values, feed them into named weak generators/encodings and compare with early coinbase public keys.

READ THE THREE STAGES

01 bounded 2009 times
02 generator families
03 coinbase comparison

WHAT WAS ACTUALLY DONE

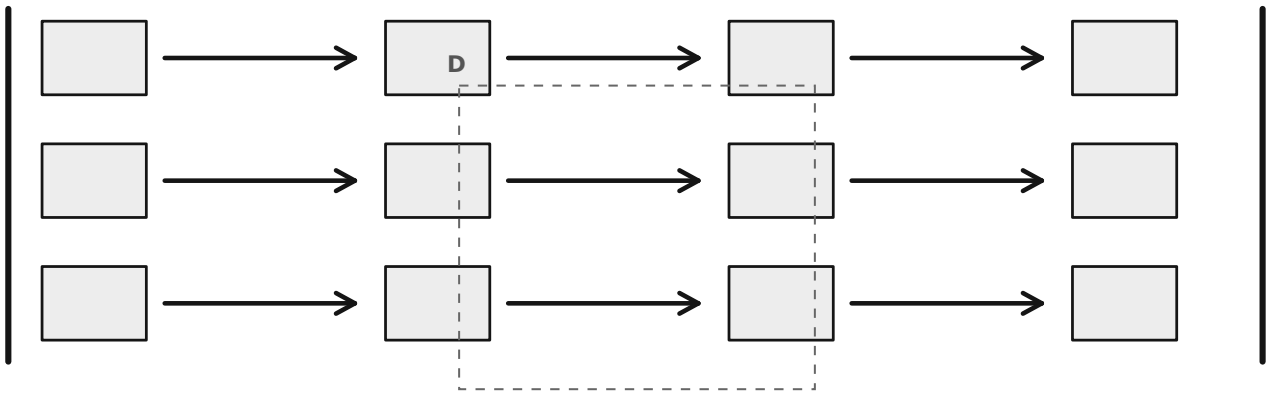
The widest saved clock-window run checks its finite 2008-2009 range and reports no matches. Actual OpenSSL machine-state seeding is not reproduced by these timestamp hypotheses.

ORIGINAL FINDING / EXECUTED RECORD

Widest saved report records 250,581,618 tested keys and no early-coinbase target recovery; true machine-state entropy is not enumerated.

M179

Weak-entropy multi-family CUDA sweep

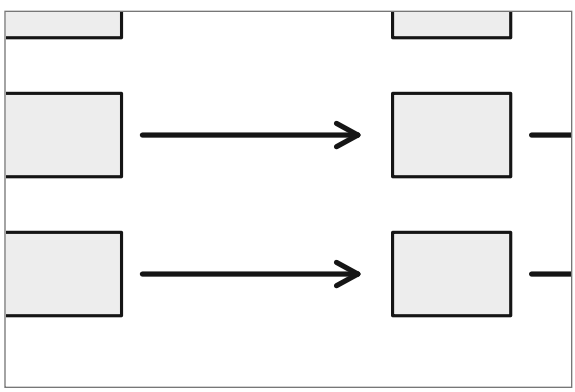


01 seed ranges

02 GPU PRNG / curve lanes

03 address-index checks

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Parallel GPU threads map seeds through PRNG, curve multiplication and hash comparison against a supported address index.

READ THE THREE STAGES

01 seed ranges
02 GPU PRNG / curve lanes
03 address-index checks

WHAT WAS ACTUALLY DONE

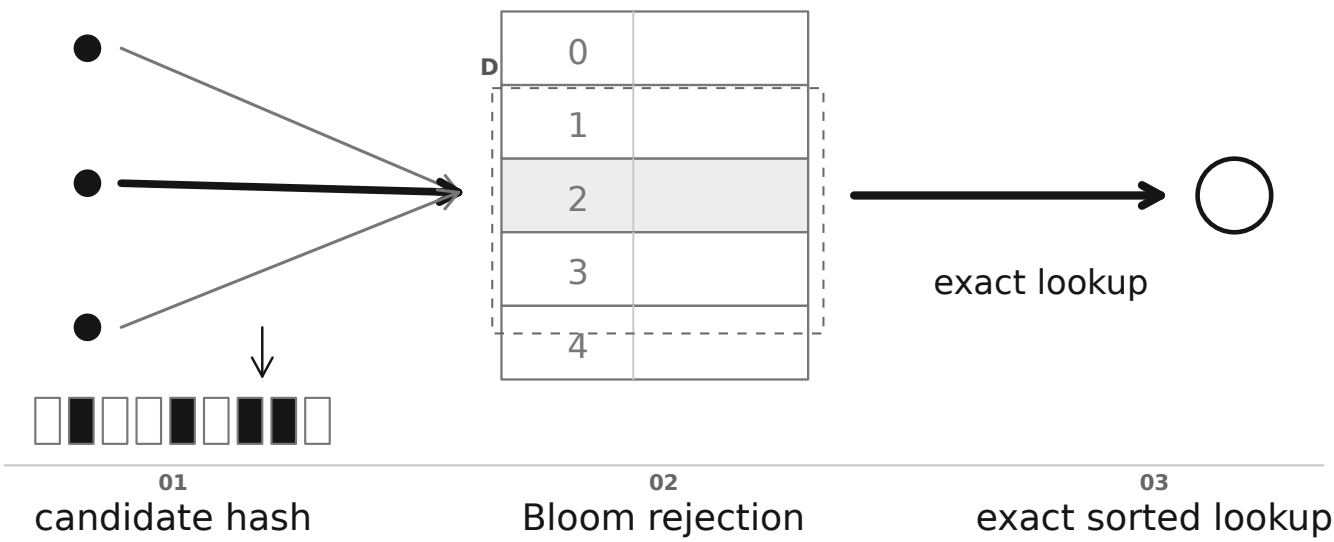
Archived multi-family runs report seed counts and planted-index detection checks. Their coverage is bounded by the selected generator families, seed intervals and supported address encodings.

ORIGINAL FINDING / RECORDED GPU SWEEP

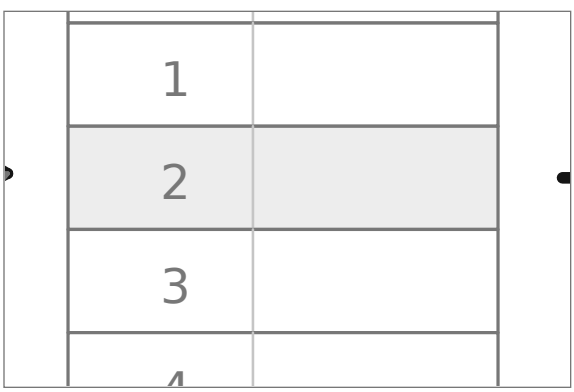
Saved report claims 27.5 billion seeds and zero funded-index hits, with planted-index detection tests. Historical record, not rerun here.

M180

Bloom plus sorted funded-address index



D / OPERATION DETAIL



Vector detail / 1.40× main-view scale

HOW THE OPERATION WORKS

A Bloom filter rapidly rejects most nonmembers; positive candidates then undergo exact lookup in a sorted hash160 index.

READ THE THREE STAGES

01 candidate hash
02 Bloom rejection
03 exact sorted lookup

WHAT WAS ACTUALLY DONE

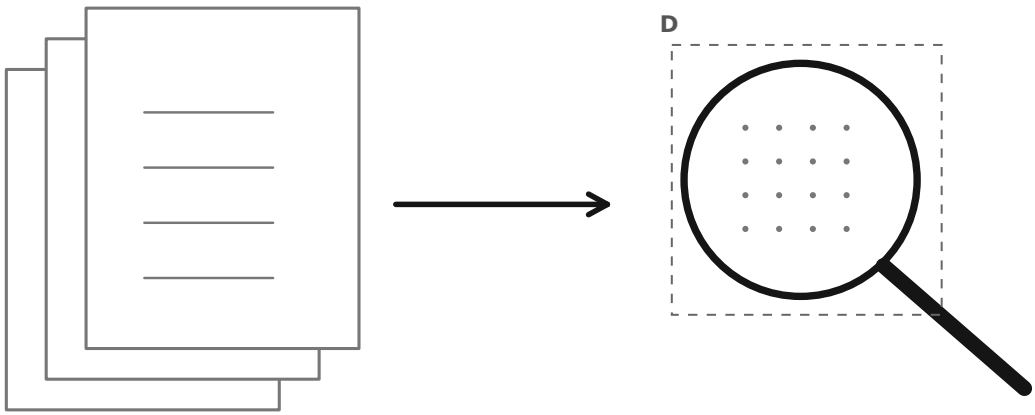
build_loyce_index.py constructs the index and GPU code performs membership checks. Bloom positives alone are not hits; the source index's date and skipped address types limit conclusions.

ORIGINAL FINDING / IMPLEMENTED

Probabilistic prefilter backed by exact lookup; index date and supported address types bound conclusions.

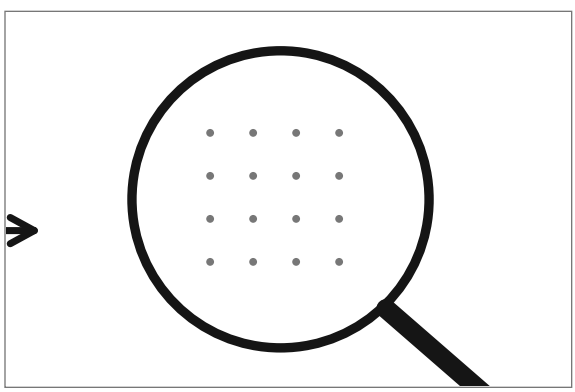
M181

Weak-RNG repository-source triage



01	02	03
wallet repositories	RNG call sites	version / dataflow triage

D / OPERATION DETAIL



Vector detail / 1.27× main-view scale

HOW THE OPERATION WORKS

Search wallet source for weak RNG calls and correlate findings with known disclosures and affected versions.

READ THE THREE STAGES

01 wallet repositories
02 RNG call sites
03 version / dataflow triage

WHAT WAS ACTUALLY DONE

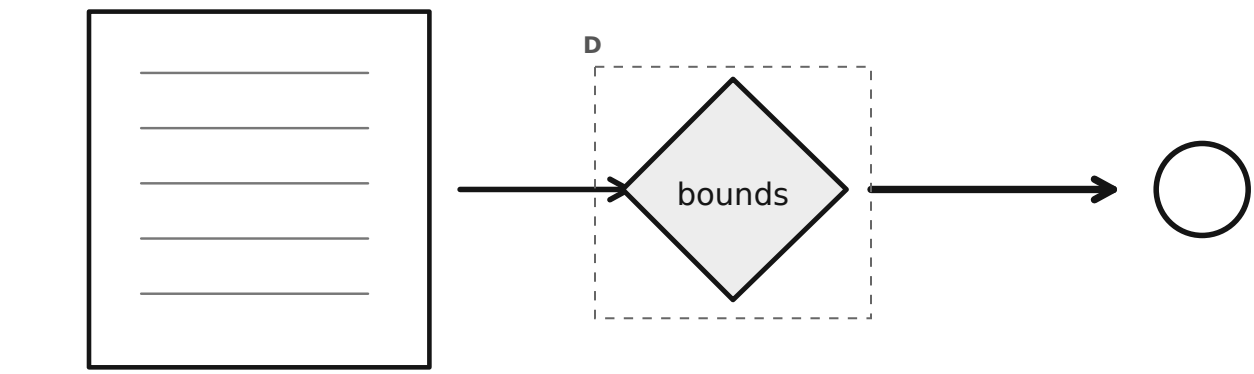
Curated repository scans record references and matching lines. A weak RNG in tests, an old dependency or a non-keygen path is not automatic evidence that current wallet keys are vulnerable.

ORIGINAL FINDING / EXECUTED RECORD

Pattern hits and disclosures locate candidate implementations; historical/current refs and dependency bugs must be distinguished.

M182

Recoverable-hit rederivation/address-type audit

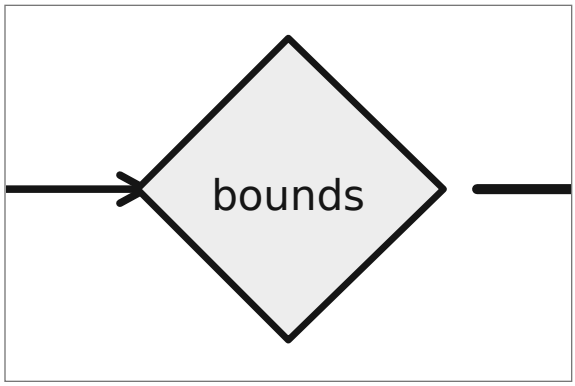


01 proposed scanner hit

02 rederive exact path

03 address-type audit

D / OPERATION DETAIL



Vector detail / 1.37× main-view scale

HOW THE OPERATION WORKS

Recompute the complete derivation and exact address type before accepting a scanner's proposed recovery.

READ THE THREE STAGES

01 proposed scanner hit
02 rederive exact path
03 address-type audit

WHAT WAS ACTUALLY DONE

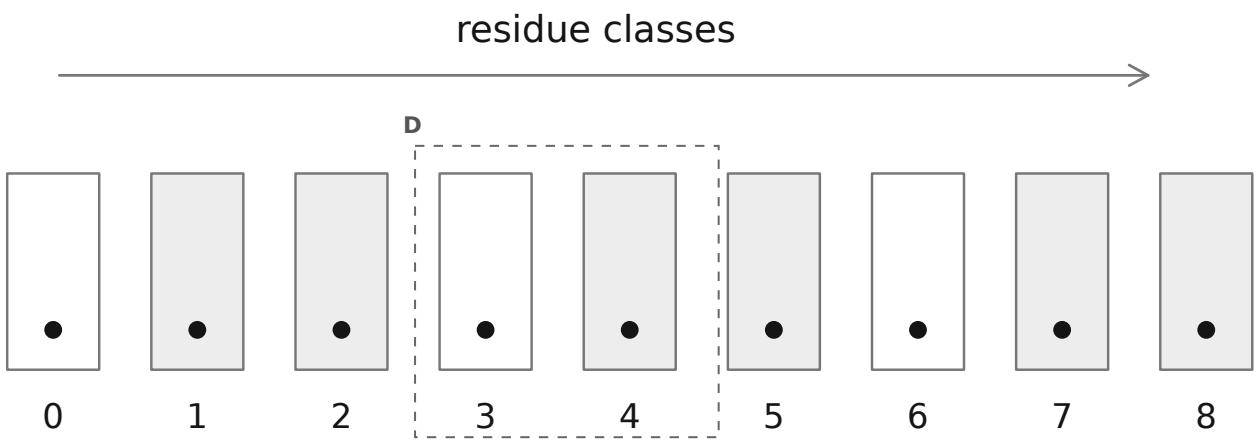
verify_scan_hits.py rejects candidates whose derivation or P2PKH/P2SH/witness interpretation does not match. It corrects the earlier tendency to label raw index associations as recovered wallets.

ORIGINAL FINDING / AUDITED/CORRECTED

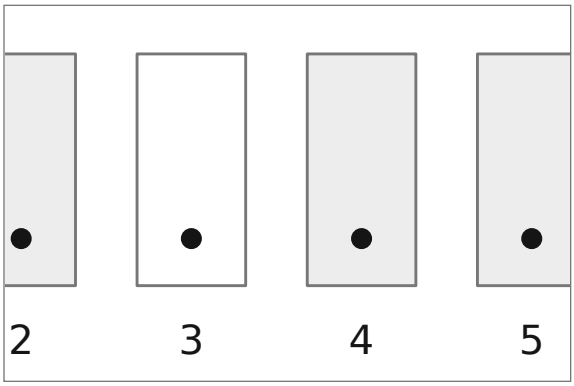
Recompute candidate addresses and reject type mismatches before reporting a recovery; prior raw hits were not all valid.

M183

Digital-root puzzle-index association



D / OPERATION DETAIL



Vector detail / 1.18× main-view scale

HOW THE OPERATION WORKS

Reduce a scalar's decimal digits repeatedly and compare the result with a puzzle-index rule.

READ THE THREE STAGES

01 scalar digits
02 reduce modulo nine
03 index association

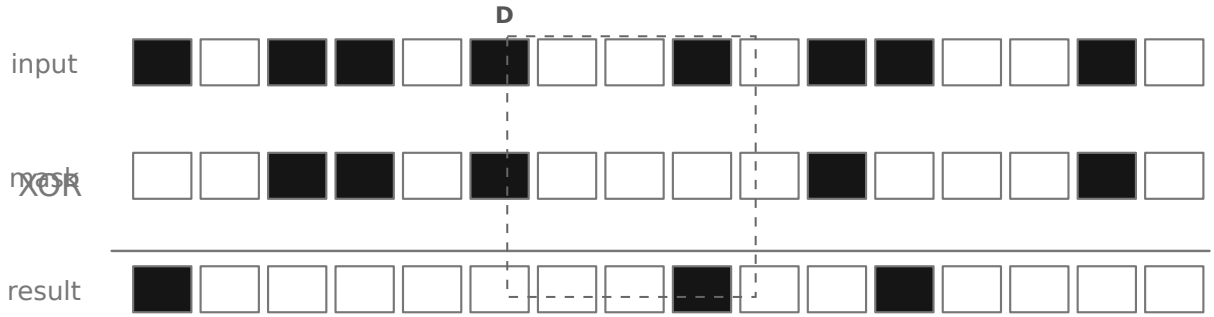
WHAT WAS ACTUALLY DONE

The creative-theory report counts observed matches and compares with controls. A small digital-root match imposes at most a weak residue condition, not the scalar's full value.

ORIGINAL FINDING / EXECUTED RECORD

One of thirteen archived creative theories; report marks it nonsignificant, not a recovered-key method.

M184
Public-key x-coordinate steganography

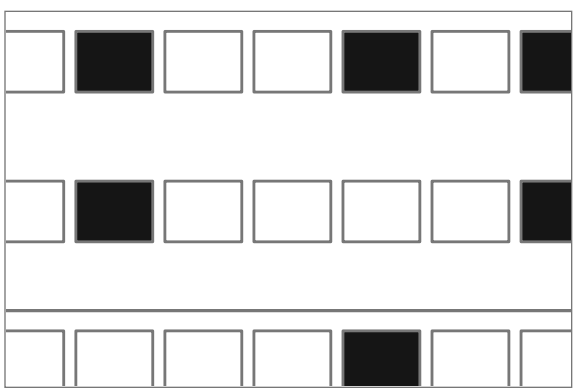


01 private scalar bits

02 public x bits

03 XOR / low-bit relation

D / OPERATION DETAIL



Vector detail / 1.32× main-view scale

HOW THE OPERATION WORKS

Compare selected low bits or XOR patterns of k and the public x-coordinate for signs of intentional embedded information.

READ THE THREE STAGES

01 private scalar bits
02 public x bits
03 XOR / low-bit relation

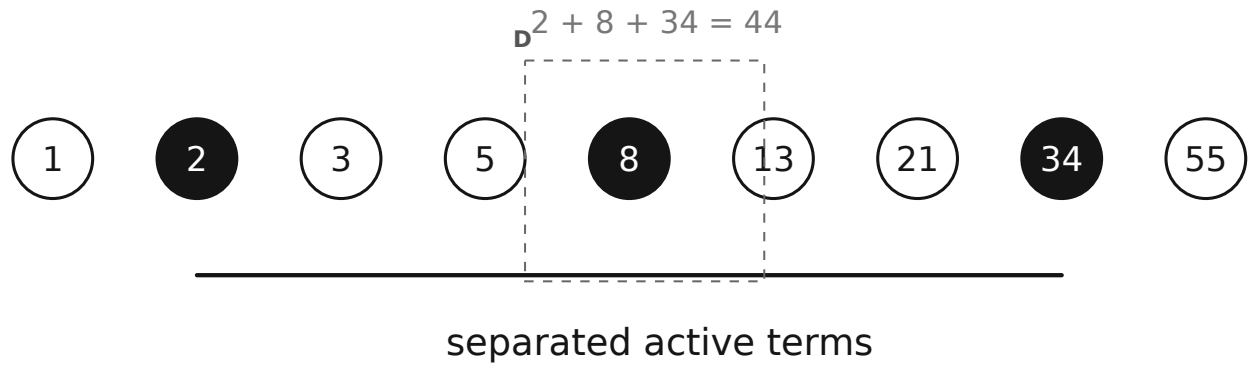
WHAT WAS ACTUALLY DONE

The report summarizes bit statistics against random keys. Public multiplication is nonlinear; apparent low-bit similarity must predict unseen scalars before it can guide recovery.

ORIGINAL FINDING / EXECUTED RECORD

Compare key/public-coordinate low-bit relations with random controls; no convincing result recorded.

M185
Zeckendorf/Fibonacci sparsity

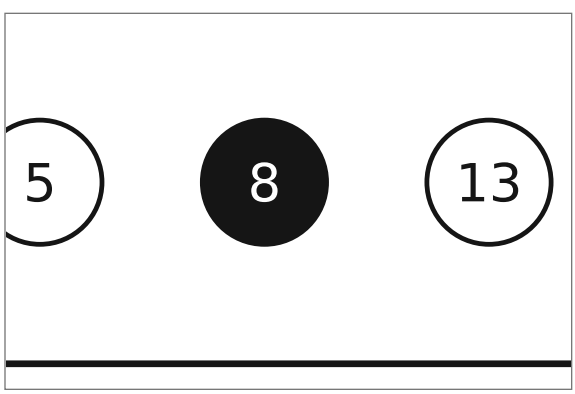


01 integer value

02 nonadjacent Fibonacci terms

03 sparse representation

D / OPERATION DETAIL



Vector detail / 1.56× main-view scale

HOW THE OPERATION WORKS

Represent integers as sums of nonadjacent Fibonacci numbers and measure how sparse that representation is.

READ THE THREE STAGES

01 integer value
02 nonadjacent Fibonacci terms
03 sparse representation

WHAT WAS ACTUALLY DONE

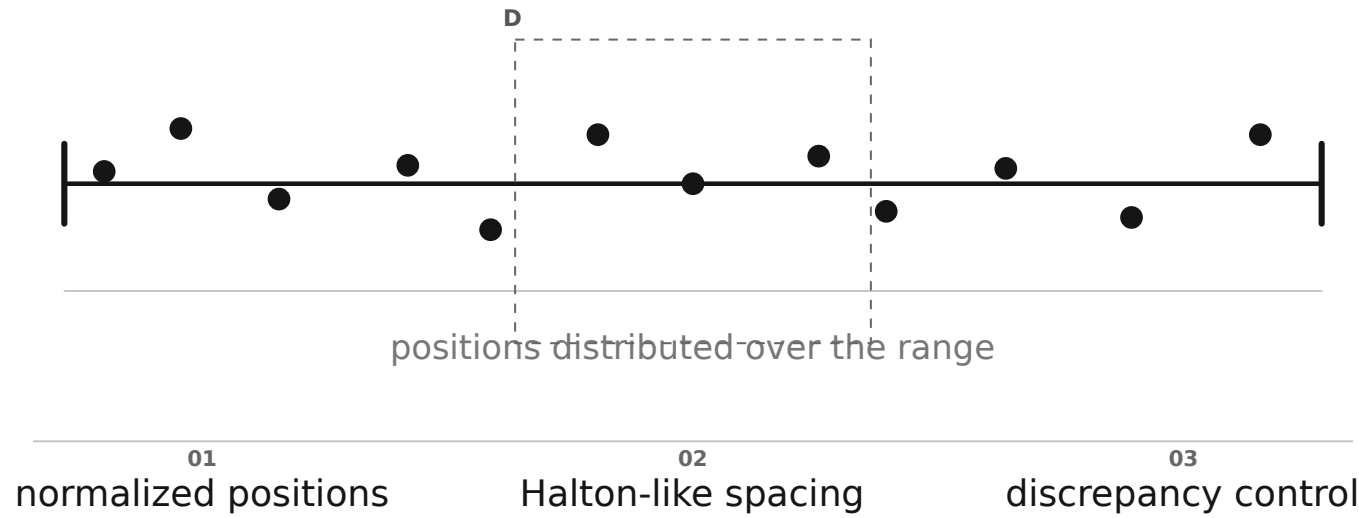
The code compares known-key weights with random-key weights. Changing numeral systems can change visible sparsity without reducing the number of possible keys.

ORIGINAL FINDING / EXECUTED RECORD

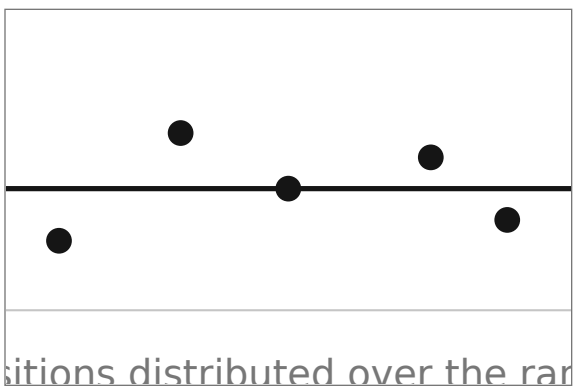
Compare Fibonacci representation weights with random keys; no significant anomaly reported.

M186

Low-discrepancy / Halton-like tier placement



D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Normalize key positions to [0,1) and measure gaps or star discrepancy against structured low-discrepancy schedules.

READ THE THREE STAGES

01 normalized positions
02 Halton-like spacing
03 discrepancy control

WHAT WAS ACTUALLY DONE

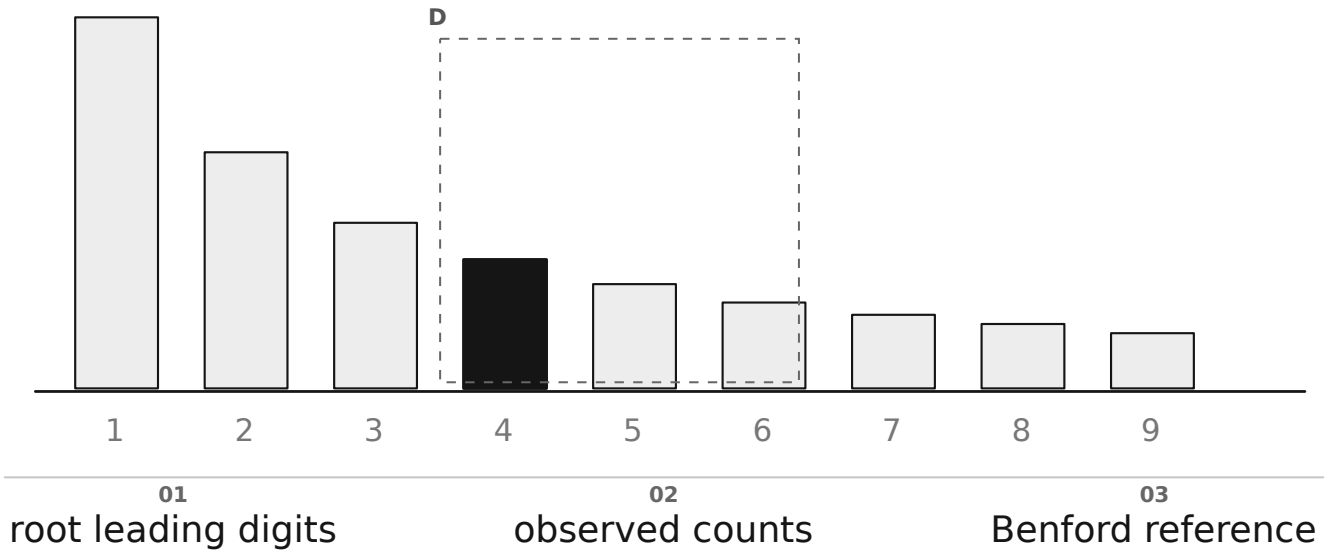
The creative report compares its discrepancy statistic with random controls. Evenly spread finite observations do not establish a deterministic next-key schedule.

ORIGINAL FINDING / EXECUTED RECORD

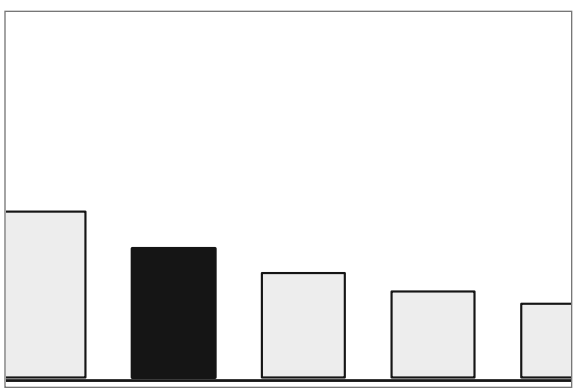
Compare normalized-position discrepancy; no significant deterministic schedule established.

M187

Benford leading digits of roots



D / OPERATION DETAIL



Vector detail / 1.00× main-view scale

HOW THE OPERATION WORKS

Compare first decimal digits of root values with a Benford-type reference distribution.

READ THE THREE STAGES

01 root leading digits
02 observed counts
03 Benford reference

WHAT WAS ACTUALLY DONE

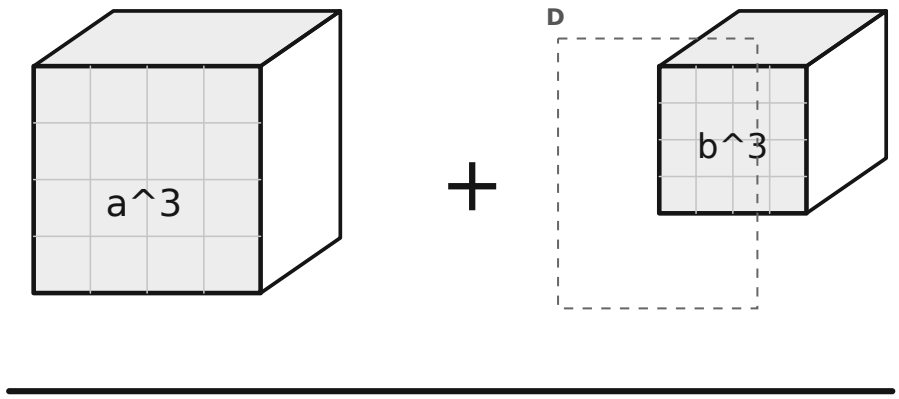
The report computes frequency/test summaries for known roots and controls. Agreement or disagreement is a distribution diagnostic, not enough information to determine unknown remainder digits.

ORIGINAL FINDING / EXECUTED RECORD

Root first-digit distribution compared with controls; no convincing signal reported.

M188

Taxicab sums of two cubes

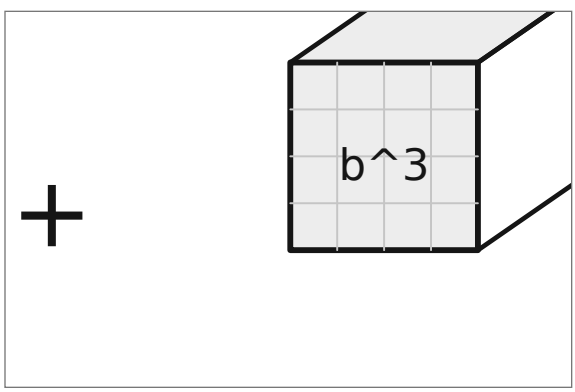


01
cube a

02
cube b

03
sum candidate

D / OPERATION DETAIL



Vector detail / 1.27× main-view scale

HOW THE OPERATION WORKS

Search whether a scalar is a sum of two integer cubes, potentially exposing a special generator family.

READ THE THREE STAGES

01 cube a
02 cube b
03 sum candidate

WHAT WAS ACTUALLY DONE

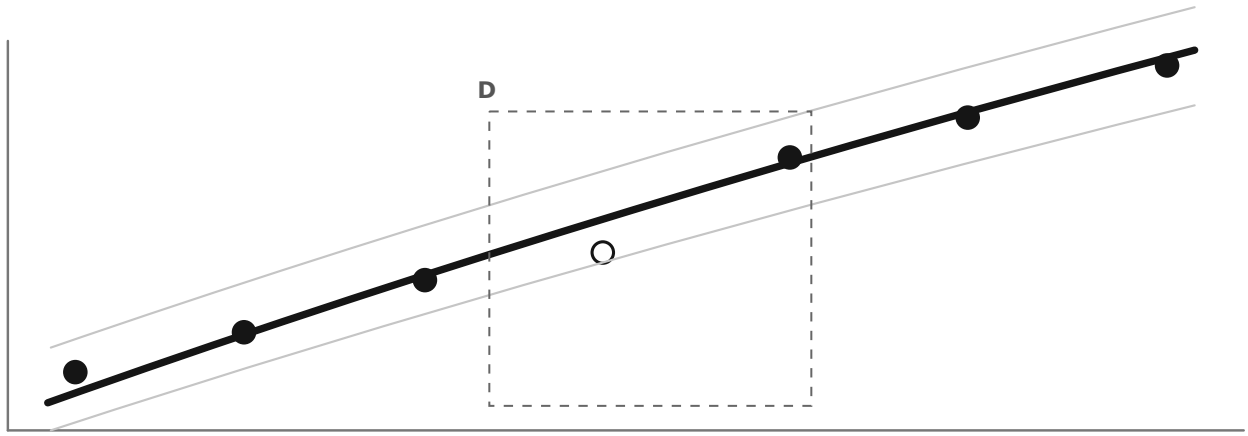
The creative report counts representations on disclosed keys and random controls. A special representation would still need independent evidence that the target was generated by that family.

ORIGINAL FINDING / EXECUTED RECORD

Search special cube-sum representations of known keys; no transferable high-index key relation.

M189

Polynomial in puzzle number

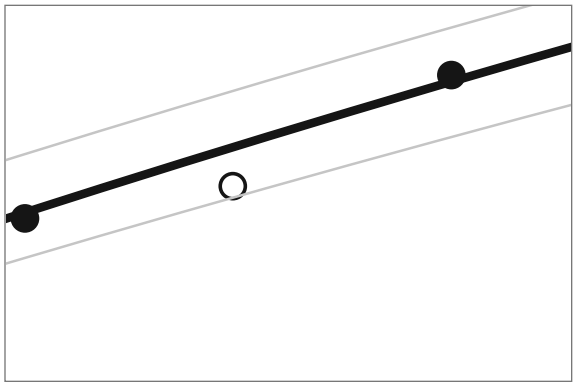


01 puzzle indices

02 polynomial fit

03 later scalar estimate

D / OPERATION DETAIL



Vector detail / 1.17× main-view scale

HOW THE OPERATION WORKS

Fit a low-degree function of puzzle number and forecast later key positions or values.

READ THE THREE STAGES

01 puzzle indices
02 polynomial fit
03 later scalar estimate

WHAT WAS ACTUALLY DONE

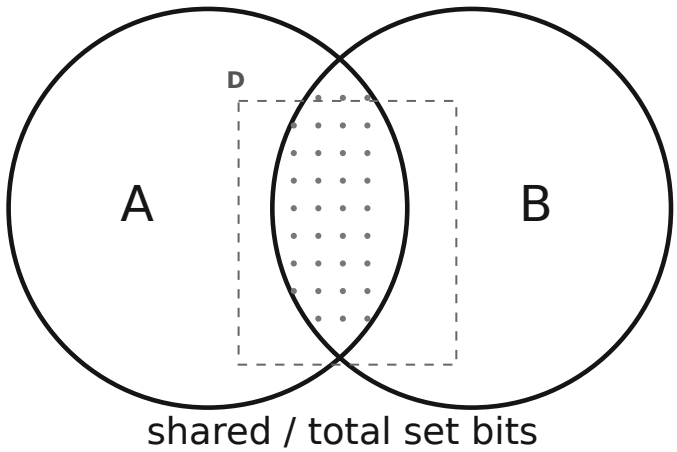
The creative theory uses leave-one-out error versus controls; separate exact continuation tests verify concrete candidates. A polynomial fitting all known points has no automatic predictive authority.

ORIGINAL FINDING / EXECUTED RECORD

Leave-one-out prediction compared with random keys; fitting known data does not determine the next scalar.

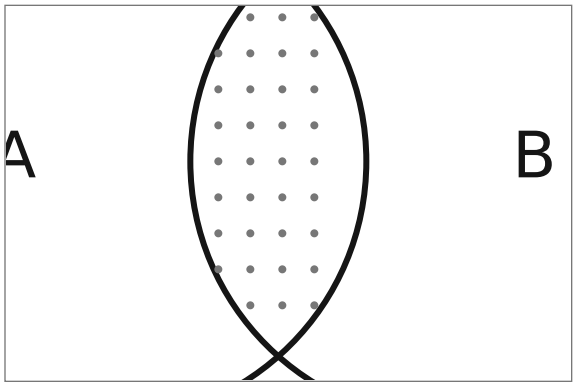
M190

Consecutive bit-Jaccard similarity



01	02	03
set bits of key A	intersection / union	set bits of key B

D / OPERATION DETAIL



Vector detail / 1.30× main-view scale

HOW THE OPERATION WORKS

View set-bit positions as sets and compute intersection divided by union for consecutive puzzle keys.

READ THE THREE STAGES

01 set bits of key A
02 intersection / union
03 set bits of key B

WHAT WAS ACTUALLY DONE

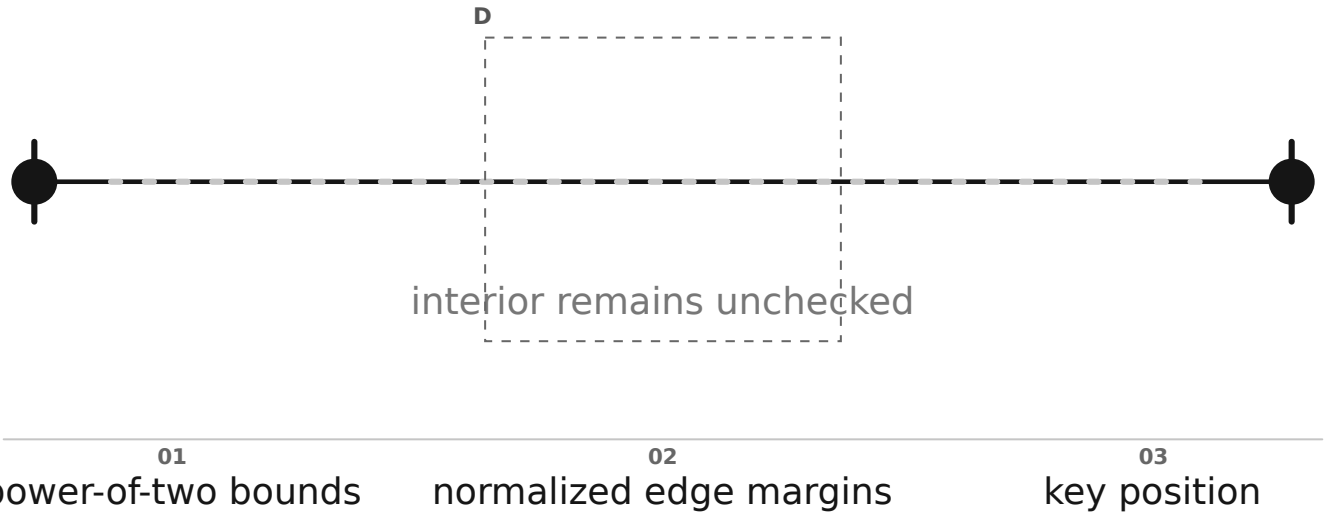
The report compares mean Jaccard overlap with random tier-matched keys. Similarity caused by bit widths must not be mistaken for inherited secret bits.

ORIGINAL FINDING / EXECUTED RECORD

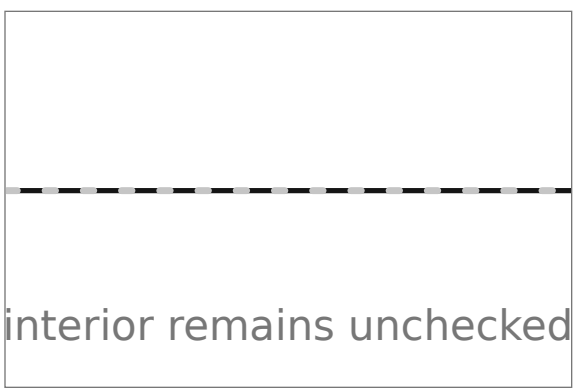
Bit-set overlap compared with random controls; no significant inheritance relation reported.

M191

Mersenne-adjacent margin hypothesis



D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Measure whether keys lie unusually close to powers of two or all-ones Mersenne-style boundaries.

READ THE THREE STAGES

01 power-of-two bounds
02 normalized edge margins
03 key position

WHAT WAS ACTUALLY DONE

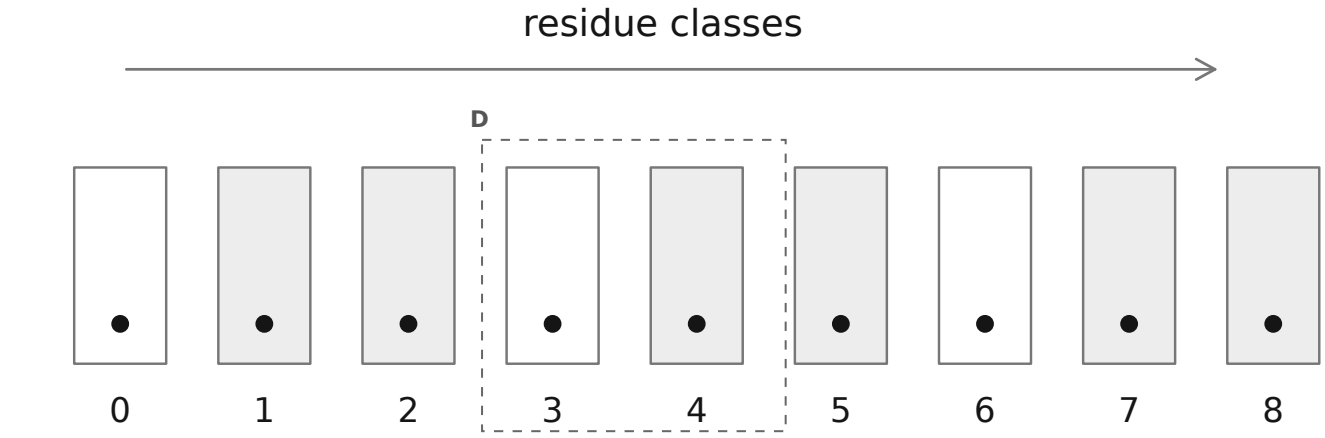
The creative test compares normalized margins with uniform controls. Small margins would identify a candidate region only under a validated generator prior.

ORIGINAL FINDING / EXECUTED RECORD

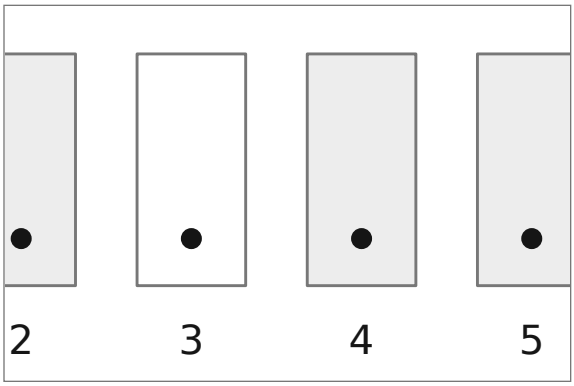
Distances to power-of-two boundaries compared with random ranges.

M192

Shared CRT residue / digital modular biases



D / OPERATION DETAIL



Vector detail / 1.18× main-view scale

HOW THE OPERATION WORKS

Count small-modulus residues and ask whether a common congruence distinguishes disclosed keys from random intervals.

READ THE THREE STAGES

01 small moduli
02 residue histograms
03 bias tests

WHAT WAS ACTUALLY DONE

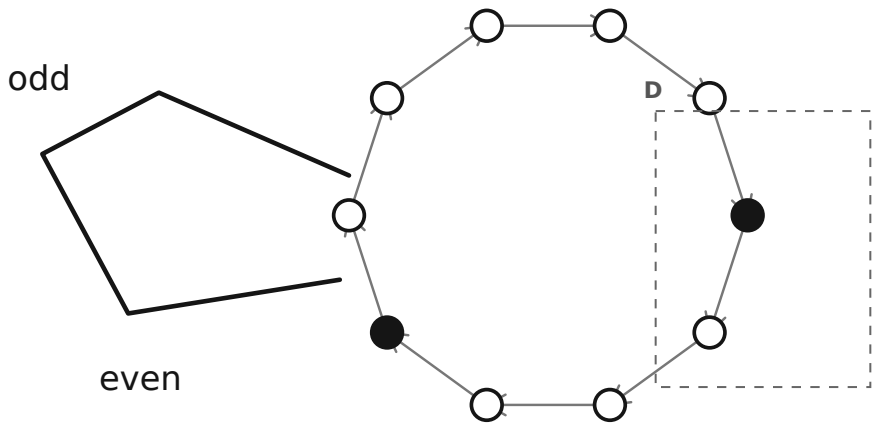
The report examines frequencies such as residues modulo three. Several weak residue preferences can arise by chance; they do not justify applying an unvalidated CRT restriction to the target.

ORIGINAL FINDING / EXECUTED RECORD

Small-modulus residue frequencies examined; exploratory bias is not proof of a target constraint.

M193

Collatz stopping-time association

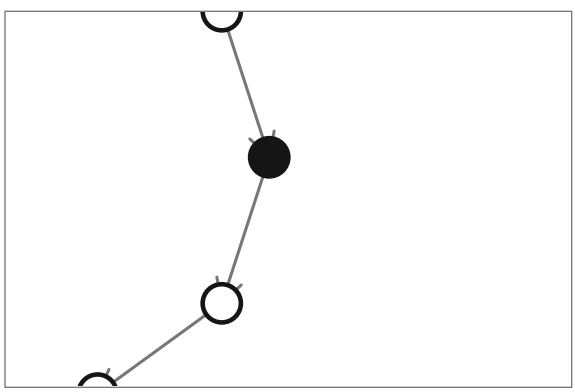


01 starting scalar

02 parity-dependent steps

03 stopping-time summary

D / OPERATION DETAIL



Vector detail / 1.25× main-view scale

HOW THE OPERATION WORKS

Map each key through the Collatz iteration and compare stopping-time statistics.

READ THE THREE STAGES

01 starting scalar
02 parity-dependent steps
03 stopping-time summary

WHAT WAS ACTUALLY DONE

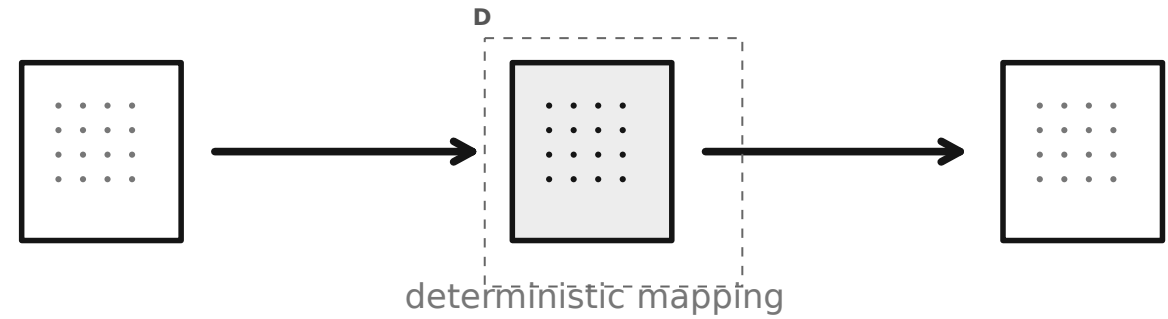
The creative report evaluates known and random keys under the same rule. The transformed statistic is not shown to be invertible from a target address or predictive of its scalar.

ORIGINAL FINDING / EXECUTED RECORD

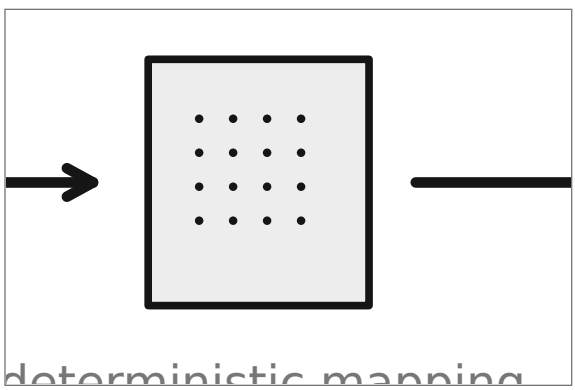
Compare stopping statistics with random-key controls; no predictive result reported.

M194

Brainwallet/BIP39 phrase schedule



D / OPERATION DETAIL



Vector detail / 1.38× main-view scale

HOW THE OPERATION WORKS

Hash or derive keys from a fixed catalog of phrases/mnemonics and apply the puzzle's interval mask.

READ THE THREE STAGES

01 phrase catalog
02 derive and mask
03 declared schedule

WHAT WAS ACTUALLY DONE

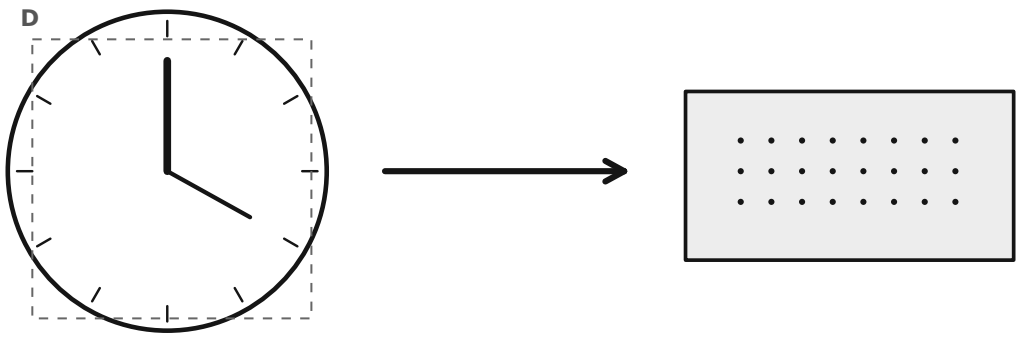
The creative family checks whether a phrase schedule reproduces significant known tiers. Tiny-tier coincidences are separated from evidence that one phrase generates later unknown keys.

ORIGINAL FINDING / EXECUTED RECORD

Catalog phrase-derived keys and masked tiers tested; archived creative report records no nontrivial hits.

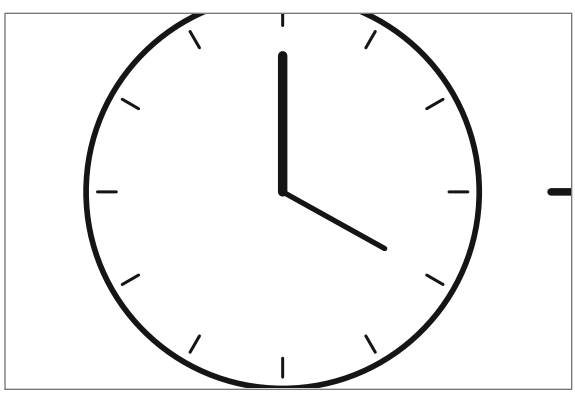
M195

Solve-date scalar encoding



01	02	03
solve-date numbers	encoding / fitting	scalar-position test

D / OPERATION DETAIL



Vector detail / 1.23× main-view scale

HOW THE OPERATION WORKS

Encode dates in scalars or compare normalized scalar positions with date-related numbers.

READ THE THREE STAGES

01 solve-date numbers
02 encoding / fitting
03 scalar-position test

WHAT WAS ACTUALLY DONE

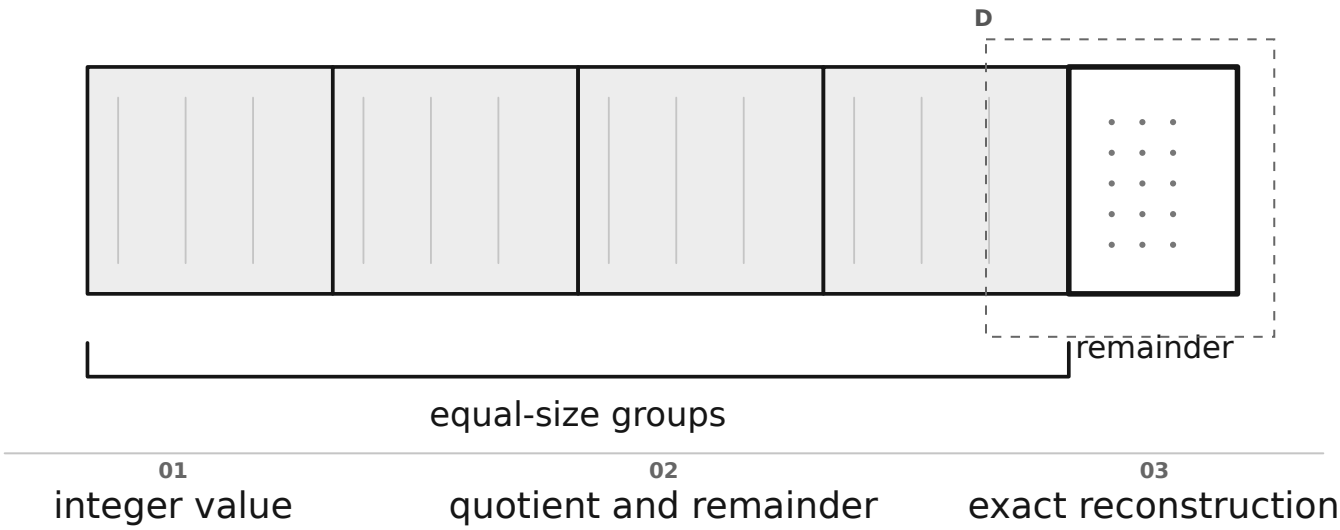
The report counts date-pattern matches and errors over known solve records. Solve date can reflect search effort rather than generation, so correlation would require independent validation.

ORIGINAL FINDING / EXECUTED RECORD

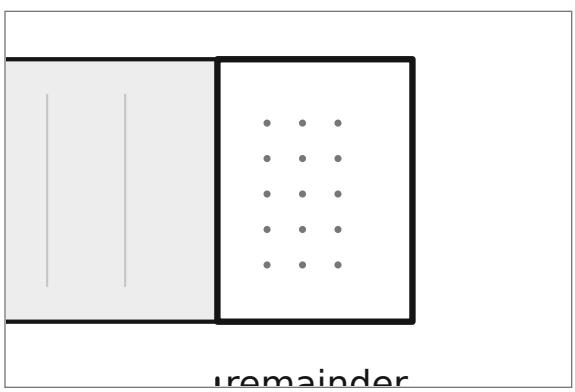
Test date patterns in keys; no significant association recorded.

M196

UniMath-inspired arithmetic identities



D / OPERATION DETAIL



Vector detail / 1.16× main-view scale

HOW THE OPERATION WORKS

Re-express arithmetic facts such as Euclidean division and reconstruction as executable exact assertions on known data.

READ THE THREE STAGES

01 integer value
02 quotient and remainder
03 exact reconstruction

WHAT WAS ACTUALLY DONE

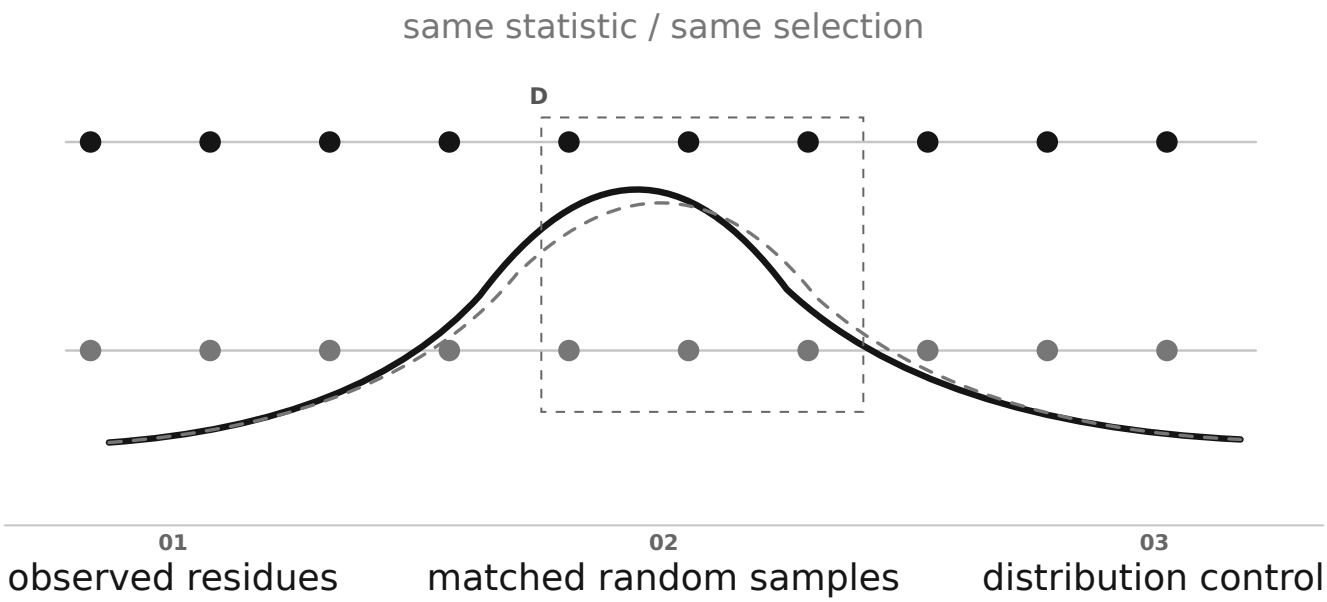
unimath_corpora.py mirrors named mathematical ideas in Python. Passing these checks validates that implementation and corpus, not a machine-checked Agda proof or an unknown-key extractor.

ORIGINAL FINDING / EXECUTED RECORD

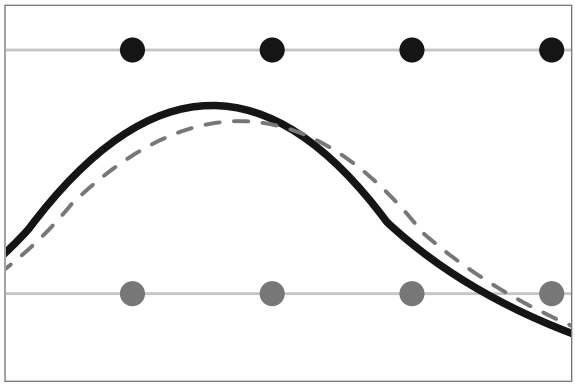
Exact division, reconstruction, congruence and remainder assertions on known keys; arithmetic checks are not an Agda proof or prediction.

M197

Residue/Legendre/coefficient distribution controls



D / OPERATION DETAIL



Vector detail / 1.17× main-view scale

HOW THE OPERATION WORKS

Compare coefficient, residue, Legendre-symbol and remainder distributions with matched random samples.

READ THE THREE STAGES

01 observed residues
02 matched random samples
03 distribution control

WHAT WAS ACTUALLY DONE

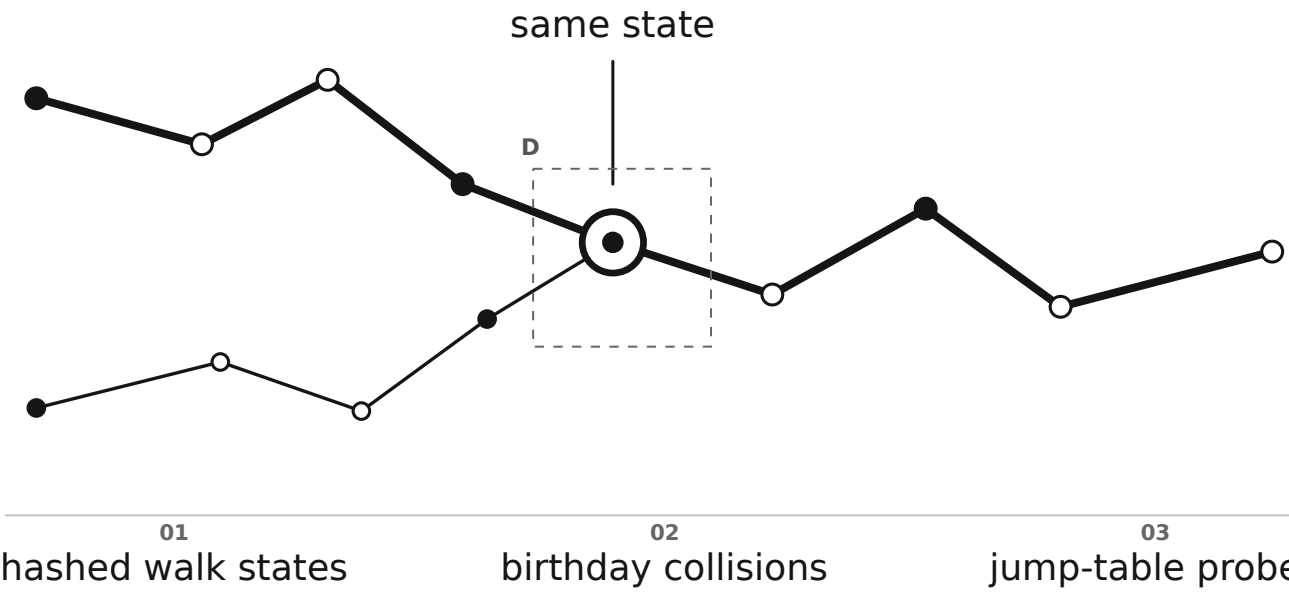
unimath_insights.py produces statistical summaries and controls. It tests distributional hypotheses rather than deriving new bits directly from the target public point.

ORIGINAL FINDING / IMPLEMENTED

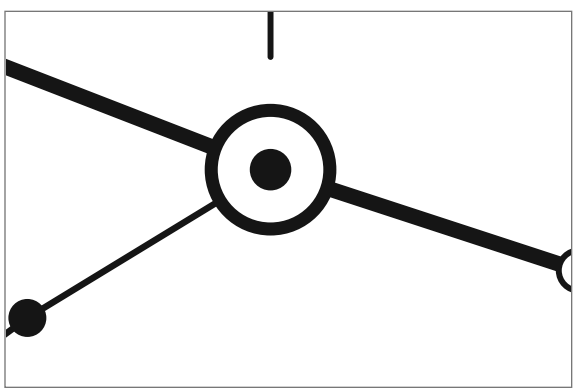
Random controls and KS/chi-square-style summaries distinguish transformation artifacts from candidate key bias.

M198

DP hashing/birthday/URAND jump probes



D / OPERATION DETAIL



Vector detail / 1.93× main-view scale

HOW THE OPERATION WORKS

Measure hash-table collisions, birthday behavior and candidate jump tables to improve practical walk mechanics.

READ THE THREE STAGES

01 hashed walk states
02 birthday collisions
03 jump-table probes

WHAT WAS ACTUALLY DONE

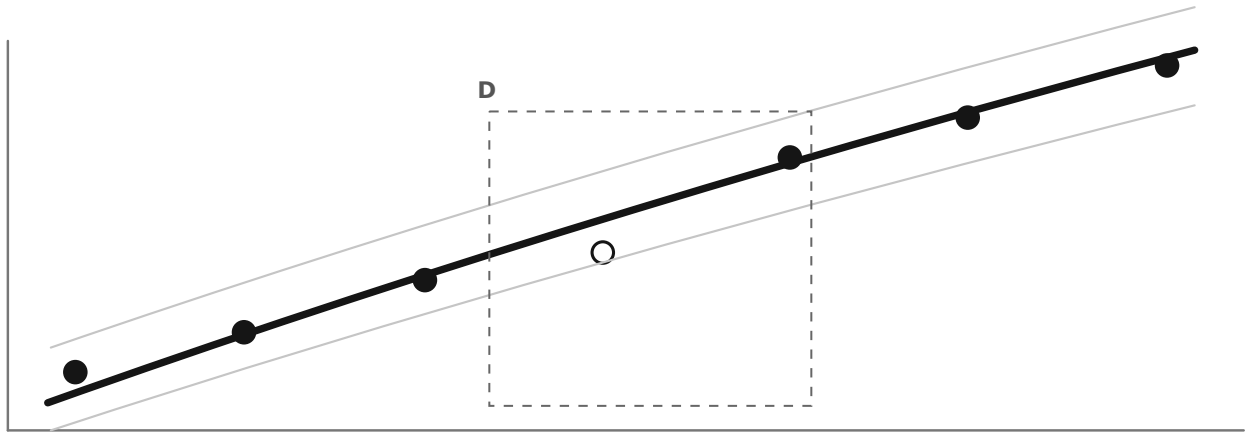
Bitsavers-inspired probes model open addressing and first-collision counts, then compare URAND-derived jumps. These are solver engineering experiments, not secret-generator observations.

ORIGINAL FINDING / IMPLEMENTED

Bitsavers-inspired hash-table, collision and jump-distribution experiments assess solver mechanics.

M199

Regression SVD/subset/interval arithmetic variants

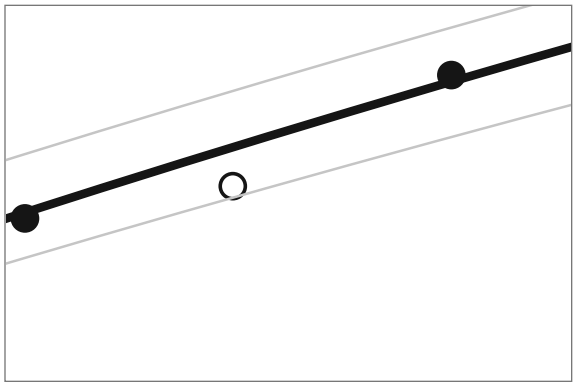


01 data subsets

02 SVD / interval rounding

03 forecast bounds

D / OPERATION DETAIL



Vector detail / 1.17× main-view scale

HOW THE OPERATION WORKS

Refit numerical predictors using alternative regression subsets/SVD and propagate rounding intervals through formulas.

READ THE THREE STAGES

01 data subsets
02 SVD / interval rounding
03 forecast bounds

WHAT WAS ACTUALLY DONE

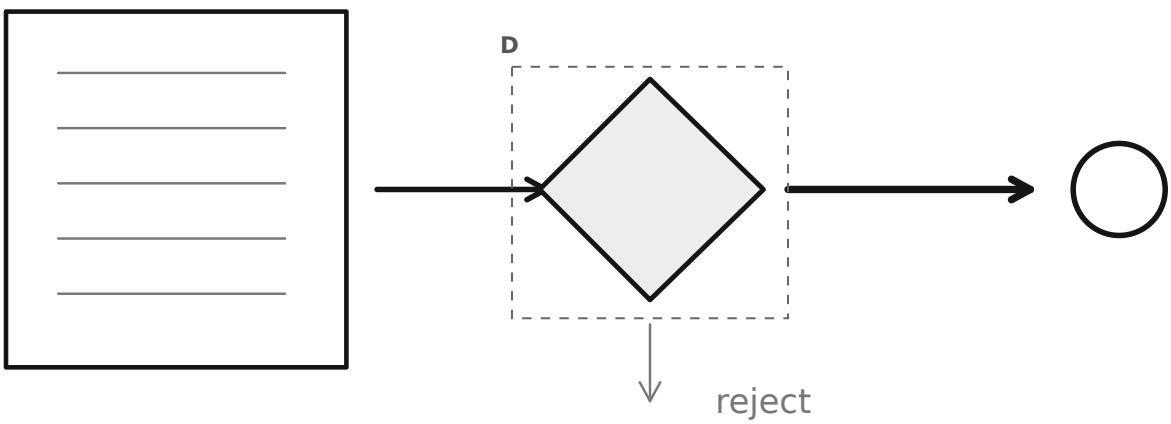
The historical methods module compares fit stability, numerical bounds and selected regressions. It can diagnose approximation error but cannot manufacture predictive information absent from the data.

ORIGINAL FINDING / IMPLEMENTED

Historical numerical-method variants audit R2 fits and rounding; not new information about the target.

M200

MOV and finite-field index-calculus applicability

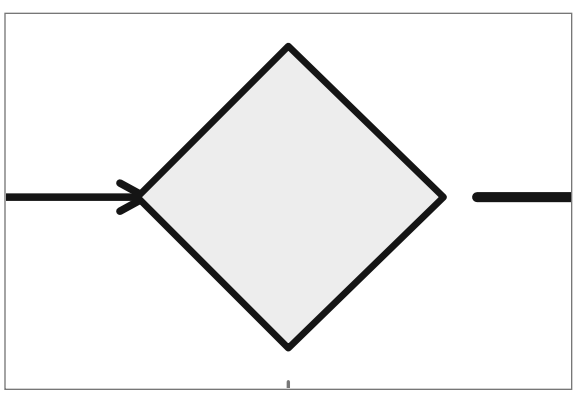


01
curve / field parameters

02
embedding-degree conditions

03
MOV applicability

D / OPERATION DETAIL



Vector detail / 1.37× main-view scale

HOW THE OPERATION WORKS

MOV reduces certain elliptic-curve logs to finite fields when the embedding degree is suitably small; index calculus has domain-specific requirements.

READ THE THREE STAGES

01 curve / field parameters
02 embedding-degree conditions
03 MOV applicability

WHAT WAS ACTUALLY DONE

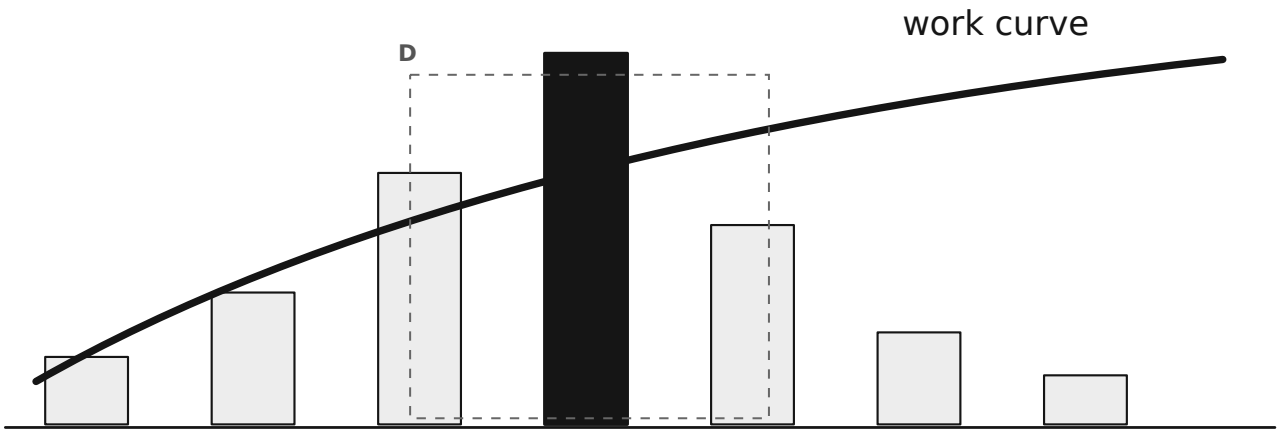
The registry estimates/checks applicability for secp256k1 and records why named structural attacks do not straightforwardly apply. It is not an executed transfer solving the target log.

ORIGINAL FINDING / APPLICABILITY REVIEW

Registry checks distinguish attacks requiring unsuitable curve structure from applicable interval methods.

M201

DDL preprocessing tradeoff bounds

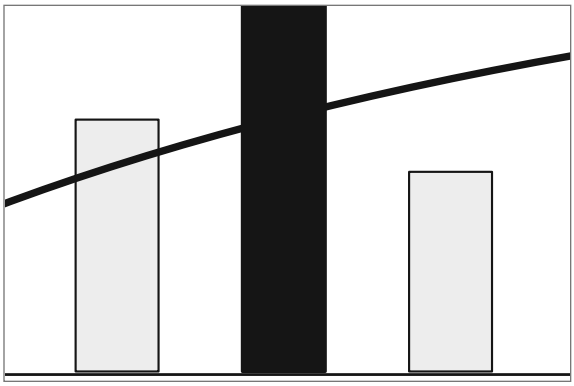


01 preprocessing work

02 memory / online work

03 amortized tradeoff

D / OPERATION DETAIL



Vector detail / 1.00× main-view scale

HOW THE OPERATION WORKS

Restricted discrete-log methods trade preprocessing, memory and online work under stated target/setup assumptions.

READ THE THREE STAGES

01 preprocessing work
02 memory / online work
03 amortized tradeoff

WHAT WAS ACTUALLY DONE

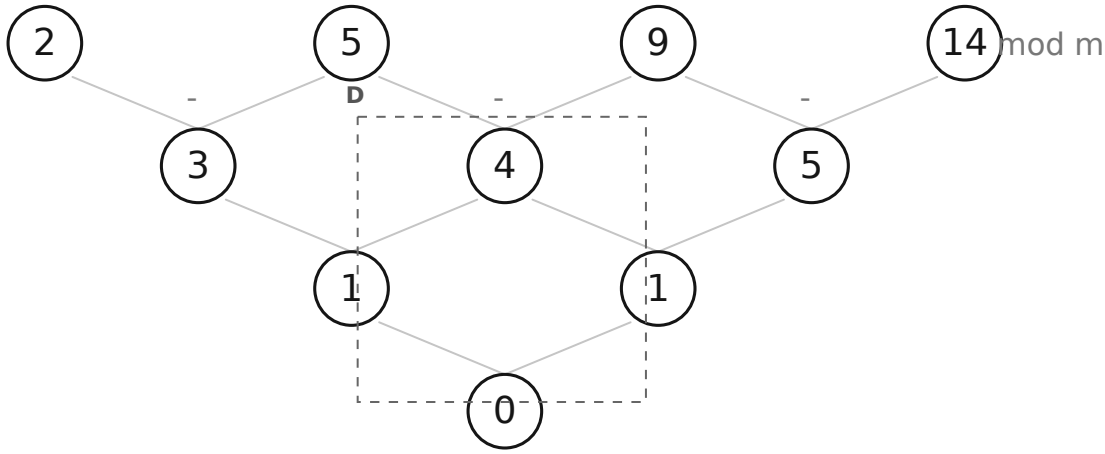
The JoC module evaluates heuristic DDL bounds and synthetic scenarios. Estimated online savings must include required preprocessing and oracles before being compared with an actual puzzle search.

ORIGINAL FINDING / SYNTHETIC/APPLICABILITY

Analysis estimates restricted discrete-log tradeoffs under additional setup assumptions; no blind puzzle recovery demonstrated.

M202

Finite modular difference triangles

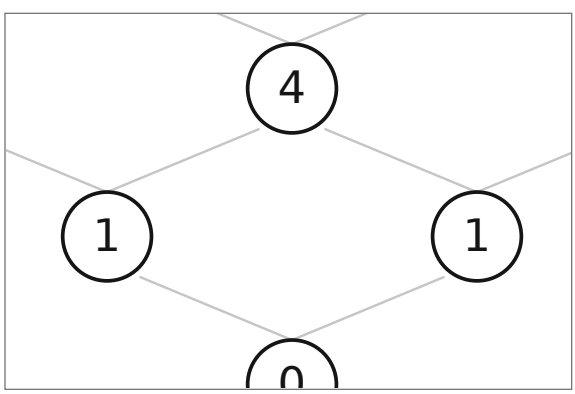


01 finite residue row

02 neighbor subtraction

03 shrinking triangle

D / OPERATION DETAIL



Vector detail / 1.21× main-view scale

HOW THE OPERATION WORKS

Repeatedly subtract neighboring residues modulo m; finite rows shrink by one entry at each level.

READ THE THREE STAGES

01 finite residue row
02 neighbor subtraction
03 shrinking triangle

WHAT WAS ACTUALLY DONE

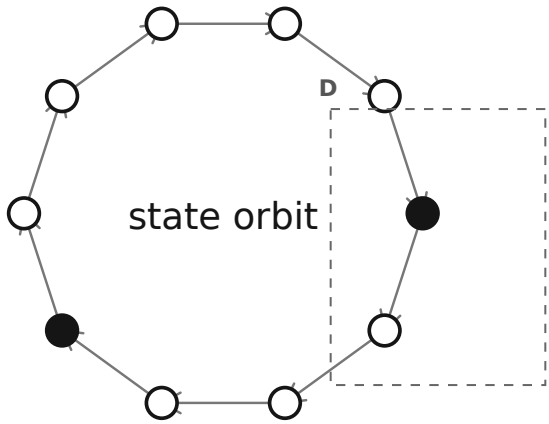
The experiment follows declared polynomial/exponential residue families through their full finite triangles. Reaching zero on supplied samples does not determine an unknown continuation without a recurrence assumption.

ORIGINAL FINDING / EXECUTED RECORD

Exact finite-row experiments track polynomial/exponential residues through all levels; they do not reconstruct an unknown puzzle key.

M203

Cyclic difference orbit classification

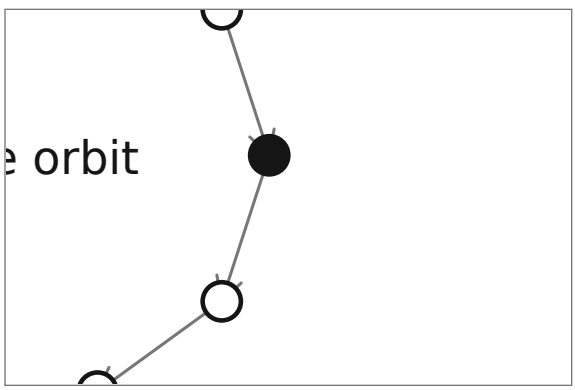


01
cyclic difference row

02
wraparound subtraction

03
finite-state cycle

D / OPERATION DETAIL



Vector detail / 1.25× main-view scale

HOW THE OPERATION WORKS

Wrap the difference operator cyclically so row length stays fixed, then detect exact zero or repeated states in a finite state graph.

READ THE THREE STAGES

01 cyclic difference row
02 wraparound subtraction
03 finite-state cycle

WHAT WAS ACTUALLY DONE

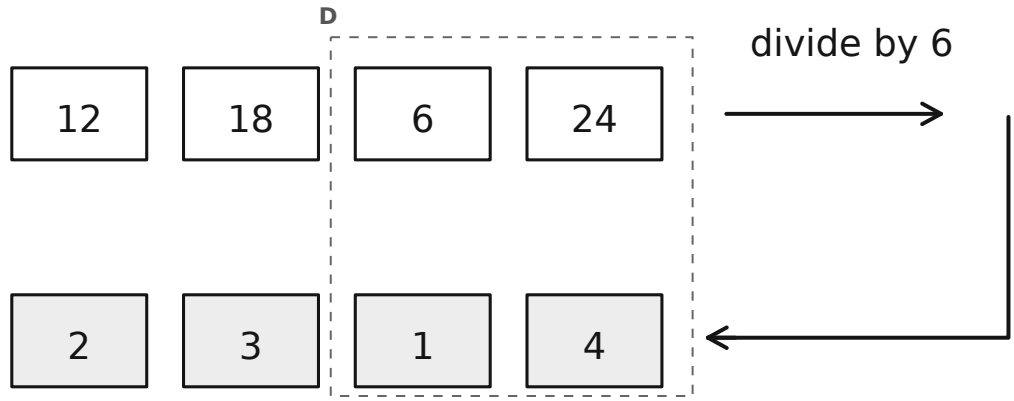
The saved study enumerates all starting rows within its declared small configurations and classifies their orbits. This is an exhaustive finite dynamics result, not cryptographic inversion.

ORIGINAL FINDING / EXECUTED RECORD

Exhaustive finite study classified 19,617 starting rows over 35 configurations into zero/cycles.

M204

Signed quotient hierarchy with common-factor removal

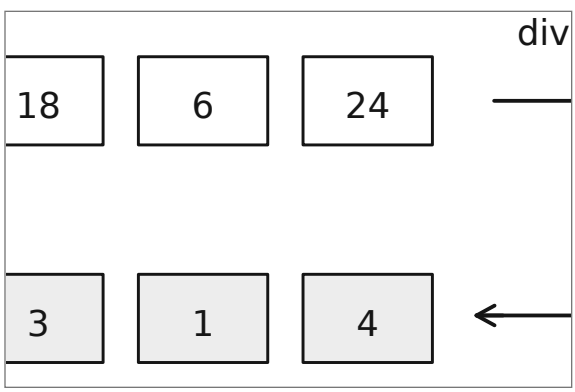


01 signed difference row

02 remove common powers

03 primitive next row

D / OPERATION DETAIL



Vector detail / 0.96× main-view scale

HOW THE OPERATION WORKS

Keep signed integer differences and divide a row by m whenever every entry shares that factor before taking the next difference.

READ THE THREE STAGES

01 signed difference row
02 remove common powers
03 primitive next row

WHAT WAS ACTUALLY DONE

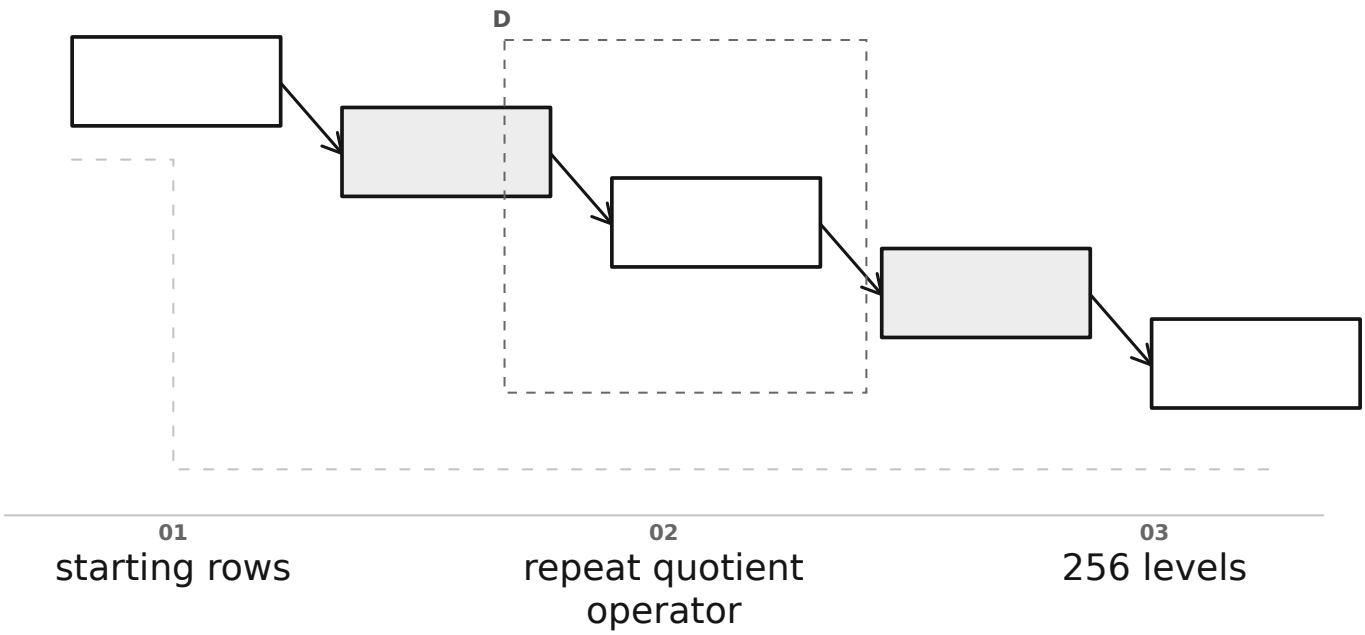
The quotient experiment records divisions and exact normalized states. Removing common factors exposes structure but can still leave growing values or unresolved trajectories.

ORIGINAL FINDING / EXECUTED RECORD

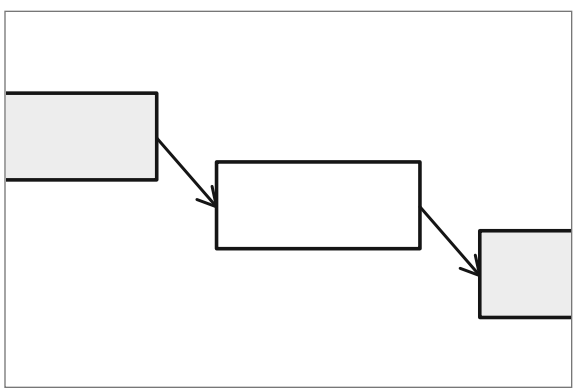
Remove factors of the full modulus before continuing signed differences; zero, cycle and capped outcomes remain separate.

M205

256-level quotient continuation



D / OPERATION DETAIL



Vector detail / 0.98× main-view scale

HOW THE OPERATION WORKS

Extend the same bounded quotient experiment to more levels without changing the starting rows or normalization rule.

READ THE THREE STAGES

01 starting rows
02 repeat quotient operator
03 256 levels

WHAT WAS ACTUALLY DONE

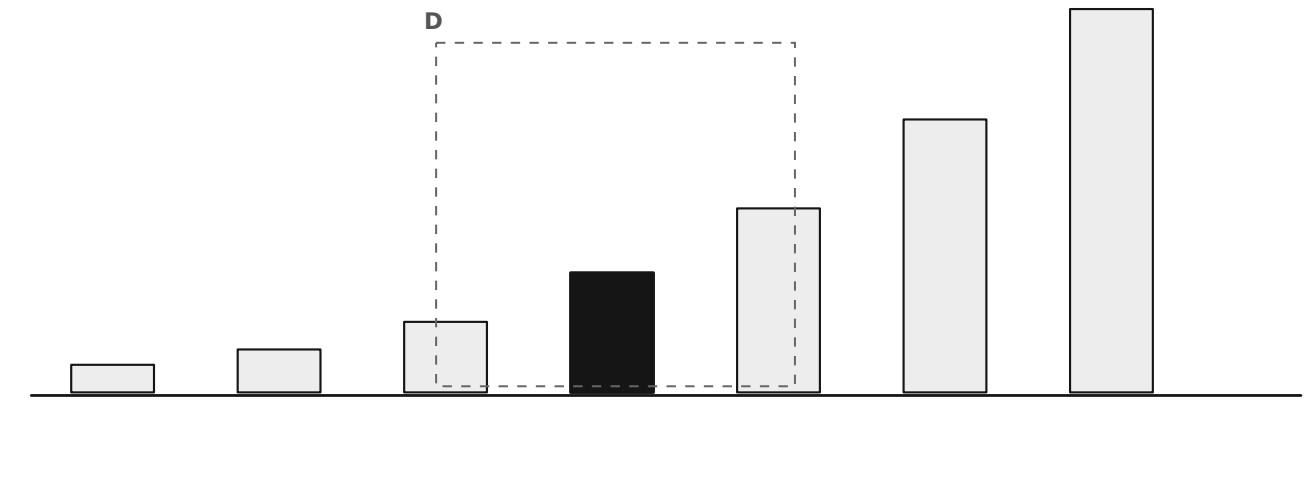
The 256-level survey compares outcomes with the earlier 128-level cap. Capped rows remain unresolved; lack of termination by the cap is not proof of infinite nontermination.

ORIGINAL FINDING / EXECUTED RECORD

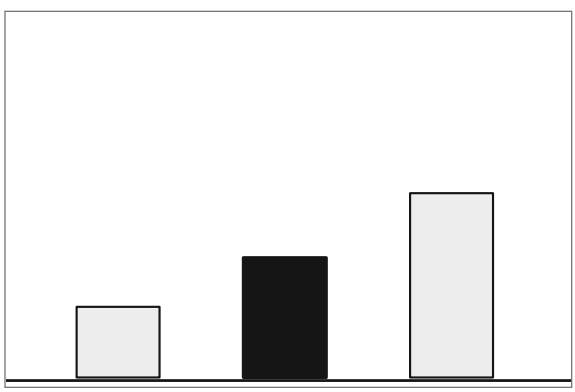
Saved survey records 163 zero, 600 cycles and 18,854 unresolved rows; no new resolutions after the earlier cap.

M206

All-value magnitude/growth tracking



D / OPERATION DETAIL



Vector detail / 1.00× main-view scale

HOW THE OPERATION WORKS

Track magnitudes, signs, divisibility and growth of every entry across every difference level, including after an earlier event.

READ THE THREE STAGES

01 all signed values
02 track every level
03 magnitude growth

WHAT WAS ACTUALLY DONE

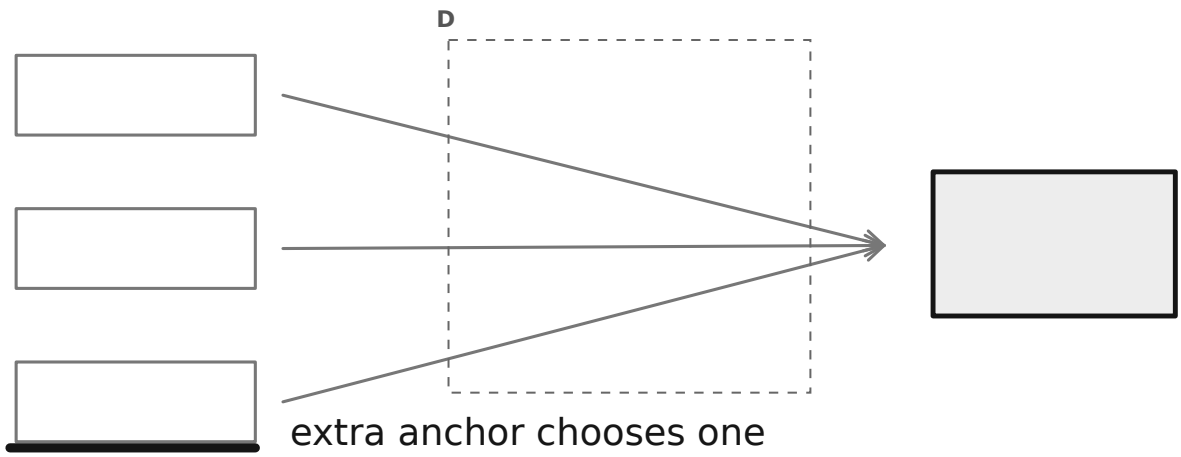
The all-values run records more than 23 million observations and reports extreme bit lengths. It measures behavior of those exact finite trajectories, not secret-key information gained.

ORIGINAL FINDING / EXECUTED RECORD

More than 23 million entry observations distinguish growth from termination; unresolved finite caps are not nontermination proofs.

M207

Inverse-difference kernel/anchor tests

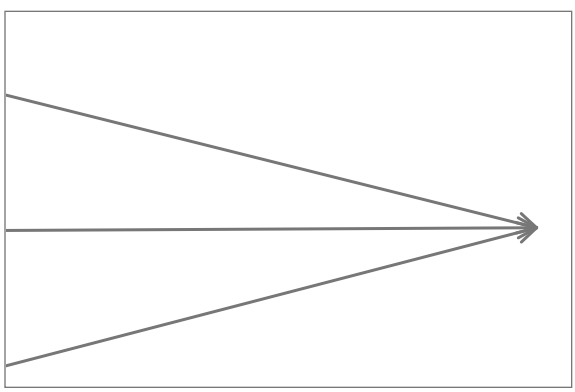


01 shared difference row

02 many possible originals

03 anchor selects one

D / OPERATION DETAIL



Vector detail / 0.98× main-view scale

HOW THE OPERATION WORKS

Group all starting rows that produce the same observed difference row. Additional anchors can select among these inverse candidates.

READ THE THREE STAGES

01 shared difference row
02 many possible originals
03 anchor selects one

WHAT WAS ACTUALLY DONE

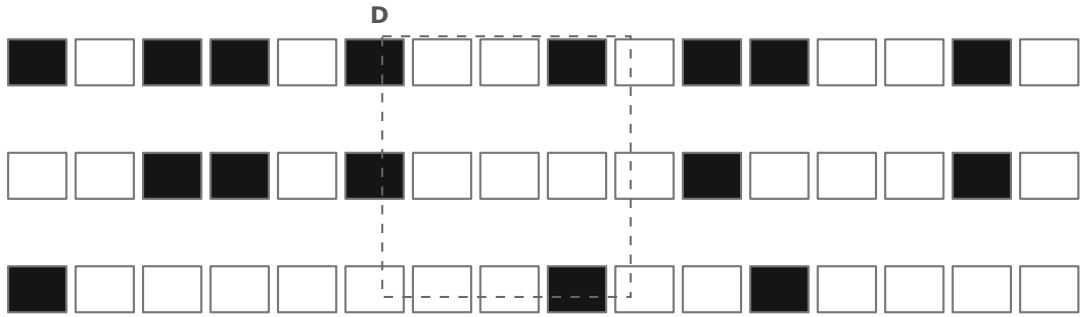
inverse_capability.py exhaustively counts compatible inputs in small finite systems. If several remain, the transformed row alone has lost information and cannot uniquely reconstruct the original.

ORIGINAL FINDING / EXECUTED RECORD

Enumerate compatible starting rows; anchors can resolve lost constants, but the residual hierarchy supplies no missing observations.

M208

Deterministic index, byte and bit evolution

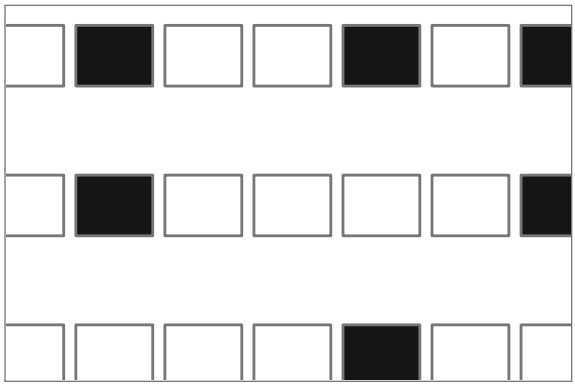


01 seed and index

02 byte / bit updates

03 deterministic evolution

D / OPERATION DETAIL



Vector detail / 1.32× main-view scale

HOW THE OPERATION WORKS

Define reversible or simple deterministic maps from an index into bitstrings, including increments and fixed byte/bit changes.

READ THE THREE STAGES

01 seed and index
02 byte / bit updates
03 deterministic evolution

WHAT WAS ACTUALLY DONE

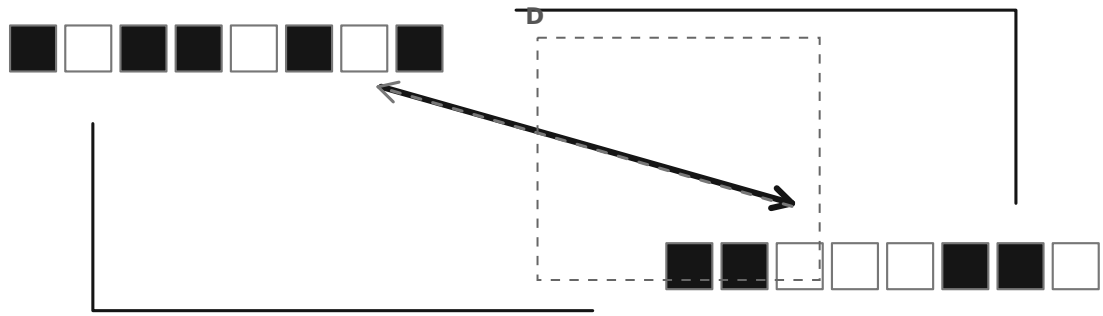
The unified framework executes registered sequence maps on artificial states. It generates controlled trajectories rather than learning the unknown puzzle creator's generation rule.

ORIGINAL FINDING / IMPLEMENTED

Unified framework includes index mapping, linear increments, byte/bit steps and monotonic traversal; code is not an unknown-key predictor.

M209

Masking, rotation and coupled sequences

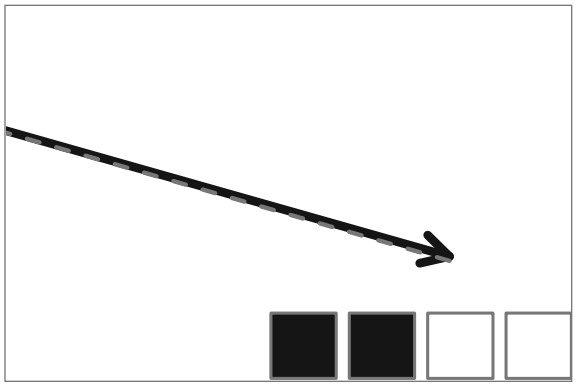


01
paired states

02
mask / rotation coupling

03
next paired states

D / OPERATION DETAIL



Vector detail / 1.42× main-view scale

HOW THE OPERATION WORKS

Apply masks, rotate bit positions or couple two evolving states according to an explicit deterministic rule.

READ THE THREE STAGES

01 paired states
02 mask / rotation coupling
03 next paired states

WHAT WAS ACTUALLY DONE

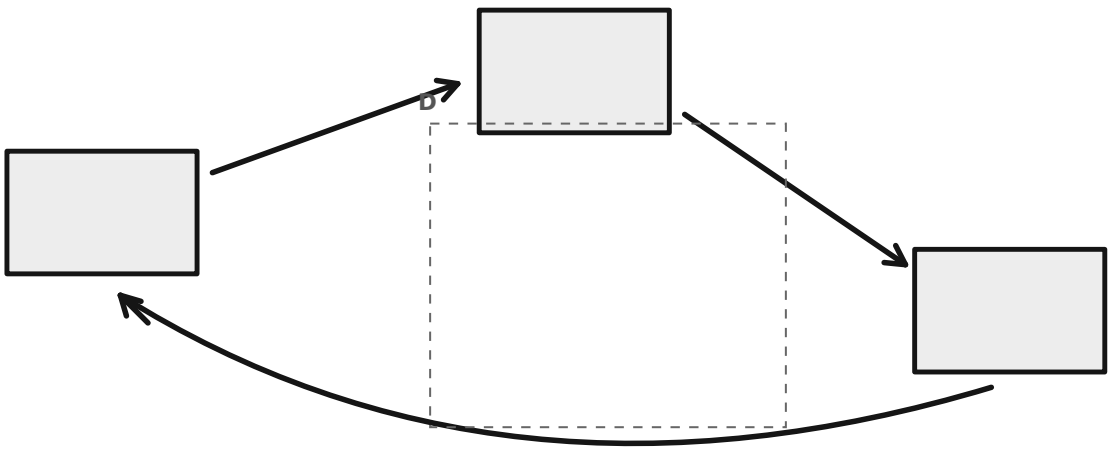
Registered transforms create particular orbits; some masks are lossy. Comparing their shapes cannot establish that a real target key belongs to one of those orbits.

ORIGINAL FINDING / IMPLEMENTED

Unified framework includes mask application, dual-sequence coupling, mask rotation and structured 256-step schedules.

M210

First/second-difference and drift schedulers

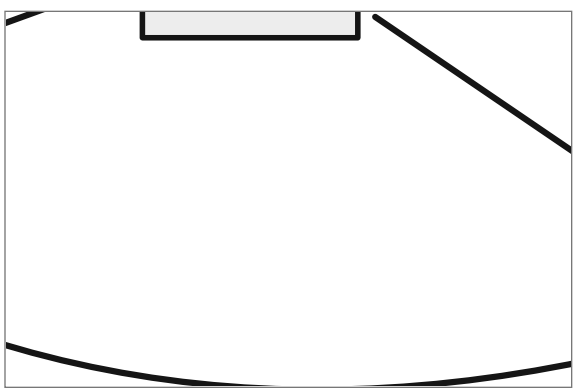


01
first / second differences

02
drift summaries

03
choose next operation

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Measure first/second differences and choose the next transformation based on drift or entropy summaries.

READ THE THREE STAGES

01 first / second differences
02 drift summaries
03 choose next operation

WHAT WAS ACTUALLY DONE

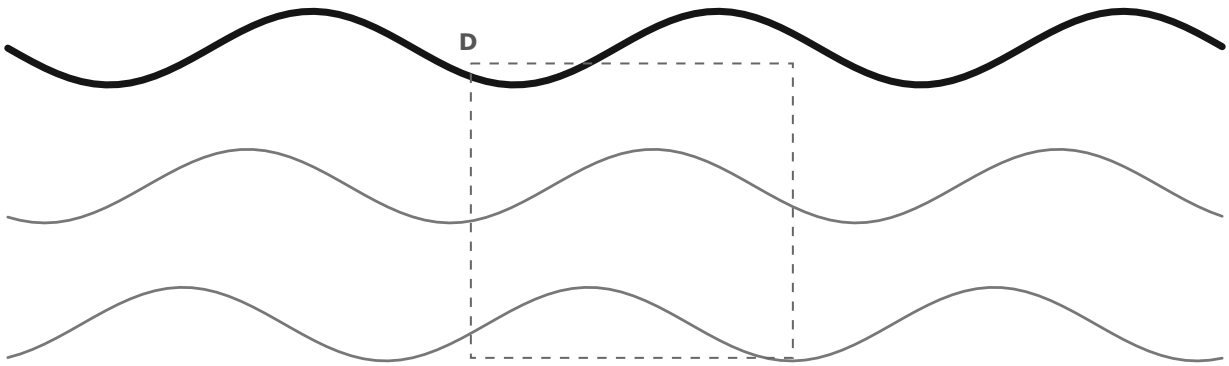
The framework's schedulers switch among existing maps using their configured rules. Adaptation optimizes trajectory diagnostics; it is not rewarded by an independently discovered target-key relation.

ORIGINAL FINDING / IMPLEMENTED

Analyze trajectory differences or switch transformations using entropy thresholds; diagnostic evolution is not key recovery.

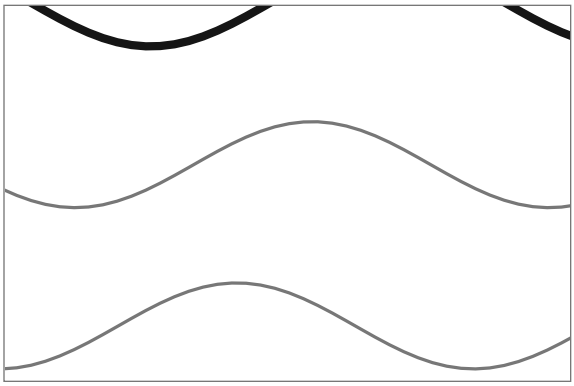
M211

Entropy, cycles, compression and spectral diagnostics



01	02	03
bit trajectories	entropy / cycles / spectra	random controls

D / OPERATION DETAIL



Vector detail / 1.17× main-view scale

HOW THE OPERATION WORKS

Compute bit entropy, compression, cycles, nearest-neighbor spacings and spectral summaries, then compare with random trajectories.

READ THE THREE STAGES

01 bit trajectories
02 entropy / cycles / spectra
03 random controls

WHAT WAS ACTUALLY DONE

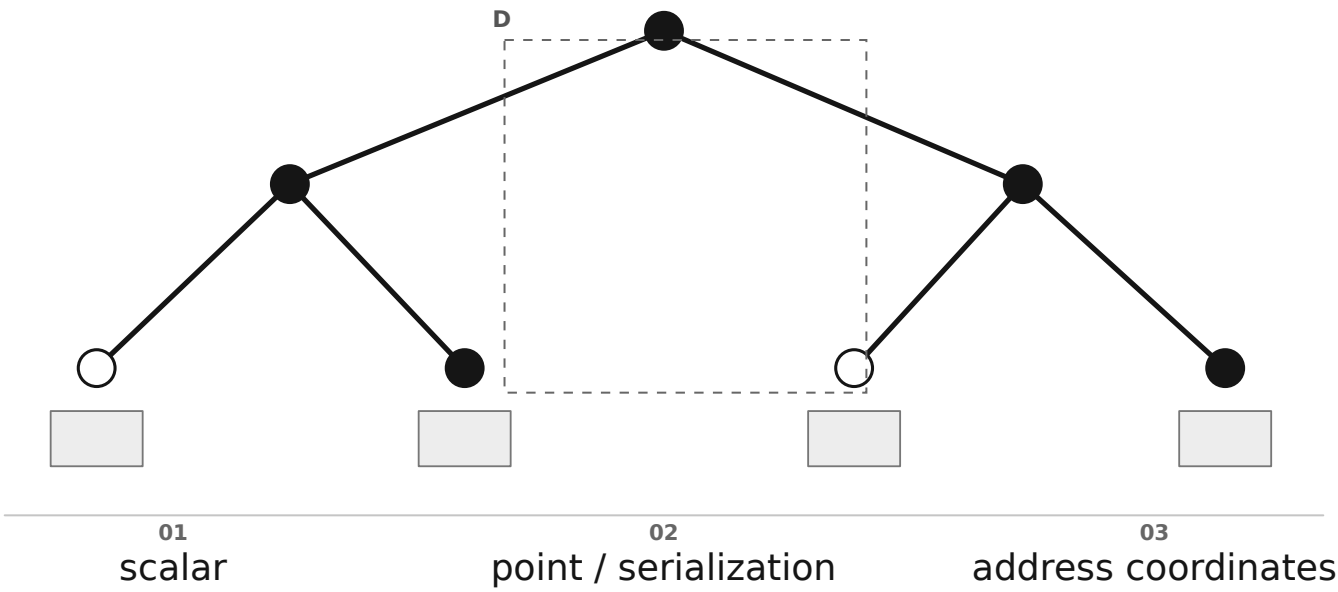
Classical arrays and statistics produce these measurements. Quantum-chaos terminology denotes analogy/diagnostics, not a quantum computation or an inverse map from address hashes to private scalars.

ORIGINAL FINDING / EXECUTED RECORD

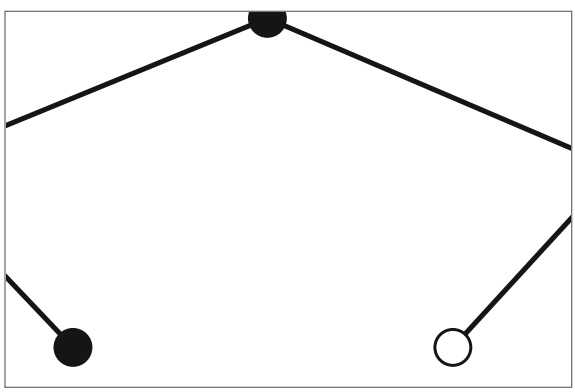
Compare deterministic trajectories with uniform controls, including cycle tests, spacings and spectral summaries; no quantum execution implied.

M212

Four-dimensional key/public/address visualization



D / OPERATION DETAIL



Vector detail / 0.98× main-view scale

HOW THE OPERATION WORKS

Place scalar, public-point and address-hash objects into shared visualization coordinates and connect their derivation edges.

READ THE THREE STAGES

01 scalar
02 point / serialization
03 address coordinates

WHAT WAS ACTUALLY DONE

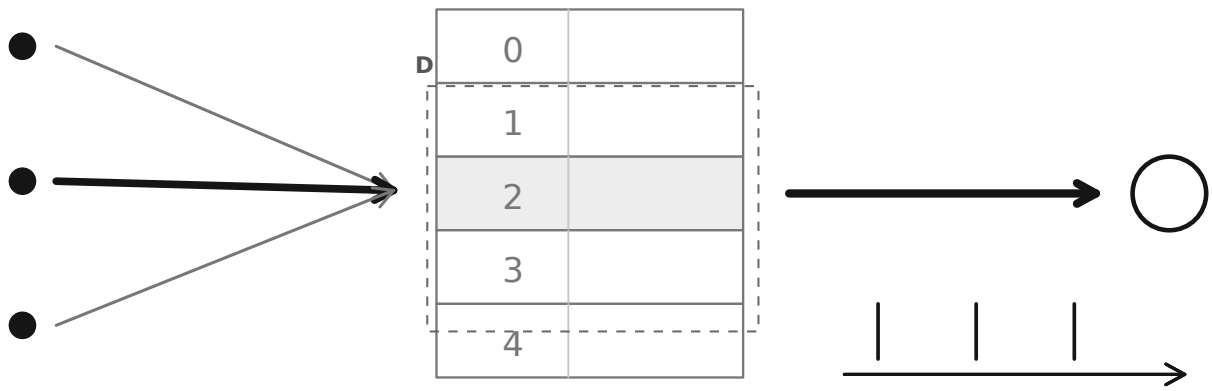
keyspace_4d.py supports raw/ring-style layouts. Normalized coordinates can illustrate hashing's loss of visible geometry without revealing an exploitable inverse relationship.

ORIGINAL FINDING / IMPLEMENTED

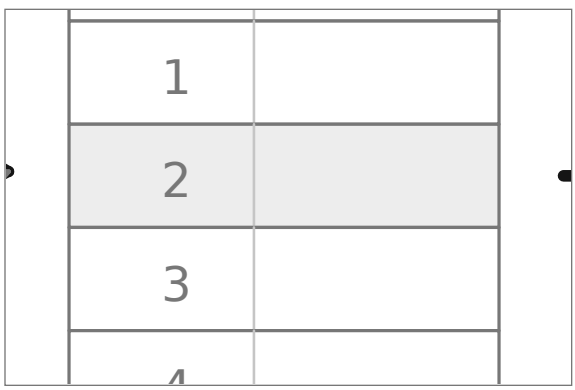
Display scalar-to-point-to-hash relationships in alternate layouts; no invertible address-to-scalar map is established.

M213

Classical BSGS multiplicative-order finder



D / OPERATION DETAIL



Vector detail / 1.40× main-view scale

HOW THE OPERATION WORKS

BSGS finds an exponent r with a^r=1 modulo N by matching stored small powers against larger steps.

READ THE THREE STAGES

01 small modular powers
02 matching table
03 large exponent steps

WHAT WAS ACTUALLY DONE

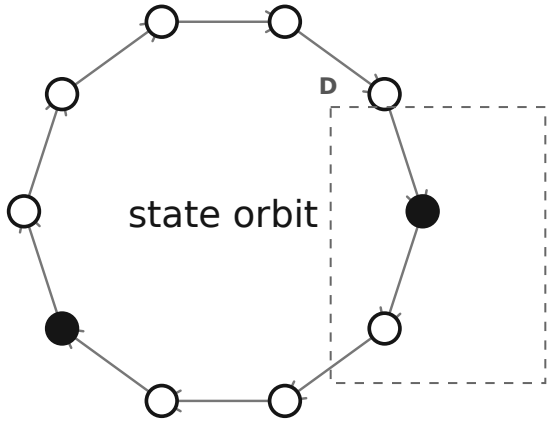
Classical Python/native order routines operate on bounded integer examples. In the factoring reduction they replace the quantum order-finding oracle; no QFT is executed.

ORIGINAL FINDING / EXECUTED RECORD

Exact small-integer order traces and backend comparisons; stands in for an order oracle, not a quantum device.

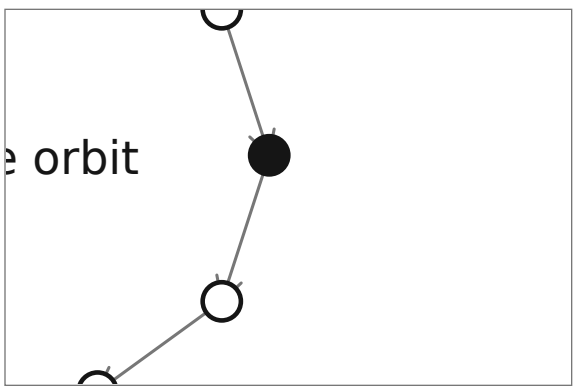
M214

Brent cycle-detection order finder



01	02	03
multiply modulo N	Brent checkpoints	return period

D / OPERATION DETAIL



Vector detail / 1.25× main-view scale

HOW THE OPERATION WORKS

Brent's cycle detector follows repeated multiplication modulo N while using a low-memory schedule to detect a return period.

READ THE THREE STAGES

01 multiply modulo N
02 Brent checkpoints
03 return period

WHAT WAS ACTUALLY DONE

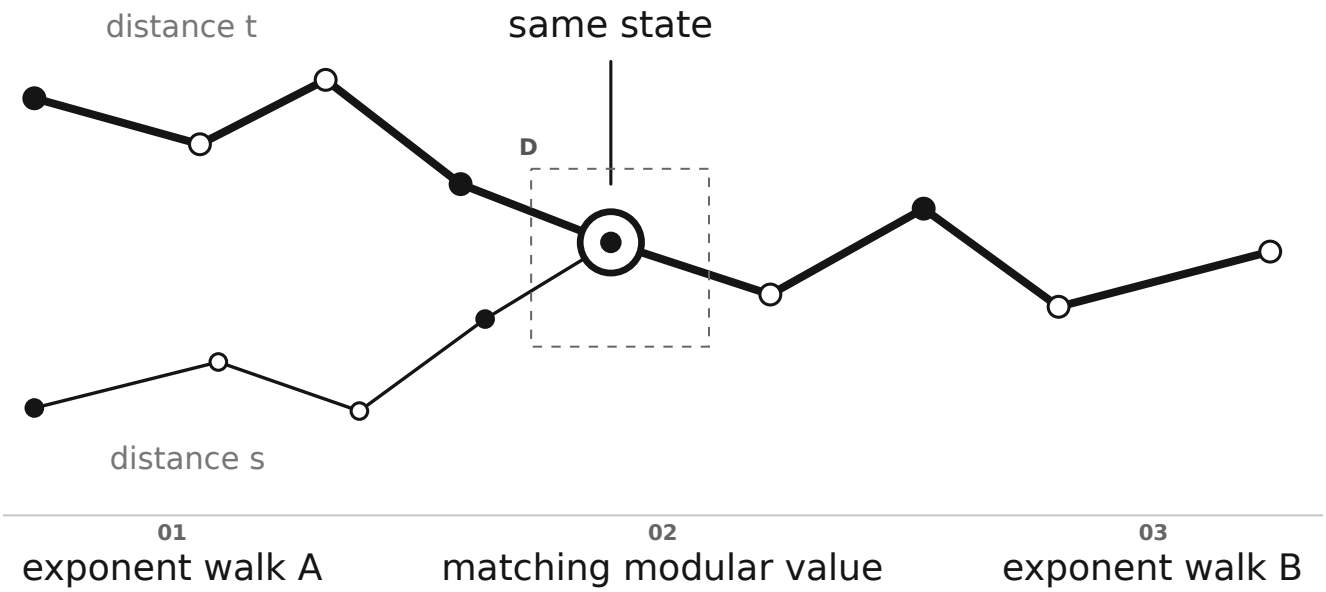
The implementation measures this classical order finder against shared small inputs. Its runtime grows with the period and should not be mistaken for Shor's quantum complexity.

ORIGINAL FINDING / EXECUTED RECORD

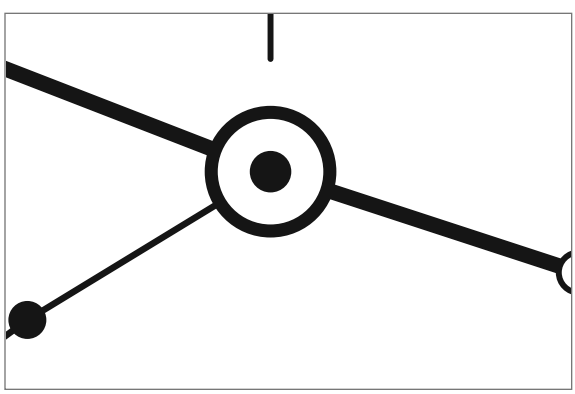
Classical linear-period comparison backend; measured on bounded small semiprimes.

M215

Kangaroo multiplicative-order finder



D / OPERATION DETAIL



Vector detail / 1.93× main-view scale

HOW THE OPERATION WORKS

Search successively larger exponent intervals with collision walks to locate a multiple/order relation.

READ THE THREE STAGES

01 exponent walk A
02 matching modular value
03 exponent walk B

WHAT WAS ACTUALLY DONE

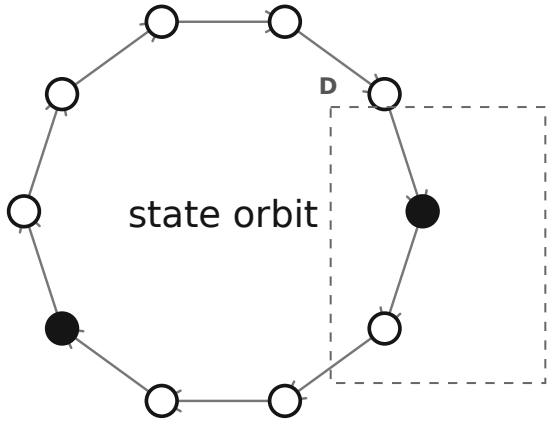
The classical kangaroo order backend is benchmarked separately from elliptic-curve kangaroo. Matching the name does not mean these factoring experiments recovered Bitcoin private keys.

ORIGINAL FINDING / EXECUTED RECORD

Expanding exponent-interval method compared with BSGS and Brent; not a secp256k1 scalar recovery.

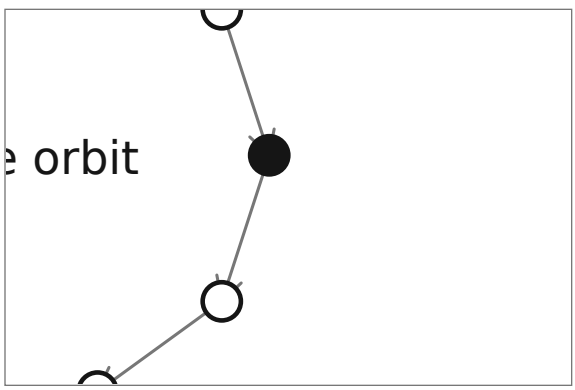
M216

Pollard-rho factoring fast path



01	02	03
modular iterations	cycle-related pair	gcd factor test

D / OPERATION DETAIL



Vector detail / 1.25× main-view scale

HOW THE OPERATION WORKS

Pollard rho searches for a nontrivial common divisor through repeated modular iteration and cycle-related gcds.

READ THE THREE STAGES

01 modular iterations
02 cycle-related pair
03 gcd factor test

WHAT WAS ACTUALLY DONE

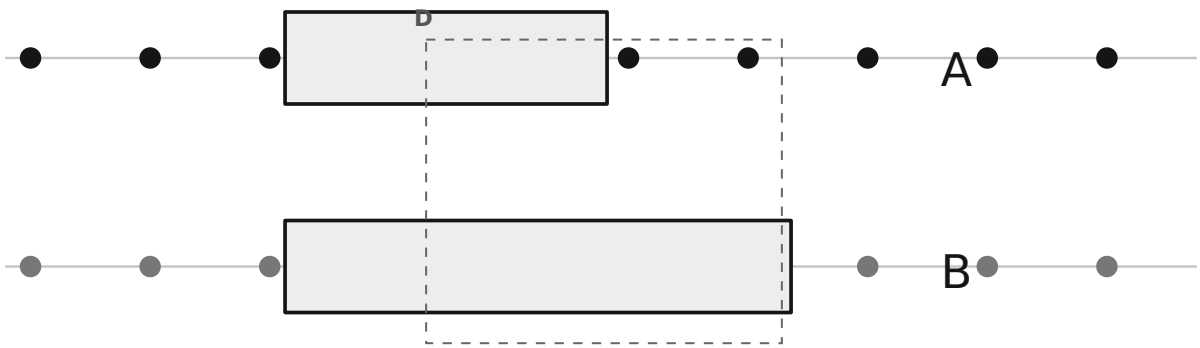
The high-level factor command can take this shortcut before order finding. Its reported factorization of 3233 is correctly labeled rho, not evidence of quantum or even BSGS execution on that path.

ORIGINAL FINDING / EXECUTED RECORD

Canonical factor command reports its shortcut explicitly; factoring 3233 must not be relabeled as a quantum Shor run.

M217

Python/GMP/Rust controlled order benchmarks

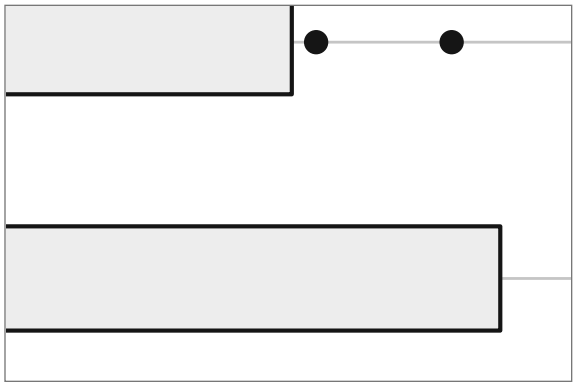


01 same modulus and base

02 Python / GMP / Rust

03 order and time checks

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Hold modulus, base and repetition schedule fixed while replacing the order-finding arithmetic backend.

READ THE THREE STAGES

01 same modulus and base

02 Python / GMP / Rust

03 order and time checks

WHAT WAS ACTUALLY DONE

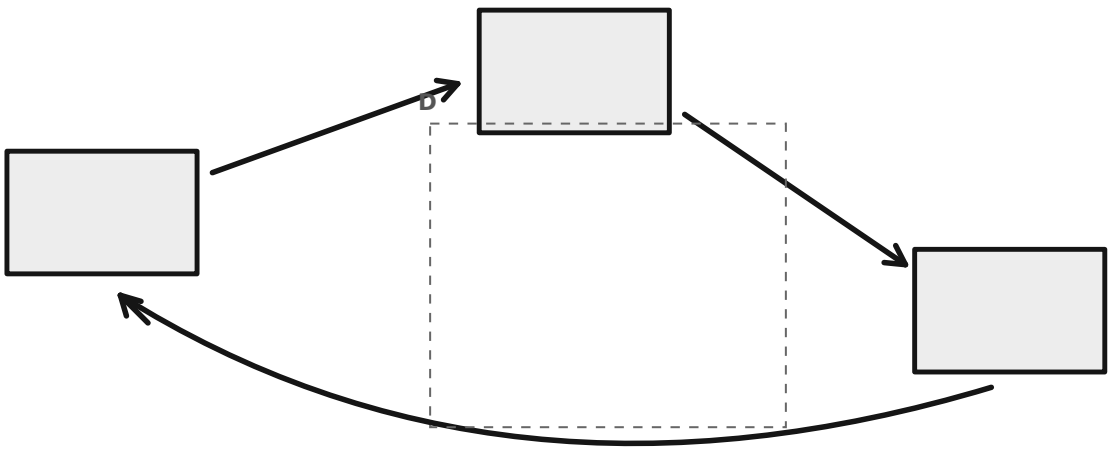
The saved matrix records per-backend times and verifies identical orders. This isolates local implementation constants; differences across operand sizes are not a clean universal scaling-law estimate.

ORIGINAL FINDING / EXECUTED RECORD

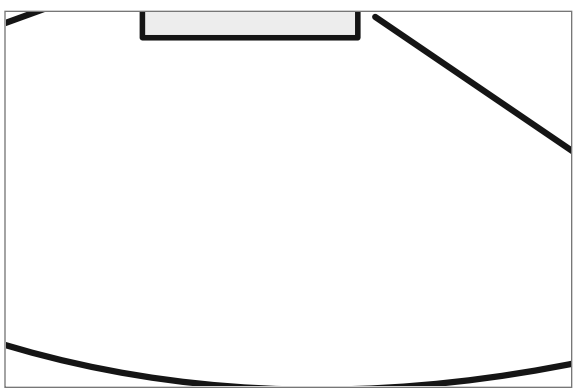
Shared modulus/base backend matrix records finite speedups; source/runtime differences are pinned in historical manifests.

M218

Adaptive Shor simulation trial budgets



D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

Measure trial success, compute a confidence-aware required budget and adjust the number of repetitions to meet a target.

READ THE THREE STAGES

01 trial outcomes
02 estimate success rate
03 adapt repetition budget

WHAT WAS ACTUALLY DONE

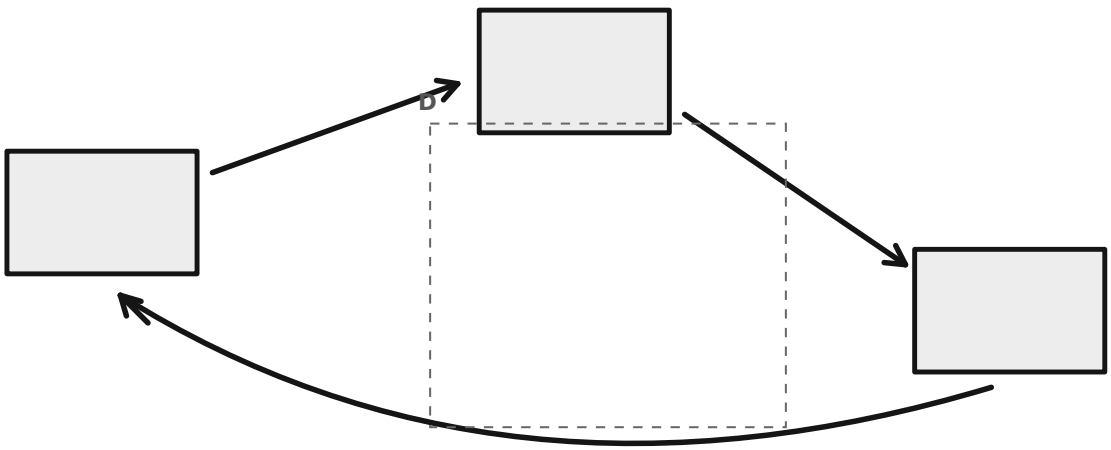
The factoring loop changes trial counts and ramps a bounded test size. The feedback improves a classical experimental procedure, not the underlying cryptographic problem's entropy.

ORIGINAL FINDING / EXECUTED RECORD

Empirical/analysis/improvement loop adjusts trial counts with Wilson-bound and runtime metrics.

M219

UCB/EWMA/early-stop method selection

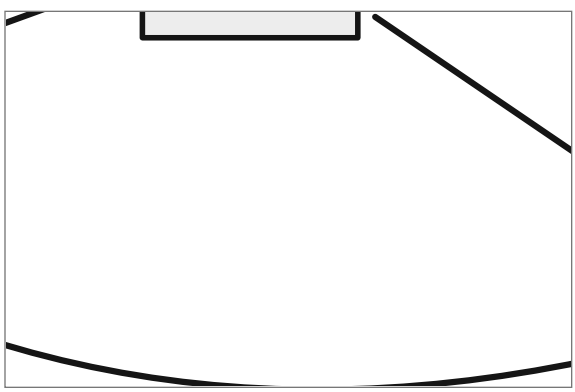


01 backend reward streams

02 UCB / EWMA policy

03 explore or stop

D / OPERATION DETAIL



Vector detail / 1.13× main-view scale

HOW THE OPERATION WORKS

An upper-confidence policy explores backend choices while EWMA smooths noisy measurements and early stopping limits unnecessary trials.

READ THE THREE STAGES

01 backend reward streams
02 UCB / EWMA policy
03 explore or stop

WHAT WAS ACTUALLY DONE

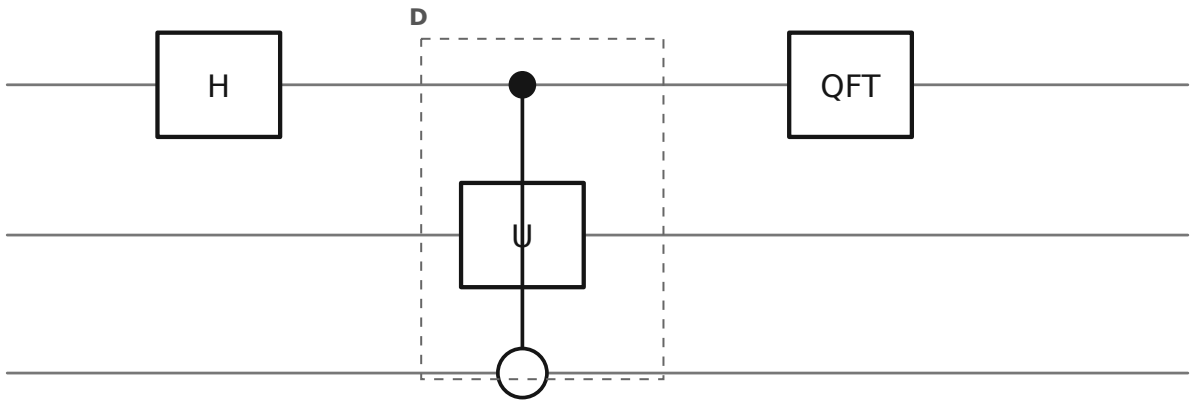
Archive modules describe/use these policy choices within simulation loops. Better simulated reward can reflect assumptions in the model and needs real execution validation before any cryptanalytic interpretation.

ORIGINAL FINDING / BRANCH/IMPLEMENTATION

Archive describes adaptive backend policy and smoothing; simulator optimization is not target-scale quantum cryptanalysis.

M220

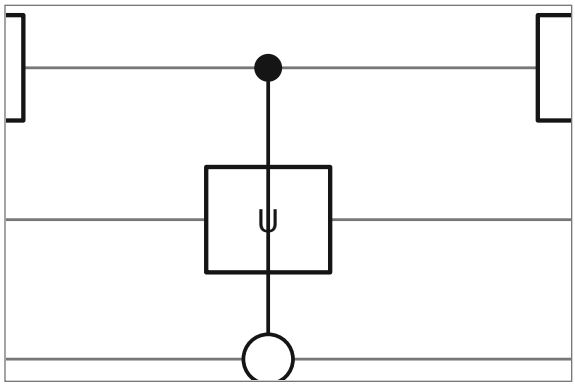
Semiprime-per-puzzle circuit/manifest analogy



schematic resource pipeline

01	02	03
puzzle-derived semiprime	order / gcd pipeline	resource analogy

D / OPERATION DETAIL



Vector detail / 1.01× main-view scale

HOW THE OPERATION WORKS

Construct a deterministic semiprime challenge from puzzle data to illustrate a comparable input size for factoring-resource calculations.

READ THE THREE STAGES

01 puzzle-derived semiprime
02 order / gcd pipeline
03 resource analogy

WHAT WAS ACTUALLY DONE

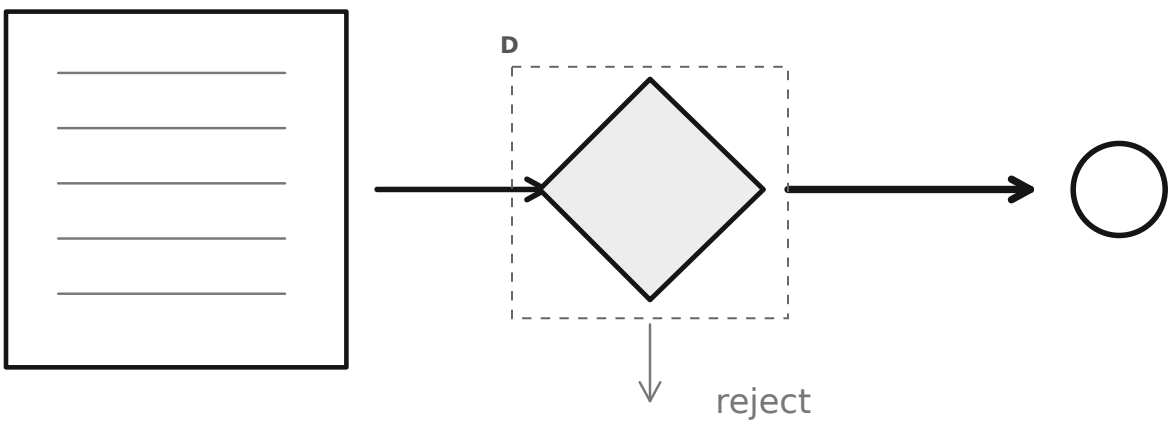
Circuit/manifest commands prepare that separate factoring task. A factor of the constructed integer is not mathematically shown to disclose the original elliptic-curve scalar.

ORIGINAL FINDING / AUDITED/CORRECTED

Factoring challenges derived from puzzle data are resource-scale analogies, not reductions recovering puzzle scalars.

M221

secp256k1 quantum ECDLP resource mode

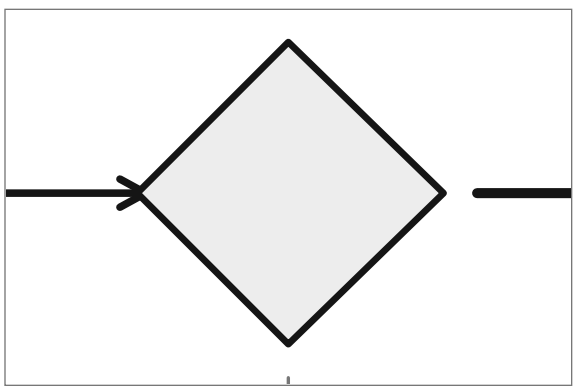


01
declared problem size

02
exact-small / model-large

03
modeled success

D / OPERATION DETAIL



Vector detail / 1.37× main-view scale

HOW THE OPERATION WORKS

The ECDLP mode combines real small-range BSGS with a probability-and-time oracle model beyond configured exact limits.

READ THE THREE STAGES

01 declared problem size
02 exact-small / model-large
03 modeled success

WHAT WAS ACTUALLY DONE

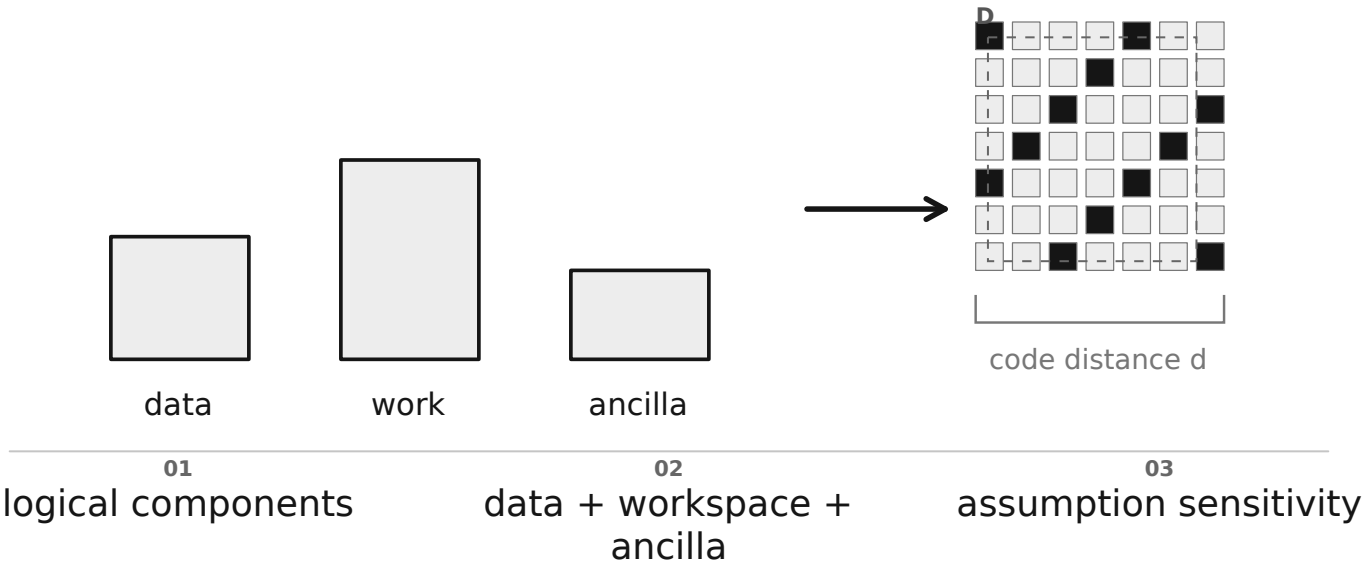
In shor_sim/ecdsa.py, exact branches call the BSGS routine; larger Shor/hybrid trials draw success from rng.random() and return formula-based elapsed time. Even the small kangaroo-labeled profile calls BSGS in its exact branch.

ORIGINAL FINDING / IMPLEMENTED/MODEL

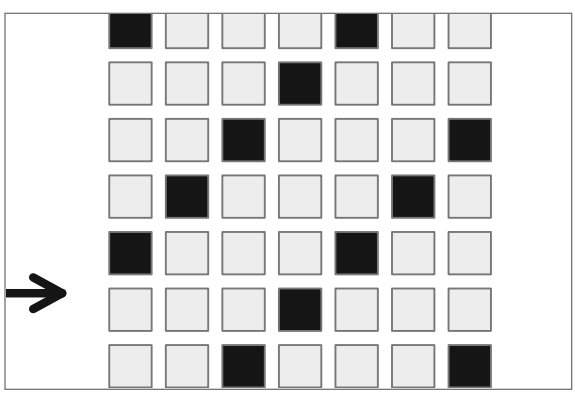
Models the relevant discrete-log task separately from integer factoring; no executed target-scale quantum circuit is established.

M222

520-logical-qubit resource sensitivity model



D / OPERATION DETAIL



Vector detail / 1.54× main-view scale

HOW THE OPERATION WORKS

Estimate logical resources under a simplified model and vary physical error/architecture assumptions to see their effect.

READ THE THREE STAGES

01 logical components
02 data + workspace + ancilla
03 assumption sensitivity

WHAT WAS ACTUALLY DONE

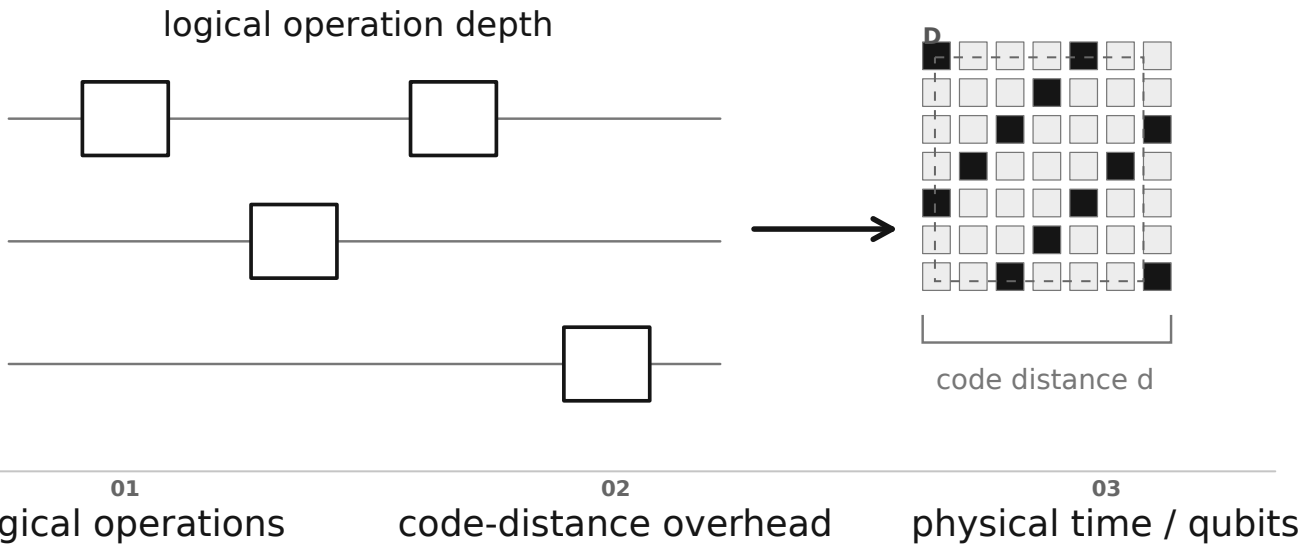
The standalone audit identifies omitted reversible-arithmetic workspace and other architecture costs. A 520-qubit headline is a sensitivity-model output, not a complete executable Bitcoin-breaking circuit.

ORIGINAL FINDING / AUDITED/CORRECTED

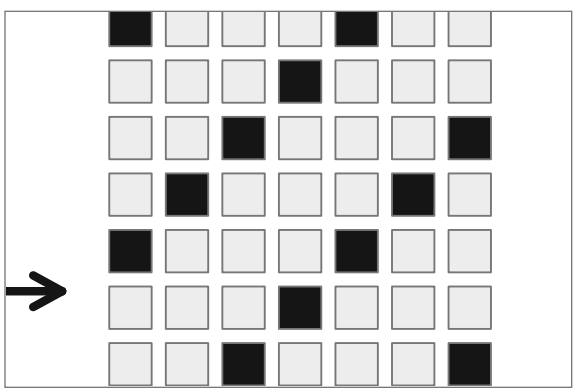
Standalone audit excludes required reversible workspace and full architecture; headline counts are not a complete Bitcoin-breaking machine.

M223

Surface-code/platform timing estimates



D / OPERATION DETAIL



Vector detail / 1.54× main-view scale

HOW THE OPERATION WORKS

Map logical operation counts to physical qubits and runtime using code distance, gate/cycle assumptions and platform parameters.

READ THE THREE STAGES

01 logical operations
02 code-distance overhead
03 physical time / qubits

WHAT WAS ACTUALLY DONE

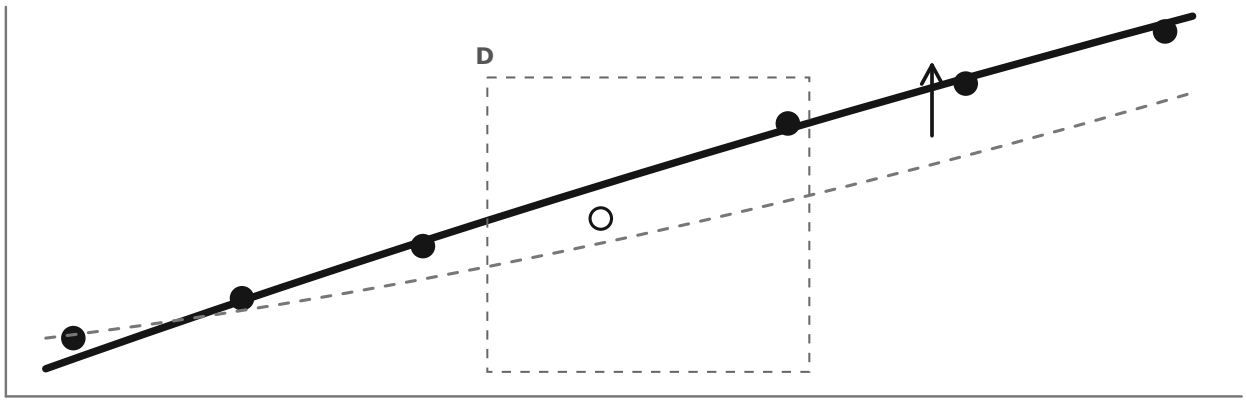
quantum.py calculates resource estimates for chosen profiles. The outputs are model-dependent arithmetic, not measured throughput on an available quantum computer.

ORIGINAL FINDING / IMPLEMENTED/MODEL

Compare physical errors, distances and cycle times for several platforms; estimates depend on explicit architecture assumptions.

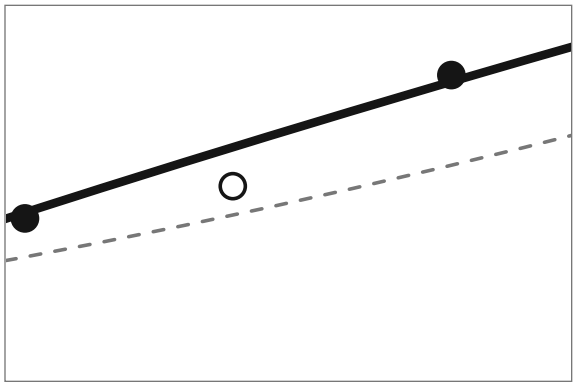
M224

Wolfram/mask sensitivity extensions



01	02	03
baseline assumptions	change mask / parameters	compare estimates

D / OPERATION DETAIL



Vector detail / 1.17× main-view scale

HOW THE OPERATION WORKS

Feed alternative masks, assumptions or external analytical outputs into the resource/sensitivity model and compare derived estimates.

READ THE THREE STAGES

01 baseline assumptions
02 change mask / parameters
03 compare estimates

WHAT WAS ACTUALLY DONE

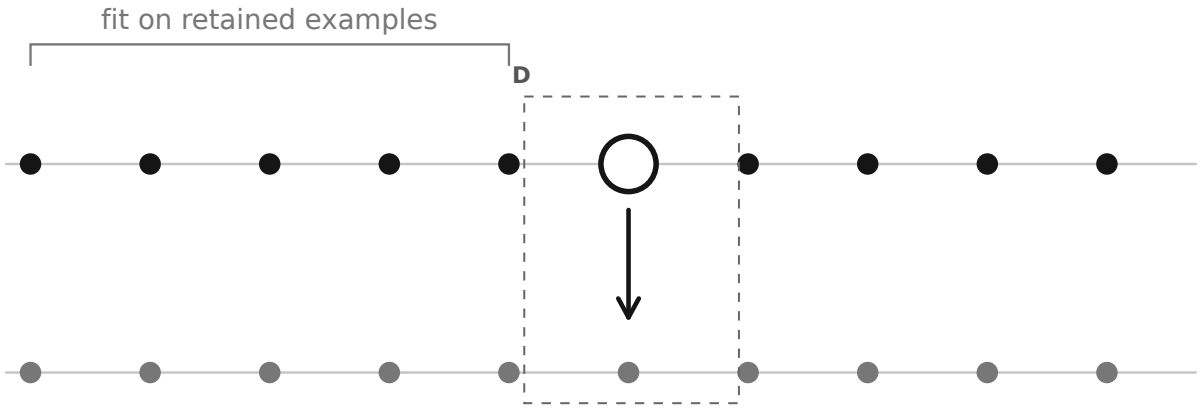
The standalone Wolfram-named modules implement follow-up model calculations. They do not construct or execute a target-sized quantum state evolution that reveals a puzzle key.

ORIGINAL FINDING / IMPLEMENTED/MODEL

Standalone modules explore model sensitivity and follow-up calculations, not recovered puzzle keys.

M225

Train/holdout adaptive-trial study

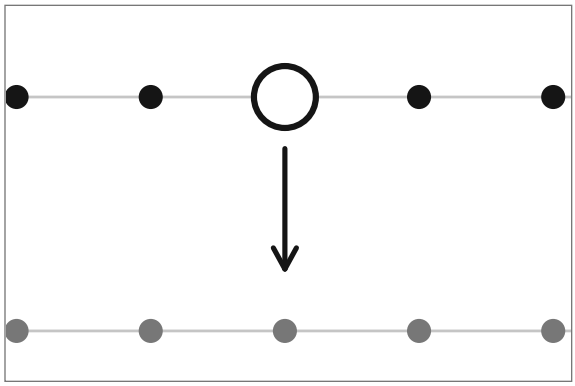


01 policy training stream

02 freeze trial budget

03 held-out streams

D / OPERATION DETAIL



Vector detail / 1.12× main-view scale

HOW THE OPERATION WORKS

Freeze a trial-budget policy on one stream, then evaluate it on additional held-out random streams to check whether tuning generalizes.

READ THE THREE STAGES

01 policy training stream
02 freeze trial budget
03 held-out streams

WHAT WAS ACTUALLY DONE

The latest factoring report records its bounded train/holdout trials and limitations. Repeated success on small semiprimes validates that experiment, not quantum-scale secp256k1 recovery.

ORIGINAL FINDING / EXECUTED RECORD

Latest factoring report includes additional controlled trial streams; conclusions remain limited to classical simulation.